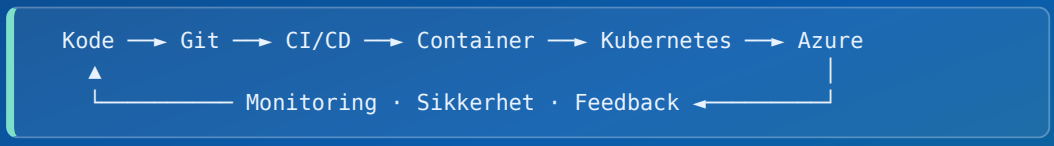
LÆREBOK · NOROFF DEVOPS ENGINEERING

DEVOPS, CLOUD & MICROSOFT AZURE

Den komplette guiden fra nybegynner til profesjonell



IT-grunnlag · Nettverk · Linux · Python · Git · Cloud · Azure (AZ-900/AZ-104)
DevOps · Docker · Kubernetes · Terraform · CI/CD · Sikkerhet · MLOps
12 praktiske prosjekter · Quiz med fasit · Ordliste · 12-måneders læringsplan

Utgave oktober 2026 · Skrevet på norsk med engelske fagbegreper

Innholdsfortegnelse

Introduksjon

- Velkommen
- Hvordan bruke boken
- En ærlig forventningsavklaring
- Om kostnader og oppdatert informasjon
- Læringsreisen – oversikt
- Produksjonsplan og kapitteloversikt

Del 1 – Forstå IT fra bunnen

- Hva er IT?
- Hvordan fungerer en datamaskin?
- Hva er et operativsystem?
- Hva er en server?
- Hva er en applikasjon?
- Hva er en database?
- Hvordan fungerer internett?
- Hva er en IP-adresse?
- Hva er DNS?
- Hva er HTTP og HTTPS?
- Hva er et API?
- Hva skjer når du åpner en nettside?
- Klient–server-arkitektur
- Hva er virtualisering?
- Hvordan alt henger sammen
- Kapitteloppsummering
- Øvingsoppgaver
- Quiz

Del 2 – Nettverk for DevOps

- OSI- og TCP/IP-modellen – kartet over nettverk
- IP-adresser: IPv4 og IPv6
- Subnetting og CIDR
- DNS – internettets telefonkatalog
- DHCP
- TCP og UDP
- Porter
- Ruter og svitsj
- Brannmur (*firewall*)
- NAT – Network Address Translation
- VPN
- Lastbalansering (*load balancing*)
- Proxy og reverse proxy

SSL/TLS og sertifikater
HTTP-statuskoder
SSH – Secure Shell
Nettverksfeilsøking – en systematisk metode
Hvordan en DevOps Engineer bruker nettverk i praksis
Kapitteloppsummering
Øvingsoppgaver
Quiz

Del 3 – Linux fra nybegynner til profesjonell

Hva er Linux, og hvorfor bruker DevOps det?
Slik får du en Linux-maskin å øve på
Terminal og shell
Filsystemet
Grunnleggende kommandoer
Brukere og grupper
Rettigheter (*permissions*)
Prosesser
Tjenester (*services*) med systemd
Pakkehåndtering (*package management*)
SSH i praksis
Miljøvariabler (*environment variables*)
Bash-scripting
Cron – planlagte jobber
Logger
Systemovervåking
Linux-sikkerhet – grunnregler
Feilsøking på Linux – metode
Kapitteloppsummering
Øvingsoppgaver
Quiz

Del 4 – Programmering for DevOps

Hva er programmering egentlig?
Python fra bunnen
JSON og YAML
API-kall med Python
Feilhåndtering (*error handling*)
Logging
Kommandolinjeargumenter
Kjøre andre programmer fra Python
Bash eller Python?
Python for Cloud: Azure SDK
Prosjektskript: helsesjekk med rapport
Testing – en smakebit
Kodekvalitet

Kapitteloppsummering
Øvingsoppgaver
Quiz

Del 5 – Git, GitHub og versjonskontroll

Hva er versjonskontroll?
Git og GitHub – ikke det samme
Installasjon og oppsett
Kjernebegrepene
Jobbe med GitHub: remote, clone, push og pull
Pull requests
Angre ting i Git
Git-arbeidsflyter (*workflows*)
Samarbeid på GitHub
GitHub Actions – en første smakebit
Praktisk prosjekt: fra null til samarbeid
Kapitteloppsummering
Øvingsoppgaver
Quiz

Del 6 – Cloud Computing fra A til Å

Hva er Cloud Computing?
Hvordan fungerer skyen fysisk?
Sky vs. on-premises
Deployeringsmodeller: public, private, hybrid
Tjenestemodeller: IaaS, PaaS, SaaS
Regioner og tilgjengelighetssoner
Skalering, elastisitet, høy tilgjengelighet og feiltoleranse
Sky-tjenestekategorier
Sky-sikkerhet og modellen for delt ansvar
Prising og kostnadsoptimalisering
AWS, Azure og Google Cloud – en sammenligning
Cloud-native og Well-Architected
Kapitteloppsummering
Øvingsoppgaver
Quiz

Del 7 – Microsoft Azure fra A til Å

7.1 Grunnmur, identitet og styring
7.2 Nettverk og compute
7.3 Lagring, data, sikkerhet og drift
7.4 Azure DevOps, Azure Pipelines og Bicep
Del 7 – kapitteloppsummering
Del 7 – quiz
7.5 AZ-900: Microsoft Azure Fundamentals
7.6 AZ-104: Microsoft Azure Administrator

Del 8 – DevOps Engineering

- Hva er DevOps?
- Hvorfor oppstod DevOps?
- DevOps-kultur
- Agile
- Scrum
- SDLC – livssyklusen for programvareutvikling
- DevOps-livssyklusen
- CI/CD – kontinuerlig integrasjon, levering og deploy
- Automatisering
- Infrastruktur i DevOps
- Deploy-strategier
- Monitoring og observability
- Incident management – hendelseshåndtering
- SRE – Site Reliability Engineering
- Platform Engineering
- DevSecOps
- En typisk arbeidsdag for en DevOps Engineer
- Kapitteloppsummering
- Øvingsoppgaver
- Quiz

Del 9 – Docker og containere

- Hva er containere?
- Containere vs. virtuelle maskiner
- Hva er Docker?
- Images og containere
- Dockerfile
- Volumes – data som overlever containeren
- Nettverk i Docker
- Docker Compose
- Registre (*registries*)
- Container-sikkerhet
- Feilsøking av containere
- Prosjekt: bygg, kjør og publiser en applikasjon
- Kapitteloppsummering
- Øvingsoppgaver
- Quiz

Del 10 – Kubernetes

- Hvorfor finnes Kubernetes?
- Arkitekturen: cluster og noder
- kubectl og første møte
- Pods
- Deployments
- Services

- Ingress og Gateway API
- ConfigMaps og Secrets
- Namespaces
- Persistent lagring
- Autoskalering
- Helm – pakkehåndtereren for Kubernetes
- Kubernetes-nettverk
- Kubernetes-sikkerhet
- Overvåking av Kubernetes
- Azure Kubernetes Service i praksis
- Feilsøking i Kubernetes
- Kapitteloppsummering
- Øvingsoppgaver
- Quiz

Del 11 – Infrastructure as Code

- Hva er Infrastructure as Code?
- Terraform
- Bicep – kort repetisjon
- Ansible og configuration management
- GitOps
- Kapitteloppsummering
- Øvingsoppgaver
- Quiz

Del 12 – CI/CD og automatisering

- Hva er en pipeline?
- Testing i pipelinen
- GitHub Actions i dybden
- Azure DevOps Pipelines
- Jenkins
- Artefakter
- Hemmeligheter i pipelines
- Automated deployment
- Rollback
- Blue-green deployment
- Canary deployment
- Komplett CI/CD-pipeline: fra kode til produksjon
- Kapitteloppsummering
- Øvingsoppgaver
- Quiz

Del 13 – Cloud security og monitoring

- Del A – Sikkerhet
- Del B – Monitoring og observability
- Del C – Hendelseshåndtering og feilsøking
- Kapitteloppsummering

Øvingsoppgaver
Quiz

Del 14 – AI Infrastructure og MLOps

Hva er AI-infrastruktur?
Hva er MLOps?
Hvordan kjøres AI-modeller i Cloud?
CPU vs. GPU
AI-arbeidslaster (*AI workloads*)
Model deployment og model serving
ML-pipelines
Model versioning – versjonering av modeller
Model monitoring – overvåking av modeller
Azure Machine Learning
Azure AI-tjenester og Microsoft Foundry
Kubernetes for AI
Infrastruktur for LLM-er
Scaling AI applications – skalering
AI-sikkerhet
Kostnader ved AI-infrastruktur
Fra DevOps Engineer til MLOps / AI Infrastructure Engineer
Kapitteloppsummering
Øvingsoppgaver
Quiz

Del 15 – Virkelige prosjekter

Prosjekt 1 – Sett opp og herd en Linux-server
Prosjekt 2 – Python-automatisering: Azure-kostnadsjeger
Prosjekt 3 – Azure-VM med kode: CLI-skript og cloud-init
Prosjekt 4 – Profesjonelt GitHub-repo og portefølje
Prosjekt 5 – Containeriser en app med Docker
Prosjekt 6 – Publiser en nettside på Azure med CI/CD
Prosjekt 7 – CI/CD-pipeline til Azure App Service
Prosjekt 8 – Azure-infrastruktur med Terraform og pipeline
Prosjekt 9 – Deploy en app på Kubernetes med Helm
Prosjekt 10 – Komplett Cloud/DevOps-plattform
Prosjekt 11 – MLOps: modell-API med CI/CD og overvåking
Prosjekt 12 – Observability og hendelsesøvelse (game day)
Slik presenterer du prosjektene

Vedlegg A – Ordliste

Vedlegg B – Hurtigreferanser

Linux
Nettverksfeilsøking
Git
Docker

Kubernetes (kubectl)
Azure CLI
Terraform
Python – mønstre du ofte trenger
KQL og PromQL
Subnetting

Vedlegg C – Oversikt over sertifiseringer

Vedlegg D – Kilder og videre læring
Offisiell dokumentasjon
Bøker
Gratis øvingsressurser
Norske fagmiljøer

Vedlegg E – Personlig 12-måneders læringsplan
Måned 1 – IT-grunnlag og nettverk
Måned 2 – Linux
Måned 3 – Python, Bash og Git
Måned 4 – Cloud Computing og Azure-grunnmur
Måned 5 – Azure-tjenester og AZ-900
Måned 6 – DevOps-kultur, Azure DevOps og Bicep
Måned 7 – Docker
Måned 8 – Kubernetes
Måned 9 – Infrastructure as Code
Måned 10 – CI/CD og AZ-104-forberedelse
Måned 11 – Sikkerhet, monitoring og AZ-104
Måned 12 – AI-infrastruktur, sluttprosjekt og jobbsøking
Når kan jeg begynne å søke?
Månedlig sjekkliste
Avslutning

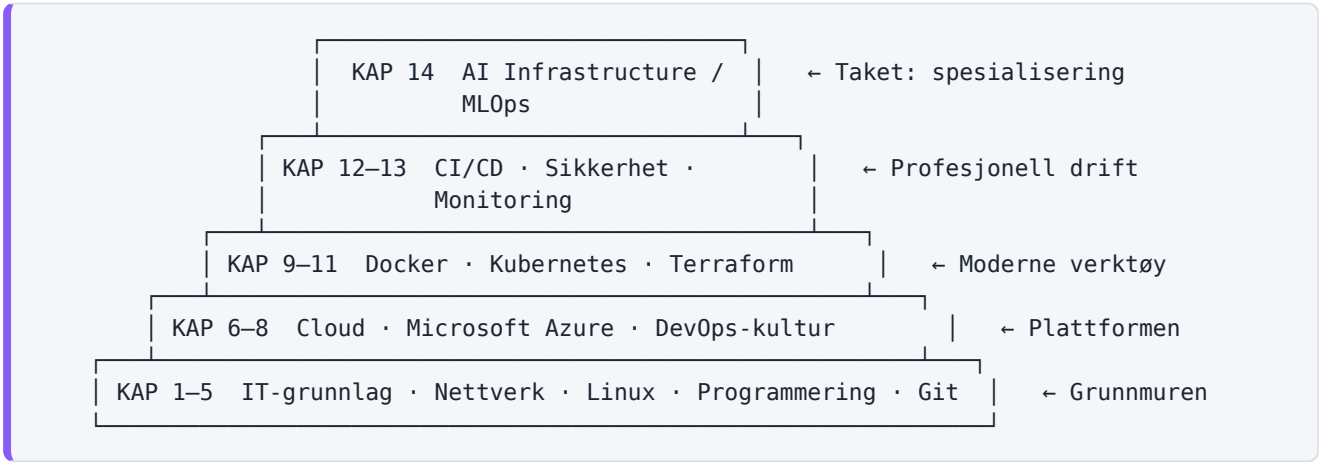
Introduksjon

Velkommen

Denne boken er skrevet for deg som studerer DevOps Engineering, er ganske ny innen IT, og ønsker å forstå **hvorfor** ting fungerer – ikke bare pugge ord.

Mange lærebøker starter med «Kubernetes er en orkestreringsplattform for containere» og forventer at du skal nikke. Denne boken gjør det motsatte. Vi starter med hva en datamaskin faktisk er, bygger videre med nettverk, Linux og programmering, og først når grunnmuren står, bygger vi Cloud, Azure, Docker, Kubernetes, Terraform, CI/CD, sikkerhet og til slutt AI-infrastruktur oppå.

Tenk på boken som et hus:



Hvis du hopper rett til taket, faller huset sammen. Hvis du bygger grunnmuren skikkelig, blir alt det andre mye enklere.

Hvordan bruke boken

Læringsmetoden i hvert kapittel

Hvert viktig konsept forklares med den samme faste oppskriften, slik at du alltid vet hvor du finner hva:

Steg	Spørsmål	Hva du får
1	**Hva er det?**	Forklaring med helt enkle ord
2	**Hvorfor finnes det?**	Problemet teknologien løser
3	**Hvordan fungerer det?**	Steg-for-steg teknisk forklaring
4	**Hverdagsanalogi**	Restaurant, lager, bygning, bil ...

Steg	Spørsmål	Hva du får
5	**Fra arbeidslivet**	Hvordan en ekte bedrift bruker det
6	**Diagram**	Visuell forklaring
7	**Praktisk**	Kode, kommandoer, oppsett
8	**Vanlige feil**	Det nybegynnere misforstår
9	**Husk**	Kort oppsummering
10	**Test deg selv**	Kontrollspørsmål med forklarte svar

For små begreper slår vi noen av stegene sammen, slik at boken ikke blir kunstig lang. For de store temaene (for eksempel Docker, Kubernetes og Azure-tjenestene) får du hele oppskriften.

Faste bokser i teksten

> **Hverdagsanalogi:** Sammenligner teknologien med noe du kjenner fra før.

> **Fra arbeidslivet:** Viser hvordan dette brukes i en ekte bedrift.

> **Vanlig feil:** Noe nybegynnere (og erfarne!) ofte gjør feil.

> **Husk:** Det viktigste du må ta med deg.

> **Kostnad:** Advarsel om ting som kan koste penger i Cloud.

> **Fordypning:** Avansert stoff. Du kan hoppe over dette første gang du leser.

Kodeblokker og terminal

Kommandoer du skal skrive i terminalen vises slik. Tegnet `$` betyr «skriv dette som vanlig bruker» og skal **ikke** skrives inn. Linjer uten `$` er resultatet du får tilbake:

```
$ whoami
student
```

Kommentarer i kode starter med `#` og forklarer hva linjen gjør. De er der for deg – maskinen ignorerer dem.

Engelske begreper

IT-bransjen snakker engelsk, også i Norge. Første gang et begrep dukker opp, får du både det norske og det engelske ordet, for eksempel «lastbalansering (*load balancing*)». Senere bruker vi ofte det engelske ordet, fordi det er det du vil høre på jobb. Bakerst i boken finner du en ordliste.

Tre måter å lese boken på

1. **Fra start til slutt** – anbefalt hvis du er helt ny. Kapitlene bygger på hverandre.
2. **Som oppslagsverk** – bruk innholdsfortegnelsen, ordlisten og hurtigreferansene.
3. **Prosjektdrevet** – start med et prosjekt i kapittel 15, og les kapitlene prosjektet viser til når du står fast.

> **Husk:** Du lærer DevOps med fingrene, ikke bare med øynene. Skriv kommandoene selv. Gjør feil. Fiks dem. Det er slik ferdighetene faktisk bygges.

En ærlig forventningsavklaring

Å lese denne boken gjør deg ikke til senioringeniør. Ingen bok kan det. Det boken *kan* gjøre, er å gi deg:

- et solid mentalt kart over hvordan alle bitene henger sammen,
- presise forklaringer du kan komme tilbake til,
- oppgaver og prosjekter som bygger ekte, praktiske ferdigheter,
- en realistisk plan for de neste tolv månedene.

Erfaring kommer av å bygge, feile, feilsøke og bygge igjen – helst sammen med andre. En god juniorkandidat er ikke en som kan alt, men en som forstår grunnprinsippene, kan lære raskt og kan vise frem ekte prosjekter.

Om kostnader og oppdatert informasjon

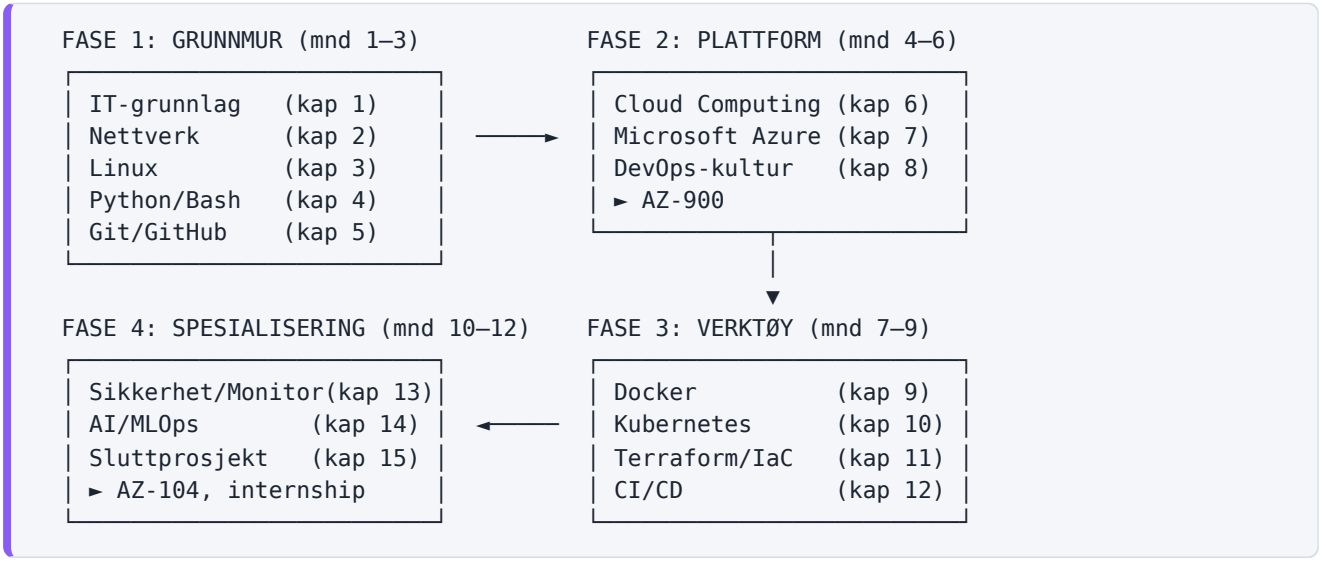
> **Kostnad:** Cloud-tjenester koster ekte penger. Gratiskontoer og studentkreditter har grenser. Hvert prosjekt i boken har en egen seksjon om **opprydding**. Følg den hver gang. Sett opp et budsjettvarsel (*budget alert*) i Azure før du gjør noe annet – det viser vi i kapittel 7.

Teknologi endrer seg raskt. Tjenester får nye navn (Azure Active Directory heter nå *Microsoft Entra ID*), priser endres, og verktøy får nye versjoner. Informasjonen i boken er skrevet ut fra offentlig dokumentasjon fra Microsoft Learn, Docker, Kubernetes, HashiCorp, GitHub og Python slik den var kjent per **oktober 2026**. Der noe er spesielt utsatt for endring, står det en merknad. **Sjekk alltid of-**

fisiell dokumentasjon før du stoler på priser, kvoter, eksamensinnhold og versjonsnumre. En samlet kildeliste med lenker finner du i vedlegg D.

Boken bruker tekstbaserte diagrammer (laget med tegn) i stedet for skjermbilder. Det er et bevisst valg: skjermbilder av Azure Portal blir raskt utdaterte fordi Microsoft endrer utseendet ofte, mens diagrammene forklarer prinsippene som ikke endrer seg. Der et skjermbilde ville vært nyttig, beskriver teksten hva du skal se etter i portalen.

Læringsreisen – oversikt



Produksjonsplan og kapitteloversikt

Boken er bygget opp i 15 deler pluss vedlegg. Her er planen for hele boken, med hva hvert kapittel inneholder og hvilke praktiske leveranser du sitter igjen med.

Del	Kapittel	Innhold	Du kan etterpå
1	Forstå IT fra bunnen	Datamaskin, OS, server, database, applikasjon, frontend/backend, internett, IP, DNS, HTTP(S), API, klient–server, virtualisering	Forklare hva som skjer når du åpner en nettside

Del	Kapittel	Innhold	Du kan etterpå
2	Nettverk for DevOps	IPv4/IPv6, subnetting/CIDR, DNS, DHCP, TCP/UDP, porter, ruter/svitsj, brannmur, VPN, NAT, lastbalansering, proxy, TLS, statuskoder, SSH, feilsøking	Planlegge et VNet og feilsøke tilkoblingsproblemer
3	Linux	Distribusjoner, shell, filsystem, brukere, rettigheter, prosesser, systemd, pakker, SSH, Bash, cron, miljøvariabler, logger, sikkerhet	Drifte en Linux-server fra terminalen
4	Programmering for DevOps	Python fra null, datatyper, løkker, funksjoner, JSON/YAML, API-kall, feilhåndtering, logging, Bash, automatisering, Azure SDK	Skrive egne automatiseringsskript
5	Git og GitHub	Repo, commit, branch, merge, PR, konflikter, arbeidsflyter, GitHub Actions, samarbeid	Jobbe profesjonelt med versjonskontroll
6	Cloud Computing	Datasentre, IaaS/PaaS/SaaS, regioner/soner, skalering, HA, DR, sikkerhet, prising, delt ansvar, AWS/Azure/GCP	Velge riktig skytjeneste for et behov
7	Microsoft Azure A–Å	30+ tjenester med opprettelse, sikkerhet, kostnad og øvelser; AZ-900 og AZ-104 med prøveeksamener	Administrere grunnleggende Azure-miljøer
8	DevOps Engineering	Kultur, Agile/Scrum, SDLC, CI/CD, deploystrategier, observability, hendelser, SRE, Platform Engineering, DevSecOps	Forklare DevOps i et intervju

Del	Kapittel	Innhold	Du kan etterpå
9	Docker	Images, containere, Dockerfile, Compose, registry, volumes, nettverk, sikkerhet, feilsøking	Containerisere en applikasjon
10	Kubernetes	Cluster, noder, pods, deployments, services, ingress, config, secrets, lagring, autoskalering, Helm, AKS	Deploye en app på Kubernetes
11	Infrastructure as Code	Terraform, state, moduler, remote state, Bicep, Ansible, GitOps	Bygge Azure-infrastruktur med kode
12	CI/CD og automatisering	GitHub Actions, Azure Pipelines, Jenkins, testing, artefakter, hemmeligheter, rollback, blue-green, canary	Bygge en pipeline fra kode til produksjon
13	Sikkerhet og overvåking	IAM, minste privilegium, kryptering, Zero Trust, skanning, backup, logger, metrikker, tracing, Prometheus, Grafana, hendelseshåndtering	Sikre og overvåke et system
14	AI Infrastructure og MLOps	GPU, modellservering, ML-pipelines, Azure ML, Azure AI-tjenester, LLM-infrastruktur, kostnader, karrierevei	Forstå hvordan AI kjøres i produksjon
15	Virkelige prosjekter	12 prosjekter med økende vanskelighetsgrad	Vise frem en sterk portefølje
A–E	Vedlegg	Ordliste, hurtigreferanser, sertifiseringer, kilder, 12-måneders læringsplan	Repetere og planlegge videre

Slik er hvert kapittel bygget opp: læringsmål → forklaringer med læringsmetoden → praktiske øvelser → kapitteloppsummering → quiz med fasit.

Kvalitetsprinsipper for boken:

- Ingen plassholdere og ingen «forklaring kommer senere».
- Alle kommandoer er ekte kommandoer fra offisielle verktøy.
- Grunnleggende stoff og fordypning holdes adskilt.
- Kostnader og gratisalternativer nevnes eksplisitt.
- Informasjon som endres ofte er merket med dato og kilde.

Del 1 – Forstå IT fra bunnen

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva CPU, RAM, lagring og operativsystem gjør, og hvordan de samarbeider,
- forklare forskjellen på en server og en vanlig PC,
- forklare hva en applikasjon, en database, frontend og backend er,
- beskrive steg for steg hva som skjer når du skriver `www.vg.no` i nettleseren,
- forklare IP-adresser, DNS, HTTP/HTTPS og API på et grunnleggende nivå,
- forklare klient–server-arkitektur og virtualisering.

Hva er IT?

Hva er det? IT (informasjonsteknologi, *information technology*) er all teknologi vi bruker for å **lagre, behandle og sende informasjon**. Det inkluderer maskinvare (*hardware*) som datamaskiner, servere og nettverksutstyr, og programvare (*software*) som operativsystemer, apper og databaser.

Hvorfor finnes det? Bedrifter og samfunnet har enorme mengder informasjon: kundedata, bestillinger, pasientjournaler, lønnsslipper. Uten IT måtte alt dette ligget i papirarkiver og blitt behandlet manuelt.

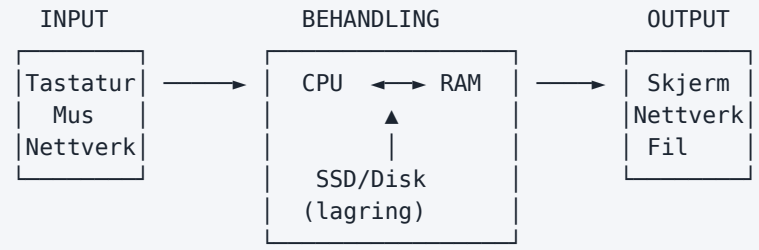
Rollene i IT. IT er et stort felt. Noen vanlige roller:

Rolle	Hva de gjør
Utvikler (*developer*)	Skriver koden som blir til applikasjoner
Driftstekniker (*operations / sysadmin*)	Holder servere og systemer i gang
Nettverksingeniør	Designer og drifter nettverk
DevOps Engineer	Bygger broen mellom utvikling og drift med automatisering
Cloud Engineer	Bygger og drifter løsninger i skyen (Azure, AWS, GCP)
Site Reliability Engineer (SRE)	Sørger for at systemer er pålitelige, ofte med kode
Sikkerhetsingeniør	Beskytter systemer mot angrep
MLOps / AI Infrastructure Engineer	Bygger plattformer som trener og kjører AI-modeller

> **Husk:** En DevOps Engineer må forstå litt av *alt* – maskinvare, nettverk, operativsystem, kode og skyen – fordi jobben er å få alle delene til å spille sammen.

Hvordan fungerer en datamaskin?

Hva er det? En datamaskin er en maskin som tar imot data (*input*), behandler dem (*processing*), lagrer dem (*storage*) og gir et resultat (*output*).



CPU – hjernen

Hva er det? CPU (*Central Processing Unit*, prosessor) utfører instruksjonene i programmene. Den regner, sammenligner og flytter data.

Hvordan fungerer det? CPU-en henter en instruksjon, tolker den og utfører den – milliarder av ganger i sekundet. Hastigheten måles i gigahertz (GHz). Moderne CPU-er har flere **kjerner** (*cores*), og hver kjerne kan jobbe med sin egen oppgave samtidig.

> **Hverdagsanalogi:** CPU-en er kokken på en restaurant. Flere kjerner er flere kokker. Fire kokker kan lage fire retter samtidig, men én enkelt rett blir ikke nødvendigvis raskere av at det står fire kokker rundt den.

> **Fra arbeidslivet:** Når du velger størrelse på en virtuell maskin i Azure, velger du antall **vCPU** (virtuelle CPU-kjerner). En liten webserver trenger kanskje 1–2 vCPU, mens en database for en nettbutikk kan trenge 8–16.

RAM – arbeidsbenken

Hva er det? RAM (*Random Access Memory*, arbeidsminne) er der datamaskinen holder det den jobber med *akkurat nå*. RAM er ekstremt rask, men **flyktig** (*volatile*): alt forsvinner når strømmen går.

> **Hverdagsanalogi:** RAM er kokkens benk. Ingrediensene kokken bruker akkurat nå ligger på benken. Er benken for liten, må kokken gå til lageret hele tiden, og alt går tregt.

> **Fra arbeidslivet:** Når en server går tom for RAM, begynner operativsystemet å flytte data til disk (*swapping*) – det gjør alt mye tregere. På Linux kan prosessen til og med bli drept av «OOM killer» (*Out Of Memory*). I Kubernetes ser du dette som statusen `OOMKilled`.

Lagring – SSD og harddisk

Hva er det? Lagring (*storage*) beholder data selv når maskinen er slått av: filer, programmer, operativsystemet.

Type	Hvordan	Fart	Typisk bruk
HDD (harddisk)	Roterende magnetiske plater	Treg	Billig masselagring, arkiv
SSD	Flash-minne, ingen bevegelige deler	Rask	Operativsystem, apper
NVMe SSD	SSD koblet direkte på PCIe-bussen	Svært rask	Databaser, krevende systemer

> **Hverdagsanalogi:** Lagring er restaurantens kjølerom. Alt ligger trygt der over natten, men det tar lengre tid å hente noe derfra enn fra benken (RAM).

Hastighetshierarkiet er viktig å forstå:

```
Raskest, minst, dyrest
  ▲   CPU-cache    (nanosekunder)
  │   RAM          (~100 nanosekunder)
  │   NVMe SSD     (mikrosekunder)
  │   HDD          (millisekunder)
  │   Nettverk/sky-lagring (millisekunder og mer)
  ▼
Tregest, størst, billigst
```

Hovedkort, nettverkskort og GPU

- **Hovedkort** (*motherboard*) kobler alle delene sammen.
- **Nettverkskort** (*NIC – Network Interface Card*) kobler maskinen til nettverket.
- **GPU** (*Graphics Processing Unit*) har tusenvis av små kjerner som er gode på å gjøre samme regneoperasjon på mye data samtidig. Opprinnelig laget for grafikk – i dag helt sentral for AI (se kapittel 14).

Hva er et operativsystem?

Hva er det? Operativsystemet (*Operating System, OS*) er programvaren som styrer maskinvaren og lar andre programmer bruke den. Eksempler: Windows, macOS, Linux, Android, iOS.

Hvorfor finnes det? Uten et OS måtte hvert program selv vite hvordan det skal snakke med akkurat din SSD, ditt nettverkskort og din skjerm. OS-et gir et felles grensesnitt.

Hva gjør OS-et?

1. **Prosessstyring** – bestemmer hvilket program som får bruke CPU-en når.
2. **Minnehåndtering** – deler ut RAM til programmene og hindrer dem i å lese hverandres minne.
3. **Filsystem** – organiserer data på disken i filer og mapper.
4. **Drivere** – snakker med maskinvaren.
5. **Sikkerhet** – brukere, passord og rettigheter.
6. **Nettverk** – sender og mottar data.



> **Hverdagsanalogi:** OS-et er restaurantsjefen. Kokkene (programmene) må spørre sjefen om plass på benken (RAM), tid ved komfyren (CPU) og hylleplass i kjølerommet (disk).

Kjernen (*kernel*) er selve hjertet i operativsystemet. Linux er egentlig navnet på en kjerne; rundt den bygges hele operativsystemer som Ubuntu og Red Hat (mer i kapittel 3).

Hva er en server?

Hva er det? En server er en datamaskin (eller et program) som **leverer tjenester til andre maskiner** over et nettverk. Ordet kommer fra engelsk *serve* – å servere.

Hvorfor finnes det? Når du åpner Instagram, ligger ikke bildene på telefonen din. De ligger på servere som leverer dem til deg og millioner av andre.

Server vs. vanlig PC

Egenskap	Vanlig PC	Server
Formål	Én bruker, interaktivt arbeid	Mange brukere, leverer tjenester
Oppetid	Slås av om kvelden	Går 24/7 i årevis
Skjerm/tastatur	Ja	Vanligvis ikke (*headless*), styres over nettverk
Operativsystem	Windows/macOS	Oftest Linux, også Windows Server
Maskinvare	Forbrukerdeler	Redundante strømforsyninger, ECC-RAM, RAID-disker
Plassering	Kontor/hjemme	Serverrom eller datasenter, i rack
Grafisk grensesnitt	Ja	Ofte bare kommandolinje

> **Vanlig feil:** Å tro at en server må være en spesiell type maskin. Teknisk sett kan din egen laptop være en server – det handler om *rollen*. Men i produksjon bruker man maskinvare som er laget for å tåle kontinuerlig drift.

> **Fra arbeidslivet:** I dag kjører de fleste «servere» som **virtuelle maskiner** eller **containere** i et datasenter eid av en skyleverandør. Du ser aldri den fysiske maskinen.

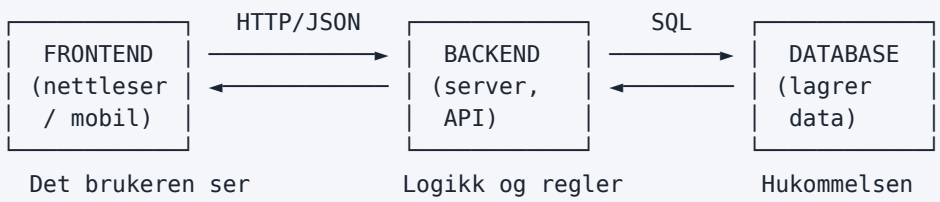
Vanlige typer servere (roller):

- **Webserver** – leverer nettsider (nginx, Apache, IIS).
- **Applikasjonsserver** – kjører forretningslogikken (Python, Java, Node.js).
- **Databaseserver** – lagrer strukturerte data (PostgreSQL, SQL Server, MySQL).
- **Filserver** – deler filer.
- **DNS-server** – oversetter navn til IP-adresser.
- **E-postserver** – sender og mottar e-post.

Hva er en applikasjon?

Hva er det? En applikasjon (app, program) er programvare som løser en oppgave for en bruker: en nettbank, Spotify, et lønnssystem.

Moderne applikasjoner består som regel av flere deler som snakker sammen:



Frontend og backend

Frontend er det brukeren ser og klikker på: knapper, skjemaer, bilder. Bygges med HTML (struktur), CSS (utseende) og JavaScript (oppførsel), ofte med rammeverk som React, Vue eller Angular. Frontend kjører i brukerens nettleser eller mobil.

Backend er det som skjer bak kulissene: sjekke passord, regne ut priser, hente data, sende e-post. Skrives i språk som Python, Java, C#, Go eller JavaScript (Node.js). Backend kjører på servere.

> **Hverdagsanalogi:** På en restaurant er **frontend** spisesalen og menyen – det gjesten ser. **Backend** er kjøkkenet. **Databasen** er lageret med råvarer. **Servitøren** som tar bestillingen fra salen til kjøkkenet er **API-et**.

> **Fra arbeidslivet:** En DevOps Engineer skriver sjelden frontend-kode, men må vite hvordan både frontend og backend bygges, testes og deployes. Frontend blir ofte statiske filer som legges på en CDN eller Azure Static Web Apps; backend kjøres i containere, App Service eller Kubernetes.

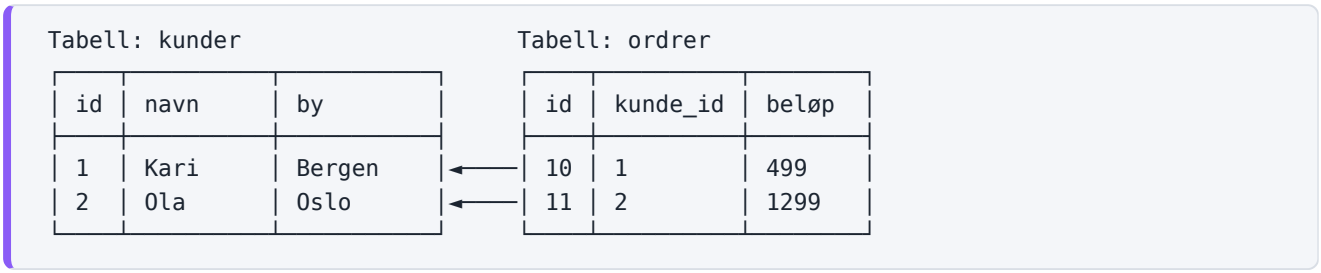
Hva er en database?

Hva er det? En database er et system for å **lagre, organisere og hente data** effektivt og trygt.

Hvorfor finnes det? Du *kunne* lagret alle kundene dine i en tekstfil. Men hva skjer når 10 000 brukere skriver til filen samtidig, når du må finne én kunde blant 50 millioner, eller når strømmen går midt i en betaling? Databaser løser dette.

Relasjonsdatabaser (SQL)

Data lagres i **tabeller** med rader og kolonner – som regneark med strenge regler. Tabeller kobles sammen med **nøkler**.



Man snakker med dem via **SQL** (*Structured Query Language*):

```sql
-- Hent navn og beløp for alle ordrer over 1000 kroner
SELECT kunder.navn, ordrer.beløp
FROM ordrer
JOIN kunder ON kunder.id = ordrer.kunde_id
WHERE ordrer.beløp > 1000;
```

Eksempler: PostgreSQL, MySQL, Microsoft SQL Server, Azure SQL Database.

NoSQL-databaser

Lagrer data i mer fleksible formater: dokumenter (JSON), nøkkel–verdi, grafer eller kolonnefamilier. Eksempler: MongoDB, Redis, Azure Cosmos DB.

	SQL (relasjonell)	NoSQL
Struktur	Fast skjema, tabeller	Fleksibel
Styrke	Konsistens, komplekse spørringer, transaksjoner	Skalering, fart, fleksible data
Typisk bruk	Bank, ordre, regnskap	Sesjoner, cache, IoT, kataloger

> **Husk:** Databaser er nesten alltid den mest kritiske delen av et system. En webserver kan du bygge på nytt på minutter. Mistet data er ofte borte for alltid. Derfor: **backup, backup, backup** – og test at du faktisk kan gjenopprette.

Hvordan fungerer internett?

Hva er det? Internett er et globalt nettverk av nettverk. Millioner av nettverk (hjemmenett, bedriftsnett, mobilnett, datasentre) er koblet sammen av internettleverandører (ISP-er) og sjøkabler, og de bruker felles regler – **protokoller** – for å snakke sammen.

Hvordan fungerer det? Data deles opp i små **pakker** (*packets*). Hver pakke har en avsender-adresse og en mottaker-adresse (IP-adresser). Rutere langs veien leser mottakeradressen og sender pakken videre i riktig retning, litt som postterminaler. Pakkene kan ta forskjellige veier og settes sammen igjen hos mottakeren.

> **Hverdagsanalogi:** Tenk deg at du skal sende en bok til en venn i Tromsø, men posten bare tar postkort. Du river ut sidene, nummererer dem, skriver adressen på hver og sender dem. Noen går via Trondheim, andre via Bodø. Vennen din sorterer dem i riktig rekkefølge og har boken igjen. Det er pakkesvitsjing.

Viktige protokoller du vil møte i hele boken:

Protokoll	Oppgave
IP	Adressering og ruting av pakker
TCP	Pålitelig levering, i riktig rekkefølge
UDP	Rask levering uten garanti
DNS	Oversetter navn til IP-adresser
HTTP/HTTPS	Overfører nettsider og API-data
TLS	Krypterer forbindelsen
SSH	Sikker fjernstyring av servere

Hva er en IP-adresse?

Hva er det? En IP-adresse er en unik adresse for en enhet på et nettverk, omtrent som en gate-adresse. Eksempel (IPv4): `192.168.1.25`.

En IPv4-adresse består av fire tall fra 0 til 255, separert med punktum. Det gir omtrent 4,3 milliarder adresser – og de er nesten brukt opp. Derfor finnes **IPv6**, med adresser som `2001:db8::1`, og et nesten uendelig antall adresser.

Det finnes **private** adresser (brukes inne i hjemmenett og bedriftsnett, f.eks. `192.168.x.x` og `10.x.x.x`) og **offentlige** adresser (synlige på internett). Dette går vi grundig gjennom i kapittel 2.

Hva er DNS?

Hva er det? DNS (*Domain Name System*) er internettets telefonkatalog. Den oversetter navn som `www.noroff.no` til IP-adresser som maskiner forstår.

Hvorfor finnes det? Mennesker husker navn; maskiner bruker tall. Dessuten kan IP-adressen bak et navn endres uten at brukerne merker noe.

```
$ nslookup www.example.com
Name:    www.example.com
Address: 93.184.215.14
```

(IP-adressen du får kan være en annen – den kan endres over tid.)

Vi går dypt inn i DNS i kapittel 2.

Hva er HTTP og HTTPS?

Hva er det? HTTP (*HyperText Transfer Protocol*) er språket nettlesere og webservere bruker for å snakke sammen. HTTPS er HTTP **kryptert med TLS**, slik at ingen underveis kan lese eller endre innholdet.

Hvordan fungerer det? Klienten sender en **forespørsel** (*request*), serveren svarer med en **respons** (*response*).

```
FORESPØRSEL (fra nettleser)          RESPONS (fra server)
GET /index.html HTTP/1.1             HTTP/1.1 200 OK
Host: www.example.com                Content-Type: text/html
User-Agent: Mozilla/5.0 ...          Content-Length: 1256

                                     <html>...</html>
```

En forespørsel har en **metode** som forteller hva klienten vil:

Metode	Betydning	Eksempel
GET	Hent data	Vis en produktside
POST	Opprett noe nytt	Send inn et skjema
PUT / PATCH	Oppdater	Endre profilen din
DELETE	Slett	Slett en kommentar

Responsen har en **statuskode**: `200` (OK), `404` (finnes ikke), `500` (serverfeil). Full liste i kapittel 2.

Du kan selv se HTTP i terminalen med `curl`:

```
$ curl -I https://www.example.com
HTTP/2 200
content-type: text/html
...
```

`-I` betyr «vis bare hodet (*headers*) i responsen».

> **Vanlig feil:** Å tro at HTTPS betyr at nettsiden er trygg. HTTPS betyr bare at *forbindelsen* er kryptert. En svindelside kan fint ha HTTPS.

Hva er et API?

Hva er det? Et API (*Application Programming Interface*) er en avtalt måte for programmer å snakke med hverandre på. Et web-API tar imot HTTP-forespørsler og svarer – oftest med data i **JSON**-format.

Hvorfor finnes det? Et program skal ikke trenge å vite *hvordan* et annet program er bygget inni. Det holder å kjenne «menyen»: hvilke forespørsler du kan sende og hva du får tilbake.

> **Hverdagsanalogi:** API-et er servitøren. Du (klienten) trenger ikke gå inn på kjøkkenet. Du bestiller fra menyen (API-dokumentasjonen), servitøren tar bestillingen til kjøkkenet (serveren) og kommer tilbake med maten (dataene).

Eksempel – hent data fra et offentlig API (GitHub):

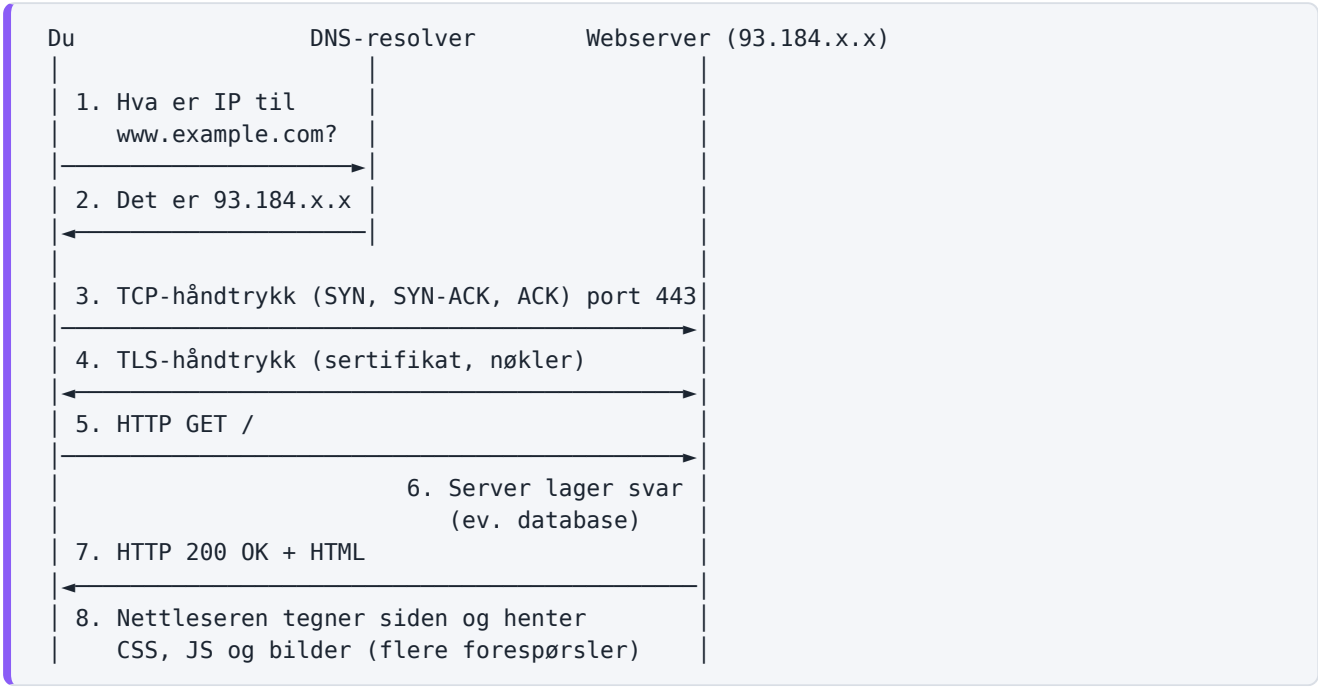
```
$ curl https://api.github.com/users/octocat
{
  "login": "octocat",
  "id": 583231,
  "name": "The Octocat",
  "public_repos": 8,
  ...
}
```

> **Fra arbeidslivet:** Nesten alt i Cloud er API-er. Når du klikker «Create» i Azure Portal, sender portalen egentlig et API-kall til Azure Resource Manager. Azure CLI, Terraform og Bicep sender de samme API-kallene. Dette er grunnen til at alt i skyen *kan* automatiseres.

REST er den vanligste stilen for web-API-er: ressurser har URL-er (`/users/42`), og man bruker HTTP-metodene for å jobbe med dem.

Hva skjer når du åpner en nettside?

Dette er et klassisk intervjuspørsmål. La oss gå gjennom hva som skjer når du skriver `https://www.example.com` og trykker Enter.

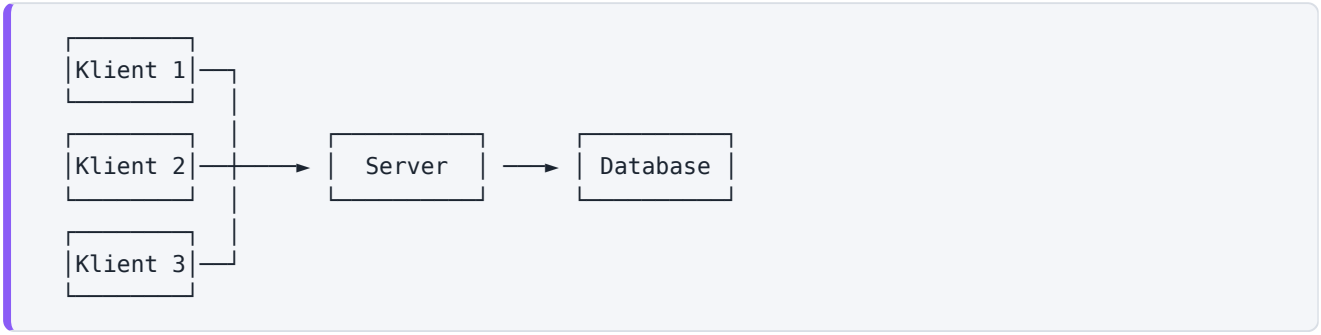


1. **URL tolkes.** Nettleseren ser protokoll (`https`), domenenavn (`www.example.com`) og sti (`/`).
2. **DNS-oppslag.** Nettleseren sjekker sin cache, deretter operativsystemets, deretter spør den en DNS-resolver (ofte hos internettleverandøren eller f.eks. 1.1.1.1). Resolveren finner IP-adressen.
3. **TCP-forbindelse.** Nettleseren åpner en forbindelse til IP-adressen på port 443 (HTTPS) med et tre-veis håndtrykk.
4. **TLS-håndtrykk.** Serveren viser sitt sertifikat; nettleseren sjekker at det er gyldig og utstedt for riktig domene. De blir enige om krypteringsnøkler.
5. **HTTP-forespørsel.** Nettleseren sender `GET /`.
6. **Serveren behandler.** Ofte går forespørselen først gjennom en lastbalanserer, så til en webserver, som kanskje spør en applikasjonsserver og en database.
7. **Respons.** Serveren svarer med statuskode og HTML.
8. **Rendering.** Nettleseren leser HTML, oppdager at den trenger CSS, JavaScript og bilder, og henter dem (ofte fra en CDN). Siden tegnes på skjermen.

> **Husk:** Hvert eneste steg kan feile, og en DevOps Engineer må vite hvordan man feilsøker hvert steg: DNS (`nslookup`), nettverk (`ping`, `traceroute`), port (`nc`, `telnet`), TLS (`openssl s_client`), HTTP (`curl -v`). Dette øver vi på i kapittel 2.

Klient–server-arkitektur

Hva er det? En modell der **klienter** (nettlesere, mobilapper, andre programmer) sender forespørsler, og **servere** svarer. Nesten hele internett er bygget slik.



Fordeler: data og logikk er samlet ett sted (lett å oppdatere og sikre), og klientene kan være enkle.

Utfordring: serveren blir et sentralt punkt. Går den ned, virker ingenting – det er et **single point of failure**. Løsningen er å ha flere servere bak en lastbalanserer (kapittel 2 og 7).

Fra enkelt til avansert finnes det flere arkitekturer:

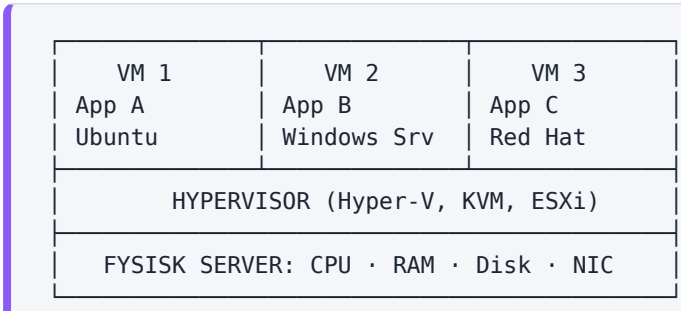
- **Monolitt** – hele applikasjonen er ett stort program. Enkelt å starte med.
- **Tre-lags arkitektur** (*three-tier*) – presentasjon, logikk og data i hvert sitt lag.
- **Mikrotjenester** (*microservices*) – applikasjonen deles i mange små tjenester som snakker via API-er. Fleksibelt, men mer komplekst å drifte. Det er her Docker og Kubernetes virkelig skinner.

Hva er virtualisering?

Hva er det? Virtualisering lar én fysisk maskin late som den er mange separate maskiner – **virtuelle maskiner** (*VM-er*).

Hvorfor finnes det? Før virtualisering kjørte bedrifter typisk én applikasjon per fysisk server (for å unngå at appene ødela for hverandre). De fleste serverne brukte 5–15 % av kapasiteten. Det var sløsing med strøm, plass og penger.

Hvordan fungerer det? En programvare kalt **hypervisor** fordeler den fysiske maskinens CPU, RAM, disk og nettverk mellom flere VM-er. Hver VM har sitt eget operativsystem og tror den er en ekte maskin.



To typer hypervisor:

- **Type 1 (bare metal)** – kjører direkte på maskinvaren. Brukes i datasentre. Eksempler: Microsoft Hyper-V (som Azure bygger på), VMware ESXi, KVM (Linux).

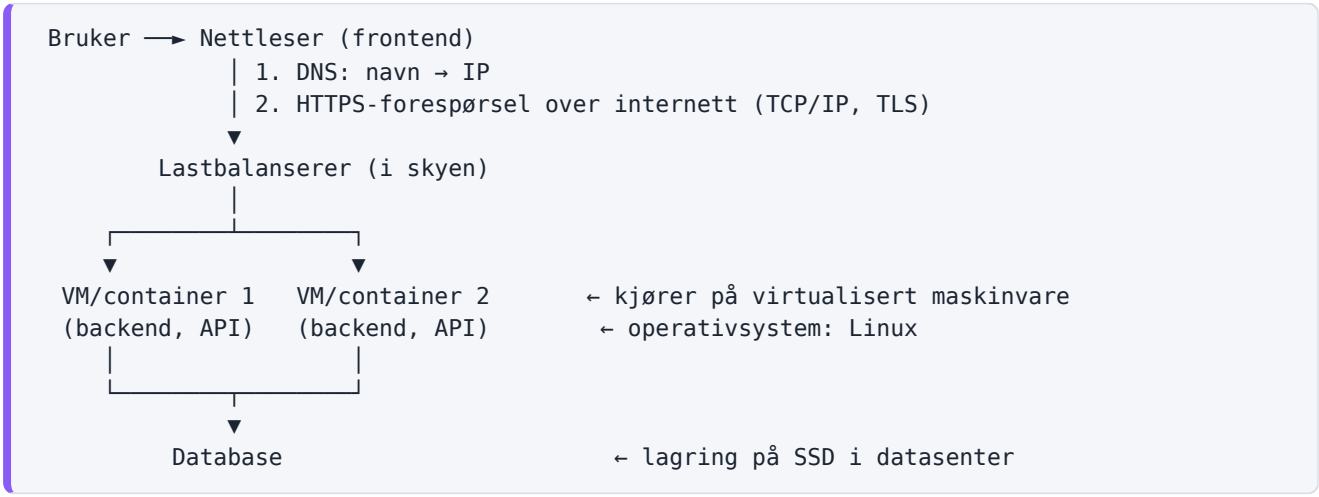
- **Type 2 (hosted)** – kjører som et program oppå et vanlig OS. Brukes på laptoper. Eksempler: VirtualBox, VMware Workstation, Parallels.

> **Hverdagsanalogi:** En fysisk server er en stor bygning. Uten virtualisering leier én bedrift hele bygningen selv om den bare bruker ett rom. Med virtualisering deles bygningen i leiligheter med egne dører og låser. Hypervisoren er vaktmesteren som fordeler strøm og vann.

> **Fra arbeidslivet:** Når du lager en VM i Azure, starter Azure en virtuell maskin på en fysisk server i et av Microsofts datasentre, ved hjelp av en spesialtilpasset Hyper-V. Hele skyen er bygget på virtualisering.

Containere er en lettere form for isolasjon der appene deler operativsystemets kjerne i stedet for å ha hvert sitt OS. Det er temaet i kapittel 9.

Hvordan alt henger sammen



Alt i denne boken handler om å bygge, automatisere, sikre og overvåke denne kjeden.

Kapitteloppsummering

- En datamaskin består av CPU (regner), RAM (raskt korttidsminne), lagring (permanent) og nettverkskort.
- Operativsystemet styrer maskinvaren og lar programmer dele den trygt.
- En server er en rolle: en maskin som leverer tjenester over nettverk, ofte uten skjerm, 24/7.
- En applikasjon består gjerne av frontend (det brukeren ser), backend (logikken) og database (dataene).

- Internett sender data i pakker mellom IP-adresser; DNS oversetter navn til IP; HTTP(S) er språket for web.
- API-er lar programmer snakke sammen; all Cloud-automatisering bygger på API-er.
- Virtualisering lar én fysisk server kjøre mange VM-er – fundamentet for Cloud.

Øvingsoppgaver

1. Åpne Oppgavebehandling (Windows) eller Aktivitetsmonitor (macOS). Finn hvor mange CPU-kjerner, hvor mye RAM og hvor stor disk maskinen din har. Hvilket program bruker mest RAM?
2. Kjør `nslookup www.noroff.no` i terminalen. Hvilken IP-adresse fikk du?
3. Kjør `curl -I https://www.noroff.no`. Hvilken statuskode fikk du? Hvilke *headers* ser du?
4. Tegn (på papir) hva som skjer når du åpner en nettside, med alle åtte stegene. Forklar det høyt for en venn.
5. Last ned VirtualBox og installer Ubuntu Server i en VM. (Dette er grunnlaget for kapittel 3.)

Quiz

1. Hva er forskjellen på RAM og SSD?
2. Hvorfor kjører de fleste servere uten skjerm?
3. Hva er forskjellen på frontend og backend?
4. Hva gjør DNS?
5. Hva betyr statuskode 404?
6. Hva er forskjellen på HTTP og HTTPS?
7. Hva er et API, forklart med en analogi?
8. Hva er en hypervisor?
9. Hva er et *single point of failure*?
10. Hvorfor kan alt i skyen automatiseres?

Fasit

1. **RAM** er raskt, flyktig arbeidsminne som tømmes når strømmen går. **SSD** er permanent lagring som beholder data, men er tregere enn RAM.
2. Fordi de styres over nettverket (f.eks. via SSH), står i datasentre uten fysisk tilgang, og skjerm/grafikk ville bare brukt ressurser.
3. **Frontend** er det brukeren ser og interagerer med (kjører i nettleser/mobil). **Backend** er logikken og dataene bak (kjører på servere).
4. DNS oversetter domenenavn (som `www.noroff.no`) til IP-adresser.
5. 404 betyr at ressursen du ba om ikke finnes (*Not Found*).

6. HTTPS er HTTP kryptert med TLS. Innholdet kan ikke leses eller endres underveis, og serverens identitet bekreftes med et sertifikat.
7. Et API er en avtalt måte for programmer å snakke sammen. Som en servitør som tar bestillingen din (forespørselen) til kjøkkenet (serveren) og kommer tilbake med maten (svaret), uten at du trenger å vite hvordan kjøkkenet fungerer.
8. Programvare som deler en fysisk maskins ressurser mellom flere virtuelle maskiner.
9. En komponent som stopper hele systemet hvis den feiler. Løses med redundans (flere eksemplarer).
10. Fordi alle handlinger i skyen – også klikk i portalen – er API-kall. Det du kan klikke, kan du også gjøre med kode.

Del 2 – Nettverk for DevOps

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

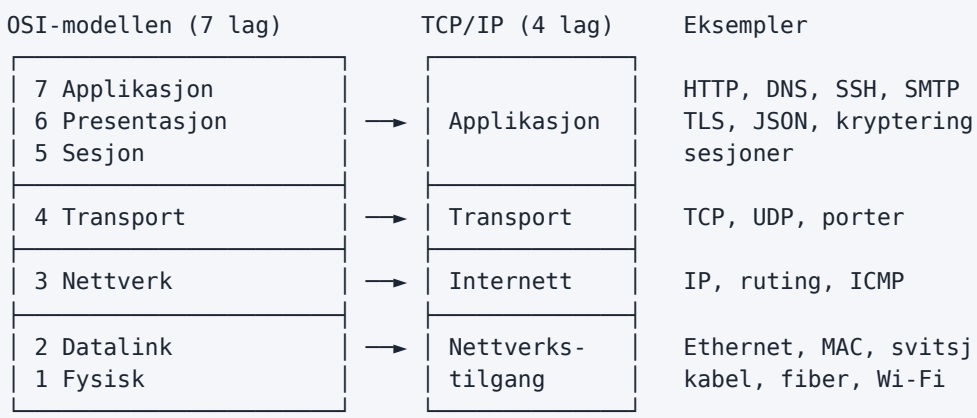
- lese og forstå IPv4- og IPv6-adresser, og skille private fra offentlige,
- regne ut subnett med CIDR-notasjon (for eksempel hvor mange adresser `/24` gir),
- forklare DNS, DHCP, TCP, UDP og porter,
- forklare ruter, svitsj, brannmur, NAT, VPN, lastbalanserer, proxy og reverse proxy,
- forstå TLS-sertifikater og HTTP-statuskoder,
- bruke SSH og feilsøke nettverksproblemer systematisk med standardverktøy.

> **Hvorfor må en DevOps Engineer kunne nettverk?** Fordi «det virker ikke» i 50 % av tilfellene betyr «noe i nettverket er feil»: en brannmurregel, en DNS-oppføring, en port som ikke er åpen, et subnett som er for lite. I Azure designer du selv virtuelle nettverk, subnett og sikkerhetsregler. Uten nettverkskunnskap blir det gjetting.

OSI- og TCP/IP-modellen – kartet over nettverk

Hva er det? En modell som deler nettverkskommunikasjon i **lag** (*layers*). Hvert lag har én jobb og bruker laget under seg.

Hvorfor finnes det? For å dele et svært komplekst problem i håndterbare biter – og for å gi et felles språk for feilsøking: «Er dette et lag 3-problem (IP) eller et lag 7-problem (HTTP)?»



> **Hverdagsanalogi:** Å sende et brev. Lag 7 er innholdet du skriver. Lag 4 er at du nummererer sidene og ber om kvittering. Lag 3 er adressen på konvolutten. Lag 2 er postbilen som kjører mellom to terminaler. Lag 1 er selve veien.

> **Fra arbeidslivet:** Du vil høre «lag 4-lastbalanserer» (ser bare IP og port – f.eks. Azure Load Balancer) og «lag 7-lastbalanserer» (forstår HTTP, kan rute på URL – f.eks. Azure Application Gateway). Det er denne modellen de refererer til.

IP-adresser: IPv4 og IPv6

IPv4

En IPv4-adresse er 32 bit, skrevet som fire tall (oktetter) fra 0 til 255:

```
  192   .   168   .    1    .   25
11000000.10101000.00000001.00011001   ← samme adresse i binært (32 bit)
```

Hver oktett er 8 bit, og 8 bit kan uttrykke $2^8 = 256$ verdier (0–255).

En IP-adresse har to deler:

- **Nettverksdelen** – hvilket nettverk (gate) enheten tilhører.
- **Vertsdelen** (*host*) – hvilken enhet (husnummer) på nettverket.

Hvor skillet går, bestemmes av **subnettmasken** eller **CIDR-prefikset** (se under).

IPv6

IPv6 er 128 bit, skrevet som åtte grupper heksadesimale tall:

```
2001:0db8:0000:0000:0000:ff00:0042:8329
2001:db8::ff00:42:8329          ← forkortet: ledende nuller og én serie nuller fjernes
```

Hvorfor? IPv4 har ca. 4,3 milliarder adresser – for få til alle telefoner, PC-er og IoT-enheter i verden. IPv6 har 2^{128} adresser ($\approx 3{,}4 \times 10^{38}$). I praksis lever IPv4 og IPv6 side om side (*dual stack*), og de fleste bedriftsnett og skymiljøer bruker fortsatt primært IPv4 internt.

Private og offentlige adresser

Offentlige (*public*) IP-adresser er unike på hele internett og kan nås derfra. **Private** adresser er reservert for interne nettverk (RFC 1918) og rutes aldri på internett:

Område	CIDR	Antall adresser	Typisk bruk
10.0.0.0 – 10.255.255.255	10.0.0.0/8	~16,7 millioner	Bedrifter, Azure VNets
172.16.0.0 – 172.31.255.255	172.16.0.0/12	~1 million	Bedrifter, Docker-nettverk

Område	CIDR	Antall adresser	Typisk bruk
192.168.0.0 – 192.168.255.255	192.168.0.0/16	65 536	Hjemmenettverk

Andre spesielle adresser du bør kjenne:

- `127.0.0.1` (*localhost*, *loopback*) – maskinen selv. IPv6: `::1`.
- `0.0.0.0` – «alle adresser på denne maskinen» når en tjeneste lytter; «hvilken som helst» i regler.
- `169.254.x.x` – automatisk adresse når DHCP feilet (et tegn på problemer). I Azure er `169.254.169.254` metadata-tjenesten for VM-er.

> **Hverdagsanalogi:** En offentlig IP er bedriftens gateadresse. Private IP-er er romnumrene inne i bygget. Posten utenfra kjenner bare gateadressen; resepsjonen (NAT-ruteren) vet hvilket rom brevet skal til.

Subnetting og CIDR

Dette er et tema mange gruer seg for. Det er lettere enn det ser ut.

Hva er det? Subnetting er å dele et stort nettverk i mindre nettverk (subnett). **CIDR** (*Classless Inter-Domain Routing*) er skrivemåten `10.0.1.0/24`, der tallet etter skråstreken sier **hvor mange bit som er nettverksdelen**.

Hvorfor?

- **Organisering:** webservere i ett subnett, databaser i et annet.
- **Sikkerhet:** du kan sette brannmurregler mellom subnett («bare web-subnettet får snakke med database-subnettet»).
- **Effektivitet:** ikke sløse adresser.

Hvordan fungerer det? En IPv4-adresse har 32 bit. Hvis prefikset er `/24`, er 24 bit nettverk og 32 − 24 = **8 bit** igjen til verter. 8 bit gir 2^8 = **256** adresser.

Formelen du må kunne:

```
Antall adresser i et subnett = 2^(32 – prefiks)
```

CIDR	Subnettmaske	Adresser	Typisk bruk
/8	255.0.0.0	16 777 216	Svært stort nettverk
/16	255.255.0.0	65 536	Et helt VNet

CIDR	Subnettmaske	Adresser	Typisk bruk
/20	255.255.240.0	4 096	Stort subnett (f.eks. AKS)
/22	255.255.252.0	1 024	Mellomstort subnett
/24	255.255.255.0	256	Vanlig subnett
/26	255.255.255.192	64	Lite subnett
/27	255.255.255.224	32	Lite subnett (f.eks. gateway)
/28	255.255.255.240	16	Veldig lite
/29	255.255.255.248	8	Minste subnett Azure tillater
/32	255.255.255.255	1	Én enkelt adresse (i regler)

> **Husk:** Mindre prefikstall = **større** nettverk. `/16` er mye større enn `/24`. Hver gang prefikset øker med 1, halveres antall adresser.

Eksempel: dele et VNet i subnett

Du får `10.0.0.0/16` (65 536 adresser) og trenger tre subnett:

```
VNet 10.0.0.0/16
├── snet-web   10.0.1.0/24   →  10.0.1.0  – 10.0.1.255
├── snet-app   10.0.2.0/24   →  10.0.2.0  – 10.0.2.255
└── snet-db    10.0.3.0/24   →  10.0.3.0  – 10.0.3.255
```

Reserverte adresser

I vanlige nettverk er første adresse **nettverksadressen** og siste er **broadcast**, så et `/24` gir 254 brukbare adresser.

I Azure reserveres **5 adresser** i hvert subnett: de fire første og den siste. I `10.0.1.0/24`:

Adresse	Brukes til
10.0.1.0	Nettverksadresse
10.0.1.1	Standard gateway (Azure)
10.0.1.2 og 10.0.1.3	Azure DNS-funksjoner
10.0.1.255	Broadcast

Altså: et `/24` i Azure gir **251** brukbare adresser. Et `/29` (8 adresser) gir bare 3. Dette er et klassisk AZ-104-spørsmål.

Regneøvelse steg for steg

Hvor mange adresser er i `10.0.0.0/22`, og hvilket område dekker det?

1. Vertsbit: 32 − 22 = 10.
2. Adresser: 2^{10} = 1 024.
3. 1 024 adresser = 4 × 256, altså fire «/24-blokker».
4. Området: `10.0.0.0` – `10.0.3.255`.

Overlapper `10.0.2.0/24` med `10.0.0.0/22`? Ja – `10.0.2.x` ligger innenfor `10.0.0.0` – `10.0.3.255`. Overlappende adresseområder er en vanlig feil når man kobler VNets sammen (*peering*) eller kobler skyen til kontornettet – det fungerer da ikke.

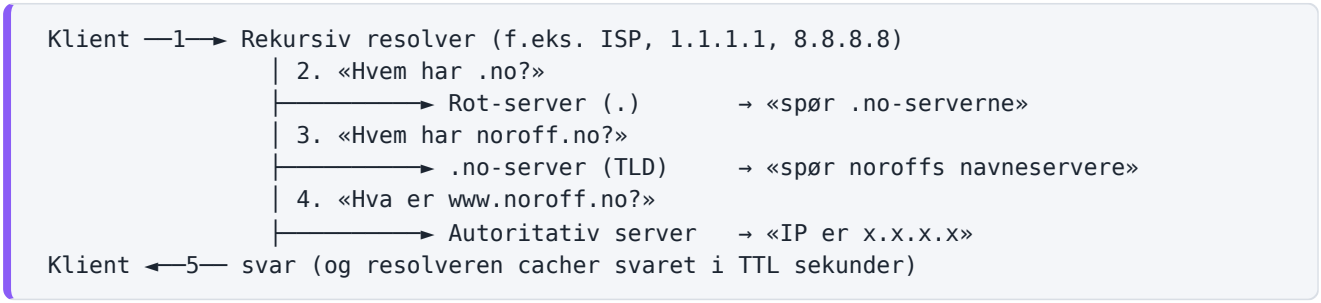
> **Vanlig feil:** Å lage VNets som er for små. Kubernetes (AKS) kan bruke svært mange IP-adresser – én per pod med noen nettverksmodeller. Planlegg romslig fra start; det er vanskelig å endre senere.

> **Fordypning:** Subnettmasken `255.255.255.192` i binært er `11111111.11111111.11111111.11000000` – 26 enere, altså `/26`. Masken og prefikset er to måter å skrive det samme på.

DNS – internettets telefonkatalog

Hva er det? Et distribuert, hierarkisk system som oversetter navn (`www.noroff.no`) til IP-adresser og annen informasjon.

Hvordan fungerer et oppslag?



1. Klienten spør en **rekursiv resolver**.
2. Resolveren spør en **rot-server**, som henviser til serverne for toppnivådomenet (TLD) `.no`.
3. `.no`-serverne henviser til den **autoritative** navneserveren for `noroff.no`.
4. Den autoritative serveren gir svaret.
5. Resolveren lagrer svaret i **cache** i en viss tid (**TTL** – *Time To Live*).

DNS-posttyper (*record types*)

Type	Betydning	Eksempel
A	Navn → IPv4	`www → 20.50.1.10`
AAAA	Navn → IPv6	`www → 2001:db8::10`
CNAME	Navn → annet navn (alias)	`www → minapp.azurewebsites.net`
MX	E-postserver for domenet	`@ → mail.example.com`
TXT	Fritekst, ofte verifisering	`"v=spf1 ..."`
NS	Hvilke navneservere som er autoritative	`ns1-01.azure-dns.com`
SOA	Informasjon om sonen	Serienummer, TTL-verdier
PTR	IP → navn (omvendt oppslag)	`10.1.50.20 → server.example.com`
SRV	Tjeneste og port	Brukes av f.eks. Active Directory

Praktisk

```
$ nslookup www.example.com          # enkelt oppslag (Windows, macOS, Linux)
$ dig www.example.com               # detaljert (Linux/macOS)
$ dig +short www.example.com A      # bare svaret
$ dig example.com MX                # hvilken e-postserver?
$ dig @1.1.1.1 www.example.com      # spør en bestemt DNS-server
```

Vanlig feil: «Jeg endret DNS, men ingenting skjer!» Svaret er ofte **TTL/cache**. Hvis TTL var 3600 sekunder, kan gamle svar leve i en time. Senk TTL *før* en planlagt flytting.

Fra arbeidslivet: Når du flytter en nettside fra en gammel server til Azure App Service, lager du en CNAME fra `www.firma.no` til `firma.azurewebsites.net`, verifiserer domenet med en TXT-post, og legger på et TLS-sertifikat.

Husk: «It's always DNS» er et ekte ordtak blant driftsfolk. Sjekk DNS tidlig når noe ikke kan nås.

DHCP

Hva er det? DHCP (*Dynamic Host Configuration Protocol*) deler automatisk ut IP-adresse, subnett-maske, standard gateway og DNS-server til enheter som kobler seg til et nettverk.

Hvordan? Fire steg, ofte husket som **DORA**:

```
Klient                          DHCP-server
  │── Discover (broadcast) ──────►│   «Er det en DHCP-server her?»
  │◄──────────────── Offer ────────│   «Du kan få 192.168.1.50»
  │── Request ────────────────────►│   «Jeg tar den!»
  │◄──────────────── Ack ──────────│   «Bekreftet, leid i 24 timer»
```

Hverdagsanalogi: Et hotell. Du kommer til resepsjonen (Discover), får tilbud om rom 214 (Offer), sier ja (Request), og får nøkkelkortet (Ack). Når du sjekker ut, kan rommet gis til en annen. Det er et *lease*.

Fra arbeidslivet: I Azure får VM-er sin private IP via DHCP fra Azure-plattformen. Du kan sette adressen til **statisk** i Azure (på nettverkskortet), men du skal *ikke* endre IP-innstillingene manuelt inne i operativsystemet på en Azure-VM – da mister du kontakten med den.

TCP og UDP

Begge er protokoller på **transportlaget** (lag 4). De bestemmer *hvordan* data leveres.

TCP – den pålitelige

TCP (*Transmission Control Protocol*) garanterer at alt kommer frem, i riktig rekkefølge, uten feil. Mistede pakker sendes på nytt.

Før data sendes, gjøres et **tre-veis håndtrykk** (*three-way handshake*):

```
Klient                    Server
  │─── SYN ──────────────►│   «Vil du snakke?»
  │◄── SYN-ACK ──────────│   «Ja, vil du?»
  │─── ACK ──────────────►│   «Ja!»  → forbindelsen er åpen
```

UDP – den raske

UDP (*User Datagram Protocol*) sender pakkene og håper det beste. Ingen håndtrykk, ingen garanti, men lav forsinkelse.

	TCP	UDP
Pålitelighet	Garantert levering og rekkefølge	Ingen garanti
Hastighet	Litt tregere (overhead)	Raskere
Forbindelse	Ja (håndtrykk)	Nei

	TCP	UDP
Brukes til	Web (HTTP/1.1, HTTP/2), SSH, e-post, databaser	DNS-oppslag, video, VoIP, spill, HTTP/3 (QUIC)

> **Hverdagsanalogi:** TCP er rekommandert post med kvittering. UDP er å rope gjennom et vindu – raskt, men du vet ikke om noen hørte det.

Porter

Hva er det? En port er et nummer (0–65535) som identifiserer **hvilken tjeneste** på en maskin trafikken skal til. IP-adressen er bygget; porten er døren.

En forbindelse identifiseres av: **protokoll + kilde-IP + kildeport + mål-IP + målport**.

Port	Protokoll	Tjeneste
20/21	TCP	FTP (filoverføring, ukryptert – unngå)
22	TCP	SSH (sikker fjerninnlogging)
25	TCP	SMTP (e-post mellom servere)
53	UDP/TCP	DNS
80	TCP	HTTP
123	UDP	NTP (tidssynkronisering)
443	TCP (og UDP for HTTP/3)	HTTPS
1433	TCP	Microsoft SQL Server / Azure SQL
3306	TCP	MySQL
3389	TCP	RDP (Windows fjernskrivebord)
5432	TCP	PostgreSQL
6379	TCP	Redis
6443	TCP	Kubernetes API-server
8080	TCP	Vanlig alternativ HTTP-port for apper
9090	TCP	Prometheus

- **Velkjente porter** (*well-known*): 0–1023 (krever ofte root-rettigheter for å lytte på i Linux).
- **Registrerte porter**: 1024–49151.
- **Dynamiske/midlertidige porter** (*ephemeral*): brukes som kildeport av klienter.

```
        $ ss -tlnp            # Linux: hvilke TCP-porter lytter maskinen på?
        State   Recv-Q  Send-Q  Local Address:Port   Peer Address:Port  Process
        LISTEN  0       511     0.0.0.0:80           0.0.0.0:*          users:
(("nginx",pid=812,fd=6))
        LISTEN  0       128     0.0.0.0:22           0.0.0.0:*          users:
(("sshd",pid=640,fd=3))
```

`-t` TCP, `-l` lyttende, `-n` vis tall i stedet for navn, `-p` vis prosess.

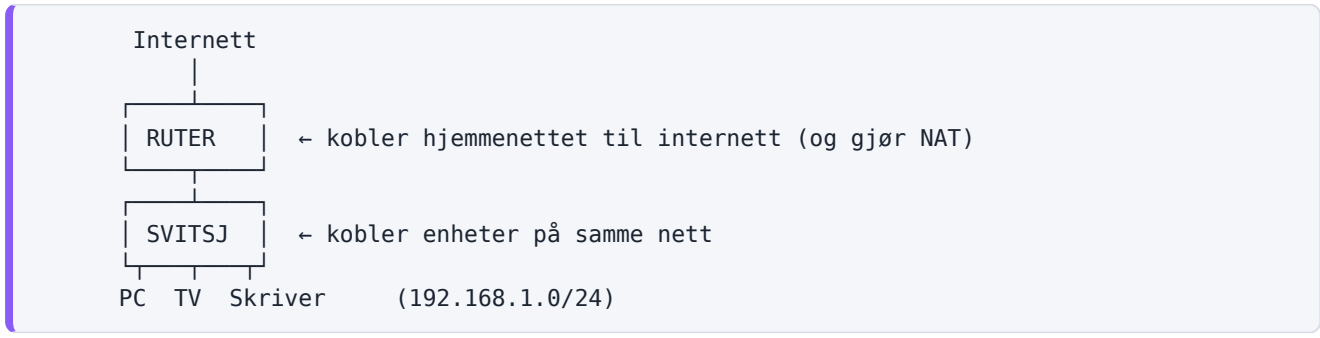
> **Vanlig feil:** En app som lytter på `127.0.0.1:8080` kan **bare** nås fra maskinen selv. For å nå den utenfra må den lytte på `0.0.0.0:8080` (alle grensesnitt). Dette er en klassiker med Docker-containere og Flask-apper.

> **Kostnad og sikkerhet:** Åpne aldri port 22 (SSH) eller 3389 (RDP) mot hele internett (`0.0.0.0/0`). Automatiserte roboter prøver å logge inn på disse portene hele døgnet. Bruk Azure Bastion, VPN eller begrens til din egen IP-adresse.

Ruter og svitsj

Svitsj (*switch*, lag 2) kobler sammen enheter i **samme** lokale nettverk. Den lærer hvilke **MAC-adresser** (fysiske adresser på nettverkskort) som sitter på hvilken port, og sender trafikken kun dit den skal.

Ruter (*router*, lag 3) kobler sammen **forskjellige** nettverk og bestemmer hvilken vei pakker skal gå, basert på IP-adresser og en **rutetabell**.



Standard gateway (*default gateway*) er ruteren en enhet sender trafikk til når målet *ikke* er på det lokale nettet. Hjemme er det typisk `192.168.1.1`.

```
        $ ip route            # Linux: vis rutetabellen
        default via 192.168.1.1 dev eth0
        192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.50
```

Les: «Alt til 192.168.1.0/24 går direkte ut på eth0. Alt annet (*default*) sendes til ruteren 192.168.1.1.»

> **Fra arbeidslivet:** I Azure finnes ingen fysiske rutere du ser, men det finnes **systemruter** som automatisk ruter mellom subnett i et VNet og ut til internett. Du kan overstyre dem med **User Defined Routes (UDR)** i en **route table** – for eksempel for å tvinge all trafikk gjennom en brannmur.

Brannmur (*firewall*)

Hva er det? En brannmur tillater eller blokkerer nettverkstrafikk basert på regler.

Hvordan? Regler ser typisk på: retning (inn/ut), kilde-IP, mål-IP, port, protokoll, og om trafikken er en del av en eksisterende forbindelse (*stateful*).

```
Prioritet  Retning  Kilde          Port   Handling
100        Inn      Internett      443    TILLAT
110        Inn      84.20.10.5/32  22     TILLAT   (bare kontor-IP)
4096       Inn      Alle           Alle   AVVIS    (standard: avvis alt annet)
```

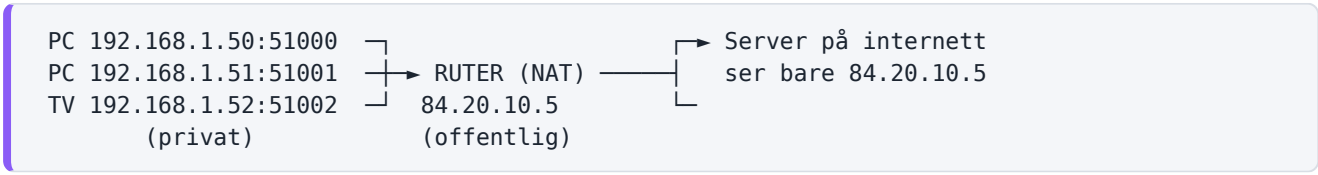
Typer:

- **Vertsbasert** – på selve maskinen: `ufw` / `iptables` / `nftables` på Linux, Windows Defender Firewall.
- **Nettverksbasert** – foran nettverket: maskinvarebrannmur, **Azure Network Security Group (NSG)**, **Azure Firewall**.
- **WAF** (*Web Application Firewall*) – lag 7, forstår HTTP og stopper angrep som SQL-injeksjon. Finnes i Azure Application Gateway og Azure Front Door.

> **Husk:** Prinsippet er **«avvis alt, tillat det som trengs»** (*default deny*). Hver åpning skal ha en grunn.

NAT – Network Address Translation

Hva er det? NAT oversetter IP-adresser når trafikk passerer en ruter. Vanligst: mange private adresser deler én offentlig adresse.



Ruteren husker hvem som sendte hva (i en NAT-tabell), slik at svarene kommer tilbake til riktig enhet.

Hvorfor? Det sparer offentlige IPv4-adresser, og interne enheter kan ikke nås direkte utenfra uten videre.

> **Fra arbeidslivet:** I Azure gir **NAT Gateway** VM-er i et subnett felles, stabil utgående IP-adresse. Det er nyttig når en ekstern partner bare vil tillate trafikk fra kjente IP-er. Microsoft har dessuten endret standarden slik at nye VNets/subnett ikke lenger får «skjult» utgående internettilgang automatisk (*default outbound access*); utgående trafikk skal settes opp eksplisitt med NAT Gateway, offentlig IP, lastbalanserer eller brannmur. Sjekk Microsoft Learn («Default outbound access in Azure») for gjeldende status.

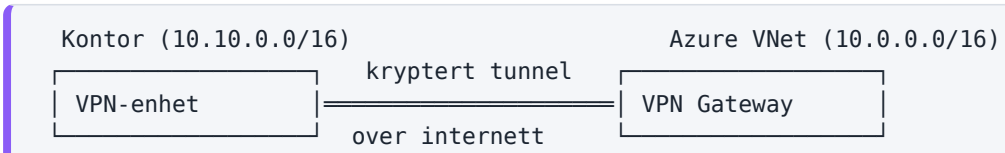
Port forwarding (*DNAT*) er det motsatte: trafikk til ruterens offentlige IP på en bestemt port sendes videre til en intern maskin.

VPN

Hva er det? Et VPN (*Virtual Private Network*) lager en **kryptert tunnel** over et usikkert nettverk (internett), slik at to steder kan snakke sammen som om de var på samme private nettverk.

To hovedtyper:

- **Site-to-site** – kobler et helt kontornett til et annet nett (f.eks. kontoret til Azure VNet). I Azure: **VPN Gateway**.
- **Point-to-site** (*remote access*) – kobler én enkelt bruker/PC til nettverket.



Legg merke til at adresseområdene **ikke** overlapper. Det må de ikke gjøre.

> **Fordypning: Azure ExpressRoute** er en privat, dedikert forbindelse mellom bedriften og Azure som *ikke* går over det offentlige internett – høyere ytelse og forutsigbarhet, men dyrere.

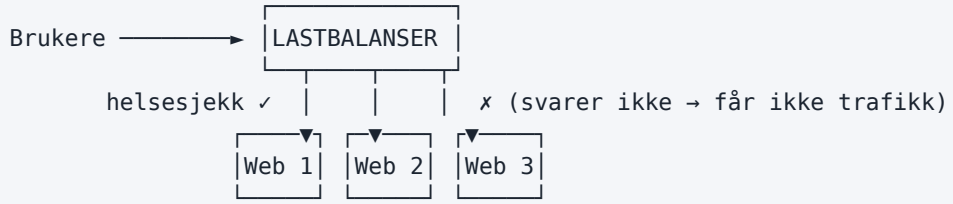
Lastbalansering (*load balancing*)

Hva er det? En lastbalanserer fordeler innkommende trafikk mellom flere servere.

Hvorfor?

1. **Skalering:** én server klarer ikke all trafikken.
2. **Høy tilgjengelighet:** dør én server, sendes trafikken til de andre.

3. **Vedlikehold uten nedetid:** ta én server ut, oppdater, sett inn igjen.



Helsesjekker (*health probes*): lastbalansereren spør jevnlig hver server, f.eks. `GET /health`. Svarer den ikke, slutter den å få trafikk.

Algoritmer: *round robin* (på rad), *least connections* (minst belastet), *hash* på kilde-IP (samme bruker til samme server).

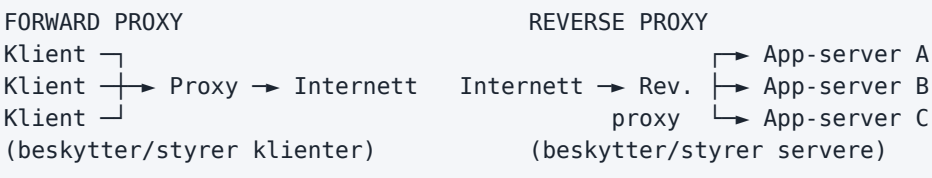
	Lag 4	Lag 7
Ser på	IP og port	HTTP: URL, headers, cookies
Kan	Fordele TCP/UDP raskt	Rute `/api` og `/bilder` til ulike servere, TLS-terminering, WAF
Azure	Azure Load Balancer	Application Gateway, Front Door

> **Hverdagsanalogi:** En hovmester i en travel restaurant som fordeler gjestene på servitørene, og som slutter å sende gjester til en servitør som har gått hjem syk.

Proxy og reverse proxy

Proxy (forward proxy) sitter foran **klientene** og sender forespørsler ut på deres vegne. Brukes i bedrifter for å filtrere og logge internettbruk, eller cache innhold.

Reverse proxy sitter foran **serverne** og tar imot forespørsler på deres vegne. Klienten vet ikke hvilken server som egentlig svarer.



En reverse proxy gjør ofte:

- **TLS-terminering** (håndterer HTTPS, så appene slipper),
- lastbalansering,
- caching og komprimering,

- ruting basert på URL (`/api` → backend, `/` → frontend),
- skjuler interne servere.

Eksempler: **nginx**, HAProxy, Traefik, Envoy, Azure Application Gateway, Kubernetes Ingress-kontrollere.

Et enkelt nginx-oppsett som reverse proxy:

```
server {
    listen 80;                          # lytt på port 80
    server_name minapp.example.com;     # for dette domenet

    location / {
        proxy_pass http://127.0.0.1:8000;          # send videre til appen på port 8000
        proxy_set_header Host $host;               # behold opprinnelig vertsnavn
        proxy_set_header X-Real-IP $remote_addr;   # gi appen klientens ekte IP
    }
}
```

SSL/TLS og sertifikater

Hva er det? TLS (*Transport Layer Security*) krypterer trafikk mellom klient og server. SSL er det gamle navnet (alle SSL-versjoner er utdaterte og usikre); folk sier fortsatt «SSL-sertifikat», men det er TLS som brukes. Bruk TLS 1.2 eller 1.3.

TLS gir tre ting:

1. **Konfidensialitet** – ingen kan lese trafikken.
2. **Integritet** – ingen kan endre den uten at det oppdages.
3. **Autentisering** – du vet at du snakker med riktig server (via sertifikatet).

Sertifikater. Et sertifikat er et digitalt «pass» for et domene. Det inneholder domenenavnet, en **offentlig nøkkel**, gyldighetsperiode og en signatur fra en **sertifikatutsteder** (*Certificate Authority, CA*) som nettleseren stoler på.

```
Nettleseren stoler på  ──►  Rot-CA (innebygd i OS/nettleser)
                                │ signerer
                            Mellom-CA (intermediate)
                                │ signerer
                            Sertifikat for www.firma.no
```

Asymmetrisk kryptering, enkelt forklart: Serveren har et nøkkelpar – en **offentlig nøkkel** alle kan få, og en **privat nøkkel** som aldri forlater serveren. Noe som er låst med den offentlige nøkkelen kan bare låses opp med den private. I TLS brukes dette til å bli enige om en felles, rask **symmetrisk** nøkkel som så krypterer resten av samtalen.

> **Hverdagsanalogi:** Den offentlige nøkkelen er en åpen postkasse med smal sprekk – alle kan legge brev inn. Bare eieren har nøkkelen (den private) som åpner den.

Praktisk:

```
        # Se sertifikatet til en nettside (utløpsdato, utsteder)
        $ openssl s_client -connect www.example.com:443 -servername www.example.com
</dev/null 2>/dev/null \
            | openssl x509 -noout -subject -issuer -dates
```

> **Vanlig feil: Utløpte sertifikater** er en av de vanligste årsakene til nedetid i hele bransjen. Automatiser fornyelse (f.eks. Let's Encrypt med `certbot`, App Service Managed Certificates, eller Azure Key Vault med automatisk fornyelse) og overvåk utløpsdatoer. Merk at bransjen har vedtatt gradvis kortere maksimal levetid for offentlige sertifikater fremover – automatisering blir derfor obligatorisk i praksis.

HTTP-statuskoder

Statuskoder er tresifrede tall i HTTP-svar. Første siffer sier hvilken kategori:

Kategori	Betydning
1xx	Informasjon
2xx	Suksess
3xx	Omdirigering
4xx	Klientfeil – feilen er i forespørselen
5xx	Serverfeil – feilen er på serversiden

De viktigste:

Kode	Navn	Hva det betyr i praksis
200	OK	Alt gikk bra
201	Created	Noe ble opprettet (etter POST)
204	No Content	Gikk bra, men ingen data tilbake
301	Moved Permanently	Permanent flyttet (f.eks. http → https)
302 / 307	Found / Temporary Redirect	Midlertidig omdirigering
304	Not Modified	Bruk cachet versjon

Kode	Navn	Hva det betyr i praksis
400	Bad Request	Forespørselen er feilformatert
401	Unauthorized	Ikke autentisert – du må logge inn
403	Forbidden	Autentisert, men ikke lov
404	Not Found	Finnes ikke
405	Method Not Allowed	Feil HTTP-metode
409	Conflict	Konflikt, f.eks. finnes allerede
429	Too Many Requests	Du blir begrenset (*rate limiting*)
500	Internal Server Error	Appen krasjet / ukjent feil
502	Bad Gateway	Proxyen fikk ugyldig svar fra backend (backend nede?)
503	Service Unavailable	Tjenesten er overbelastet eller i vedlikehold
504	Gateway Timeout	Proxyen ventet for lenge på backend

> **Fra arbeidslivet:** Statuskoder er gull ved feilsøking. **502** fra Application Gateway eller nginx betyr nesten alltid at *appen bak* ikke svarer – ikke at proxyen er ødelagt. **401 vs. 403**: 401 = «hvem er du?», 403 = «jeg vet hvem du er, men du får ikke lov».

SSH – Secure Shell

Hva er det? SSH er en kryptert protokoll for å logge inn på og styre en annen maskin over nettverket, typisk Linux-servere. Bruker port 22.

Passord vs. nøkler. Passord kan gjettes. **SSH-nøkler** er et nøkkelpar: den private nøkkelen ligger på din PC, den offentlige legges på serveren. Det er mye sikrere, og standard i bransjen.

```
        # 1. Lag et nøkkelpar (ed25519 er moderne og anbefalt)
        $ ssh-keygen -t ed25519 -C "student@noroff"
        #    Lagres som ~/.ssh/id_ed25519 (PRIVAT – del aldri!) og ~/.ssh/id_ed25519.pub
(offentlig)

        # 2. Kopier den offentlige nøkkelen til serveren
        $ ssh-copy-id student@203.0.113.10

        # 3. Logg inn
        $ ssh student@203.0.113.10

        # 4. Kopier en fil til serveren
        $ scp rapport.txt student@203.0.113.10:/home/student/
```

En `~/.ssh/config`-fil sparer deg for skriving:

```
Host webserver
    HostName 203.0.113.10
    User student
    IdentityFile ~/.ssh/id_ed25519
```

Nå holder det med `ssh webserver`.

> **Vanlig feil:** Feil rettigheter på nøkkelfilen gir feilmeldingen «UNPROTECTED PRIVATE KEY FILE!». Løsning: `chmod 600 ~/.ssh/id_ed25519`.

> **Husk:** Den **private** nøkkelen deles aldri, sjekkes aldri inn i Git, og sendes aldri på e-post eller chat.

(IP-adressene `203.0.113.x` i eksemplene er reservert for dokumentasjon og finnes ikke på ekte.)

Nettverksfeilsøking – en systematisk metode

Når «nettsiden virker ikke», jobb deg **nedenfra og opp** i lagene, og test én ting om gangen:

```
 1. Har jeg nettverk?           ip addr / ipconfig      → har jeg IP-adresse?
 2. Når jeg gatewayen?          ping 192.168.1.1
 3. Når jeg internett?          ping 1.1.1.1
 4. Virker DNS?                 nslookup / dig minside.no
 5. Er porten åpen?             nc -zv minside.no 443
 6. Hvilken vei går trafikken?  traceroute / tracert / mtr
 7. Virker TLS?                 openssl s_client -connect minside.no:443
 8. Hva svarer HTTP?            curl -v https://minside.no
 9. Lytter tjenesten?           ss -tlnp (på serveren)
10. Hva sier loggene?           journalctl, /var/log/nginx/error.log
```

Verktøyene

Verktøy	Hva det gjør	Eksempel
`ip addr` / `ipconfig`	Vis IP-adresser	`ip addr show`
`ping`	Sjekk om en vert svarer (ICMP)	`ping -c 4 1.1.1.1`
`traceroute` / `tracert`	Vis veien pakkene tar	`traceroute vg.no`
`mtr`	ping + traceroute kontinuerlig	`mtr vg.no`
`nslookup` / `dig`	DNS-oppslag	`dig +short vg.no`
`nc` (netcat)	Test om en TCP-port er åpen	`nc -zv vg.no 443`
`curl`	HTTP-forespørsler	`curl -v https://vg.no`
`ss` / `netstat`	Vis åpne porter og forbindelser	`ss -tlnp`
`tcpdump`	Fang nettverkstrafikk	`sudo tcpdump -i eth0 port 53`
`openssl s_client`	Test TLS	se over

> **Vanlig feil:** Å konkludere at serveren er nede fordi `ping` ikke svarer. Mange brannmurer (også Azure-standarder) blokkerer ICMP. Test heller porten med `nc` eller `curl`.

Eksempel på feilsøking

Situasjon: Du har satt opp en webserver på en Azure-VM, men `http://<offentlig-ip>` henger.

1. `curl -v http://<ip>` fra din PC → henger på «Trying ...» → trafikken kommer ikke frem. Mistenk brannmur.
2. Sjekk NSG-reglene i Azure → port 80 er ikke åpnet. Legg til regel → prøv igjen.
3. Nå får du «Connection refused» → trafikken kommer frem, men ingenting lytter.
4. SSH inn, kjør `sudo ss -tlnp` → nginx kjører ikke. `sudo systemctl status nginx` viser feil i konfigurasjonen.
5. `sudo nginx -t` viser linjen med feilen. Fiks, `sudo systemctl restart nginx` → `200 OK`.

> **Husk:** «Henger/timeout» betyr ofte **brannmur** (pakkene droppes). «Connection refused» betyr at maskinen svarte, men **ingen tjeneste lytter** på porten. Den forskjellen sparer deg mye tid.

Hvordan en DevOps Engineer bruker nettverk i praksis

- Designe Azure VNets og subnett uten overlapp, med plass til vekst.

- Skrive NSG-regler og brannmurregler etter minste-privilegium-prinsippet.
- Sette opp DNS for nye tjenester og TLS-sertifikater med automatisk fornyelse.
- Konfigurere lastbalanserere og helsesjekker.
- Koble skymiljøet sammen med kontornettet (VPN/ExpressRoute) eller andre VNets (*peering*).
- Bruke private endepunkter (*Private Endpoints*) slik at databaser og lagring ikke er eksponert mot internett.
- Feilsøke «hvorfor får ikke tjeneste A kontakt med tjeneste B?» – kanskje den vanligste support-saken i Kubernetes og Cloud.

Kapitteloppsummering

- Nettverk beskrives i lag (OSI/TCP-IP); feilsøk nedenfra og opp.
- IPv4 er 32 bit; private områder er 10/8, 172.16/12 og 192.168/16. IPv6 er 128 bit.
- Antall adresser i et subnett = 2^(32 − prefiks). Azure reserverer 5 adresser per subnett.
- DNS oversetter navn til IP; husk TTL og cache. DHCP deler ut adresser automatisk.
- TCP er pålitelig med håndtrykk; UDP er raskt uten garanti. Porter identifiserer tjenester.
- Ruter kobler nettverk; svitsj kobler enheter; brannmur filtrerer; NAT oversetter adresser; VPN lager krypterte tunneler.
- Lastbalanserere fordeler trafikk og bruker helsesjekker; reverse proxy står foran servere.
- TLS gir kryptering, integritet og autentisering med sertifikater. Automatiser fornyelse.
- SSH med nøkler er standard for Linux-servere.

Øvingsoppgaver

1. Regn ut antall adresser i `/25`, `/23` og `/27`. Hvor mange er brukbare i Azure?
2. Del `172.16.0.0/16` i fire like store subnett. Skriv opp CIDR og område for hvert.
3. Kjør `dig noroff.no MX` og `dig noroff.no NS`. Hva forteller svarene?
4. Kjør `traceroute vg.no` (eller `tracert` på Windows). Hvor mange «hopp» er det?
5. Lag et SSH-nøkkelpar med `ssh-keygen -t ed25519`. Se på innholdet i `.pub`-filen.
6. Bruk `curl -v https://www.noroff.no` og identifiser DNS, TCP, TLS og HTTP i utskriften.
7. Kjør `ss -tlnp` (Linux) eller `netstat -ano` (Windows). Hvilke porter lytter maskinen din på?

Quiz

1. Hvor mange adresser er det i et `/26`-subnett?
2. Hvilket av disse er en privat adresse: `8.8.8.8`, `10.20.30.40`, `172.32.0.1`?
3. Hva er forskjellen på en A-post og en CNAME-post?

4. Hvorfor bruker DNS ofte UDP?
5. Hvilken port bruker HTTPS? SSH? PostgreSQL?
6. Hva betyr «Connection refused» sammenlignet med «timeout»?
7. Hva er forskjellen på en forward proxy og en reverse proxy?
8. Hva er forskjellen på 401 og 403?
9. Hvorfor bør du ikke åpne port 22 mot `0.0.0.0/0`?
10. To VNets har `10.0.0.0/16` og `10.0.128.0/17`. Kan de kobles sammen med peering?

Fasit

1. 2^(32−26) = 2^6 = **64** adresser (59 brukbare i Azure).
2. **`10.20.30.40`**. `172.32.0.1` er utenfor 172.16.0.0/12 (som slutter på 172.31.255.255), så den er offentlig.
3. En **A-post** peker et navn direkte til en IPv4-adresse. En **CNAME** peker et navn til et *annet navn* (alias), som så slås opp videre.
4. Fordi et DNS-oppslag er ett lite spørsmål og ett lite svar – håndtrykket i TCP ville gjort det tregere. (Store svar og soneoverføringer bruker TCP.)
5. HTTPS: **443**. SSH: **22**. PostgreSQL: **5432**.
6. **Refused**: maskinen nås, men ingen tjeneste lytter på porten. **Timeout**: pakkene kommer ikke frem eller blir droppet – ofte brannmur eller ruting.
7. **Forward proxy** står foran klientene og sender forespørsler ut for dem. **Reverse proxy** står foran serverne og tar imot forespørsler for dem.
8. **401**: ikke autentisert (vi vet ikke hvem du er). **403**: autentisert, men mangler tilgang.
9. Fordi roboter konstant prøver å bryte seg inn via SSH. Begrens til kjente IP-er, bruk Bastion/VPN og nøkkelbasert innlogging.
10. **Nei.** `10.0.128.0/17` ligger innenfor `10.0.0.0/16` – adresseområdene overlapper, og overlappende VNets kan ikke peeres.

Del 3 – Linux fra nybegynner til profesjonell

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva Linux er og hvorfor nesten hele skyen kjører på det,
- navigere i filsystemet og jobbe med filer fra terminalen,
- administrere brukere, grupper og rettigheter,
- styre prosesser og tjenester med `ps`, `top` og `systemctl`,
- installere programvare med pakkehåndterere,
- skrive enkle Bash-skript, planlegge jobber med cron og lese logger,
- sikre og feilsøke en Linux-server.

Hva er Linux, og hvorfor bruker DevOps det?

Hva er det? Linux er et gratis operativsystem med åpen kildekode. Strengt tatt er «Linux» navnet på **kjernen** (*kernel*), skrevet av Linus Torvalds i 1991. Kjernen pluss verktøy, pakkesystem og programmer utgjør en **distribusjon** (*distro*), som Ubuntu eller Red Hat.

Hvorfor bruker DevOps Linux?

- Det store flertallet av servere på internett og i skyen kjører Linux – også mange arbeidslaster i Azure.
- Containere (Docker) og Kubernetes er bygget på Linux-kjernens funksjoner.
- Alt kan styres fra kommandolinjen og automatiseres med skript.
- Gratis, stabilt, sikkert, og enormt godt dokumentert.

> **Hverdagsanalogi:** Windows er som en ferdigmøblert leilighet: komfortabelt, men vanskelig å bygge om. Linux er som en verktøykasse og et lager av byggeklosser: du kan bygge akkurat det du trenger, og du kan automatisere byggingen.

Distribusjoner

Distro	Familie	Pakkeverktøy	Typisk bruk
Ubuntu (LTS)	Debian	`apt`	Svært populær i sky og blant utviklere
Debian	Debian	`apt`	Stabil, grunnlag for mange container-images

Distro	Familie	Pakkeverktøy	Typisk bruk
Red Hat Enterprise Linux (RHEL)	Red Hat	`dnf`	Store bedrifter, kommersiell support
Rocky Linux / AlmaLinux	Red Hat	`dnf`	Gratis RHEL-kompatible alternativer
Fedora	Red Hat	`dnf`	Nyeste teknologi
Alpine	Egen	`apk`	Svært små container-images
Azure Linux	Egen (Microsoft)	`tdnf`	Microsofts distro, brukes bl.a. på AKS-noder

LTS (*Long Term Support*) betyr at versjonen får sikkerhetsoppdateringer i mange år. Ubuntu kommer med en LTS hvert annet år (f.eks. 22.04, 24.04, 26.04). Bruk LTS på servere.

> **Husk:** Lær én familie godt (Ubuntu/Debian er et godt valg), men vit at Red Hat-familien bruker `dnf` i stedet for `apt`. Resten er stort sett likt.

Slik får du en Linux-maskin å øve på

Du har flere gratis muligheter:

1. **WSL** (*Windows Subsystem for Linux*) på Windows: åpne PowerShell som administrator og kjør `wsl --install`. Du får Ubuntu i et vindu.
2. **Virtuell maskin** med VirtualBox eller UTM (Mac) og Ubuntu Server.
3. **macOS-terminalen** – macOS er Unix og har mange av de samme kommandoene (men ikke alt, f.eks. ikke `systemctl`).
4. **Azure Cloud Shell** – en Linux-terminal i nettleseren, direkte i Azure Portal.
5. **En liten Azure-VM** (se kapittel 7; husk kostnader og opprydding).

Terminal og shell

Hva er det?

- **Terminalen** er vinduet du skriver i.
- **Shellet** er programmet som tolker det du skriver og kjører kommandoer. Det vanligste er **Bash**; macOS bruker **zsh** som standard. Begge fungerer nesten likt for det vi gjør.

Hvorfor kommandolinje når det finnes grafiske grensesnitt? Servere har som regel ikke grafisk grensesnitt. Kommandoer kan dessuten **skriptes, gjentas, dokumenteres og automatiseres**. Ti

klikk i et GUI kan bli én linje i et skript som kjøres på 500 servere.

Anatomien til en kommando

```
  ls   -l -a    /var/log
  |    |        |
  |    |        └── argument: hva kommandoen jobber med
  |    └─────────── flagg/opsjoner: endrer oppførselen (-la kan kombineres)
  └──────────────── kommandoen
```

Prompten

```
student@webserver:~$
|       |         | └── $ = vanlig bruker   (# = root)
|       |         └──── ~ = du er i hjemmemappen din
|       └────────────── maskinens navn
└──────────────────────── brukernavnet ditt
```

Overlevelsestips

Tast/kommando	Gjør
`Tab`	Fullfører fil- og kommandonavn. Bruk den hele tiden!
`↑` / `↓`	Bla i tidligere kommandoer
`Ctrl + C`	Avbryt kommandoen som kjører
`Ctrl + L` eller `clear`	Tøm skjermen
`Ctrl + R`	Søk i kommandohistorikken
`history`	Vis historikken
`man ls`	Manualen for `ls` (avslutt med `q`)
`ls --help`	Kort hjelp

Filsystemet

Hva er det? Linux organiserer alt i ett tre som starter i **roten** `/`. Det finnes ingen `C:`-disk; andre disker «monteres» inn som mapper.

```
/                   ← roten
├── bin, sbin       ← grunnleggende programmer (ofte lenker til /usr/bin)
├── boot            ← kjernen og oppstartsfiler
├── dev             ← enheter (disker: /dev/sda, /dev/nvme0n1)
├── etc             ← KONFIGURASJONSFILER (nginx, ssh, brukere)
├── home            ← brukernes hjemmemapper (/home/student)
├── opt             ← tredjeparts programvare
├── proc, sys       ← virtuelle filer med info om kjernen og prosesser
├── root            ← hjemmemappen til root-brukeren
├── tmp             ← midlertidige filer (slettes ofte ved omstart)
├── usr             ← programmer og biblioteker (/usr/bin, /usr/lib)
└── var             ← data som endres: LOGGER (/var/log), nettsider (/var/www)
```

Husk: Tre mapper du kommer til å bruke hele tiden: **`/etc`** (konfigurasjon), **`/var/log`** (logger) og **`/home`** (brukerfiler).

«Alt er en fil»: I Linux er nesten alt representert som filer – også disker (`/dev/sda`), prosesser (`/proc/1234`) og systeminformasjon (`/proc/cpuinfo`). Derfor kan de samme verktøyene (`cat` , `grep`) brukes på nesten alt.

Absolutte og relative stier

- **Absolutt sti** starter med `/` : `/home/student/prosjekt/app.py` – virker uansett hvor du står.
- **Relativ sti** starter der du er: `prosjekt/app.py` .
- `.` = mappen du står i. `..` = mappen over. `~` = hjemmemappen din.

Grunnleggende kommandoer

Her er kommandoene du *må* kunne. For hver: hva du skriver, hva det betyr, hva du får, og når du bruker den.

`pwd` – hvor er jeg?

```
$ pwd
/home/student
```

Print Working Directory. Bruk når du er usikker på hvor du er.

`ls` – hva finnes her?

```
$ ls
dokumenter  prosjekt  notater.txt

$ ls -la
total 24
drwxr-xr-x 4 student student 4096 Oct 30 10:12 .
drwxr-xr-x 3 root    root    4096 Oct 28 09:00 ..
-rw------- 1 student student  220 Oct 28 09:00 .bash_history
drwxr-xr-x 2 student student 4096 Oct 30 10:10 dokumenter
-rw-r--r-- 1 student student  512 Oct 30 10:12 notater.txt
drwxr-xr-x 3 student student 4096 Oct 30 10:11 prosjekt
```

`-l` = lang liste med detaljer, `-a` = vis også skjulte filer (de som starter med punktum), `-h` = lesbare størrelser (KB, MB). Vi forklarer kolonnene under «Rettigheter».

`cd` – bytt mappe

```
$ cd prosjekt          # gå inn i mappen prosjekt
$ cd ..                # gå én mappe opp
$ cd /var/log          # gå til en absolutt sti
$ cd ~                 # gå hjem (bare `cd` gjør det samme)
$ cd -                 # gå tilbake dit du var sist
```

`mkdir` – lag mapper

```
$ mkdir backup                    # lag én mappe
$ mkdir -p prosjekt/src/utils     # lag hele stien; -p = lag foreldremapper ved
behov
```

`touch`, `cat`, `less`, `head`, `tail` – lag og les filer

```
$ touch tom-fil.txt               # lag en tom fil (eller oppdater tidsstempelet)
$ cat notater.txt                 # skriv ut hele filen
$ less /var/log/syslog            # bla i store filer (q = avslutt, / = søk)
$ head -n 20 fil.txt              # de 20 første linjene
$ tail -n 50 /var/log/syslog      # de 50 siste linjene
$ tail -f /var/log/nginx/access.log   # FØLG filen live – helt sentralt ved
feilsøking
```

`cp`, `mv`, `rm` – kopier, flytt, slett

```
$ cp notater.txt notater-backup.txt       # kopier fil
$ cp -r prosjekt prosjekt-kopi            # kopier mappe (-r = rekursivt)
$ mv notater.txt dokumenter/              # flytt fil
$ mv gammelt-navn.txt nytt-navn.txt       # gi nytt navn (= flytt til nytt navn)
$ rm tom-fil.txt                          # slett fil
$ rm -r gammel-mappe                      # slett mappe med innhold
$ rmdir tom-mappe                         # slett tom mappe
```

> **Vanlig feil:** `rm` har **ingen papirkurv**. `rm -rf` sletter alt uten å spørre. Kjør aldri `rm -rf` med variabler du ikke har sjekket, og aldri som root uten å dobbeltsjekke stien. Bruk `rm -ri` (spør for hver fil) når du er usikker.

Redigere filer: `nano` og `vim`

```
$ nano /etc/hosts      # enkel editor. Ctrl+O = lagre, Ctrl+X = avslutt
$ vim fil.txt          # kraftig editor. i = skriv, Esc, :wq = lagre og avslutt,
:q! = avslutt uten å lagre
```

Lær deg nok `vim` til å overleve – den finnes på nesten alle servere.

`grep` – søk i tekst

```
$ grep "error" /var/log/syslog            # linjer som inneholder "error"
$ grep -i "error" app.log                 # -i = ignorer store/små bokstaver
$ grep -r "DATABASE_URL" ./prosjekt       # -r = søk rekursivt i mapper
$ grep -n "listen" /etc/nginx/nginx.conf  # -n = vis linjenummer
$ grep -v "DEBUG" app.log                 # -v = linjer som IKKE matcher
$ grep -c "404" access.log                # -c = tell treff
```

`find` – finn filer

```
$ find /var/log -name "*.log"                # alle .log-filer under /var/log
$ find . -type f -size +100M                 # filer større enn 100 MB her
$ find /tmp -type f -mtime +7                # filer endret for mer enn 7 dager
siden
$ find . -name "*.tmp" -delete               # finn og slett (vær forsiktig!)
```

Pipes og omdirigering – superkraften

Pipe (`|`) sender resultatet fra én kommando inn i neste. Små verktøy kombineres til kraftige.

```
        # Hvilke 5 IP-adresser har gjort flest forespørsler mot webserveren?
        $ cat /var/log/nginx/access.log | awk '{print $1}' | sort | uniq -c | sort -rn |
head -5
           1532 203.0.113.44
            821 198.51.100.7
            ...
```

Les det steg for steg: hent loggen → plukk ut første kolonne (IP) → sorter → tell like linjer → sorter etter antall, største først → vis topp 5.

Omdirigering:

```
        $ echo "hei" > fil.txt        # > skriver til fil (OVERSKRIVER)
        $ echo "igjen" >> fil.txt     # >> legger til på slutten
        $ kommando 2> feil.log        # 2> sender feilmeldinger (stderr) til fil
        $ kommando > alt.log 2>&1     # både vanlig utskrift og feil til samme fil
        $ kommando > /dev/null        # kast utskriften
```

> **Fordypning – stdin, stdout, stderr:** Hvert program har tre «strømmer»: inn (0, stdin), ut (1, stdout) og feil (2, stderr). `2>&1` betyr «send strøm 2 dit strøm 1 går».

Nyttige småverktøy

```
        $ wc -l fil.txt          # tell linjer
        $ sort fil.txt           # sorter linjer
        $ uniq                   # fjern like linjer som står etter hverandre
        $ cut -d',' -f2 data.csv # plukk ut kolonne 2 fra en CSV
        $ df -h                  # ledig diskplass per filsystem
        $ du -sh /var/log        # hvor mye plass bruker en mappe?
        $ free -h                # RAM-bruk
        $ uname -a               # kjerneversjon
        $ cat /etc/os-release    # hvilken distro og versjon
        $ which python3          # hvor ligger programmet?
        $ tar -czf backup.tar.gz mappe/   # pakk og komprimer
        $ tar -xzf backup.tar.gz          # pakk ut
```

Brukere og grupper

Hva er det? Linux er et flerbrukersystem. Hver **bruker** har et brukernavn, en ID (UID) og en hjemmemappe. **Grupper** samler brukere slik at de kan få felles rettigheter.

Spesielle brukere:

- **root** (UID 0) – superbrukeren som kan alt. Farlig å bruke direkte.
- **Systembrukere** – for tjenester, f.eks. `www-data` (nginx/Apache), `postgres`. De kan ikke logge inn, og begrenser skaden hvis tjenesten blir hacket.

```
$ whoami                                # hvem er jeg?
$ id                                    # min UID, GID og grupper
uid=1000(student) gid=1000(student) groups=1000(student),27(sudo)

$ sudo adduser ola                      # lag ny bruker (Debian/Ubuntu, interaktivt)
$ sudo useradd -m -s /bin/bash kari     # lag bruker (alle distroer); -m = lag
hjemmemappe
$ sudo passwd kari                      # sett passord
$ sudo groupadd utviklere               # lag gruppe
$ sudo usermod -aG utviklere kari       # legg kari i gruppen (-aG = append til
Group)
$ sudo usermod -aG docker student       # vanlig: gi deg tilgang til docker
$ groups kari                           # hvilke grupper er kari med i?
$ sudo deluser ola                      # slett bruker
```

Informasjonen lagres i `/etc/passwd` (brukere), `/etc/group` (grupper) og `/etc/shadow` (krypterte passord, bare lesbar for root).

`sudo` – låne superkrefter

`sudo` (*superuser do*) kjører én kommando med root-rettigheter, hvis brukeren din har lov (er i gruppen `sudo` på Ubuntu eller `wheel` på Red Hat).

```
$ apt update
E: Could not open lock file ... - open (13: Permission denied)
$ sudo apt update        # nå virker det
```

Vanlig feil: Å logge inn som root og gjøre alt der. Bruk en vanlig bruker og `sudo` for enkeltkommandoer. Det gir sporbarhet (logges) og hindrer mange uhell.

Vanlig feil: `usermod -G` *uten* `-a` erstatter alle gruppene til brukeren – inkludert `sudo`-gruppen. Da har du låst deg selv ute. Bruk alltid `-aG`.

Rettigheter (*permissions*)

Hva er det? Hver fil og mappe har en **eier** (*user*), en **gruppe** (*group*), og rettigheter for tre kategorier: eieren, gruppen og **alle andre** (*others*).

```
-rwxr-xr--  1  student  utviklere  2048  Oct 30 10:12  deploy.sh
|└┘└┘└┘     |        |
| |  |  |     eier     gruppe
| |  |  └── andre (others):   r-- = kan lese
| |  └───── gruppen:          r-x = kan lese og kjøre
| └──────── eieren (user):    rwx = kan lese, skrive og kjøre
└─────────── filtype: - = fil, d = mappe, l = lenke
```

Bokstav	På en fil	På en mappe
`r` (read)	Lese innholdet	Liste innholdet (`ls`)
`w` (write)	Endre innholdet	Lage/slette filer i mappen
`x` (execute)	Kjøre som program	Gå inn i mappen (`cd`)

Tallnotasjon

Hver rettighet har en verdi: **r = 4, w = 2, x = 1**. Legg sammen for hver kategori:

```
rwx = 4+2+1 = 7      r-x = 4+0+1 = 5      r-- = 4      rw- = 6
rwxr-xr-- = 754      rw-r--r-- = 644      rw------- = 600
```

Tall	Betyr	Typisk bruk
755	Alle kan lese og kjøre, bare eier kan endre	Skript, programmer, mapper
644	Alle kan lese, bare eier kan skrive	Vanlige filer, konfig
600	Bare eier kan lese og skrive	Private SSH-nøkler, hemmeligheter
700	Bare eier har tilgang	`~/.ssh`-mappen
777	Alle kan alt	**Nesten aldri riktig!**

`chmod` og `chown`

```
$ chmod +x deploy.sh                # gjør skriptet kjørbart
$ chmod 600 ~/.ssh/id_ed25519       # privat nøkkel: bare eier
$ chmod -R 755 /var/www/minside     # -R = rekursivt
$ chmod u+w,g-w,o-rwx fil.txt       # symbolsk: legg til/fjern for u(ser),
g(roup), o(thers)

$ sudo chown www-data:www-data /var/www/minside -R   # endre eier og gruppe
$ sudo chown student fil.txt                          # bare eier
```

Vanlig feil: «Det virker ikke, så jeg kjørte `chmod 777`.» Da kan hvem som helst endre filen – et alvorlig sikkerhetshull. Finn ut *hvilken* bruker som trenger tilgang, og gi akkurat den tilgangen.

Fra arbeidslivet: En typisk feil etter deploy: nginx viser `403 Forbidden` fordi filene i `/var/www` eies av brukeren som kopierte dem, og `www-data` ikke kan lese dem. Løsning: riktig eier/gruppe og `644` / `755`.

Prosesser

Hva er det? En prosess er et program som kjører. Hver prosess har en **PID** (*process ID*), en eier, og bruker CPU og minne.

```
$ ps aux                        # alle prosesser
USER   PID %CPU %MEM    VSZ   RSS TTY STAT START   TIME COMMAND
root     1  0.0  0.3 167640 12800 ?   Ss   09:00   0:02 /sbin/init
www-data 812 0.1 0.5 55280  9800 ?   S    09:01   0:05 nginx: worker process
...
$ ps aux | grep nginx           # finn en bestemt prosess
$ pgrep -a python               # finn prosesser ved navn

$ top                           # live oversikt (q = avslutt, P = sorter CPU, M =
sorter minne)
$ htop                          # penere versjon (må ofte installeres: sudo apt
install htop)
```

Avslutte prosesser:

```
$ kill 1234          # be prosess 1234 avslutte pent (signal SIGTERM)
$ kill -9 1234       # tving den (SIGKILL) – bare når den ikke reagerer
$ pkill -f app.py    # avslutt basert på navn/kommandolinje
```

> **Hverdagsanalogi:** `kill` er å be en gjest pent om å gå. `kill -9` er å la vekteren bære dem ut. Prøv det høflige først, slik at programmet får lagret og ryddet.

Forgrunn og bakgrunn:

```
$ python3 app.py &       # & = kjør i bakgrunnen
$ jobs                   # vis bakgrunnsjobber
$ fg %1                  # hent jobb 1 til forgrunnen
$ nohup ./langjobb.sh &  # fortsett selv om du logger ut
```

For ekte tjenester bruker vi ikke `&`, men **systemd** (neste seksjon).

Lastgjennomsnitt (*load average*)

`uptime` og `top` viser tre tall, f.eks. `load average: 0.50, 1.20, 0.80`. Det er gjennomsnittlig antall prosesser som kjører eller venter, siste 1, 5 og 15 minutter. Tommelfingerregel: sammenlign med antall CPU-kjerner (`nproc`). Konstant høyere load enn kjerner betyr at maskinen er overbelastet.

Tjenester (*services*) med systemd

Hva er det? En tjeneste er et program som kjører i bakgrunnen hele tiden (en *daemon*): webserver, database, SSH-server. På nesten alle moderne distroer styres de av **systemd**, med kommandoen

`systemctl`.

```
$ sudo systemctl status nginx      # kjører den? siste loggmeldinger
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
     Active: active (running) since Thu 2026-10-30 09:01:12 UTC; 2h ago
   Main PID: 811 (nginx)

$ sudo systemctl start nginx       # start
$ sudo systemctl stop nginx        # stopp
$ sudo systemctl restart nginx     # stopp og start
$ sudo systemctl reload nginx      # les konfigurasjonen på nytt uten å stoppe
$ sudo systemctl enable nginx      # start automatisk ved oppstart
$ sudo systemctl disable nginx     # ikke start ved oppstart
$ sudo systemctl enable --now nginx   # enable + start i ett
$ systemctl list-units --type=service --state=running   # alle kjørende tjenester
```

> **Vanlig feil:** `start` uten `enable`. Tjenesten kjører – til serveren startes på nytt. Da er den borte, og ingen skjønner hvorfor.

Lag din egen tjeneste

Har du en Python-app som skal kjøre som tjeneste, lager du en *unit-fil*:

```
# /etc/systemd/system/minapp.service
[Unit]
Description=Min Python-app
After=network.target                # start etter at nettverket er oppe

[Service]
User=minapp                         # kjør som egen bruker, ikke root
WorkingDirectory=/opt/minapp
ExecStart=/usr/bin/python3 /opt/minapp/app.py
Restart=on-failure                  # start på nytt hvis den krasjer
Environment=PORT=8000

[Install]
WantedBy=multi-user.target          # starte ved vanlig oppstart
```

```
$ sudo systemctl daemon-reload         # systemd leser inn nye/endrede unit-filer
$ sudo systemctl enable --now minapp
$ journalctl -u minapp -f              # følg loggen til tjenesten
```

Pakkehåndtering (*package management*)

Hva er det? En pakkehåndterer installerer, oppdaterer og fjerner programvare fra trygge **repositories** (pakkebrønner), og håndterer **avhengigheter** automatisk.

> **Hverdagsanalogi:** Som App Store for serveren. Du sier «installer nginx», og pakkehåndtereren henter nginx og alt nginx trenger, fra en kilde den stoler på.

Debian/Ubuntu (`apt`):

```
$ sudo apt update                # oppdater LISTEN over tilgjengelige pakker
$ sudo apt upgrade               # oppgrader installerte pakker
$ sudo apt install nginx         # installer
$ sudo apt install -y git curl   # -y = svar ja automatisk (nyttig i skript)
$ sudo apt remove nginx          # fjern
$ apt search postgresql          # søk
$ apt list --installed           # hva er installert?
```

Red Hat-familien (`dnf`):

```
$ sudo dnf check-update
$ sudo dnf install nginx
$ sudo dnf upgrade
$ sudo dnf remove nginx
```

> **Vanlig feil:** `apt install` feiler med «Unable to locate package». Ofte har du glemt `sudo apt update` først.

Andre pakkeverktøy du møter: `pip` (Python-biblioteker), `npm` (JavaScript), `snap` , og `apk` (Alpine, i containere).

SSH i praksis

Grunnleggende SSH ble forklart i kapittel 2. Her er det du trenger for serverdrift:

```
$ ssh -i ~/.ssh/id_ed25519 azureuser@20.100.1.2    # bruk en bestemt nøkkel
$ ssh -p 2222 bruker@server                         # annen port
$ ssh bruker@server 'df -h'                         # kjør én kommando eksternt
$ scp -r ./site bruker@server:/tmp/                 # kopier mappe
$ rsync -avz ./site/ bruker@server:/var/www/site/   # synkroniser effektivt (bare
endringer)
$ ssh -L 8080:localhost:80 bruker@server            # tunnel: din port 8080 →
serverens port 80
```

Herde SSH-serveren i `/etc/ssh/sshd_config` (eller en fil i `/etc/ssh/sshd_config.d/`):

```
PermitRootLogin no              # ingen direkte root-innlogging
PasswordAuthentication no       # bare nøkler
PubkeyAuthentication yes
```

```
        $ sudo sshd -t                 # test konfigurasjonen før restart!
        $ sudo systemctl restart ssh   # tjenesten heter "ssh" på Ubuntu, "sshd" på Red
Hat
```

> **Vanlig feil:** Å slå av passordinnlogging *før* du har testet at nøkkelinnlogging virker. Hold alltid én SSH-økt åpen mens du tester i en ny – ellers kan du låse deg ute.

Miljøvariabler (*environment variables*)

Hva er det? Navngitte verdier som prosesser kan lese. Brukes til konfigurasjon: hvilken database å koble til, hvilket miljø (dev/prod) appen kjører i, osv.

```
        $ echo $HOME                # /home/student
        $ echo $PATH                # mappene shellet leter etter programmer i
        $ env                       # vis alle miljøvariabler
        $ export APP_ENV=production  # sett variabel for dette shellet og prosesser det
starter
        $ echo $APP_ENV
        production
        $ unset APP_ENV             # fjern den
```

For å gjøre en variabel permanent for brukeren din, legg `export`-linjen i `~/.bashrc` og kjør `source ~/.bashrc`.

`PATH` er spesielt viktig: når du skriver `python3`, leter shellet gjennom mappene i `PATH` (separert med `:`) etter et program med det navnet. «command not found» betyr ofte at programmet ikke ligger i `PATH`.

> **Fra arbeidslivet:** Prinsippet «konfigurasjon i miljøet, ikke i koden» (fra *The Twelve-Factor App*) brukes overalt: samme container-image kjører i test og produksjon, bare med forskjellige miljøvariabler. App Service, Kubernetes og GitHub Actions bruker alle miljøvariabler.

> **Vanlig feil:** Å skrive passord direkte i `.bashrc` eller i skript som sjekkes inn i Git. Hemmeligheter hører hjemme i et hemmelighetshvelv (Azure Key Vault, kapittel 7 og 13).

Bash-scripting

Hva er det? Et Bash-skript er en tekstfil med kommandoer som kjøres i rekkefølge. Alt du kan skrive i terminalen, kan du legge i et skript.

Hvorfor? Gjenta oppgaver feilfritt, dokumentere fremgangsmåter, og automatisere.

Ditt første skript

```
        #!/usr/bin/env bash
        # hilsen.sh – et første skript
        # Linje 1 («shebang») forteller hvilket program som skal kjøre skriptet.

        navn="Student"                      # variabel: INGEN mellomrom rundt =
        echo "Hei, $navn!"                  # $navn henter verdien
        echo "I dag er det $(date +%A)"     # $( ) kjører en kommando og setter inn
resultatet
```

```
        $ chmod +x hilsen.sh
        $ ./hilsen.sh
        Hei, Student!
        I dag er det torsdag
```

Argumenter, betingelser og løkker

```
        #!/usr/bin/env bash
        set -euo pipefail
        # set -e: stopp ved første feil
        # set -u: feil hvis du bruker en variabel som ikke finnes
        # set -o pipefail: en feil midt i en pipe regnes som feil

        # $1 er første argument. ${1:-} gir tom streng hvis det mangler (pga. set -u).
        if [[ -z "${1:-}" ]]; then
            echo "Bruk: $0 <mappe>" >&2       # >&2 = skriv til stderr
            exit 1                            # avslutt med feilkode
        fi

        mappe="$1"

        if [[ -d "$mappe" ]]; then            # -d = finnes mappen?
            echo "Mappen $mappe finnes."
        else
            echo "Lager $mappe ..."
            mkdir -p "$mappe"
        fi

        # for-løkke over filer
        for fil in "$mappe"/*.log; do
            [[ -e "$fil" ]] || continue       # hopp over hvis ingen treff
            linjer=$(wc -l < "$fil")
            echo "$fil har $linjer linjer"
        done
```

Nyttige tester i `[[ ]]`:

Test	Sann hvis
`-f fil`	fil finnes og er en vanlig fil
`-d mappe`	mappe finnes

Test	Sann hvis
`-z "$var"`	variabelen er tom
`-n "$var"`	variabelen er ikke tom
`"$a" == "$b"`	strengene er like
`$a -eq $b`, `-lt`, `-gt`	tall er lik / mindre / større

Returkoder (*exit codes*). Hver kommando returnerer et tall: **0 = suksess**, alt annet = feil. Du ser det med `echo $?`. CI/CD-pipelines bruker dette: et steg som returnerer noe annet enn 0, stopper pipelinen.

Funksjoner

```
logg() {
    # $* = alle argumentene
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"
}

logg "Starter backup"
```

Et nyttig skript: backup med rotasjon

```
#!/usr/bin/env bash
# backup.sh – lager komprimert backup av en mappe og sletter gamle backuper
set -euo pipefail

KILDE="/var/www/minside"          # hva skal tas backup av
MAL="/var/backups/minside"        # hvor backupene lagres
BEHOLD_DAGER=7                    # hvor lenge vi beholder backuper

mkdir -p "$MAL"
fil="$MAL/minside-$(date +%Y%m%d-%H%M%S).tar.gz"

tar -czf "$fil" -C "$(dirname "$KILDE")" "$(basename "$KILDE")"
echo "Backup laget: $fil ($(du -h "$fil" | cut -f1))"

# slett backuper eldre enn BEHOLD_DAGER
find "$MAL" -name "minside-*.tar.gz" -mtime +"$BEHOLD_DAGER" -print -delete
```

Vanlig feil: Å glemme anførselstegn rundt variabler. `rm -rf $MAPPE/` med en tom `$MAPPE` blir `rm -rf /`. Bruk alltid `"$variabel"` og `set -u`. Verktøyet **ShellCheck** (`sudo apt install shellcheck`) finner slike feil automatisk – bruk det.

Cron – planlagte jobber

Hva er det? Cron kjører kommandoer automatisk på faste tidspunkter.

```
$ crontab -e      # rediger dine cron-jobber
$ crontab -l      # vis dem
```

Formatet er fem tidsfelt etterfulgt av kommandoen:

```
┌──────── minutt (0–59)
│ ┌────── time (0–23)
│ │ ┌──── dag i måneden (1–31)
│ │ │ ┌── måned (1–12)
│ │ │ │ ┌ ukedag (0–7, 0 og 7 = søndag)
│ │ │ │ │
30 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
```

Uttrykk	Betyr
`30 2 * * *`	Hver dag kl. 02:30
`*/15 * * * *`	Hvert 15. minutt
`0 9 * * 1-5`	Kl. 09:00 mandag–fredag
`0 0 1 * *`	Midnatt den 1. hver måned
`@reboot`	Ved oppstart

Vanlig feil: Skriptet virker når du kjører det selv, men ikke i cron. Årsak: cron har en minimal `PATH` og et annet miljø. Bruk **fulle stier** (`/usr/bin/python3`), og send utskrift til en loggfil slik at du ser feilene. Husk også at serverens tidssone kan være UTC.

Fra arbeidslivet: Den samme cron-syntaksen brukes i GitHub Actions (`on: schedule`), Kubernetes CronJobs og Azure Functions (med et ekstra sekundfelt). Lær den én gang, bruk den overalt.

Systemd timers er et moderne alternativ til cron, med bedre logging. Du vil se begge deler.

Logger

Hvor finner jeg loggene?

Plassering	Innhold
`journalctl`	systemd-journalen: logger fra alle tjenester

Plassering	Innhold
`/var/log/syslog` (Ubuntu) eller `/var/log/messages` (Red Hat)	Generelle systemmeldinger
`/var/log/auth.log` (Ubuntu) eller `/var/log/secure` (Red Hat)	Innlogginger, sudo
`/var/log/nginx/access.log` og `error.log`	Webserveren
`/var/log/apt/`	Pakkeinstallasjoner
`dmesg`	Meldinger fra kjernen (maskinvare, OOM-killer)

```
        $ journalctl -u nginx                    # logger for én tjeneste
        $ journalctl -u nginx -f                 # følg live
        $ journalctl -u nginx --since "1 hour ago"
        $ journalctl -p err -b                   # bare feil siden siste oppstart
        $ sudo tail -f /var/log/auth.log         # se innloggingsforsøk live
        $ sudo grep "Failed password" /var/log/auth.log | wc -l   # mislykkede
innlogginger
        $ sudo dmesg -T | grep -i "out of memory"
```

Loggrotasjon. Logger vokser. `logrotate` roterer, komprimerer og sletter gamle logger automatisk (konfig i `/etc/logrotate.d/`). Fulle disker på grunn av logger er en klassisk årsak til nedetid.

Systemovervåking

```
        $ uptime                 # oppetid og load average
        $ top / htop             # CPU, minne og prosesser live
        $ free -h                # minne
        $ df -h                  # diskplass per filsystem
        $ df -i                  # inoder (kan gå tomt selv om det er plass!)
        $ du -sh /var/* | sort -h   # hvilke mapper er størst?
        $ iostat -x 2            # diskaktivitet (pakken sysstat)
        $ vmstat 2               # CPU, minne, swap hvert 2. sekund
        $ ss -tunap              # nettverksforbindelser
```

Manuell overvåking er greit for feilsøking. I produksjon samles slike metrikker automatisk av verktøy som Azure Monitor, Prometheus og Grafana (kapittel 13).

Linux-sikkerhet – grunnregler

1. **Oppdater jevnlig:** `sudo apt update && sudo apt upgrade`. Vurder `unattended-upgrades` for automatiske sikkerhetsoppdateringer.
2. **Ingen root-innlogging og ingen passord over SSH** – bare nøkler.

3. **Brannmur:** på Ubuntu er `ufw` enkelt:

```
$ sudo ufw allow OpenSSH        # IKKE glem denne før du slår på!
$ sudo ufw allow 80/tcp
$ sudo ufw allow 443/tcp
$ sudo ufw enable
$ sudo ufw status verbose
```

4. **Minste privilegium:** tjenester kjører som egne brukere, ikke root. Bare de som trenger det, får `sudo`.
5. **Fail2ban:** blokkerer IP-er som prøver å gjette passord.
6. **Fjern det du ikke bruker:** færre pakker og åpne porter = mindre angrepsflate.
7. **Overvåk logger:** `auth.log` forteller hvem som prøver å komme inn.
8. **SELinux/AppArmor:** ekstra sikkerhetslag som begrenser hva programmer får gjøre. Ikke slå dem av for å «få ting til å virke».

Feilsøking på Linux – metode

```
1. Hva er symptomet?       «Nettsiden gir 502»
2. Kjører tjenesten?       systemctl status <tjeneste>
3. Hva sier loggene?       journalctl -u <tjeneste> -n 100 / tail /var/log/...
4. Ressurser?              df -h · free -h · top
5. Nettverk/porter?        ss -tlnp · curl localhost:<port>
6. Rettigheter?            ls -l · id <bruker>
7. Hva er endret sist?     deploy? oppdatering? konfig? (history, git log)
8. Endre ÉN ting, test, dokumenter.
```

Eksempel – «disken er full»:

```
$ df -h
/dev/root        30G   30G     0 100% /
$ sudo du -xh / --max-depth=1 2>/dev/null | sort -h | tail -5
...
 18G  /var
$ sudo du -xh /var --max-depth=2 | sort -h | tail -3
 17G  /var/log/minapp
$ ls -lh /var/log/minapp
-rw-r--r-- 1 minapp minapp 17G Oct 30 11:00 debug.log
```

Årsak: appen logger på debug-nivå uten rotasjon. Løsning: senk loggnivået, sett opp `logrotate`, og legg inn diskvarsel i overvåkingen.

> **Vanlig feil:** Å slette en stor loggfil som et program fortsatt skriver til. Plassen frigjøres ikke før programmet lukker filen. Tøm den heller: `sudo truncate -s 0 debug.log`.

Kapitteloppsummering

- Linux er kjernen; distroer (Ubuntu, RHEL, Debian, Alpine) bygger komplette systemer rundt den.
- Filsystemet starter i `/`. `/etc` = konfig, `/var/log` = logger, `/home` = brukere.
- Kjernekommandoer: `pwd ls cd mkdir cp mv rm cat less tail grep find` + pipes og omdirigering.
- Rettigheter: eier/gruppe/andre × r/w/x. 644 for filer, 755 for skript/mapper, 600 for hemmeligheter.
- Tjenester styres med `systemctl`; logger leses med `journalctl` og i `/var/log`.
- Programvare installeres med `apt` eller `dnf`.
- Bash-skript automatiserer; bruk `set -euo pipefail`, anførselstegn og ShellCheck.
- Cron planlegger jobber; bruk fulle stier og logg utskriften.
- Sikkerhet: oppdater, SSH-nøkler, brannmur, minste privilegium.

Øvingsoppgaver

1. Lag mappestrukturen `~/lab/{logs,scripts,data}` med én kommando (tips: `mkdir -p` og klammeparenteser).
2. Lag en bruker `testbruker`, en gruppe `team`, legg brukeren i gruppen, og bekreft med `id testbruker`.
3. Lag en fil som bare eier kan lese og skrive. Bekreft med `ls -l`.
4. Installer nginx, start og enable den, og bekreft med `curl localhost`.
5. Skriv et skript `diskvarsel.sh` som skriver en advarsel hvis diskbruken på `/` er over 80 %. (Tips: `df --output=pcent / | tail -1 | tr -dc '0-9'`.)
6. Legg skriptet i cron slik at det kjører hvert 10. minutt og logger til `~/lab/logs/disk.log`.
7. Finn de 10 største filene under `/var` med `find` eller `du`.

Quiz

1. Hva er forskjellen på Linux-kjernen og en distribusjon?
2. Hvor ligger konfigurasjonsfiler, og hvor ligger logger?
3. Hva betyr `chmod 640 fil.txt`?
4. Hva er forskjellen på `>` og `>>`?
5. Hva er forskjellen på `systemctl start` og `systemctl enable`?
6. Hvorfor bør du bruke `sudo` i stedet for å logge inn som root?
7. Hva gjør `set -euo pipefail` i et Bash-skript?
8. Hva betyr cron-uttrykket `0 */6 * * *`?
9. Hva betyr returkode 0?

10. Et program sier «Permission denied» når det skal lese `/etc/minapp/config.yaml`. Hva sjekker du?

Fasit

1. **Kjernen** er kjernen i OS-et som styrer maskinvaren. En **distribusjon** er kjernen pluss verktøy, pakkesystem og programmer, satt sammen til et komplett operativsystem.
2. Konfigurasjon i `/etc`, logger i `/var/log` (og i systemd-journalen via `journalctl`).
3. Eier: les+skriv (6), gruppe: les (4), andre: ingenting (0).
4. `>` **overskriver** filen; `>>` **legger til** på slutten.
5. `start` starter tjenesten **nå**. `enable` gjør at den starter **automatisk ved oppstart**.
6. Det gir sporbarhet (hvem gjorde hva), begrenser root-makten til enkeltkommandoer, og reduserer risikoen for katastrofale uhell.
7. Stopp ved feil (`-e`), feil ved udefinerte variabler (`-u`), og at feil midt i en pipe teller (`pipefail`).
8. På minutt 0 hver sjette time: 00:00, 06:00, 12:00 og 18:00.
9. Suksess. Alt annet enn 0 er en feil.
10. Sjekk hvilken bruker programmet kjører som (`ps aux`, unit-filen), og filens eier og rettigheter (`ls -l`). Sjekk også rettighetene på mappen `/etc/minapp` (krever `x` for å gå inn). Gi riktig gruppe/eier – ikke `chmod 777`.

Del 4 – Programmering for DevOps

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva programmering er og hvordan kode blir kjørt,
- skrive Python-programmer med variabler, datatyper, betingelser, løkker og funksjoner,
- jobbe med lister, ordbøker, JSON og YAML,
- gjøre API-kall, håndtere feil og logge på en profesjonell måte,
- skrive automatiseringsskript i Python og Bash,
- forstå hvordan Python brukes mot Azure.

Hva er programmering egentlig?

Hva er det? Programmering er å skrive **presise instruksjoner** som en datamaskin kan følge. Datamaskinen er ekstremt rask, men helt uten sunn fornuft: den gjør *nøyaktig* det du skriver – ikke det du mente.

> **Hverdagsanalogi:** En oppskrift for en robotkokk. «Stek til den er gyllen» fungerer for et menneske. Roboten trenger: «Sett platen på nivå 6. Vent 180 sekunder. Snu. Vent 120 sekunder.» Programmering er å skrive oppskrifter for roboter.

Hvordan blir kode kjørt?

- **Kompilerte språk** (C, Go, Rust, Java/C# i en variant) oversettes først til maskinkode eller bytekode. Det gir rask kjøring.
- **Tolkede språk** (Python, Bash, JavaScript) leses og kjøres av en *tolk* (*interpreter*) linje for linje. Det gir raskere utvikling og enkel testing.

Hvorfor må en DevOps Engineer programmere? Du skal ikke nødvendigvis bygge store applikasjoner, men du skal:

- automatisere repeterende oppgaver (opprydding, rapporter, deploys),
- lese og forstå koden til utviklerne du jobber med,
- skrive pipelines, infrastrukturkode og konfigurasjon (YAML, HCL, Bicep),
- bruke API-er for Azure, GitHub, Slack osv.,
- feilsøke applikasjoner i produksjon.

Hvilke språk? Start med **Python** (lettlest, enormt mange biblioteker, brukes overalt i DevOps, sky og AI) og **Bash** (for Linux-automatisering). Senere kan **Go** være nyttig (Docker, Kubernetes og Terraform er skrevet i Go), og **PowerShell** er nyttig i Windows-tunge miljøer.

Python fra bunnen

Installasjon og første program

Sjekk om Python er installert:

```
$ python3 --version
Python 3.12.3
```

(Versjonen din kan være en annen. Bruk Python 3.10 eller nyere for eksemplene i boken.) På Windows kan du installere fra python.org eller Microsoft Store; på Ubuntu: `sudo apt install python3 python3-pip python3-venv`.

Lag filen `hei.py`:

```
print("Hei, verden!")
```

```
$ python3 hei.py
Hei, verden!
```

`print()` er en **funksjon** som skriver ut det du gir den. Teksten i anførselstegn er en **streng** (*string*).

Bruk en god editor. Visual Studio Code (gratis) med Python-utvidelsen gir fargekoding, feilmarkering og feilsøking.

Det interaktive skallet. Skriv bare `python3` for å teste kode linje for linje. Avslutt med `exit()`.

Virtuelle miljøer

Hva er det? Et virtuelt miljø (*virtual environment*) er en egen, isolert mappe med Python-pakker for ett prosjekt.

Hvorfor? Prosjekt A trenger kanskje `requests` versjon 2.28, prosjekt B en annen. Uten isolasjon krasjer de.

```
$ python3 -m venv .venv            # lag miljøet i mappen .venv
$ source .venv/bin/activate        # aktiver (Windows: .venv\Scripts\activate)
(.venv) $ pip install requests     # installer pakker inn i miljøet
(.venv) $ pip freeze > requirements.txt   # lagre listen over pakker
(.venv) $ deactivate               # gå ut av miljøet
```

En annen person (eller en CI-pipeline) gjenskaper miljøet med `pip install -r requirements.txt`.

> **Vanlig feil:** Å sjekke inn `.venv`-mappen i Git. Legg den i `.gitignore`; sjekk heller inn `requirements.txt`.

Variabler

Hva er det? En variabel er et **navn** som peker på en **verdi**. Som en merkelapp på en eske.

```
servernavn = "web-01"      # en tekst (str)
antall_cpu = 4             # et heltall (int)
cpu_last = 72.5            # et desimaltall (float)
er_aktiv = True            # en sannhetsverdi (bool): True eller False

print(servernavn)                          # web-01
print(f"{servernavn} har {antall_cpu} CPU-er")  # f-streng: setter inn verdier
```

Linje for linje:

- `servernavn = "web-01"` – lag navnet `servernavn` og la det peke på teksten `"web-01"`. `=` betyr «tilordne», ikke «er lik».
- `f"..."` – en **f-streng**. Alt inni `{ }` byttes ut med verdien.

Navneregler: små bokstaver og understrek (`antall_cpu`), ikke start med tall, ingen mellomrom, ikke æøå (det virker, men er uvanlig i kode). Velg beskrivende navn: `antall_feil` er bedre enn `x`.

Datatyper

Type	Eksempel	Brukes til
`str`	`"nginx"`	Tekst
`int`	`443`	Heltall
`float`	`99.9`	Desimaltall
`bool`	`True`	Ja/nei
`list`	`["web-01", "web-02"]`	Ordnet samling
`dict`	`{"navn": "web-01", "cpu": 4}`	Nøkkel → verdi
`None`	`None`	«Ingen verdi»

```
port = "8080"            # dette er TEKST, ikke et tall
print(port + 1)          # TypeError! Kan ikke legge tall til tekst
print(int(port) + 1)     # 8081 – int() konverterer tekst til tall
print(type(port))        # <class 'str'>
```

> **Vanlig feil:** Verdier fra miljøvariabler, filer og brukerinput er **alltid tekst**. Konverter dem med `int()` eller `float()` før du regner med dem.

Strengoperasjoner du ofte trenger:

```
        linje = "  ERROR: Disk full on /dev/sda1  "
        print(linje.strip())               # fjern mellomrom i endene
        print(linje.lower())               # små bokstaver
        print("ERROR" in linje)            # True – inneholder teksten "ERROR"?
        print(linje.strip().split(":"))    # ['ERROR', ' Disk full on /dev/sda1']
        print("web-01".replace("01", "02"))  # web-02
```

Betingelser (*conditions*)

Hva er det? Kode som bare kjøres hvis noe er sant.

```
        cpu_last = 91

        if cpu_last > 90:                      # hvis lasten er over 90
            print("KRITISK: skaler opp!")      # innrykk (4 mellomrom) = denne blokken
hører til if
        elif cpu_last > 70:                    # ellers, hvis over 70
            print("ADVARSEL: høy last")
        else:                                  # i alle andre tilfeller
            print("OK")
```

> **Husk:** I Python er **innrykk** (*indentation*) en del av språket. Linjene som hører til `if`, må ha samme innrykk. Bruk 4 mellomrom.

Sammenligninger: `==` (lik), `!=` (ulik), `<`, `>`, `<=`, `>=`. Kombiner med `and`, `or`, `not`:

```
        miljø = "prod"
        er_godkjent = False
        if miljø == "prod" and not er_godkjent:
            print("Stopp! Deploy til produksjon krever godkjenning.")
```

Løkker (*loops*)

`for`-løkke – gjør noe for hvert element i en samling:

```
        servere = ["web-01", "web-02", "db-01"]

        for server in servere:                 # server får hver verdi etter tur
            print(f"Sjekker {server} ...")

        for i in range(3):                     # range(3) gir 0, 1, 2
            print(f"Forsøk nummer {i + 1}")
```

`while`-løkke – gjenta så lenge noe er sant:

```
import time

forsøk = 0
while forsøk < 5:
    forsøk += 1                          # samme som forsøk = forsøk + 1
    print(f"Prøver å koble til (forsøk {forsøk}) ...")
    time.sleep(1)                        # vent ett sekund
```

`break` avslutter løkken, `continue` hopper til neste runde.

> **Vanlig feil:** En `while`-løkke som aldri blir usann, kjører for alltid. Avbryt med `Ctrl + C`.

Funksjoner

Hva er det? En navngitt kodeblokk du kan gjenbruke. Den tar imot **parametere** og kan gi tilbake (**returnere**) en verdi.

Hvorfor? Unngå å skrive samme kode flere ganger, gjøre koden lesbar, og gjøre det mulig å teste deler hver for seg.

```
def status_for_last(cpu_prosent: float, grense: float = 80.0) -> str:
    """Returnerer en status basert på CPU-last.

    cpu_prosent: målt last i prosent
    grense: når vi regner lasten som høy (standard 80)
    """
    if cpu_prosent >= grense:
        return "HØY"
    return "NORMAL"

print(status_for_last(95))              # HØY
print(status_for_last(60))              # NORMAL
print(status_for_last(60, grense=50))   # HØY
```

Forklaring:

- `def` – definerer en funksjon.
- `cpu_prosent: float` – parameter med **typehint** (forteller leseren og verktøy hvilken type som forventes; Python håndhever det ikke).
- `grense: float = 80.0` – parameter med **standardverdi**.
- `-> str` – funksjonen returnerer en streng.
- `"""..."""` – **docstring**: dokumentasjon av funksjonen.
- `return` – gir verdien tilbake og avslutter funksjonen.

Lister og ordbøker

Lister (*lists*) er ordnede samlinger:

```
porter = [22, 80, 443]
porter.append(8080)          # legg til på slutten → [22, 80, 443, 8080]
print(porter[0])             # 22 — første element har indeks 0!
print(porter[-1])            # 8080 — siste element
print(len(porter))           # 4
porter.remove(22)            # fjern verdien 22
print(80 in porter)          # True

# List comprehension: lag en ny liste i én linje
høye = [p for p in porter if p > 100]   # [443, 8080]
```

Ordbøker (*dictionaries*, *dict*) lagrer **nøkkel → verdi**:

```
vm = {
    "navn": "web-01",
    "region": "norwayeast",
    "størrelse": "Standard_B2s",
    "tags": {"miljø": "test", "eier": "student"},
}

print(vm["navn"])                    # web-01
print(vm["tags"]["miljø"])           # test — ordbok inni ordbok
vm["status"] = "running"             # legg til / endre en nøkkel
print(vm.get("ip", "ingen IP"))      # .get() gir standardverdi hvis nøkkelen
mangler

for nøkkel, verdi in vm.items():     # gå gjennom alle par
    print(f"{nøkkel}: {verdi}")
```

Vanlig feil: `vm["ip"]` gir `KeyError` hvis nøkkelen ikke finnes. Bruk `vm.get("ip")` når nøkkelen kan mangle.

Husk: Lister og ordbøker er de viktigste datastrukturene i DevOps-skripting, fordi JSON (fra API-er) blir nettopp lister og ordbøker i Python.

Moduler og biblioteker

Python har et stort **standardbibliotek**, og du kan installere mer med `pip`:

```
import os                    # operativsystemet: miljøvariabler, filer
import sys                   # argumenter, avslutning
import json                  # JSON
import subprocess            # kjøre andre programmer
from pathlib import Path     # moderne filstier
from datetime import datetime

print(os.environ.get("HOME"))              # les miljøvariabel
print(Path("/var/log").exists())           # finnes mappen?
print(datetime.now().isoformat())          # nåtid, f.eks. 2026-10-30T10:15:00
```

Lese og skrive filer

```
from pathlib import Path

# Skriv til fil. "with" sørger for at filen lukkes automatisk.
with open("servere.txt", "w", encoding="utf-8") as f:
    f.write("web-01\nweb-02\ndb-01\n")

# Les linje for linje
with open("servere.txt", encoding="utf-8") as f:
    for linje in f:
        print(linje.strip())

# Kortversjon med pathlib
innhold = Path("servere.txt").read_text(encoding="utf-8")
```

JSON og YAML

Dette er de to dataformatene du kommer til å se *hele tiden* i DevOps.

JSON

Hva er det? JSON (*JavaScript Object Notation*) er et tekstformat for data. Nesten alle API-er bruker det.

```
{
  "navn": "web-01",
  "cpu": 4,
  "aktiv": true,
  "porter": [80, 443],
  "tags": { "miljø": "prod" }
}
```

Regler: nøkler i **doble** anførselstegn, ingen komma etter siste element, ingen kommentarer.

```
import json

tekst = '{"navn": "web-01", "porter": [80, 443]}'
data = json.loads(tekst)               # JSON-tekst → Python-dict
print(data["porter"][1])               # 443

data["porter"].append(8080)
print(json.dumps(data, indent=2))      # Python → pen JSON-tekst

with open("vm.json", "w") as f:        # skriv til fil
    json.dump(data, f, indent=2)
with open("vm.json") as f:             # les fra fil
    data = json.load(f)
```

På kommandolinjen er `jq` gull verdt:

```
        $ curl -s https://api.github.com/repos/kubernetes/kubernetes | jq
'.stargazers_count'
        $ az vm list | jq '.[].name'                        # navnet på alle VM-er
```

(Azure CLI har også sitt eget spørrespråk: `az vm list --query "[].name" -o tsv`.)

YAML

Hva er det? YAML (*YAML Ain't Markup Language*) er et mer menneskevennlig dataformat. Det brukes i Kubernetes, GitHub Actions, Azure Pipelines, Docker Compose og Ansible.

```
        # Dette er en kommentar – YAML tillater kommentarer
        navn: web-01            # nøkkel: verdi
        cpu: 4
        aktiv: true
        porter:                 # en liste
          - 80
          - 443
        tags:                   # en ordbok (innrykk med mellomrom!)
          miljø: prod
          eier: devops-team
        beskrivelse: |          # | = flerlinjers tekst, linjeskift beholdes
          Webserver for
          nettbutikken.
```

Samme data i JSON og YAML:

```
JSON                                YAML
{"porter": [80, 443],               porter:
 "tags": {"miljø": "prod"}}           - 80
                                      - 443
                                    tags:
                                      miljø: prod
```

Vanlig feil: YAML bruker **mellomrom, aldri tab**, og innrykket bestemmer strukturen. Ett mellomrom feil = feil struktur eller ugyldig fil. Bruk en editor med YAML-støtte og et verktøy som `yamllint`.

Vanlig feil: YAML gjetter typer. `versjon: 1.10` blir tallet 1.1. `land: NO` kan i eldre YAML-tolkere bli `false`. Sett tekst i anførselstegn når du er i tvil: `versjon: "1.10"`.

```
        import yaml          # pip install pyyaml

        with open("config.yaml") as f:
            config = yaml.safe_load(f)        # safe_load: aldri yaml.load på ukjente
filer
        print(config["tags"]["miljø"])
```

API-kall med Python

Biblioteket `requests` (`pip install requests`) gjør HTTP enkelt.

```
        import requests

        url = "https://api.github.com/repos/python/cpython"
        svar = requests.get(url, timeout=10)      # ALLTID timeout – ellers kan skriptet
henge for alltid

        print(svar.status_code)                   # 200
        svar.raise_for_status()                   # kaster unntak hvis 4xx/5xx
        data = svar.json()                        # JSON → dict
        print(f"{data['full_name']} har {data['stargazers_count']} stjerner")
```

Med autentisering og POST:

```
        import os
        import requests

        token = os.environ["GITHUB_TOKEN"]        # hemmelighet fra miljøet, ALDRI i koden
        headers = {
            "Authorization": f"Bearer {token}",
            "Accept": "application/vnd.github+json",
        }
        body = {"title": "Disk nesten full på web-01", "body": "Automatisk opprettet av
overvåkingsskript."}

        svar = requests.post(
            "https://api.github.com/repos/MITT-BRUKERNAVN/MITT-REPO/issues",
            headers=headers, json=body, timeout=10,
        )
        print(svar.status_code)                   # 201 = opprettet
```

> **Vanlig feil:** Å lime inn tokens og passord direkte i koden og pushe til GitHub. Roboter skanner GitHub etter nøkler kontinuerlig, og en lekket nøkkel kan misbrukes innen minutter. GitHub har *secret scanning* som kan stoppe noen av disse, men ikke stol på det. Bruk miljøvariabler eller et hemmelighetshvelv.

Feilhåndtering (*error handling*)

Hva er det? Når noe går galt, kaster Python et **unntak** (*exception*). Uten håndtering krasjer programmet. Med `try` / `except` kan du reagere kontrollert.

```
        import requests

        def hent_status(url: str) -> int | None:
            """Returner HTTP-statuskode for url, eller None hvis den ikke kan nås."""
            try:
                svar = requests.get(url, timeout=5)
                return svar.status_code
            except requests.exceptions.Timeout:
                print(f"Tidsavbrudd mot {url}")
            except requests.exceptions.ConnectionError:
                print(f"Kan ikke koble til {url}")
            return None

        print(hent_status("https://www.example.com"))
```

- `try:` – prøv denne koden.
- `except Type:` – hvis *denne typen* feil skjer, gjør dette.
- `finally:` – (valgfri) kjøres uansett, f.eks. for opprydding.
- `raise` – kast et unntak selv.

Gjentatte forsøk med ventetid (*retry with backoff*) er et viktig mønster i skyen, fordi nettverk og API-er av og til feiler midlertidig:

```
        import time
        import requests

        def get_med_retry(url: str, forsøk: int = 4) -> requests.Response:
            for n in range(forsøk):
                try:
                    svar = requests.get(url, timeout=5)
                    if svar.status_code < 500:        # 5xx kan være midlertidig; 4xx er
det ikke
                        return svar
                except requests.exceptions.RequestException as e:
                    print(f"Feil: {e}")
                vent = 2 ** n                         # 1, 2, 4, 8 sekunder
(eksponentiell backoff)
                print(f"Prøver igjen om {vent} s ...")
                time.sleep(vent)
            raise RuntimeError(f"Ga opp etter {forsøk} forsøk mot {url}")
```

> **Vanlig feil:** `except:` uten type, eller `except Exception: pass`. Da svelges *alle* feil, også de du burde visst om. Fang de feilene du forventer, og logg resten.

Logging

Hvorfor ikke bare `print`? I produksjon trenger du tidsstempel, alvorlighetsgrad, og muligheten til å skru detaljnivået opp og ned uten å endre koden.

```
        import logging

        logging.basicConfig(
            level=logging.INFO,                                    # vis INFO og
alvorligere
            format="%(asctime)s %(levelname)s %(name)s: %(message)s",
        )
        log = logging.getLogger("diskmonitor")

        log.debug("Detaljer for feilsøking")         # vises ikke på INFO-nivå
        log.info("Starter sjekk av disker")
        log.warning("Disk / er 85 % full")
        log.error("Klarte ikke lese /mnt/data")
```

```
2026-10-30 10:20:01,123 INFO diskmonitor: Starter sjekk av disker
2026-10-30 10:20:01,130 WARNING diskmonitor: Disk / er 85 % full
2026-10-30 10:20:01,131 ERROR diskmonitor: Klarte ikke lese /mnt/data
```

Nivå	Bruk
DEBUG	Detaljer for utviklere ved feilsøking
INFO	Normale hendelser («startet», «ferdig»)
WARNING	Noe uventet, men programmet fortsetter
ERROR	En operasjon feilet
CRITICAL	Programmet kan ikke fortsette

Fra arbeidslivet: I skyen logger man ofte i **JSON-format** (*strukturert logging*) til `stdout`. Plattformen (Kubernetes, App Service, Azure Monitor) samler loggene, og du kan søke på felt som `level`, `user_id` eller `request_id`. Logg aldri passord, tokens eller personopplysninger.

Kommandolinjeargumenter

Profesjonelle skript tar imot argumenter i stedet for hardkodede verdier. `argparse` er innebygd:

```
        import argparse

        parser = argparse.ArgumentParser(description="Sjekk om nettsider svarer.")
        parser.add_argument("urler", nargs="+", help="En eller flere URL-er")
        parser.add_argument("--timeout", type=int, default=5, help="Sekunder før
tidsavbrudd")
        args = parser.parse_args()

        print(args.urler, args.timeout)
```

```
$ python3 sjekk.py https://example.com https://noroff.no --timeout 3
$ python3 sjekk.py --help       # gratis hjelpetekst!
```

Kjøre andre programmer fra Python

```python
import subprocess

resultat = subprocess.run(
    ["df", "-h", "/"],          # kommando som liste – trygt mot injeksjon
    capture_output=True,        # fang utskriften
    text=True,                  # som tekst, ikke bytes
    check=True,                 # kast unntak hvis kommandoen feiler
)
print(resultat.stdout)
```

> **Vanlig feil:** `subprocess.run(f"rm -rf {mappe}", shell=True)` med data fra brukeren. Det åpner for *command injection*. Bruk liste-formen og unngå `shell=True`.

Bash eller Python?

Bruk Bash når ...	Bruk Python når ...
Du limer sammen Linux-kommandoer	Du har logikk, datastrukturer, mange betingelser
Skriptet er kort (under ~50 linjer)	Du jobber med JSON/API-er
Du installerer pakker, kopierer filer	Du trenger god feilhåndtering og tester
Det skal kjøre i en minimal container	Skriptet skal vedlikeholdes over tid

Bash ble grundig gjennomgått i kapittel 3. Her er et lite Bash-skript som bruker et API, for sammenligning:

```bash
#!/usr/bin/env bash
set -euo pipefail
# helsesjekk.sh – sjekker at URL-er svarer 200
for url in "$@"; do
  kode=$(curl -s -o /dev/null -w "%{http_code}" --max-time 5 "$url" || echo "000")
  if [[ "$kode" == "200" ]]; then
    echo "OK   $url"
  else
    echo "FEIL $url ($kode)" >&2
  fi
done
```

`-s` stille, `-o /dev/null` kast innholdet, `-w "%{http_code}"` skriv bare statuskoden.

Python for Cloud: Azure SDK

Microsoft har offisielle Python-biblioteker for Azure. Autentisering skjer typisk med `DefaultAzureCredential`, som automatisk prøver flere metoder: miljøvariabler, managed identity (i Azure), og din Azure CLI-innlogging (lokalt).

```
$ pip install azure-identity azure-mgmt-resource
$ az login        # logg inn lokalt, så kan DefaultAzureCredential bruke CLI-innloggingen
```

```
import os
from azure.identity import DefaultAzureCredential
from azure.mgmt.resource import ResourceManagementClient

subscription_id = os.environ["AZURE_SUBSCRIPTION_ID"]   # sett med: export AZURE_SUBSCRIPTION_ID=...
credential = DefaultAzureCredential()                   # finner innlogging automatisk
client = ResourceManagementClient(credential, subscription_id)

# List alle ressursgrupper med region og tags
for rg in client.resource_groups.list():
    print(f"{rg.name:30} {rg.location:15} {rg.tags}")
```

> **Fra arbeidslivet:** Et klassisk skript er en «kostnadsjeger» som finner ressursgrupper uten `eier`-tag eller med `utløper`-dato i fortiden, og varsler eller sletter dem. Det bygger du i prosjekt 2.

Prosjektskript: helsesjekk med rapport

Her er et komplett, profesjonelt lite verktøy som samler mye av kapitlet.

```
        #!/usr/bin/env python3
        """uptime_check.py – sjekker en liste nettsider og skriver en JSON-rapport.

        Bruk:
            python3 uptime_check.py sites.yaml --out rapport.json
        """
        import argparse
        import json
        import logging
        import sys
        import time
        from datetime import datetime, timezone

        import requests
        import yaml

        log = logging.getLogger("uptime")


        def sjekk(url: str, timeout: int) -> dict:
            """Sjekk én URL og returner resultatet som en ordbok."""
            start = time.perf_counter()                      # start stoppeklokke
            try:
                svar = requests.get(url, timeout=timeout)
                ms = round((time.perf_counter() - start) * 1000)
                ok = svar.status_code < 400
                return {"url": url, "status": svar.status_code, "ms": ms, "ok": ok}
            except requests.exceptions.RequestException as e:
                return {"url": url, "status": None, "ms": None, "ok": False, "feil":
type(e).__name__}


        def main() -> int:
            parser = argparse.ArgumentParser()
            parser.add_argument("config", help="YAML-fil med liste over nettsider")
            parser.add_argument("--out", default="rapport.json")
            parser.add_argument("--timeout", type=int, default=5)
            args = parser.parse_args()

            logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %
(message)s")

            with open(args.config, encoding="utf-8") as f:
                urler = yaml.safe_load(f)["sites"]           # forventer: sites: [url1,
url2, ...]

            resultater = []
            for url in urler:
                r = sjekk(url, args.timeout)
                nivå = logging.INFO if r["ok"] else logging.ERROR
                log.log(nivå, "%s → %s (%s ms)", url, r["status"], r["ms"])
                resultater.append(r)

            rapport = {
                "tidspunkt": datetime.now(timezone.utc).isoformat(),
                "antall_ok": sum(r["ok"] for r in resultater),   # True teller som 1
                "antall_feil": sum(not r["ok"] for r in resultater),
                "resultater": resultater,
            }
```

```
            with open(args.out, "w", encoding="utf-8") as f:
                json.dump(rapport, f, indent=2, ensure_ascii=False)

            return 0 if rapport["antall_feil"] == 0 else 1      # returkode 1 hvis noe
feilet


        if __name__ == "__main__":       # kjør main() bare når filen startes direkte
            sys.exit(main())
```

`sites.yaml`:

```
        sites:
          - https://www.example.com
          - https://www.noroff.no
          - https://finnes-ikke.example.invalid
```

Legg merke til profesjonelle detaljer: argumenter i stedet for hardkoding, timeout, feilhåndtering, logging, strukturert JSON-utdata og **returkode** som en CI-pipeline eller cron kan reagere på.

Testing – en smakebit

Kode du ikke tester, vet du ikke om virker. `pytest` er standardverktøyet:

```
        # test_uptime.py
        from uptime_check import sjekk

        def test_ugyldig_url_gir_ok_false():
            r = sjekk("https://finnes-ikke.example.invalid", timeout=2)
            assert r["ok"] is False
```

```
        $ pip install pytest
        $ pytest -q
        .                                                    [100%]
        1 passed in 0.41s
```

I kapittel 12 kjører vi slike tester automatisk i en CI-pipeline ved hver endring.

Kodekvalitet

- **Formatering:** `ruff format` eller `black` – alle får lik kodestil.
- **Linting:** `ruff check` (Python), `shellcheck` (Bash), `yamllint` (YAML) – finner feil og dårlig praksis.
- **Lesbarhet:** beskrivende navn, korte funksjoner, kommentarer som forklarer *hvorfor*, ikke *hva*.

Kapitteloppsummering

- Programmering er presise instruksjoner. Python og Bash er de viktigste språkene for en DevOps-nybegynner.
- Python-grunnlag: variabler, datatyper, `if` / `elif` / `else`, `for` / `while`, funksjoner, lister, ordbøker.
- JSON (API-er) og YAML (konfigurasjon) blir lister og ordbøker i Python. YAML er følsomt for innrykk.
- Bruk `requests` med timeout for API-kall, `try` / `except` for feil, `logging` for logging, `argparse` for argumenter.
- Hemmeligheter skal aldri stå i koden.
- Returkoder lar skript samarbeide med cron og CI/CD.
- Azure SDK med `DefaultAzureCredential` lar Python styre Azure.

Øvingsoppgaver

1. Skriv en funksjon `er_privat_ip(ip: str) -> bool` som sjekker om en IPv4-adresse er i et privat område. (Tips: modulen `ipaddress` har `ipaddress.ip_address(ip).is_private`.)
2. Les en fil `servere.txt` med ett servernavn per linje, og skriv ut navnene sortert alfabetisk uten duplikater.
3. Hent `https://api.github.com/users/<ditt-brukernavn>/repos` og skriv ut navnet og antall stjerner for hvert repo.
4. Konverter en JSON-fil til YAML med Python.
5. Utvid `uptime_check.py` slik at den også skriver en advarsel hvis responstiden er over 1000 ms.
6. Skriv en pytest-test for oppgave 1.

Quiz

1. Hva er forskjellen på en liste og en ordbok?
2. Hva skriver `print(["a", "b", "c"][1])` ut?
3. Hvorfor gir `"5" + 5` en feil i Python?
4. Hva er galt med denne YAML-en?

```
        tags:
            miljø: prod
```

5. Hvorfor bør du alltid bruke `timeout` i `requests.get`?
6. Hva er forskjellen på `json.load` og `json.loads`?

7. Hvorfor er `except: pass` en dårlig idé?
8. Hva gjør `if __name__ == "__main__":`?
9. Hvorfor returnerer skriptet `1` når en nettside feiler?
10. Hvor skal et API-token ligge når skriptet kjører?

Fasit

1. En **liste** er en ordnet samling elementer hentet med indeks (0, 1, 2 …). En **ordbok** lagrer par av nøkkel → verdi og hentes med nøkkelen.
2. `b` – indekser starter på 0.
3. Fordi `"5"` er tekst og `5` er et tall, og Python blander ikke typer automatisk. Bruk `int("5") + 5`.
4. Den bruker **tab** til innrykk. YAML tillater bare mellomrom.
5. Uten timeout kan skriptet vente for alltid hvis serveren aldri svarer, og f.eks. blokkere en pipeline eller cron-jobb.
6. `json.load` leser fra en **fil**; `json.loads` leser fra en **streng** (s = string).
7. Den skjuler alle feil, også alvorlige og uventede, slik at du ikke får vite at noe er galt.
8. Koden under kjøres bare når filen startes direkte, ikke når den importeres som modul (f.eks. fra en test).
9. Slik at det som kaller skriptet (cron, CI/CD, et annet skript) kan oppdage at noe gikk galt. 0 = suksess, alt annet = feil.
10. I en miljøvariabel eller et hemmelighetshvelv (f.eks. Azure Key Vault, GitHub Secrets) – aldri i koden eller i Git.

Del 5 – Git, GitHub og versjonskontroll

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva versjonskontroll er og hvorfor alle profesjonelle team bruker det,
- bruke Git til å lage commits, branches og merges,
- samarbeide via GitHub med clone, push, pull og pull requests,
- løse merge-konflikter,
- velge en Git-arbeidsflyt og skrive gode commit-meldinger,
- lage en enkel GitHub Actions-workflow.

Hva er versjonskontroll?

Hva er det? Versjonskontroll er et system som **husker alle endringer** i filer over tid: hvem endret hva, når og hvorfor. Du kan gå tilbake til enhver tidligere versjon.

Hvorfor finnes det? Uten versjonskontroll ender man med mapper som dette:

```
rapport.docx
rapport_v2.docx
rapport_v2_FINAL.docx
rapport_v2_FINAL_ekte_final.docx
rapport_v2_FINAL_ekte_final_Karis_endringer.docx
```

Tenk deg det med 500 kodefiler og 30 utviklere. Versjonskontroll løser:

- **Historikk** – hva endret seg, og hvorfor?
- **Samarbeid** – mange kan jobbe samtidig uten å overskrive hverandre.
- **Angring** – noe gikk galt i produksjon? Gå tilbake til forrige fungerende versjon.
- **Sporbarhet** – hvilken endring introduserte feilen?

> **Hverdagsanalogi:** Git er som lagringspunkter i et dataspill. Før du går inn i et farlig område, lagrer du. Går det galt, laster du det siste lagringspunktet. Branches er som å lage en kopi av spillet for å teste en annen vei, uten å ødelegge hovedspillet.

Git og GitHub – ikke det samme

	Git	GitHub
Hva	Et **verktøy** for versjonskontroll	En **nettjeneste** som lagrer Git-repoer

	Git	GitHub
Hvor	Kjører lokalt på din maskin	I skyen (github.com)
Laget av	Linus Torvalds (2005)	GitHub Inc. (eid av Microsoft)
Tilbyr	Commits, branches, historikk	Deling, pull requests, issues, Actions (CI/CD), sikkerhetsskanning
Alternativer	(Git er dominerende)	GitLab, Bitbucket, Azure Repos

Git er **distribuert**: hver utvikler har en full kopi av hele historikken. GitHub er bare et felles møtepunkt.

Installasjon og oppsett

```
        $ git --version                    # er Git installert?
        $ sudo apt install git             # Ubuntu; Windows: git-scm.com; macOS: xcode-
select --install

        # Fortell Git hvem du er – dette står i hver commit du lager
        $ git config --global user.name "Kari Nordmann"
        $ git config --global user.email "kari@example.com"
        $ git config --global init.defaultBranch main     # kall hovedbranchen "main"
        $ git config --list                                # se innstillingene
```

Autentisering mot GitHub. GitHub tar ikke imot vanlig passord for Git-operasjoner. Bruk enten SSH-nøkkel (legg `~/.ssh/id_ed25519.pub` inn under *Settings → SSH and GPG keys* på GitHub), eller **GitHub CLI** (`gh auth login`), eller et *personal access token* over HTTPS.

Kjernebegrepene

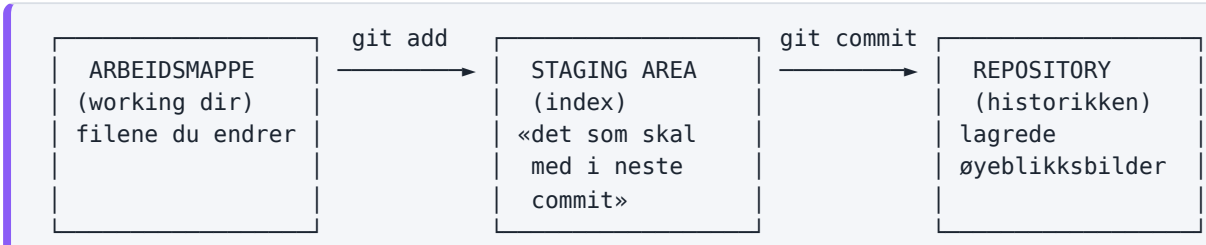
Repository (repo)

Hva er det? Et repository er et prosjekt under versjonskontroll: alle filene pluss en skjult mappe `.git` som inneholder hele historikken.

```
        $ mkdir mitt-prosjekt && cd mitt-prosjekt
        $ git init                 # gjør mappen til et Git-repo
        Initialized empty Git repository in /home/student/mitt-prosjekt/.git/
```

De tre områdene

Dette er det viktigste å forstå i Git:



Hverdagsanalogi: Du pakker en flyttekasse. Arbeidsmappen er rommet ditt. Staging er kassen – du velger hva som skal oppi. Commit er å teipe igjen kassen og skrive en etikett på den. Kassen (commiten) endres ikke etterpå.

Commit

Hva er det? Et **øyeblikksbilde** (*snapshot*) av prosjektet, med en melding, forfatter, tidspunkt og en unik ID (en *hash*, som `a1b2c3d`).

```
$ echo "# Mitt prosjekt" > README.md
$ git status                          # hva er endret?
Untracked files:
        README.md

$ git add README.md                   # legg i staging
$ git commit -m "Legg til README"     # lag commit med melding
[main (root-commit) 3f2a1b9] Legg til README
 1 file changed, 1 insertion(+)

$ git log --oneline                   # vis historikken kort
3f2a1b9 Legg til README
```

Nyttige varianter:

```
$ git add .                 # legg til ALLE endringer i mappen
$ git add -p                # velg bit for bit hva som skal med (anbefalt!)
$ git diff                  # hva har jeg endret (ikke staget)?
$ git diff --staged         # hva ligger i staging?
$ git log --oneline --graph --all   # historikk som tre
$ git show 3f2a1b9          # se hva en commit endret
```

Gode commit-meldinger

En god commit-melding forklarer **hva** og **hvorfor**, i imperativ form («Legg til», ikke «La til»):

```
DÅRLIG                      BRA
"fix"                       "Fiks nullpekerfeil i innloggingen ved tomt passord"
"endringer"                 "Øk timeout mot betalings-API fra 5 til 15 sekunder"
"asdf"                      "Legg til helsesjekk-endepunkt /health"
```

Mange team bruker **Conventional Commits**: `feat: legg til eksport til CSV`, `fix: rett beregning av mva`, `docs: oppdater README`, `ci: kjør tester på pull requests`. Det gjør historikken lettere å lese og kan brukes til automatisk versjonering.

> **Husk:** Små, hyppige commits som gjør *én* ting er bedre enn én kjempecommit med alt. Da er det lett å forstå, gjennomgå og angre.

.gitignore

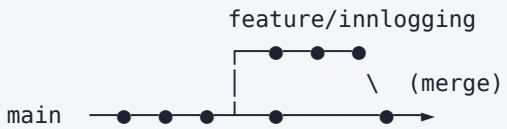
En fil som forteller Git hva som **ikke** skal versjonskontrolleres:

```
# .gitignore
.venv/
__pycache__/
*.log
.env              # lokale hemmeligheter!
node_modules/
.terraform/
*.tfstate         # Terraform-state kan inneholde hemmeligheter
.DS_Store
```

> **Vanlig feil:** Å committe `.env` med passord. Selv om du sletter filen i neste commit, ligger den i **historikken**. Hvis en hemmelighet har havnet i Git: **roter (bytt) hemmeligheten umiddelbart**. Å skrive om historikken er ikke nok når den allerede er pushet.

Branch

Hva er det? En branch (gren) er en **parallell utviklingslinje**. Du kan jobbe med en ny funksjon uten å påvirke hovedbranchen (`main`).



Teknisk er en branch bare en lett «peker» til en commit – derfor er branches billige og raske i Git.

```
$ git branch                          # list branches (* = den du er på)
$ git switch -c feature/innlogging    # lag ny branch og bytt til den
$ git switch main                     # bytt tilbake til main
$ git branch -d feature/innlogging    # slett branch (etter merge)
```

(`git checkout -b` gjør det samme som `git switch -c` – du vil se begge i guider.)

Merge

Hva er det? Å slå sammen endringene fra én branch inn i en annen.

```
$ git switch main
$ git merge feature/innlogging
```

To typer:

- **Fast-forward**: `main` har ikke endret seg siden branchen ble laget. Git flytter bare pekeren frem.
- **Merge commit (tre-veis merge)**: begge har endret seg. Git lager en ny commit som kombinerer dem.

Rebase er et alternativ: flytter dine commits slik at de ligger «oppå» siste versjon av main, som gir en rett historikk.

```
$ git switch feature/innlogging
$ git rebase main
```

> **Vanlig feil:** Rebase skriver om historikk. **Rebase aldri commits som andre allerede har hentet** (f.eks. `main`). Bruk rebase på din egen, lokale branch.

Merge-konflikter

Hva er det? Når to branches har endret **samme linjer** i samme fil, kan ikke Git vite hvilken versjon som er riktig. Du må bestemme.

```
$ git merge feature/farge
Auto-merging style.css
CONFLICT (content): Merge conflict in style.css
Automatic merge failed; fix conflicts and then commit the result.
```

Filen ser nå slik ut:

```
body {
<<<<<<< HEAD
  background: white;          ← slik det er på main (der du står)
=======
  background: lightblue;      ← slik det er på branchen du merger inn
>>>>>>> feature/farge
}
```

Løs konflikten:

1. Åpne filen. Bestem hva som er riktig (én av dem, begge, eller noe nytt).
2. Fjern markørene `<<<<<<<`, `=======` og `>>>>>>>`.
3. `git add style.css`

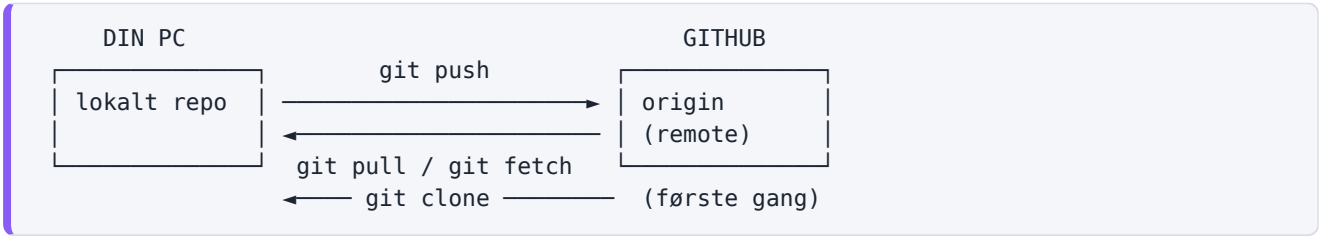
4. `git commit` (Git foreslår en melding).

Angre hele mergen: `git merge --abort`.

> **Husk:** Konflikter er normalt og ikke farlig. De blir færre og mindre hvis du merger små endringer ofte. VS Code har et godt verktøy for å løse konflikter visuelt.

Jobbe med GitHub: remote, clone, push og pull

Remote er en ekstern kopi av repoet, typisk på GitHub. Standardnavnet er `origin`.




```
# Hente et eksisterende repo første gang
$ git clone git@github.com:brukernavn/repo.git
$ cd repo

# Koble et lokalt repo til et nytt, tomt GitHub-repo
$ git remote add origin git@github.com:brukernavn/mitt-prosjekt.git
$ git push -u origin main         # -u: husk at lokal main følger origin/main

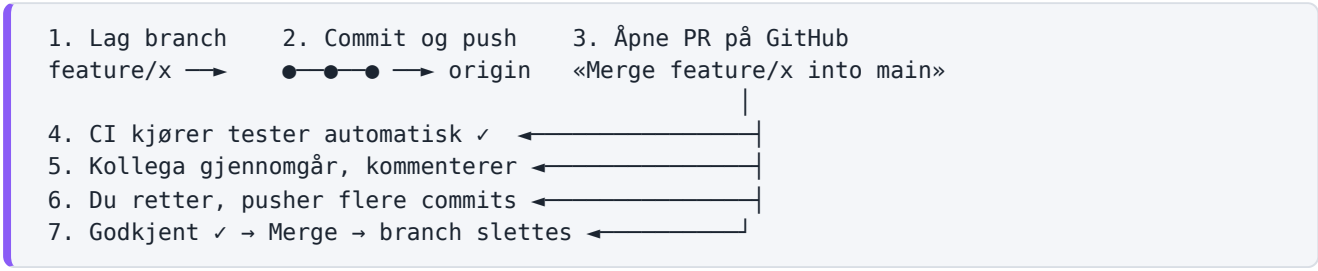
$ git push                        # send dine commits til GitHub
$ git fetch                       # hent nyheter, men ikke flett dem inn
$ git pull                        # fetch + merge (hent og flett inn)
$ git remote -v                   # vis remotes
```

> **Vanlig feil:** `git push` blir avvist med «Updates were rejected because the remote contains work that you do not have locally». Noen har pushet før deg. Kjør `git pull` (løs eventuelle konflikter), og push igjen. **Ikke** bruk `git push --force` på delte brancher – da sletter du kollegaenes arbeid.

Pull requests

Hva er det? En pull request (PR) er en **forespørsel om å merge** en branch inn i en annen (vanligvis `main`), med mulighet for **kodegjennomgang** (*code review*), diskusjon og automatiske tester før endringen slippes inn. (GitLab kaller det *merge request*.)

Hvorfor? Fire øyne ser mer enn to. PR-er fanger feil, sprer kunnskap i teamet, og sikrer at tester kjører før noe havner i `main`.



En god PR:

- er liten og handler om én ting,
- har en beskrivelse: hva, hvorfor, hvordan testet,
- refererer til en issue (`Closes #42` lukker issue 42 automatisk ved merge),
- har grønne tester.

Merge-metoder på GitHub:

- *Create a merge commit* – beholder alle commits pluss en merge commit.
- *Squash and merge* – slår alle commits i PR-en sammen til én. Gir ren historikk; svært vanlig.
- *Rebase and merge* – legger commitene rett på main uten merge commit.

Branch protection / rulesets. I profesjonelle repoer beskyttes `main`: ingen kan pushe direkte, PR krever godkjenning og grønne tester. Dette settes opp under *Settings → Rules* (rulesets) eller *Branches* i GitHub.

Angre ting i Git

Situasjon	Kommando
Forkast endringer i en fil (ikke staget)	`git restore fil.txt`
Ta en fil ut av staging	`git restore --staged fil.txt`
Endre siste commit-melding (ikke pushet)	`git commit --amend`
Angre en commit som er pushet (trygt)	`git revert <hash>` – lager en ny commit som gjør det motsatte
Flytt branchen tilbake (lokalt, farlig)	`git reset --hard <hash>` – forkaster alt etter
Legg arbeid til side midlertidig	`git stash` … `git stash pop`
Finn tapte commits	`git reflog`

> **Husk:** På delte brancher: bruk **`revert`**, ikke `reset`. `revert` legger til historikk; `reset` skriver den om.

Git-arbeidsflyter (*workflows*)

GitHub Flow (anbefalt for de fleste)



1. `main` er alltid i en tilstand som kan deployes.
2. Lag en kortlevd branch for hver endring.
3. Åpne PR, få review og grønne tester.
4. Merge til `main` → deploy automatisk.

Trunk-based development

Alle integrerer til `main` (*trunken*) svært ofte – minst daglig – med svært kortlevde brancher eller direkte. Uferdige funksjoner skjules med **feature flags**. Brukes av mange høytytende team og henger tett sammen med Continuous Integration.

Git Flow (eldre, mer kompleks)

Egne brancher for `develop`, `feature/*`, `release/*` og `hotfix/*`. Passer for produkter med planlagte, versjonerte utgivelser (f.eks. programvare som installeres hos kunder), men er ofte for tungt for webtjenester med kontinuerlig deploy.

> **Fra arbeidslivet:** De fleste moderne skyteam bruker GitHub Flow eller trunk-based. Hvis du får spørsmålet i intervju, vis at du kjenner forskjellen og hvorfor korte brancher reduserer konflikter og risiko.

Navngivning av brancher

`feature/eksport-csv`, `fix/innlogging-timeout`, `docs/readme`, `chore/oppgrader-python`. Gjerne med issue-nummer: `feature/123-eksport-csv`.

Tags og versjoner

```
$ git tag -a v1.2.0 -m "Versjon 1.2.0"
$ git push origin v1.2.0
```

Semantisk versjonering (*SemVer*): `MAJOR.MINOR.PATCH`. Øk MAJOR ved endringer som brekker kompatibilitet, MINOR ved ny funksjonalitet, PATCH ved feilrettinger.

Samarbeid på GitHub

- **Issues** – oppgaver, feil og ideer. Kan tildeles, merkes og kobles til PR-er.
- **Projects** – kanban-tavler for planlegging.
- **README.md** – forsiden til repoet. Første inntrykk for arbeidsgivere!
- **CODEOWNERS** – bestemmer hvem som automatisk blir bedt om review for hvilke filer.
- **Forks** – din egen kopi av noen andres repo. Brukes for å bidra til open source.
- **Dependabot** – foreslår automatisk oppdateringer av avhengigheter og varsler om sårbarheter.
- **Secret scanning og code scanning** – finner lekkede hemmeligheter og sårbarheter.

GitHub Actions – en første smakebit

Hva er det? GitHub Actions er GitHubs innebygde CI/CD-system. Du beskriver automatiske jobber i YAML-filer i mappen `.github/workflows/`. De kjøres på hendelser som `push` og `pull_request`.

```
hendelse (push/PR) ──► workflow ──► job(s) ──► steps
                                      │
                                kjører på en «runner»
                                (en VM hos GitHub eller din egen)
```

`.github/workflows/ci.yml`:

```yaml
name: CI                                    # navn som vises på GitHub

on:                                         # når skal workflowen kjøre?
  push:
    branches: [main]
  pull_request:                             # på alle pull requests

jobs:
  test:                                     # navnet på jobben
    runs-on: ubuntu-latest                  # GitHub-hostet Linux-maskin
    steps:
      - uses: actions/checkout@v4           # hent koden (en ferdig "action")
      - uses: actions/setup-python@v5       # installer Python
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt   # kjør en kommando
      - run: pytest -q                          # feiler testene, blir jobben rød
```

Når du pusher, ser du resultatet under fanen **Actions** i repoet, og som en grønn ✓ eller rød × på PR-en.

> **Kostnad:** GitHub Actions er gratis for offentlige repoer på GitHub-hostede standard-runnere. For private repoer følger det med et visst antall gratis minutter per måned avhengig av plan; over det betaler du. Sjekk «About billing for GitHub Actions» i GitHub Docs for gjeldende kvoter.

> **Merk om versjoner:** `@v4` / `@v5` viser til hovedversjoner av offisielle actions. Nye hovedversjoner kommer jevnlig (f.eks. har `actions/checkout` fått v5). Sjekk siden til hver action på GitHub Marketplace og bruk en aktivt vedlikeholdt versjon.

CI/CD med GitHub Actions blir gjennomgått i dybden i kapittel 12.

Praktisk prosjekt: fra null til samarbeid

Dette prosjektet tar deg gjennom en komplett, realistisk arbeidsflyt. Tid: ca. 2 timer. Kostnad: gratis.

Steg 1 – lag repoet

1. Logg inn på github.com → **New repository** → navn `devops-notater`, *Public*, huk av for **Add a README**, velg `.gitignore`-malen **Python** → **Create repository**.
2. Klon det:

```
$ git clone git@github.com:<brukernavn>/devops-notater.git
$ cd devops-notater
```

Steg 2 – første funksjon på en branch

```
$ git switch -c feature/kommandoliste
$ mkdir -p notater
$ cat > notater/linux.md <<'EOF'
# Linux-kommandoer jeg kan
- `pwd` – vis hvor jeg er
- `ls -la` – list alle filer med detaljer
EOF
$ git add notater/linux.md
$ git commit -m "docs: legg til liste over Linux-kommandoer"
$ git push -u origin feature/kommandoliste
```

Steg 3 – pull request

1. GitHub viser en gul boks: **Compare & pull request**. Klikk.
2. Skriv en beskrivelse: *Hva:* liste over Linux-kommandoer. *Hvorfor:* repetisjon til eksamen.
3. **Create pull request** → **Squash and merge** → **Delete branch**.

```
$ git switch main
$ git pull                        # hent den mergede endringen
$ git branch -d feature/kommandoliste
```

Steg 4 – fremprovoser og løs en konflikt

```
# Branch A
$ git switch -c fix/tittel-a
$ sed -i 's/# Linux-kommandoer jeg kan/# Mine Linux-kommandoer/' notater/linux.md
$ git commit -am "docs: ny tittel (A)"
$ git switch main

# Branch B endrer samme linje
$ git switch -c fix/tittel-b
$ sed -i 's/# Linux-kommandoer jeg kan/# Linux – jukselapp/' notater/linux.md
$ git commit -am "docs: ny tittel (B)"

$ git switch main
$ git merge fix/tittel-a          # går fint (fast-forward)
$ git merge fix/tittel-b          # KONFLIKT
$ git status                      # viser filen med konflikt
```

Åpne `notater/linux.md`, velg tittelen du vil ha, fjern markørene, og:

```
$ git add notater/linux.md
$ git commit                      # godta foreslått melding
$ git push
```

(`git commit -am` = legg til alle *sporede* endrede filer og commit i ett.)

Steg 5 – legg til CI

Lag `.github/workflows/lint.yml` som sjekker Markdown-filene:

```yaml
name: Lint
on: [push, pull_request]
jobs:
  markdown:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: DavidAnson/markdownlint-cli2-action@v20
        with:
          globs: "**/*.md"
```

Push via en branch og PR. Se jobben kjøre under *Actions*. Hvis den feiler, les loggen, fiks, og push igjen.

> **Merk:** Tredjeparts actions (som `DavidAnson/markdownlint-cli2-action`) har egne versjonsnumre. Sjekk siste versjon på GitHub Marketplace. I sikkerhetsbevisste team låser man tredjeparts actions til en **full commit-hash** i stedet for en tag.

Steg 6 – beskytt main

Settings → *Rules* → *Rulesets* → *New branch ruleset*: mål `main`, krev pull request før merge, krev at statussjekken «Lint» er grønn. Prøv å pushe direkte til `main` – det blir nå avvist.

Steg 7 – presentasjon

Skriv en god README: hva repoet er, hvordan det er organisert, og et merke (*badge*) som viser CI-status:

```
        ![Lint](https://github.com/<brukernavn>/devops-
notater/actions/workflows/lint.yml/badge.svg)
```

På CV-en: «Bruker Git og GitHub daglig med feature-brancher, pull requests, kodegjennomgang, beskyttede brancher og GitHub Actions.»

Kapitteloppsummering

- Git er et lokalt versjonskontrollverktøy; GitHub er en nettjeneste for å dele og samarbeide om Git-repoer.
- Arbeidsmappe → `git add` → staging → `git commit` → historikk.
- Branches er billige parallelle linjer; merge slår dem sammen; konflikter løses manuelt.
- `clone`, `push`, `pull`, `fetch` synkroniserer med GitHub.
- Pull requests gir kodegjennomgang og automatiske tester før merge.
- Bruk `revert` på delte brancher; aldri `push --force` på `main`.
- GitHub Flow og trunk-based er vanligst i moderne team.
- GitHub Actions kjører automatiske workflows definert i YAML.

Øvingsoppgaver

1. Lag et repo lokalt, gjør fem commits, og vis historikken med `git log --oneline --graph`.
2. Gjør en endring, stage den, og angre den med `git restore --staged` og `git restore`.
3. Lag en commit med en «feil», push den, og angre den trygt med `git revert`.
4. Bruk `git stash` for å bytte branch midt i arbeidet.

5. Fork et åpent prosjekt på GitHub, gjør en liten forbedring i dokumentasjonen på en branch, og åpne en PR mot din egen fork.
6. Legg til en workflow som kjører `pytest` på Python-koden fra kapittel 4.

Quiz

1. Hva er forskjellen på Git og GitHub?
2. Hva er staging area?
3. Hva er en branch, teknisk sett?
4. Når oppstår en merge-konflikt?
5. Hva er forskjellen på `git fetch` og `git pull`?
6. Hvorfor bør du ikke bruke `git push --force` på `main`?
7. Du har committet et passord og pushet. Hva er det *første* du gjør?
8. Hva er forskjellen på `git revert` og `git reset --hard`?
9. Hva gjør `Closes #12` i en PR-beskrivelse?
10. Hvor ligger GitHub Actions-workflows i et repo?

Fasit

1. **Git** er verktøyet for versjonskontroll som kjører lokalt. **GitHub** er en nettjeneste som lagrer Git-repoer og legger til samarbeid (PR-er, issues, Actions).
2. Et mellomområde der du samler endringene som skal være med i neste commit.
3. En lett, flyttbar peker til en commit.
4. Når to brancher har endret de samme linjene i samme fil (eller én har slettet en fil den andre endret), slik at Git ikke kan velge automatisk.
5. `fetch` henter nye data fra remote uten å endre dine filer. `pull` er `fetch` + `merge` (eller rebase) inn i branchen din.
6. Det overskriver historikken på remote og kan slette andres commits.
7. **Bytt (roter) passordet** umiddelbart. Rydd deretter i historikken og sett opp tiltak (secret scanning, `.gitignore`, hemmelighetshvelv).
8. `revert` lager en ny commit som opphever en tidligere – trygt på delte brancher. `reset --hard` flytter branchen tilbake og forkaster commits og endringer – skriver om historikken.
9. Lukker issue nummer 12 automatisk når PR-en merges til standardbranchen.
10. I `.github/workflows/` som `.yml`-filer.

Del 6 – Cloud Computing fra A til Å

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva Cloud Computing er, og hvordan skyen fungerer fysisk,
- sammenligne sky og egne datasentre (*on-premises*),
- skille mellom public, private og hybrid cloud, og mellom IaaS, PaaS og SaaS,
- forklare regioner, tilgjengelighetssoner, skalering, høy tilgjengelighet, feiltoleranse og katastrofegjenoppretting,
- forstå prismodeller, kostnadsoptimalisering og modellen for delt ansvar,
- sammenligne AWS, Azure og Google Cloud.

Hva er Cloud Computing?

Hva er det? Cloud Computing (skytjenester) er å **leie IT-ressurser over internett** – servere, lagring, databaser, nettverk, programvare, AI – i stedet for å eie og drifte dem selv. Du betaler for det du bruker, og kan få mer eller mindre på minutter.

Hvorfor finnes det? Før skyen måtte en bedrift som trengte en ny server:

1. få budsjett godkjent,
2. bestille maskinvare (ukers leveringstid),
3. montere den i et serverrom med strøm, kjøling og nettverk,
4. installere og sikre operativsystemet,
5. drifte og bytte deler i årevis – også når den sto ubrukt.

Og de måtte kjøpe nok kapasitet for **toppen** (julehandelen), selv om den bare trengtes noen uker i året.

> **Hverdagsanalogi:** Strøm. For hundre år siden hadde fabrikker egne generatorer. I dag kobler du deg til strømnettet og betaler for kilowattimene du bruker. Skyen gjør det samme med data-kraft: du trenger ikke eie kraftverket.

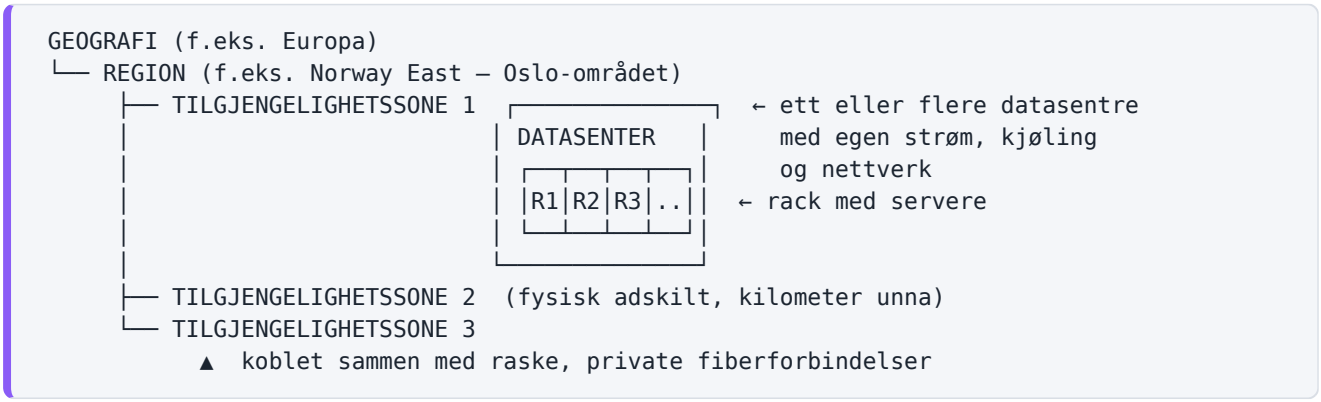
NIST-definisjonen – fem kjennetegn

Det amerikanske standardiseringsinstituttet NIST definerer skyen med fem egenskaper. De dukker ofte opp på eksamen:

Egenskap	Betyr
On-demand self-service	Du kan selv opprette ressurser når du vil, uten å ringe noen
Broad network access	Tilgjengelig over nettverk fra mange typer enheter
Resource pooling	Leverandøren deler store mengder maskinvare mellom mange kunder
Rapid elasticity	Kapasiteten kan økes og minskes raskt, ofte automatisk
Measured service	Bruken måles, og du betaler for det du bruker

Hvordan fungerer skyen fysisk?

Skyen er ikke magisk eller «i lufta». Den er **andres datamaskiner i enorme datasentre**.



Datasentre

Et datasenter er en bygning full av servere i **rack** (stativer), med:

- **Strøm**: flere uavhengige strømtilførsler, UPS-batterier og dieselaggregater.
- **Kjøling**: servere produserer enorme mengder varme. Nordiske datasentre utnytter kald luft og vann.
- **Nettverk**: flere fiberforbindelser til internett og mellom datasentrene.
- **Sikkerhet**: adgangskontroll, vakter, kameraer, ofte biometriske lås.
- **Automatisering**: programvare som overvåker maskinvaren og flytter arbeidslaster når noe feiler.

Når du oppretter en VM, finner skyens **kontrollplan** (*control plane*) en fysisk server med ledig kapasitet, og hypervisoren starter VM-en din der (kapittel 1). Du ser bare resultatet: en IP-adresse og en innlogging.

> **Fra arbeidslivet:** Microsoft har regioner i Norge: **Norway East** (Oslo-området) og **Norway West** (Stavanger-området, begrenset tilgang, brukt bl.a. for katastrofegjenoppretting). Norske virksomheter velger ofte Norway East for lav forsinkelse og for å holde data i Norge. Sjekk alltid hvilke tjenester som finnes i hvilken region – ikke alle tjenester er tilgjengelige overalt.

Sky vs. on-premises

On-premises («on-prem») betyr at bedriften eier og drifter maskinvaren selv, i eget serverrom eller i et leid datasenter (*colocation*).

	On-premises	Sky
Oppstartskostnad	Høy (kjøpe maskinvare)	Lav (ingen investering)
Kostnadstype	**CapEx** – kapitalutgift, investering	**OpEx** – driftsutgift, løpende
Tid til ny server	Uker–måneder	Minutter
Skalering	Må kjøpe for toppen	Skaler opp og ned etter behov
Vedlikehold av maskinvare	Deg	Leverandøren
Kontroll	Full	Mindre over det underliggende
Global rekkevidde	Vanskelig	Regioner over hele verden
Kompetansebehov	Maskinvare, strøm, kjøling	Sky-arkitektur, sikkerhet, kostnadsstyring

> **Vanlig feil:** «Skyen er alltid billigere.» Ikke nødvendigvis. For stabile, forutsigbare arbeidslaster kan egen maskinvare være billigere over tid. Skyen vinner på **fleksibilitet, hastighet og tjenester**, men koster mye hvis man ikke styrer kostnadene. Noen virksomheter flytter til og med arbeidslaster tilbake (*repatriation*).

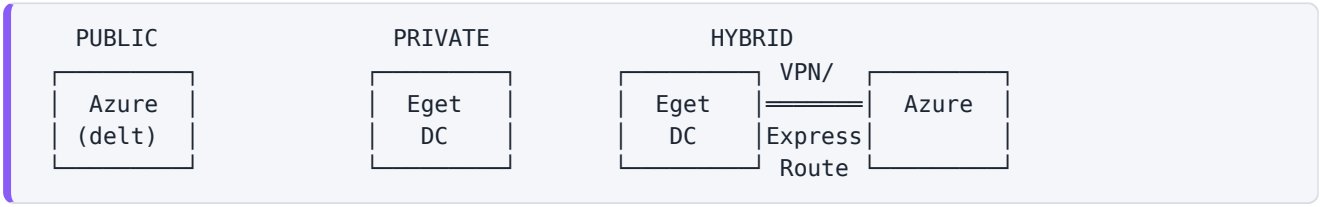
Deployeringsmodeller: public, private, hybrid

Public cloud – ressursene eies og driftes av en leverandør (Microsoft, Amazon, Google) og deles mellom mange kunder (logisk isolert). Det er dette de fleste mener med «skyen».

Private cloud – skyteknologi (selvbetjening, automatisering, virtualisering) brukt av **én** organisasjon, ofte i eget datasenter. Gir kontroll, men du må drifte alt.

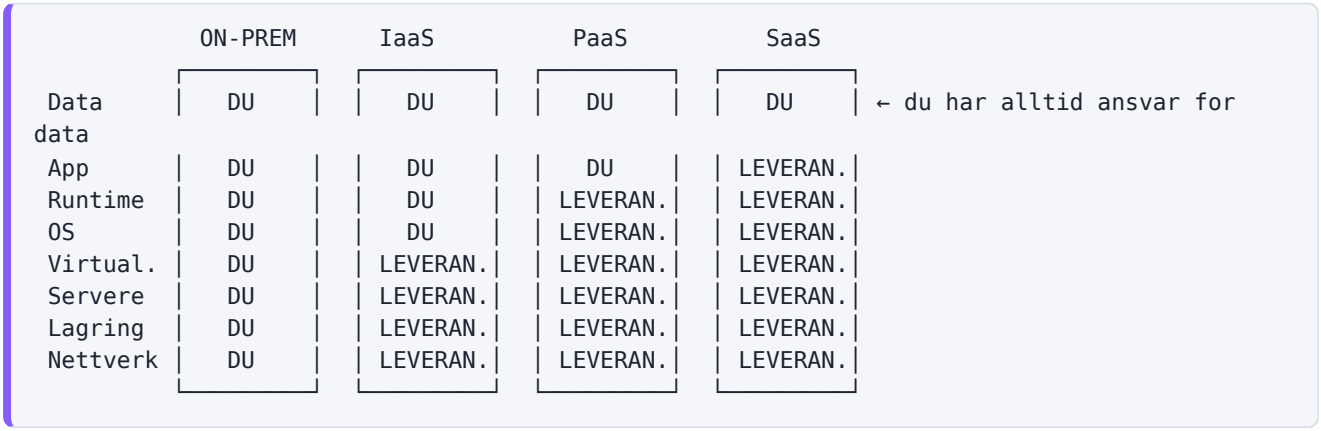
Hybrid cloud – en kombinasjon. For eksempel: kundedata må ligge i eget datasenter av regulatoriske grunner, mens nettsiden og analysene kjører i Azure. **Azure Arc** lar deg styre servere og Kubernetes-klynger utenfor Azure fra Azure-portalen.

Multi-cloud – å bruke flere offentlige skyer (f.eks. Azure + AWS) samtidig. Gir fleksibilitet, men mer kompleksitet.



Tjenestemodeller: IaaS, PaaS, SaaS

Dette er et av de viktigste begrepene i hele sky-feltet. Spørsmålet er: **hvor mye drifter du selv, og hvor mye drifter leverandøren?**



IaaS – Infrastructure as a Service

Du leier **infrastruktur**: virtuelle maskiner, disker, nettverk. Du installerer og drifter operativsystem og programvare selv.

- **Azure-eksempler:** Virtual Machines, Virtual Network, Managed Disks.
- **Fordel:** Maksimal kontroll og fleksibilitet. Lett å flytte eksisterende servere («lift and shift»).
- **Ulempe:** Du må patche OS, sikre, ta backup og skalere selv.

PaaS – Platform as a Service

Du leier en **plattform** der du bare laster opp koden eller dataene. Leverandøren drifter OS, runtime, patching og ofte skalering.

- **Azure-eksempler:** App Service, Azure SQL Database, Azure Functions, Azure Container Apps.
- **Fordel:** Mye mindre drift. Raskere utvikling. Innebygd skalering og høy tilgjengelighet.
- **Ulempe:** Mindre kontroll; du må tilpasse deg plattformens begrensninger.

SaaS – Software as a Service

Du bruker **ferdig programvare** over nettet. Du konfigurerer og bruker; alt annet er leverandørens ansvar.

- **Eksempler:** Microsoft 365, Gmail, Salesforce, Slack, GitHub.

> **Hverdagsanalogi – pizza som tjeneste: On-prem:** du lager pizza hjemme fra bunnen – kjøper ovn, ingredienser, lager deig. **IaaS:** du leier et kjøkken med ovn; du tar med ingredienser og lager pizzaen selv. **PaaS:** du bestiller pizza levert på døren; du dekker bordet og server. **SaaS:** du går på pizzarestaurant; du bare spiser.

Serverless og **containere** (Container Apps, AKS) ligger et sted mellom IaaS og PaaS; vi kommer tilbake til dem.

> **Husk:** Jo lenger mot SaaS, jo mindre drift for deg – men også mindre kontroll. Velg det høyeste abstraksjonsnivået som dekker behovet. En DevOps Engineer bør spørre: «Hvorfor trenger vi en VM her, når App Service kunne gjort jobben?»

Regioner og tilgjengelighetssoner

- **Region** – et geografisk område med ett eller flere datasentre (f.eks. *Norway East*, *West Europe*, *Sweden Central*).
- **Tilgjengelighetssone** (*Availability Zone, AZ*) – fysisk adskilte lokasjoner **innenfor en region**, hver med egen strøm, kjøling og nettverk. Som regel tre per region som støtter soner.
- **Regionpar** (*region pairs*) – noen Azure-regioner er paret med en annen region i samme geografi for katastrofegjenoppretting og koordinerte oppdateringer. (Nyere regioner har ikke alltid et par.)

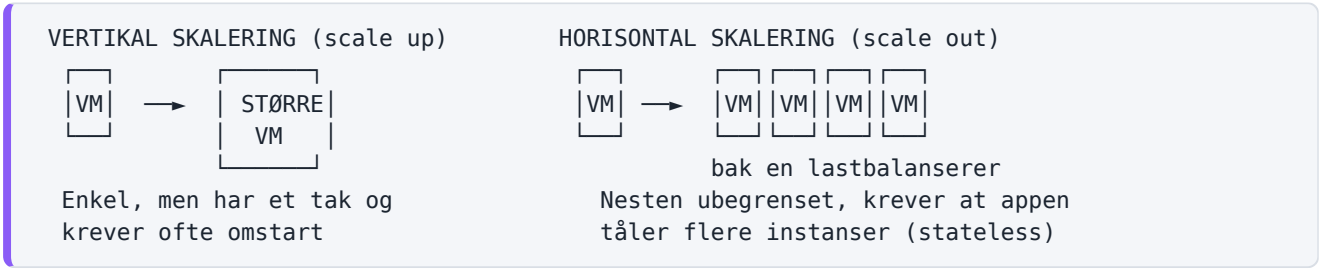
Hvordan velge region?

1. **Nærhet til brukerne** – lav forsinkelse (*latency*).
2. **Lovkrav og datasuverenitet** – f.eks. GDPR, krav om at data blir i Norge/EU.
3. **Tilgjengelige tjenester** – ikke alle tjenester og VM-størrelser finnes i alle regioner.
4. **Pris** – prisene varierer mellom regioner.
5. **Støtte for tilgjengelighetssoner.**

Skalering, elastisitet, høy tilgjengelighet og feiltoleranse

Skalerbarhet (*scalability*)

Evnen til å håndtere mer last ved å legge til ressurser.



- **Vertikal** (*scale up/down*): større eller mindre maskin.
- **Horisontal** (*scale out/in*): flere eller færre maskiner.

Elastisitet (*elasticity*)

Evnen til å skalere **automatisk** opp og ned i takt med etterspørselen. Skalerbarhet er *at* du kan skalere; elastisitet er at det skjer *automatisk og dynamisk*.

> **Fra arbeidslivet:** En nettbutikk kjører 2 instanser til vanlig, autoskalerer til 20 på Black Friday, og tilbake til 2 natten etter. De betaler bare for toppen mens den varer.

Høy tilgjengelighet (*high availability, HA*)

Systemet er tilgjengelig **så mye av tiden som mulig**, målt i prosent oppetid:

Tilgjengelighet	Maks nedetid per år	Per måned (ca.)
99 %	3,65 dager	7,2 timer
99,9 % («tre niere»)	8,76 timer	43,8 minutter
99,95 %	4,38 timer	21,9 minutter
99,99 % («fire niere»)	52,6 minutter	4,4 minutter
99,999 %	5,26 minutter	26 sekunder

En **SLA** (*Service Level Agreement*) er leverandørens løfte om tilgjengelighet, med kompensasjon (kreditt) hvis de bryter det. For eksempel gir Azure høyere SLA for VM-er spredt over flere tilgjengelighetssoner enn for én enkelt VM. Sjekk gjeldende SLA-er i Microsofts SLA-dokument («Service Level Agreements (SLA) for Online Services»), siden de oppdateres.

> **Vanlig feil:** Å tro at SLA-en til hver tjeneste er SLA-en til hele systemet. Hvis appen avhenger av tre tjenester med 99,9 % hver, blir den samlede tilgjengeligheten omtrent 0,999 × 0,999 × 0,999 ≈ **99,7 %**. Avhengigheter i serie *senker* tilgjengeligheten.

Feiltoleranse (*fault tolerance*)

Systemet **fortsetter å virke** selv om en komponent feiler, fordi det finnes **redundans** – ekstra eksemplarer som tar over. Høy tilgjengelighet handler om å minimere nedetid; feiltoleranse handler om at enkeltfeil ikke gir nedetid i det hele tatt.

Katastrofegjenoppretting (*disaster recovery, DR*)

Planen og løsningen for å gjenopprette tjenesten etter en **stor** hendelse: en hel region er nede, løsepengevirus har kryptert alt, eller noen slettet databasen.

To nøkkeltall:

- **RPO** (*Recovery Point Objective*) – hvor mye **data** har vi råd til å miste? (RPO = 1 time → backup minst hver time.)
- **RTO** (*Recovery Time Objective*) – hvor **lang tid** kan det ta før tjenesten er tilbake?



DR-strategier, fra billig/treg til dyr/rask:

Strategi	Beskrivelse	RTO
Backup & restore	Ta backup, gjenopprett ved behov	Timer–dager
Pilot light	Minimal kjerne (f.eks. replikert database) kjører i reserve-regionen	Timer
Warm standby	Nedskalert full kopi kjører	Minutter
Active-active	Fullt system i flere regioner samtidig	Nær null

> **Husk:** Backup du ikke har testet å gjenopprette, er ikke en backup – det er et håp.

Sky-tjenestekategorier

Compute (regnekraft)

Type	Hva	Azure
Virtuelle maskiner	Full OS-kontroll	Virtual Machines, VM Scale Sets
Containere	Pakkede apper	Container Apps, Container Instances, AKS
PaaS-webhosting	Last opp kode	App Service
Serverless	Kode som kjører ved hendelser	Azure Functions

Lagring (*storage*)

Type	Hva	Analogi	Azure
Blokklagring (*block*)	Rå disk for en VM	En harddisk	Managed Disks
Fillagring (*file*)	Delte mapper (SMB/NFS)	En nettverksdisk på kontoret	Azure Files
Objektlagring (*object*)	Filer (objekter) via HTTP, nesten ubegrenset	Et enormt lager med merkede esker	Blob Storage

Lagring har ofte **nivåer** (*tiers*): *hot* (ofte brukt, dyrere lagring, billig tilgang), *cool*, *cold* og *archive* (sjelden brukt, svært billig lagring, dyr og treg å hente).

Nettverk

Virtuelle nettverk (VNet), subnett, sikkerhetsgrupper, lastbalanserere, DNS, VPN-gatewayer, CDN – alt fra kapittel 2, men som programvaredefinerte tjenester du oppretter med klikk eller kode.

Databaser

- **Administrerte relasjonsdatabaser**: Azure SQL Database, Azure Database for PostgreSQL / MySQL (*Flexible Server*). Leverandøren tar seg av patching, backup og HA.
- **NoSQL**: Azure Cosmos DB.
- **Cache**: Azure Cache for Redis / Azure Managed Redis (sjekk Microsoft Learn for gjeldende produktnavn og veikart).

> **Fra arbeidslivet:** Å drifte en database selv på en VM (installere, patche, sette opp replikering og backup) er mye arbeid og risiko. De fleste team velger en **administrert database** (PaaS) med mindre de har særskilte behov.

Serverless

Hva er det? Du skriver bare kode (en *funksjon*) som kjøres når noe skjer – en HTTP-forespørsel, en fil lastes opp, en melding i en kø, et tidspunkt. Du tenker ikke på servere i det hele tatt (de finnes, men leverandøren håndterer dem), og betaler typisk per kjøring.

- **Fordeler:** null serverdrift, skalerer automatisk (også til null), svært billig ved lav eller ujevn bruk.
- **Ulemper: kaldstart** (*cold start*) – første kall etter en stille periode kan være tregt; begrensninger på kjøretid; vanskeligere lokal testing; risiko for leverandørbinding.
- **Azure:** Azure Functions, Logic Apps (arbeidsflyter med lite kode), Container Apps (skalerer også til null).

Sky-sikkerhet og modellen for delt ansvar

Hva er det? Sikkerheten i skyen er **delt** mellom leverandøren og kunden. Leverandøren sikrer **skyen** (fysiske datasentre, maskinvare, hypervisor). Du sikrer det du **legger i** skyen.

Ansvar	On-prem	IaaS	PaaS	SaaS
Data, kontoer og identiteter	Kunde	Kunde	Kunde	Kunde
Enheter (PC-er, mobiler)	Kunde	Kunde	Kunde	Kunde
Applikasjon	Kunde	Kunde	Delt	Leverandør
Nettverkskontroller	Kunde	Kunde	Delt	Leverandør
Operativsystem	Kunde	Kunde	Leverandør	Leverandør
Fysiske verter, nettverk, datasenter	Kunde	Leverandør	Leverandør	Leverandør

> **Husk:** Du er **alltid** ansvarlig for: dine **data**, dine **brukerkontoer og tilganger**, og **enhetene** som brukes. De aller fleste sikkerhetsbrudd i skyen skyldes kundens feilkonfigurasjon – åpne lagringskontoer, svake passord, manglende MFA – ikke at leverandøren ble hacket.

Grunnleggende sky-sikkerhet (utdypes i kapittel 13):

- **Identitet er den nye perimeteren**: MFA for alle, minste privilegium, ingen delte kontoer.
- **Kryptering** av data i ro (*at rest*) og under overføring (*in transit*).
- **Nettverkssegmentering** og private endepunkter.
- **Logging og overvåking** av alt.
- **Policy** som hindrer usikker konfigurasjon.

Prising og kostnadsoptimalisering

Hva koster penger i skyen?

Kategori	Eksempel
Compute	Per sekund/time en VM kjører, per vCPU/minne, per funksjonskjøring
Lagring	Per GB per måned, pluss per transaksjon
Utgående datatrafikk (*egress*)	Data **ut** av skyen til internett koster ofte; data **inn** er som regel gratis
Lisenser	Windows Server, SQL Server (kan inngå i timeprisen)
Administrerte tjenester	Lastbalanserere, gatewayer, brannmurer, offentlige IP-er – ofte per time
Støtte	Supportavtaler

Prismodeller

- **Pay-as-you-go** – betal per bruk, ingen binding. Dyrest per time, mest fleksibelt.
- **Reservasjoner** (*Reserved Instances*) – forplikt deg til 1 eller 3 år for en bestemt ressurs; stor rabatt.
- **Savings Plans** – forplikt deg til et beløp per time i 1 eller 3 år; rabatt med mer fleksibilitet.
- **Spot** – ledig kapasitet med stor rabatt, men kan bli tatt tilbake på kort varsel. Bra for batchjobber og testing.
- **Azure Hybrid Benefit** – bruk eksisterende Windows Server/SQL Server-lisenser i Azure.

Kostnadsoptimalisering – sjekkliste

1. **Slå av eller slett det du ikke bruker.** Den vanligste lekkasjen. Stoppede (*deallocated*) VM-er koster ikke compute, men disker og offentlige IP-er koster fortsatt.
2. **Riktig størrelse** (*right-sizing*): ikke kjør en 16-kjerners VM som bruker 5 % CPU.
3. **Autoskalering** og **automatisk avslåing** av dev/test-miljøer om natten.
4. **Riktig lagringsnivå** og livssyklusregler som flytter gamle data til billigere nivåer.
5. **Reservasjoner/Savings Plans** for stabile arbeidslaster.
6. **Tagging** av alle ressurser med eier, prosjekt og miljø – så du vet hvem som bruker hva.
7. **Budsjetter og varsler.**
8. **Velg PaaS/serverless** der det passer – du betaler ikke for ledig kapasitet.

> **Kostnad:** Den farligste setningen for en student er «jeg rydder opp i morgen». En glemt VM, lastbalanserer, NAT Gateway, VPN Gateway, Azure Firewall, eller et AKS-cluster kan spise opp studentkreditten på dager eller uker. Rydd opp **samme dag**, og slett hele ressursgruppen når du er ferdig.

FinOps er fagfeltet for økonomistyring i skyen: samarbeid mellom teknologi, økonomi og forretning for å få mest mulig verdi per krone.

AWS, Azure og Google Cloud – en sammenligning

De tre store leverandørene tilbyr i hovedsak de samme kategoriene av tjenester. Når du har lært én skikkelig, er det mye lettere å lære de andre.

Kategori	Microsoft Azure	Amazon Web Services (AWS)	Google Cloud (GCP)
Virtuelle maskiner	Virtual Machines	EC2	Compute Engine
Autoskalerende VM-grupper	VM Scale Sets	Auto Scaling Groups	Managed Instance Groups
Objektlagring	Blob Storage	S3	Cloud Storage
Virtuelt nettverk	Virtual Network (VNet)	VPC	VPC
Brannmurregler	Network Security Group	Security Group / NACL	VPC firewall rules
Administrert relasjonsdatabase	Azure SQL Database, Azure Database for PostgreSQL	RDS, Aurora	Cloud SQL, AlloyDB
NoSQL	Cosmos DB	DynamoDB	Firestore, Bigtable
Serverless funksjoner	Azure Functions	Lambda	Cloud Run functions
Container-plattform (enkel)	Container Apps	App Runner, ECS/Fargate	Cloud Run
Kubernetes	AKS	EKS	GKE
Container-register	Azure Container Registry	ECR	Artifact Registry
Identitet	Microsoft Entra ID, Azure RBAC	IAM, IAM Identity Center	Cloud IAM
Hemmeligheter	Key Vault	Secrets Manager, KMS	Secret Manager, Cloud KMS

Kategori	Microsoft Azure	Amazon Web Services (AWS)	Google Cloud (GCP)
Infrastruktur som kode (egen)	ARM / Bicep	CloudFormation	Infrastructure Manager (Terraform-basert)
Overvåking	Azure Monitor	CloudWatch	Cloud Monitoring/Logging
CI/CD	Azure DevOps / GitHub Actions	CodePipeline / CodeBuild	Cloud Build
AI-plattform	Azure Machine Learning, Microsoft Foundry (tidl. Azure AI Foundry)	SageMaker, Bedrock	Vertex AI

Styrker, grovt sagt:

- **Azure:** sterk i bedrifter som allerede bruker Microsoft (Windows Server, Microsoft 365, Entra ID, .NET), hybrid-løsninger, og tett integrasjon med GitHub og OpenAI-modeller. Svært utbredt i Norge og offentlig sektor.
- **AWS:** størst og eldst, bredest tjenestetilbud og enormt økosystem.
- **Google Cloud:** sterk på data, analyse, Kubernetes (Google laget Kubernetes) og AI.

> **Fra arbeidslivet:** I norske stillingsannonser er Azure svært ofte etterspurt, særlig hos konsulentselskaper, banker og offentlig sektor. Begreper og prinsipper (IaaS/PaaS, nettverk, IAM, IaC, containere) er overførbare mellom leverandører – og det er det arbeidsgivere egentlig ser etter.

Cloud-native og Well-Architected

Cloud-native betyr applikasjoner bygget for å utnytte skyen: containere, mikrotjenester, automatisering, deklarativ infrastruktur, observability og robusthet mot feil.

Alle de store leverandørene har et **Well-Architected Framework** med beste praksis. Microsofts har fem søyler:

Søyle	Spørsmål
Reliability (pålitelighet)	Tåler løsningen feil og gjenoppretter den seg?
Security (sikkerhet)	Er data og tilgang beskyttet?
Cost Optimization	Får vi mest mulig verdi for pengene?
Operational Excellence	Er drift, deploy og overvåking automatisert og god?

Søyle	Spørsmål
Performance Efficiency	Skalerer løsningen effektivt med lasten?

Bruk disse fem som sjekkliste når du designer noe – og når du svarer på arkitekturspørsmål i intervjuer.

Kapitteloppsummering

- Skyen er å leie IT-ressurser over internett, betalt etter bruk – fysisk er det enorme datasentre.
- Regioner inneholder tilgjengelighetssoner, som er fysisk adskilte datasentre.
- IaaS (du drifter OS), PaaS (du drifter app og data), SaaS (du bruker ferdig programvare).
- Public, private, hybrid og multi-cloud er måter å kombinere ressurser på.
- Skalering kan være vertikal eller horisontal; elastisitet er automatisk skalering.
- HA, feiltoleranse og DR (med RPO og RTO) handler om robusthet på ulike nivåer.
- Delt ansvar: leverandøren sikrer skyen; du sikrer data, identiteter og konfigurasjon.
- Kostnader styres med riktig størrelse, avslåing, tagging, budsjetter og riktige prismodeller.
- Azure, AWS og GCP tilbyr de samme kategoriene med ulike navn.

Øvingsoppgaver

1. Klassifiser som IaaS, PaaS eller SaaS: Azure VM, Microsoft 365, Azure App Service, Azure SQL Database, Gmail, Azure Functions.
2. En app har en SLA-kjede med App Service (99,95 %) og Azure SQL Database (99,99 %). Regn ut omtrentlig samlet tilgjengelighet.
3. En bedrift kan tåle å miste 15 minutter med data og må være oppe igjen innen 2 timer. Hva er RPO og RTO? Hvilken DR-strategi passer?
4. Bruk Azure Pricing Calculator (azure.microsoft.com/pricing/calculator) til å anslå månedskostnaden for én liten Linux-VM i Norway East. Sammenlign med samme VM i en annen region.
5. Lag en tabell med fem tjenester du kjenner fra Azure, og finn tilsvarende tjeneste i AWS og GCP.

Quiz

1. Nevn tre av NISTs fem kjennetegn ved skyen.
2. Hva er forskjellen på CapEx og OpEx?
3. Hva er en tilgjengelighetssone?
4. Hva er forskjellen på vertikal og horisontal skalering?
5. Hva er forskjellen på skalerbarhet og elastisitet?

6. Hva betyr RPO og RTO?
7. I PaaS – hvem patcher operativsystemet?
8. Hva er du *alltid* ansvarlig for, uansett tjenestemodell?
9. Hvorfor kan utgående datatrafikk bli en overraskende kostnad?
10. Hva heter Azure Blob Storage hos AWS?

Fasit

1. Hvilke som helst tre av: on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service.
2. **CapEx** er investeringer i fysiske eiendeler på forhånd (kjøpe servere). **OpEx** er løpende driftsutgifter (betale per bruk).
3. En fysisk adskilt lokasjon (ett eller flere datasentre) innenfor en region, med egen strøm, kjøling og nettverk.
4. **Vertikal**: gjøre én maskin større/mindre. **Horisontal**: legge til eller fjerne maskiner.
5. **Skalerbarhet** er evnen til å håndtere mer last ved å legge til ressurser. **Elastisitet** er at dette skjer automatisk, opp og ned, i takt med etterspørselen.
6. **RPO**: hvor mye data man har råd til å miste (målt i tid). **RTO**: hvor lang tid gjenopprettingen kan ta.
7. **Leverandøren** (f.eks. Microsoft).
8. **Data**, **kontoer/identiteter** og **enhetene** (endepunktene) som brukes.
9. Fordi data *ut* av skyen ofte faktureres per GB, og store volumer (video, backuper, nedlastinger) kan gi store regninger.
10. **Amazon S3**.

Del 7 – Microsoft Azure fra A til Å

Dette er bokens største del. Den er delt i seks kapitler:

Kapittel	Innhold
7.1 Grunnmur, identitet og styring	Konto, portal, CLI, ARM, hierarkiet, Entra ID, RBAC, managed identities, Policy, tags, låser, kostnader
7.2 Nettverk og compute	VNet, subnett, NSG, Load Balancer, Application Gateway, DNS, VM, App Service, Functions, ACR, AKS
7.3 Lagring, data, sikkerhet og drift	Storage Accounts, Blob, Files, Azure SQL, Key Vault, Monitor, Log Analytics, Backup
7.4 Azure DevOps og Bicep	Azure DevOps, Pipelines, Bicep, referansearkitektur
7.5 AZ-900	Sertifiseringsmål, øvingsoppgaver og prøveeksamen
7.6 AZ-104	Sertifiseringsmål, laboppgaver og prøveeksamen

Mal for hver tjeneste. Hver tjeneste beskrives med: *Hva den gjør · Hvorfor · Bedriftseksempel · Opprett og konfigurer · Sikkerhet og kostnad · Øvelse · Sammenheng med andre tjenester.*

> **Kostnad – les dette før du starter:** Gjør disse tre tingene før du lager noe i Azure: 1. Bruk **Azure for Students** hvis du kan (kreditt uten kredittkort, verifiseres med skole-e-post) eller en **Azure free account**. Vilkår, kreditbeløp og hvilke tjenester som er gratis endres – les gjeldende vilkår på azure.microsoft.com. 2. Lag et **budsjett med e-postvarsel** (vist i dette kapittelet). 3. Legg alt du lager i en **egen ressursgruppe per øvelse**, og slett ressursgruppen når du er ferdig.

7.1 Grunnmur, identitet og styring

Kom i gang: konto, portal, Cloud Shell og Azure CLI

Azure Portal (`portal.azure.com`) er det grafiske nettgrensesnittet. Nyttig for å lære og utforske, men profesjonelle team gjør mest mulig med kode.

Det du bør finne i portalen:

- **Søkefeltet øverst** – raskeste vei til enhver tjeneste eller ressurs.
- **Cloud Shell-ikonet** (`>_`) øverst – en Linux-terminal i nettleseren med Azure CLI, PowerShell, Terraform, kubectl og git ferdig installert, og allerede innlogget.

- **Subscriptions**, **Resource groups**, **Cost Management**, **Microsoft Entra ID**.
- **Activity log** på hver ressurs – hvem gjorde hva, når.

Azure CLI (`az`) er kommandolinjeverktøyet. Installer det lokalt (se Microsoft Learn: «How to install the Azure CLI»), eller bruk Cloud Shell.

```
        $ az login                                  # åpner nettleseren for innlogging
        $ az account list -o table                  # vis abonnementene du har tilgang
til
        $ az account set --subscription "<navn eller id>"   # velg abonnement
        $ az account show -o table                  # hvilket abonnement er aktivt?
        $ az group list -o table                    # list ressursgrupper
        $ az vm list -o table                       # list VM-er
        $ az find "create storage account"          # foreslår kommandoer
        $ az vm create --help                       # hjelp for en kommando
```

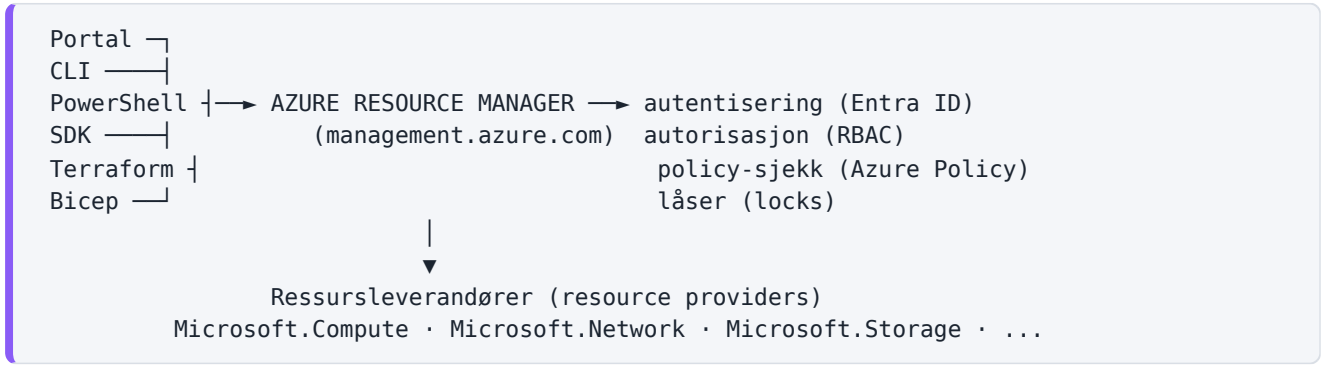
Utdataformater (`-o` / `--output`): `json` (standard), `table` (lesbart), `tsv` (for skript), `yaml`. `--query` filtrerer med JMESPath:

```
        $ az vm list --query "[].{navn:name, region:location}" -o table
        $ az group show -n rg-demo --query id -o tsv     # bare ID-en – nyttig i variabler
```

Azure PowerShell (`Az`-modulen) er et alternativ med kommandoer som `New-AzResourceGroup`. Boken bruker Azure CLI, men begge snakker med det samme API-et.

Azure Resource Manager (ARM)

Hva den gjør. Azure Resource Manager er **kontrollplanet** i Azure: tjenesten som tar imot *alle* forespørsler om å opprette, endre eller slette ressurser – fra portalen, CLI, PowerShell, SDK-er, Terraform og Bicep.



Hvorfor det er viktig. Fordi alt går gjennom ARM, får du *konsistent* sikkerhet, tilgangsstyring, policy og logging uansett verktøy. Det er også grunnen til at «klikk i portalen» og «kode» gir nøyaktig samme resultat.

Viktige begreper:

- **Ressurs** (*resource*) – én ting du administrerer: en VM, et VNet, en lagringskonto.

- **Ressursleverandør** (*resource provider*) – tjenesten som leverer en ressurstype, f.eks. `Microsoft.Compute/virtualMachines`. Noen må registreres i abonnementet før bruk: `az provider register --namespace Microsoft.ContainerService`.
- **Ressurs-ID** – full «adresse» til en ressurs:

```
/subscriptions/<sub-id>/resourceGroups/rg-
demo/providers/Microsoft.Compute/virtualMachines/vm-web01
```

- **ARM-maler** (*ARM templates*) – JSON-filer som beskriver ressurser deklarativt. **Bicep** er et enklere språk som oversettes til ARM-maler (kapittel 7.4).
- **Kontrollplan vs. dataplan**: *kontrollplanet* (ARM) er å lage og konfigurere en lagringskonto. *Dataplanet* er å lese og skrive filene i den. Tilgang styres separat – et viktig poeng for RBAC.

Hierarkiet: management groups, subscriptions, resource groups

```
Microsoft Entra-tenant (organisasjonens identitetskatalog)
└── Root management group
     ├── Management group: "Plattform"
     │    └── Subscription: "sub-connectivity"
     │         └── Resource group: rg-hub-network
     │              ├── vnet-hub
     │              └── azure-firewall
     └── Management group: "Landingssoner"
          ├── Subscription: "sub-nettbutikk-prod"
          │    ├── Resource group: rg-nettbutikk-prod-web
          │    │    ├── app-service-plan
          │    │    └── web-app
          │    └── Resource group: rg-nettbutikk-prod-data
          │         └── azure-sql-database
          └── Subscription: "sub-nettbutikk-dev"
```

Innstillinger som **policy** og **tilganger (RBAC)** som settes på et nivå, **arves nedover** til alt under.

Microsoft Entra-tenant

Organisasjonens instans av Microsoft Entra ID (identitetstjenesten). Alle abonnementer stoler på nøyaktig én tenant.

Management groups

Hva den gjør. Grupperer abonnementer slik at du kan styre policy og tilgang for mange abonnementer samtidig. Kan nøstes i opptil seks nivåer under rotgruppen.

Hvorfor. Store bedrifter har titalls eller hundrevis av abonnementer. Uten management groups måtte hver policy settes opp på hvert abonnement.

Bedriftseksempel. En bank har management groups for «Produksjon» og «Sandkasse». På «Produksjon» gjelder strenge policyer (bare Norway East og Sweden Central, ingen offentlige IP-er uten godkjenning). I «Sandkasse» får utviklere eksperimentere fritt, men med budsjettgrenser.

```
        $ az account management-group create --name mg-lab --display-name "Lab"
        $ az account management-group subscription add --name mg-lab --subscription "<sub-
id>"
```

Subscriptions (abonnementer)

Hva den gjør. Et abonnement er en **faktureringsenhet** og en **tilgangsgrense**. Alle ressurser tilhører nøyaktig ett abonnement, og kostnadene faktureres per abonnement.

Hvorfor flere abonnementer? For å skille miljøer (prod/dev), avdelinger eller prosjekter – med egne budsjetter, tilganger og **kvoter** (grenser, f.eks. hvor mange vCPU-er du kan bruke per region).

> **Vanlig feil:** Å treffe en kvotegrense og tro at noe er ødelagt. Feilmeldinger som «Operation could not be completed as it results in exceeding approved … Cores quota» betyr at du må velge mindre størrelse, annen region, eller be om økt kvote (*Quotas* i portalen). Studentabonnementer har ofte lave kvoter.

Resource groups (ressursgrupper)

Hva den gjør. En logisk **mappe** for ressurser som hører sammen og har samme livssyklus.

Regler å kunne:

- Hver ressurs ligger i **nøyaktig én** ressursgruppe.
- Ressursgrupper kan **ikke nøstes**.
- Ressursgruppen har en region (der *metadata* lagres), men ressursene i den kan ligge i andre regioner.
- **Sletter du ressursgruppen, slettes alt i den.** Perfekt for opprydding etter øvelser.
- Ressurser kan flyttes mellom grupper og abonnementer (med noen unntak).

Bedriftseksempel. «Alt for nettbutikkens testmiljø» i `rg-nettbutikk-test`. Når testperioden er over: én kommando sletter alt.

```
        $ az group create --name rg-lab-01 --location norwayeast --tags miljø=lab
eier=student
        $ az group list -o table
        $ az group delete --name rg-lab-01 --yes --no-wait      # slett alt i gruppen
```

Navnekonvensjon. Microsoft Cloud Adoption Framework anbefaler prefikser som `rg-`, `vnet-`, `snet-`, `nsg-`, `vm-`, `st` (lagringskonto), `kv-`, `app-`, `func-`, `log-`, `aks-`, `acr`. Eksempel: `rg-nettbutikk-prod-norwayeast-001`. Konsekvente navn gjør automatisering og feilsøking mye enklere.

Microsoft Entra ID

Hva den gjør. Microsoft Entra ID (tidligere **Azure Active Directory / Azure AD**) er Microsofts skybaserte **identitets- og tilgangstjeneste**. Den svarer på spørsmålet **«hvem er du?»** (autentisering) og gir tokens som andre tjenester bruker for å avgjøre **«hva får du lov til?»** (autorisasjon).

> **Hverdagsanalogi:** Entra ID er resepsjonen og ID-kortsystemet i et kontorbygg. Den sjekker hvem du er og gir deg et adgangskort. Dørene (Azure-ressurser, Microsoft 365, andre apper) leser kortet og avgjør om du slipper inn.

Hvorfor. Én sentral identitet per person gir enkel innlogging (*single sign-on*), sentral kontroll med MFA, og mulighet til å stenge all tilgang med ett klikk når noen slutter.

Viktige objekter:

Objekt	Hva
Bruker (*user*)	En person. Kan være intern (medlem) eller gjest (B2B)
Gruppe (*group*)	Samling brukere – gi tilgang til grupper, ikke enkeltpersoner
App registration	Definisjonen av en applikasjon som skal bruke Entra ID
Service principal	Identiteten en applikasjon bruker i en tenant (f.eks. en CI/CD-pipeline)
Managed identity	En service principal som Azure administrerer automatisk (se under)

Sikkerhetsfunksjoner:

- **MFA** (*multifaktorautentisering*) – passord + noe du har (app, nøkkel). Microsoft krever nå MFA for innlogging til Azure-portalen, CLI og andre administrasjonsverktøy (innført gradvis fra 2024–2025).
- **Conditional Access** – regler som «krev MFA utenfor kontoret», «blokker innlogging fra visse land», «krev administrert enhet». Krever Entra ID P1-lisens eller høyere.
- **Privileged Identity Management (PIM)** – administratorroller aktiveres *bare når de trengs*, for en begrenset tid, med godkjenning. Krever P2 / Entra ID Governance.
- **Passordløs innlogging** – passkeys, Windows Hello, Authenticator-appen.

Entra ID vs. Active Directory (on-prem). Klassisk *Active Directory Domain Services* (AD DS) i Windows-servere bruker Kerberos/LDAP og domener. Entra ID er skybasert og bruker moderne webprotokoller (OAuth 2.0, OpenID Connect, SAML). **Entra Connect** synkroniserer brukere fra lokal AD til Entra ID i hybride miljøer.

Opprett bruker og gruppe:

```
        # Domenet ditt finner du i portalen: Entra ID → Overview → Primary domain
        $ az ad user create --display-name "Test Bruker" \
            --user-principal-name testbruker@<ditt-domene>.onmicrosoft.com \
            --password "<et-sterkt-midlertidig-passord>" --force-change-password-next-
sign-in true

        $ az ad group create --display-name "devops-team" --mail-nickname "devops-team"
        $ az ad group member add --group "devops-team" \
            --member-id $(az ad user show --id testbruker@<ditt-domene>.onmicrosoft.com --
query id -o tsv)
```

Sikkerhet og kostnad. Entra ID Free følger med alle abonnementer. P1/P2-funksjoner (Conditional Access, PIM) koster per bruker. **Bruk aldri delte administratorkontoer**, og ha minst to «break-glass»-nødkontoer med sterk beskyttelse i ekte organisasjoner.

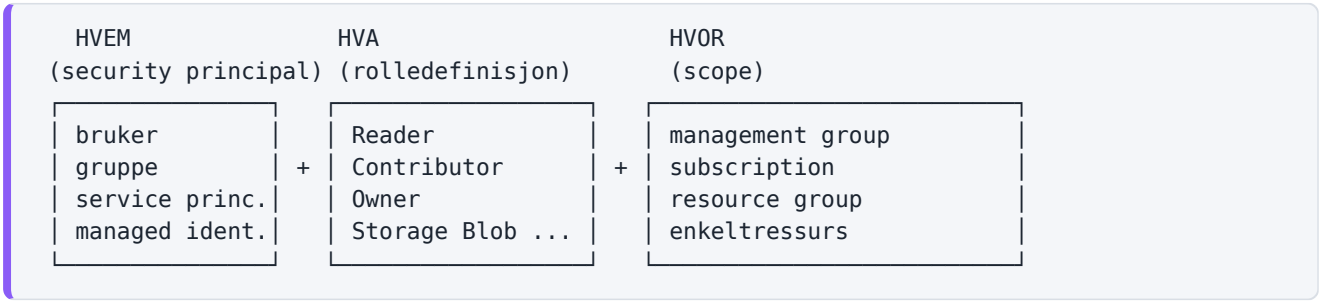
Øvelse. Opprett en testbruker og gruppen `devops-team`, legg brukeren i gruppen, og gi gruppen *Reader* på en ressursgruppe (neste seksjon). Logg inn som testbrukeren i et privat nettleservindu og bekreft at du kan se, men ikke endre.

Sammenheng. Entra ID er grunnlaget for RBAC, managed identities, Key Vault-tilgang, innlogging i AKS, og føderert innlogging fra GitHub Actions og Azure Pipelines.

Azure RBAC – rollebasert tilgangskontroll

Hva den gjør. Azure RBAC (*Role-Based Access Control*) styrer **hva** en identitet får gjøre med Azure-ressurser.

En **rolletildeling** (*role assignment*) består av tre deler:



De viktigste innebygde rollene:

Rolle	Kan
Owner	Alt, *inkludert* å gi andre tilgang
Contributor	Opprette og administrere alt, men *ikke* gi tilgang
Reader	Se alt, endre ingenting
User Access Administrator	Administrere tilganger

Rolle	Kan
Role Based Access Control Administrator	Administrere tilganger, kan begrenses med betingelser (tryggere enn UAA)
Tjenestespesifikke	f.eks. *Virtual Machine Contributor*, *Storage Blob Data Reader*, *Key Vault Secrets User*, *AcrPull*

Hvordan det evalueres:

- Tilganger **arves** nedover: Reader på abonnementet gir Reader på alle ressursgrupper i det.
- Tilganger er **additive**: Reader + Contributor = effektivt Contributor.
- **Deny assignments** (avslag) vinner over tillatelser, men lages som regel av Azure selv (f.eks. av *deployment stacks*), ikke manuelt.
- **Kontrollplan vs. dataplan**: *Contributor* på en lagringskonto lar deg konfigurere den, men å lese blob-data med Entra-innlogging krever en **data-rolle** som *Storage Blob Data Reader*.

Hvorfor. Prinsippet om **minste privilegium** (*least privilege*): gi akkurat den tilgangen som trengs, på det minste scopet, i kortest mulig tid.

Bedriftseksempel. Utviklerteamet får *Contributor* på `rg-nettbutikk-dev`, men bare *Reader* på produksjon. CI/CD-pipelinen sin identitet får *Contributor* på produksjonsgruppen – ingen mennesker endrer produksjon manuelt.

```
$ RG_ID=$(az group show -n rg-lab-01 --query id -o tsv)
$ GROUP_ID=$(az ad group show --group devops-team --query id -o tsv)

$ az role assignment create --assignee-object-id "$GROUP_ID" \
    --assignee-principal-type Group --role "Reader" --scope "$RG_ID"

$ az role assignment list --scope "$RG_ID" -o table    # hvem har tilgang her?
$ az role definition list --name "Contributor" -o json # hva inneholder rollen?
```

Vanlig feil: Å forveksle **Entra-roller** (f.eks. *Global Administrator*, *User Administrator* – styrer Entra ID selv: brukere, grupper, lisenser) med **Azure RBAC-roller** (*Owner*, *Contributor* – styrer Azure-ressurser). De er to separate systemer. En Global Administrator har ikke automatisk tilgang til Azure-ressurser (men kan gi seg selv det via «Access management for Azure resources»).

Vanlig feil: RBAC-endringer kan bruke litt tid før de slår gjennom. Får du «AuthorizationFailed» rett etter en tildeling, vent et par minutter og prøv igjen (logg eventuelt inn på nytt).

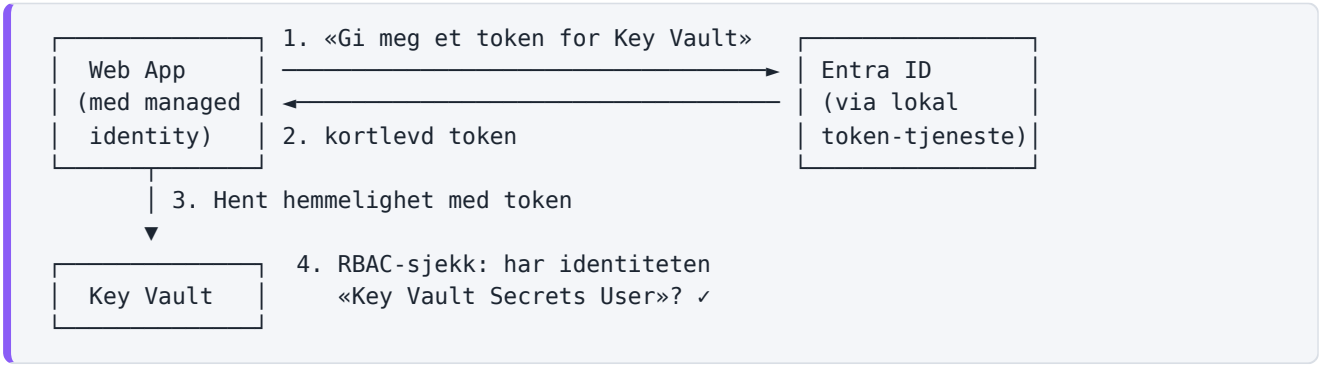
Egendefinerte roller (*custom roles*) kan lages i JSON med lister over `Actions`, `NotActions`, `DataActions` og `AssignableScopes` når ingen innebygd rolle passer.

Øvelse. Gi deg selv *Storage Blob Data Contributor* på en lagringskonto (kapittel 7.3) og bekreft at `az storage blob list --auth-mode login` virker før og ikke virker etter at du fjerner tildelingen.

Managed identities

Hva den gjør. En managed identity er en **identitet i Entra ID som Azure lager og administrerer automatisk for en ressurs** (en VM, en App Service, en Function, en AKS-pod). Ressursen kan bruke identiteten til å logge inn på andre Azure-tjenester **uten passord eller nøkler i koden**.

Hvorfor. Det klassiske problemet: appen må koble seg til databasen eller Key Vault. Hvor lagrer du passordet? I koden (farlig), i en konfigfil (farlig), i en miljøvariabel (bedre, men den må roteres)? Med managed identity finnes det **ingen hemmelighet å lekke** – Azure utsteder kortlevde tokens automatisk.



To typer:

	System-assigned	User-assigned
Opprettes	Sammen med ressursen	Som egen ressurs
Livssyklus	Slettes når ressursen slettes	Lever uavhengig
Deling	Én ressurs	Kan deles av mange ressurser
Bruk	Enkelt, én app	Mange instanser med samme tilgang, eller når identiteten må finnes før ressursen

Opprett og bruk:

```
# Slå på system-assigned identity for en web-app
$ az webapp identity assign -g rg-lab-01 -n app-demo-<unik>
{ "principalId": "1111aaaa-...", "type": "SystemAssigned" }

# Gi identiteten lesetilgang til hemmeligheter i Key Vault
$ KV_ID=$(az keyvault show -n kv-demo-<unik> --query id -o tsv)
$ az role assignment create --assignee-object-id "1111aaaa-..." \
    --assignee-principal-type ServicePrincipal \
    --role "Key Vault Secrets User" --scope "$KV_ID"
```

I Python-koden holder det med `DefaultAzureCredential()` (kapittel 4) – den finner managed identity automatisk når koden kjører i Azure, og din `az login` når du kjører lokalt. **Samme kode, ingen hemmeligheter.**

> **Husk:** Managed identity er svaret på nesten alle spørsmål av typen «hvordan bør appen autentisere seg mot en annen Azure-tjeneste?» – både i virkeligheten og på AZ-104.

Workload identity federation er en beslektet teknikk der eksterne systemer (GitHub Actions, Azure Pipelines, Kubernetes) får Azure-tokens uten lagrede hemmeligheter, basert på tillit (OIDC). Vi bruker det i kapittel 12.

Azure Policy

Hva den gjør. Azure Policy **håndhever regler** for hvordan ressurser kan konfigureres, og **rapporterer** hvilke ressurser som bryter reglene (*compliance*).

RBAC vs. Policy:

- RBAC: **hvem** får gjøre noe? («Kari får opprette VM-er.»)
- Policy: **hva** er lov å lage? («Ingen – heller ikke Kari – får lage VM-er utenfor Norway East.»)

Hvordan det fungerer.

1. En **policydefinisjon** beskriver en regel i JSON (*if* ... *then* effekt).
2. Definisjoner kan samles i et **initiativ** (*policy set*).
3. Du gjør en **tildeling** (*assignment*) av definisjonen eller initiativet til et scope (management group, abonnement, ressursgruppe).
4. ARM sjekker nye og endrede ressurser mot reglene, og Azure evaluerer eksisterende ressurser jevnlig.

Vanlige effekter:

Effekt	Gjør
`Deny`	Stopper opprettelsen
`Audit`	Tillater, men markerer som ikke-kompatibel
`Append` / `Modify`	Legger til eller endrer egenskaper (f.eks. legger til en tag)
`DeployIfNotExists`	Oppretter noe som mangler (f.eks. diagnostikkinnstillinger)
`AuditIfNotExists`	Rapporterer at noe mangler
`Disabled`	Av

Bedriftseksempler på policyer: bare tillatte regioner; påkrevd tag `kostnadssted`; ingen offentlig tilgang til lagringskontoer; bare godkjente VM-størrelser; krev TLS 1.2; send alle logger til Log Analytics.

Eksempel – «Allowed locations» på en ressursgruppe:

```
        $ RG_ID=$(az group show -n rg-lab-01 --query id -o tsv)

        # Finn den innebygde definisjonen «Allowed locations»
        $ az policy definition list --query "[?displayName=='Allowed locations'].name" -o
tsv
        e56962a6-4747-49cd-b67b-bf8b01975c4c

        $ az policy assignment create --name allowed-locations-lab \
            --policy e56962a6-4747-49cd-b67b-bf8b01975c4c \
            --scope "$RG_ID" \
            --params '{"listOfAllowedLocations": {"value": ["norwayeast",
"swedencentral"]}}'

        # Test: dette skal nå bli avvist med "RequestDisallowedByPolicy"
        $ az storage account create -n sttest$RANDOM -g rg-lab-01 -l westus --sku
Standard_LRS
```

En policyregel i JSON ser slik ut (forenklet):

```
        {
          "if": {
            "allOf": [
              { "field": "type", "equals": "Microsoft.Storage/storageAccounts" },
              { "field": "Microsoft.Storage/storageAccounts/allowBlobPublicAccess",
"equals": true }
            ]
          },
          "then": { "effect": "deny" }
        }
```

Sikkerhet og kostnad. Azure Policy for Azure-ressurser er gratis. Det er et av de viktigste verktøyene for **governance** (styring) i skyen.

> **Fordypning: Unntak** (*exemptions*) lar deg unnta enkeltressurser. **Remediation tasks** retter eksisterende ressurser for `Modify` / `DeployIfNotExists`. Microsoft Defender for Cloud bruker Azure Policy under panseret for sikkerhetsanbefalinger.

Tags og ressurslåser

Tags er nøkkel–verdi-merkelapper på ressurser og ressursgrupper: `miljø=prod`, `eier=team-beta-ling`, `kostnadssted=4410`, `utløper=2026-12-01`.

- Brukes til kostnadsrapportering (Cost Management kan gruppere på tags), automatisering og oversikt.

- Tags **arves ikke automatisk** fra ressursgruppen til ressursene – bruk Azure Policy (f.eks. «Inherit a tag from the resource group») for å få det til.

```
$ az group update -n rg-lab-01 --tags miljø=lab eier=student utløper=2026-11-30
$ az resource list --tag miljø=lab -o table
```

Ressurslåser (*resource locks*) beskytter mot uhell, **også for Owner**:

Lås	Effekt
`CanNotDelete` (*Delete*)	Kan endres, men ikke slettes
`ReadOnly`	Kan verken endres eller slettes

```
$ az lock create --name ikke-slett --lock-type CanNotDelete --resource-group rg-prod-data
$ az lock list -g rg-prod-data -o table
$ az lock delete --name ikke-slett --resource-group rg-prod-data
```

> **Vanlig feil:** `ReadOnly` kan ha uventede bivirkninger. En `ReadOnly`-lås på en lagringskonto hindrer for eksempel listing av tilgangsnøkler (det er en POST-operasjon), og på en App Service kan den hindre ting som å se konfigurasjon i portalen. Les Microsoft Learn («Lock your resources») før du bruker den.

Azure-priser og Cost Management

Hva det gjør. Cost Management + Billing viser hva du har brukt, hva du kommer til å bruke (prognose), og lar deg sette budsjetter og varsler. **Azure Pricing Calculator** estimerer kostnader *før* du bygger. **Azure Advisor** gir anbefalinger om sparing (f.eks. «denne VM-en er nesten ubrukt»).

Hva påvirker prisen? Ressurstype og størrelse, region, tid i bruk, lagringsmengde og -nivå, utgående trafikk, lisenser, og prismodell (pay-as-you-go, reservasjoner, savings plan, spot). Se kapittel 6.

Lag et budsjett (portal) – gjør dette nå:

1. Søk etter **Cost Management** → **Budgets** → **+ Add**.
2. Scope: abonnementet ditt.
3. Navn `budsjett-student`, periode *Monthly*, beløp f.eks. 20 (i valutaen din).
4. **Alert conditions**: *Actual* 50 %, *Actual* 90 %, *Forecasted* 100 %.
5. Legg inn e-postadressen din → **Create**.

> **Vanlig feil:** Å tro at et budsjett **stopper** forbruket. Et budsjett **varsler** bare. (Kreditbaserte student- og gratisabonnementer har en egen utgiftsgrense som stanser ressursene når kreditten er brukt opp – men stol ikke på det; rydd opp selv.) For automatisk handling kan et budsjett kobles til en *action group* som starter en automatisering.

Kostnadsanalyse: *Cost Management* → *Cost analysis* → grupper etter **Resource group**, **Service name** eller **Tag**. Gjør dette ukentlig mens du studerer.

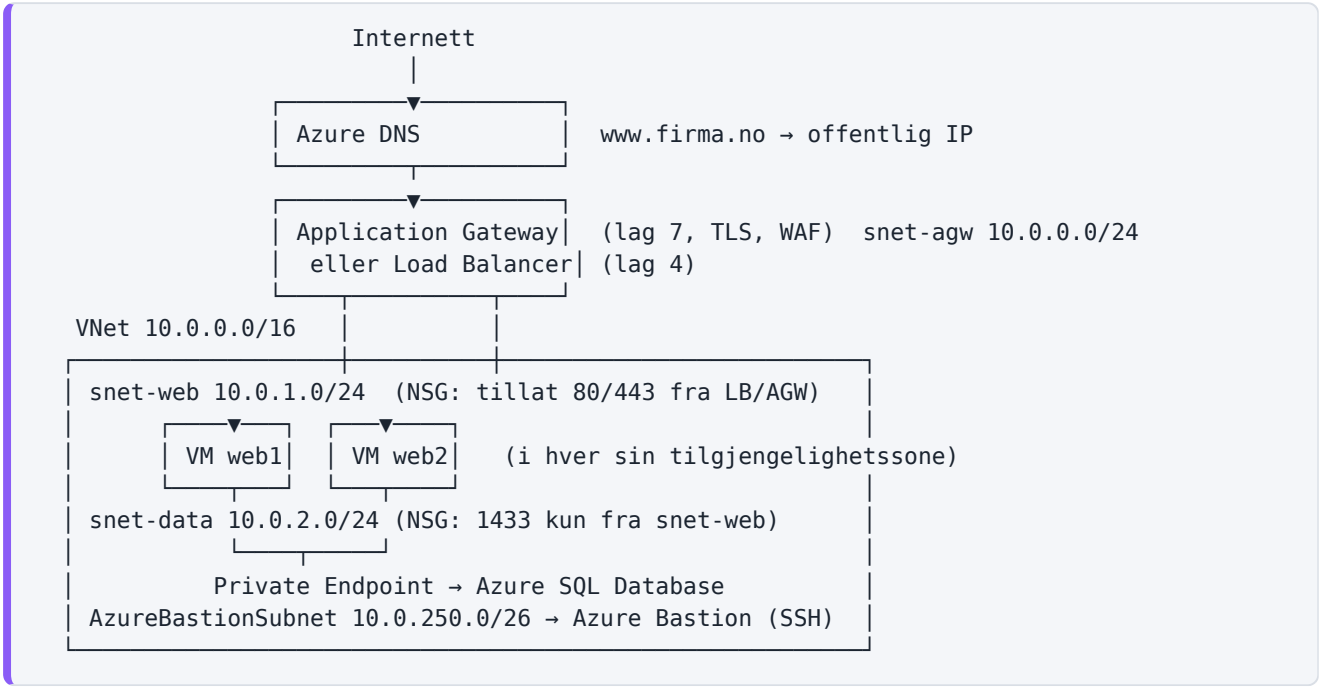
Øvelse. Sett opp budsjettet over. Lag deretter en Pricing Calculator-estimering for arkitekturen i prosjekt 10 (kapittel 15), og lagre den.

Oppsummering av 7.1

- Alt i Azure går gjennom **Azure Resource Manager** – samme API for portal, CLI og kode.
- Hierarki: **tenant → management groups → subscriptions → resource groups → ressurser**. Policy og RBAC arves nedover.
- **Entra ID** = identitet (hvem er du). **RBAC** = tilgang til Azure-ressurser (hva får du gjøre, hvor). **Policy** = regler for hva som er lov å lage.
- **Managed identities** fjerner behovet for passord mellom Azure-tjenester.
- **Tags** gir oversikt og kostnadsfordeling; **låser** hindrer uhell.
- **Budsjetter** varsler, de stopper ikke forbruk.

7.2 Nettverk og compute

I dette kapitlet bygger vi steg for steg denne klassiske arkitekturen, og lærer hver tjeneste underveis:



Før vi starter, sett noen variabler som brukes i eksemplene (i Bash/Cloud Shell):

```
RG=rg-lab-nett
LOC=norwayeast
az group create -n $RG -l $LOC --tags miljø=lab eier=student
```

Virtual Networks (VNet)

Hva den gjør. Et Azure Virtual Network er ditt **private nettverk i Azure**. Ressurser i samme VNet kan snakke med hverandre over private IP-adresser, isolert fra andre kunder.

Hvorfor. VM-er, databaser med private endepunkter, AKS og mye annet trenger et nettverk med kontrollert adresseplan, ruting og sikkerhet – akkurat som et bedriftsnett (kapittel 2), men programvaredefinert.

Viktige egenskaper:

- Et VNet tilhører **én region** og **ett abonnement**, men strekker seg over alle tilgjengelighetssonene i regionen.
- Har ett eller flere **adresseområder** (CIDR), typisk fra private områder som `10.0.0.0/16`.
- Deles i **subnett**.
- Trafikk mellom subnett i samme VNet rutes automatisk (systemruter).
- **VNet peering** kobler VNets sammen (også på tvers av regioner) over Microsofts nett – adresseområdene kan **ikke** overlappe. Peering er **ikke transitivt**: A↔B og B↔C betyr ikke A↔C.
- **Hub-and-spoke**: et sentralt «hub»-VNet med delte tjenester (brannmur, VPN-gateway, Bastion) og flere «spoke»-VNets for arbeidslastene, koblet med peering. Det vanligste bedriftsoppsettet.

Bedriftseksempel. En kommune har ett hub-VNet koblet til rådhusets nett med VPN, og egne spoke-VNets for saksbehandlingssystemet og innbyggerportalen. All trafikk ut til internett går via Azure Firewall i huben.

Opprett:

```
        az network vnet create -g $RG -n vnet-lab \
          --address-prefix 10.0.0.0/16 \
          --subnet-name snet-web --subnet-prefixes 10.0.1.0/24
```

Portal: *Virtual networks* → *+ Create* → velg ressursgruppe, navn og region → fanen *IP addresses*: adresseområde og subnett → *Review + create*.

Sikkerhet og kostnad. Selve VNet-et og subnettene er gratis. Du betaler for ting du kobler til (gatewayer, offentlige IP-er, NAT Gateway, Firewall) og for trafikk over **peering** (per GB).

Sammenheng. Nesten alt av IaaS lever i et VNet. PaaS-tjenester (SQL, Storage, Key Vault) kobles inn med **Private Endpoints** eller **service endpoints**; App Service og Functions kan sende trafikk *inn* i et VNet med **VNet integration**.

Subnett

Hva den gjør. Et subnett er en del av VNet-ets adresseområde. Ressurser plasseres i subnett, og sikkerhet (NSG) og ruting (route tables) knyttes ofte til subnettet.

Viktige regler:

- Azure reserverer **5 adresser** i hvert subnett (kapittel 2).
- Noen tjenester krever **dedikerte subnett med bestemte navn eller størrelser**: `GatewaySubnet` (VPN/ExpressRoute Gateway), `AzureBastionSubnet` (minst `/26`), `AzureFirewallSubnet` (minst `/26`), Application Gateway (eget subnett).
- **Subnet delegation** gir en tjeneste (f.eks. App Service VNet integration, Container Apps) rett til å bruke subnettet.

```
        az network vnet subnet create -g $RG --vnet-name vnet-lab -n snet-data --address-
prefixes 10.0.2.0/24
        az network vnet subnet create -g $RG --vnet-name vnet-lab -n AzureBastionSubnet --
address-prefixes 10.0.250.0/26
        az network vnet subnet list -g $RG --vnet-name vnet-lab -o table
```

> **Vanlig feil:** Å bruke hele VNet-et til ett subnett (`10.0.0.0/16` både som VNet og subnett). Da er det ikke plass til flere subnett senere. Legg igjen ledig adresserom.

> **Merk – utgående internettilgang:** Microsoft har faset ut «default outbound access» for nye virtuelle nettverk: nye subnett blir private som standard, og VM-er uten offentlig IP, NAT Gateway, lastbalanserer med utgående regler eller brannmur får da ikke automatisk tilgang ut til internett. Sjekk Microsoft Learn («Default outbound access for VMs in Azure») for nøyaktig status og datoer. Får en VM ikke installert pakker med `apt`, er dette en vanlig årsak.

Network Security Groups (NSG)

Hva den gjør. En NSG er en **tilstandsbasert brannmur** (*stateful*) på lag 3/4 med regler som tillater eller avviser trafikk basert på kilde, mål, port og protokoll. Knyttes til et **subnett** og/eller et **nettverkskort (NIC)**.

Hvordan reglene evalueres:

- Hver regel har en **prioritet** fra 100 til 4096. **Lavest tall evalueres først**, og første treff vinner.
- Separate regler for **inngående** og **utgående** trafikk.
- *Stateful*: hvis inngående trafikk er tillatt, er svaret automatisk tillatt.
- Standardregler (prioritet 65000+) som du ikke kan slette, men kan overstyre:

Retning	Standardregel	Effekt
Inn	AllowVnetInBound	Tillat trafikk innen VNet (og peerede VNets)
Inn	AllowAzureLoadBalancerInBound	Tillat helsesjekker fra Azure Load Balancer
Inn	DenyAllInBound	**Avvis alt annet**
Ut	AllowVnetOutBound	Tillat til VNet
Ut	AllowInternetOutBound	Tillat ut til internett
Ut	DenyAllOutBound	Avvis alt annet

- **Service tags** er navn på grupper av IP-adresser som Microsoft vedlikeholder: `Internet`, `VirtualNetwork`, `AzureLoadBalancer`, `Storage`, `Sql.NorwayEast`, `AzureCloud` …
- **Application Security Groups (ASG)** lar deg gruppere VM-er logisk («webservere») og bruke gruppen i regler i stedet for IP-adresser.
- Hvis det er NSG både på subnett og NIC, må trafikken tillates av **begge**.

```
Inngående: Internett → [NSG på subnett] → [NSG på NIC] → VM
Utgående:  VM → [NSG på NIC] → [NSG på subnett] → ...
```

Opprett og knytt til subnett:

```
        az network nsg create -g $RG -n nsg-web

        # Tillat HTTP og HTTPS fra internett
        az network nsg rule create -g $RG --nsg-name nsg-web -n allow-web \
          --priority 100 --direction Inbound --access Allow --protocol Tcp \
          --source-address-prefixes Internet --destination-port-ranges 80 443

        # Knytt NSG-en til subnettet
        az network vnet subnet update -g $RG --vnet-name vnet-lab -n snet-web --network-
security-group nsg-web

        # Database-subnett: bare SQL fra web-subnettet
        az network nsg create -g $RG -n nsg-data
        az network nsg rule create -g $RG --nsg-name nsg-data -n allow-sql-from-web \
          --priority 100 --direction Inbound --access Allow --protocol Tcp \
          --source-address-prefixes 10.0.1.0/24 --destination-port-ranges 1433
        az network vnet subnet update -g $RG --vnet-name vnet-lab -n snet-data --network-
security-group nsg-data
```

Feilsøking. *Network Watcher → IP flow verify* svarer på «blir denne pakken tillatt, og av hvilken regel?». *Effective security rules* på et nettverkskort viser de samlede reglene. Flow logs kan sendes til lagring/Log Analytics.

Sikkerhet og kostnad. NSG-er er gratis. Åpne aldri 22/3389 mot `Internet`; bruk Bastion eller begrens til din IP (`--source-address-prefixes <din-ip>/32`).

Øvelse. Lag en regel med prioritet 90 som **avviser** port 80 fra internett. Hva skjer med `allow-web` på prioritet 100? Bekreft med *IP flow verify*. (Svar: prioritet 90 vinner, port 80 avvises, 443 tillates fortsatt.)

Azure Virtual Machines

Hva den gjør. En Azure VM er en **virtuell server** (IaaS) med Linux eller Windows, der du har full kontroll over operativsystemet.

Hvorfor. Når du trenger full kontroll: spesiell programvare, eldre applikasjoner som flyttes fra eget datasenter («lift and shift»), egne agenter, eller arbeidslaster som ikke passer i PaaS.

Bestanddeler av en VM:

```
VM ─┬─ størrelse (vCPU, RAM)          f.eks. Standard_B2s, Standard_D2s_v5
    ├─ image (OS)                      Ubuntu 24.04, Windows Server 2025 ...
    ├─ OS-disk (managed disk)          Standard HDD / Standard SSD / Premium SSD
    ├─ datadisker (valgfritt)
    ├─ nettverkskort (NIC) → subnett   privat IP, ev. NSG
    ├─ offentlig IP (valgfritt)
    └─ identitet, extensions, boot diagnostics, sikkerhetstype (Trusted Launch)
```

VM-størrelser – familier:

Serie	Profil	Typisk bruk
B	*Burstable* – billig, bygger opp CPU-kreditt	Test, små webservere, lav jevn last
D	Generell bruk, balansert	De fleste applikasjoner
E	Mye minne	Databaser, cache
F	Mye CPU	Beregninger, batch
L	Lagringsoptimalisert	Store, raske lokale disker
N (NC, ND, NV)	GPU	AI, grafikk (kapittel 14)

Navnet `Standard_D4s_v5` leses: familie **D**, **4** vCPU, **s** = støtter Premium SSD, versjon **5**.

Disker: Standard HDD (billigst, treg) → Standard SSD → Premium SSD → Premium SSD v2 / Ultra Disk (raskest). Diskene er **managed disks** – Azure håndterer lagringen og replikerer dem.

Tilgjengelighet for VM-er:

Alternativ	Beskytter mot
Én VM med Premium SSD	Ingenting utover maskinvarefeil-flytting – lavest SLA
Availability set	Feil på ett rack/én vert (fault domains) og planlagt vedlikehold (update domains) i samme datasenter
Availability zones	Feil på et helt datasenter
Virtual Machine Scale Sets (VMSS)	Automatisk skalering av mange like VM-er, ofte over flere soner

Opprett en Linux-VM med nginx:

```
        az vm create -g $RG -n vm-web01 \
          --image Ubuntu2404 \
          --size Standard_B1s \
          --vnet-name vnet-lab --subnet snet-web \
          --admin-username azureuser \
          --generate-ssh-keys \
          --public-ip-sku Standard \
          --nsg ""
        # --image Ubuntu2404: alias for Ubuntu Server 24.04 LTS (list aliaser med: az vm
image list -o table)
        # --generate-ssh-keys: bruker ~/.ssh/id_rsa.pub eller lager et nytt nøkkelpar
        # --nsg "": ikke lag egen NSG på NIC – vi bruker NSG-en på subnettet
        # Resultatet viser publicIpAddress
```

Installer nginx uten å logge inn (Run Command):

```
az vm run-command invoke -g $RG -n vm-web01 --command-id RunShellScript \
  --scripts "sudo apt-get update && sudo apt-get install -y nginx"

curl http://<publicIpAddress>          # Welcome to nginx!
```

Alternativt kan du gi **cloud-init** ved opprettelse (`--custom-data cloud-init.yaml`) – en fil som kjøres ved første oppstart:

```
#cloud-config
package_update: true
packages:
  - nginx
runcmd:
  - echo "Hei fra $(hostname)" > /var/www/html/index.html
```

Daglig drift:

```
az vm stop -g $RG -n vm-web01          # stopper OS – du BETALER FORTSATT for
compute!
az vm deallocate -g $RG -n vm-web01    # frigjør maskinvaren – compute koster ikke
az vm start -g $RG -n vm-web01
az vm resize -g $RG -n vm-web01 --size Standard_B2s
az vm auto-shutdown -g $RG -n vm-web01 --time 1900   # slå av kl. 19:00 UTC hver
dag
az vm list -d -g $RG -o table          # -d viser IP og strømstatus
```

> **Vanlig feil:** *Stopped* vs. *Stopped (deallocated)*. Slår du av VM-en fra innsiden av OS-et (`sudo shutdown`), står den som «Stopped» og **koster fortsatt**. Bruk **Stop** i portalen eller `az vm deallocate` .

Sikker innlogging med Azure Bastion. Bastion gir SSH/RDP til VM-er via portalen eller CLI **uten offentlig IP på VM-en**. Det finnes flere SKU-er; *Developer*-SKU er gratis, men har begrensninger og er ikke tilgjengelig i alle regioner. Basic/Standard faktureres per time. Alternativt: **Just-In-Time VM access** i Microsoft Defender for Cloud åpner porten bare midlertidig.

Sikkerhet:

- SSH-nøkler, ikke passord. Ingen åpen 22 mot internett.
- **Trusted Launch** (Secure Boot + vTPM) er standard for nye VM-er der det støttes.
- Hold OS-et oppdatert – **Azure Update Manager** kan planlegge patching.
- Diskkryptering er på som standard (server-side encryption med plattformnøkler).
- Gi VM-en en **managed identity** i stedet for å lagre nøkler på den.

Kostnad. Du betaler per sekund VM-en er allokert (størrelse + eventuell OS-lisens), pluss disker (alltid, også når VM-en er deallokert), offentlig IP, og utgående trafikk. Azure free account har inkludert begrenset bruk av enkelte små B-serie-VM-er de første 12 månedene – sjekk gjeldende liste. **B1s** er billig, men har bare 1 GiB RAM.

Øvelse. Opprett `vm-web01` som over, installer nginx, endre `index.html`, deallokér VM-en, sjekk i *Cost analysis* dagen etter hva den kostet. Slett ressursgruppen når du er ferdig.

Sammenheng. VM-er lever i VNet/subnett, beskyttes av NSG, får trafikk fra Load Balancer/Application Gateway, overvåkes av Azure Monitor (via Azure Monitor Agent), sikkerhetskopieres av Azure Backup, og bygges ofte med Terraform/Bicep.

Azure Load Balancer

Hva den gjør. En **lag 4**-lastbalanserer (TCP/UDP) som fordeler trafikk til et **backend pool** av VM-er eller VMSS-instanser, basert på **helsesjekker** (*health probes*).

Typer:

- **Public** – har offentlig IP, tar trafikk fra internett.
- **Internal** – har privat IP, fordeler trafikk inne i VNet-et (f.eks. til app-laget).

SKU-er: Bruk **Standard**. Basic-SKU ble pensjonert i september 2025. Standard er sikker som standard (*secure by default*): trafikk må eksplisitt tillates med NSG, og den støtter tilgjengelighetssoner.

Bestanddeler:

```
Frontend IP ──► Load balancing rule (port 80 → 80) ──► Backend pool [vm1, vm2]
                          │
                  Health probe (HTTP GET / på port 80 hvert 5. s)
(+ inbound NAT rules for å nå enkelt-VM-er, + outbound rules for utgående trafikk)
```

Opprett:

```
az network public-ip create -g $RG -n pip-lb --sku Standard --zone 1 2 3

az network lb create -g $RG -n lb-web --sku Standard \
  --public-ip-address pip-lb --frontend-ip-name fe-web --backend-pool-name be-web

az network lb probe create -g $RG --lb-name lb-web -n probe-http \
  --protocol Http --port 80 --path /

az network lb rule create -g $RG --lb-name lb-web -n rule-http \
  --protocol Tcp --frontend-port 80 --backend-port 80 \
  --frontend-ip-name fe-web --backend-pool-name be-web --probe-name probe-http

# Legg VM-ens nettverkskort i backend pool (az vm create kaller NIC-en <vm>VMNic
# og IP-konfigurasjonen ipconfig<vm>)
az network nic ip-config address-pool add -g $RG \
  --nic-name vm-web01VMNic --ip-config-name ipconfigvm-web01 \
  --lb-name lb-web --address-pool be-web
```

Bedriftseksempel. Tre VM-er med et eldre Java-system i tre soner bak en Standard Load Balancer. Når én sone har problemer, slutter helsesjekken å svare, og trafikken går til de to andre.

Sikkerhet og kostnad. Standard Load Balancer faktureres per regel og per behandlet datamengde. NSG på backend må tillate både klienttrafikken og `AzureLoadBalancer` (helsesjekkene, tillatt i standardreglene).

Øvelse. Lag to VM-er (`vm-web01`, `vm-web02`) med hver sin `index.html` som viser maskinnavnet. Legg begge i backend-poolen og kjør `curl http://<lb-ip>` flere ganger. Stopp nginx på én av dem – hva skjer?

Application Gateway

Hva den gjør. En **lag 7**-lastbalanserer for HTTP/HTTPS med:

- **URL-basert ruting** (`/api/*` → API-servere, `/bilder/*` → bildeservere),
- **flere nettsteder** på samme gateway (*multi-site*, basert på vertsnavn),
- **TLS-terminering** og ende-til-ende-TLS,
- **Web Application Firewall (WAF)** mot vanlige angrep (OWASP-regler),
- autoskalering, tilgjengelighetssoner, omskriving av headers og omdirigering (http → https).

Bestanddeler: frontend-IP → **listener** (port/protokoll/vertsnavn/sertifikat) → **routing rule** → **back-end pool** + **backend settings** (port, protokoll, timeout, helsesjekk).

Hvorfor velge Application Gateway over Load Balancer? Når du trenger å forstå HTTP: ruting på sti/vertsnavn, TLS-håndtering eller WAF. For ren TCP/UDP-trafikk holder Load Balancer.

Relaterte tjenester:

Tjeneste	Lag	Rekkevidde	Typisk bruk
Load Balancer	4	Regional	TCP/UDP til VM-er
Application Gateway	7	Regional	Webapper i ett VNet, WAF
Azure Front Door	7	**Global** (Microsofts kantnett)	Globale nettsteder, CDN, WAF, failover mellom regioner
Traffic Manager	DNS	Global	DNS-basert fordeling mellom regioner

Opprett (forenklet – krever eget subnett):

```
        az network vnet subnet create -g $RG --vnet-name vnet-lab -n snet-agw --address-
prefixes 10.0.0.0/24
        az network public-ip create -g $RG -n pip-agw --sku Standard

        az network application-gateway create -g $RG -n agw-web \
          --sku Standard_v2 --capacity 1 \
          --vnet-name vnet-lab --subnet snet-agw \
          --public-ip-address pip-agw \
          --frontend-port 80 --http-settings-port 80 --http-settings-protocol Http \
          --servers 10.0.1.4 10.0.1.5 \
          --priority 100
```

(`--servers` er de private IP-ene til VM-ene. Opprettelsen tar ofte flere minutter.) For læring er **portalen** ofte enklere her: *Application gateways → + Create*, og veiviseren går gjennom frontend, backend og routing rule.

Sikkerhet og kostnad. v2-SKU-ene faktureres per time pluss kapasitetsenheter. **WAF_v2** koster mer. Dette er en av ressursene studenter ofte glemmer – slett den etter øvelsen.

> **Vanlig feil: 502 Bad Gateway** fra Application Gateway betyr nesten alltid at backend er usunn: appen kjører ikke, NSG blokkerer, feil port i backend settings, eller helsesjekken går mot en sti som ikke returnerer 200–399. Se *Backend health* i portalen.

Azure DNS

Hva den gjør. Azure DNS hoster **DNS-soner** (kapittel 2) i Azure, slik at du administrerer DNS-poster med samme verktøy, RBAC og automatisering som resten av infrastrukturen.

To typer:

- **Public DNS zones** – for domener på internett (`firma.no`). Du kjøper domenet hos en registrar og peker NS-postene til Azures navneservere.
- **Private DNS zones** – navneoppslag **inne i VNets** (`intern.firma.no`, eller `privatelink.database.windows.net` for private endepunkter). Knyttes til VNets med *virtual network links*, og kan registrere VM-navn automatisk.

Opprett:

```
        az network dns zone create -g $RG -n lab-eksempel.no
        az network dns record-set a add-record -g $RG -z lab-eksempel.no -n www -a
<offentlig-ip>
        az network dns record-set cname set-record -g $RG -z lab-eksempel.no -n app -c
minapp.azurewebsites.net
        az network dns zone show -g $RG -n lab-eksempel.no --query nameServers   # legg
disse inn hos registraren

        # Privat sone for et VNet
        az network private-dns zone create -g $RG -n intern.lab
        az network private-dns link vnet create -g $RG -z intern.lab -n link-lab \
          --virtual-network vnet-lab --registration-enabled true
```

Alias-poster kan peke direkte på en Azure-ressurs (offentlig IP, Front Door, Traffic Manager), og oppdateres automatisk hvis IP-en endres. De kan også brukes på **sone-toppen** (`firma.no` uten `www`), der vanlig CNAME ikke er lov.

Kostnad. Lav: per sone per måned og per million spørringer.

Sammenheng. Private DNS zones er **avgjørende** for Private Endpoints: når en app slår opp `minserver.database.windows.net`, må svaret bli den private IP-en, ellers går trafikken mot den offentlige adressen.

Azure App Service

Hva den gjør. App Service er en **PaaS for webapplikasjoner og API-er**. Du gir koden (Python, Node.js, .NET, Java, PHP) eller en container, og Azure drifter servere, OS, patching, lastbalansering og skalering.

Hvorfor. Mye mindre drift enn VM-er, innebygd HTTPS, egne domener, autoskalering, *deployment slots*, integrasjon med GitHub Actions og Azure DevOps, og innebygd autentisering («Easy Auth»).

Bestanddeler:

```
App Service Plan (maskinene – du betaler for planen)
├── Web App 1   (minapp.azurewebsites.net)
├── Web App 2
└── ...   ← flere apper kan dele samme plan og dermed samme kapasitet
```

App Service Plan-nivåer (forenklet):

Nivå	Typisk bruk	Merk
Free (F1) / Shared (D1)	Læring	Begrenset CPU-tid, ingen SLA, ingen egendefinerte TLS-sertifikater / slots
Basic (B1–B3)	Små apper, test	Egne domener, manuell skalering
Standard (S1–S3)	Produksjon	Autoskalering, deployment slots

Nivå	Typisk bruk	Merk
Premium v3 / v4 (P*v3/v4)	Krevende produksjon	Raskere maskiner, flere slots, bedre skalering
Isolated v2 (App Service Environment)	Strenge krav om isolasjon	Dedikert miljø i eget VNet

(Sjekk gjeldende nivåer og funksjoner på Microsoft Learn – nye generasjoner introduseres jevnlig.)

Opprett og deploy en Python-app:

```
        # Enkel Flask-app lokalt:
        mkdir flaskapp && cd flaskapp
        cat > app.py <<'EOF'
        from flask import Flask
        app = Flask(__name__)

        @app.route("/")
        def hjem():
            return "Hei fra Azure App Service!"

        @app.route("/health")
        def health():
            return {"status": "ok"}
        EOF
        echo "flask" > requirements.txt

        # Én kommando: lager plan, web app og deployer koden
        az webapp up --name app-lab-<unik> --resource-group $RG --location $LOC \
          --runtime "PYTHON:3.12" --sku F1

        curl https://app-lab-<unik>.azurewebsites.net
```

(`az webapp list-runtimes --os linux` viser gyldige runtime-verdier.)

Konfigurasjon og drift:

```
        az webapp config appsettings set -g $RG -n app-lab-<unik> --settings APP_ENV=test
# blir miljøvariabler
        az webapp log config -g $RG -n app-lab-<unik> --application-logging filesystem --
docker-container-logging filesystem
        az webapp log tail -g $RG -n app-lab-<unik>      # live logger
        az webapp restart -g $RG -n app-lab-<unik>
```

Deployment slots (Standard og høyere): du deployer til en `staging`-slot, tester den på `minapp-staging.azurewebsites.net`, og gjør en **swap** – produksjon og staging bytter plass nesten uten nedetid. Går noe galt, swapper du tilbake. Dette er blue-green deployment (kapittel 12).

```
        az webapp deployment slot create -g $RG -n app-lab-<unik> --slot staging
        az webapp deployment slot swap -g $RG -n app-lab-<unik> --slot staging --target-
slot production
```

Hemmeligheter: Bruk **Key Vault-referanser** i app settings, kombinert med managed identity:

```
DB_PASSWORD = @Microsoft.KeyVault(VaultName=kv-lab-<unik>;SecretName=DbPassword)
```

Sikkerhet: Slå på *HTTPS Only*, minimum TLS 1.2, managed identity, og vurder *Access restrictions* og Private Endpoints. **Kostnad:** du betaler for **planen**, ikke per app. F1 er gratis med begrensninger.

Øvelse. Deploy Flask-appen, sett app setting `APP_ENV`, og endre appen til å vise verdien (`os.environ.get("APP_ENV")`). Se loggene med `az webapp log tail`.

Sammenheng. Kode fra GitHub → GitHub Actions/Azure Pipelines → App Service. Henter hemmeligheter fra Key Vault via managed identity. Snakker med Azure SQL via VNet integration + Private Endpoint. Overvåkes med Application Insights.

Azure Functions

Hva den gjør. Azure Functions er **serverless** kode som kjører når en **trigger** skjer: HTTP-kall, en tidsplan, en fil i Blob Storage, en melding i en kø, en hendelse i Event Grid. Du skriver bare funksjonen.

Hvorfor. For små, hendelsesdrevne oppgaver er det svært effektivt: ingen servere, automatisk skalering (også til null), og du betaler for faktisk bruk på serverless-planene.

Begreper:

- **Trigger** – hva som starter funksjonen (nøyaktig én per funksjon).
- **Bindings** – deklarative koblinger til data inn/ut (les fra en kø, skriv til Blob) uten egen tilkoblingskode.
- **Function App** – beholderen for én eller flere funksjoner, med felles konfigurasjon.

Hosting-planer (forenklet):

Plan	Egenskaper
Flex Consumption	Serverless, skalerer til null, raskere skalering og VNet-støtte – Microsofts anbefalte serverless-plan for nye apper
Consumption (klassisk)	Serverless; Microsoft har kunngjort utfasing av Linux-varianten – sjekk Learn
Premium	Forhåndsoppvarmede instanser (ingen kaldstart), VNet, lengre kjøretid
Dedicated (App Service Plan)	Kjører på en App Service Plan du allerede betaler for
Container Apps	Functions i Azure Container Apps-miljø

Eksempel – HTTP-funksjon i Python (programmeringsmodell v2):

```
        # function_app.py
        import azure.functions as func

        app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)

        @app.route(route="hei")                                    # HTTP-trigger på
/api/hei
        def hei(req: func.HttpRequest) -> func.HttpResponse:
            navn = req.params.get("navn", "verden")                # les ?navn=... fra URL-
en
            return func.HttpResponse(f"Hei, {navn}!", status_code=200)

        @app.timer_trigger(schedule="0 0 6 * * *", arg_name="timer")   # hver dag 06:00
UTC (6 felt: sek min time ...)
        def morgenjobb(timer: func.TimerRequest) -> None:
            print("God morgen! Her kunne vi ryddet opp i gamle ressurser.")
```

Lokalt og deploy (krever Azure Functions Core Tools):

```
        func init funcapp --python && cd funcapp     # lag prosjekt
        # lim inn koden over i function_app.py
        func start                                   # kjør lokalt:
http://localhost:7071/api/hei?navn=Kari

        az storage account create -n stfunc<unik> -g $RG -l $LOC --sku Standard_LRS
        az functionapp create -g $RG -n func-lab-<unik> --storage-account stfunc<unik> \
          --flex-consumption-location <region-som-støtter-flex> --runtime python --
runtime-version 3.12
        func azure functionapp publish func-lab-<unik>
```

(`az functionapp list-flexconsumption-locations -o table` viser hvilke regioner som støtter Flex Consumption.)

> **Vanlig feil:** Timer-triggere i Functions bruker **seks felt** (med sekunder først), i motsetning til vanlig cron med fem. `0 */5 * * * *` = hvert 5. minutt.

Sikkerhet og kostnad. Serverless-planene har en månedlig gratiskvote for kjøringer og ressursbruk; over det betales per bruk. Function App trenger en lagringskonto (koster litt). Bruk managed identity mot andre tjenester og *function keys* eller Entra-autentisering for HTTP-funksjoner som ikke skal være åpne.

Bedriftseksempel. Når en kunde laster opp en PDF-faktura til Blob Storage, starter en Function som leser fakturaen, lagrer data i databasen og legger en melding i en kø til regnskapssystemet.

Azure Container Registry (ACR)

Hva den gjør. Et **privat register for container-images** (og andre OCI-artefakter som Helm-charts). Som Docker Hub, men privat, i din Azure-tenant, og integrert med Entra ID.

Hvorfor. Produksjonsimages skal ikke ligge åpent. ACR gir tilgangsstyring, nærhet til AKS/App Service (raskere nedlasting), geo-replikering og sårbarhetsskanning (via Microsoft Defender for Containers).

SKU-er: Basic (læring/små), Standard, Premium (geo-replikering, private endepunkter, mer lagring og ytelse).

Opprett og bruk:

```
        az acr create -g $RG -n acrlab<unik> --sku Basic       # navnet må være globalt
unikt, kun bokstaver/tall
        az acr login -n acrlab<unik>                          # logger Docker inn i
registeret

        docker build -t acrlab<unik>.azurecr.io/flaskapp:v1 .
        docker push acrlab<unik>.azurecr.io/flaskapp:v1

        # Alternativ: bygg i skyen uten lokal Docker (ACR Tasks)
        az acr build -r acrlab<unik> -t flaskapp:v1 .

        az acr repository list -n acrlab<unik> -o table
        az acr repository show-tags -n acrlab<unik> --repository flaskapp -o table
```

> **Merk:** ACR Tasks (`az acr build`) kan være begrenset på enkelte gratis- og studentabonnementer. Får du en feilmelding om dette, bygg lokalt med Docker og bruk `docker push` .

Sikkerhet. Slå **av** admin-brukeren (standard). Gi AKS/App Service tilgang med managed identity og rollen **AcrPull**; gi CI/CD-pipelinen **AcrPush**. **Kostnad:** fast pris per dag per SKU, pluss lagring utover inkludert mengde.

Sammenheng. CI bygger image → push til ACR → App Service / Container Apps / AKS henter derfra.

Azure Kubernetes Service (AKS)

Hva den gjør. AKS er Azures **administrerte Kubernetes**. Microsoft drifter kontrollplanet (API-server, etcd, scheduler); du administrerer nodene (VM-er) og arbeidslastene. Kubernetes forklares grundig i kapittel 10 – her ser vi på Azure-siden.

Hvorfor. Kubernetes er standarden for å kjøre mange containere i produksjon med selvreparasjon, skalering og rullerende oppdateringer. Å drifte kontrollplanet selv er krevende; AKS tar den jobben.

Bestanddeler:

```
AKS-cluster
 ├── Kontrollplan (administrert av Microsoft; Free/Standard/Premium-tier)
 ├── System node pool (VM-er for systemkomponenter, f.eks. CoreDNS)
 ├── User node pool(s) (VM-er for dine apper; kan skalere, kan være spot/GPU)
 ├── Nettverk: Azure CNI (Overlay) / kubenet (eldre), i et VNet
 ├── Identitet: managed identity for clusteret, Entra ID-innlogging, Workload Identity for
pods
 └── Integrasjoner: ACR, Key Vault (CSI-driver), Azure Monitor (Container Insights), Azure
Policy
```

Opprett (minimalt lab-cluster):

```
        az aks create -g $RG -n aks-lab \
          --tier free \
          --node-count 1 --node-vm-size Standard_B2s \
          --generate-ssh-keys \
          --attach-acr acrlab<unik>
        # --attach-acr gir clusterets identitet AcrPull på registeret

        az aks get-credentials -g $RG -n aks-lab        # skriver kubeconfig for kubectl
        kubectl get nodes
```

(`Standard_B2s` holder for læring; produksjon bruker typisk D-serien og minst tre noder spredt over soner. Velg en størrelse som er tilgjengelig i regionen og innenfor kvoten din: `az vm list-skus -l $LOC --resource-type virtualMachines -o table`.)

AKS Automatic er en variant der Microsoft også håndterer mye av nodehåndteringen, skalering og sikkerhetsinnstillinger med anbefalte standarder – et godt valg for team som vil ha Kubernetes med mindre drift.

Sikkerhet: Entra ID-integrasjon og Kubernetes RBAC, Workload Identity for pods, nettverkspolicyer, private clustere, Azure Policy for Kubernetes, oppdater Kubernetes-versjonen jevnlig (Kubernetes-versjoner har begrenset støttetid i AKS).

Kostnad. Kontrollplanet er gratis i *Free*-tier (uten SLA); *Standard*-tier koster per cluster-time og gir SLA. **Nodene er vanlige VM-er** og faktureres som det – det er der kostnaden ligger. Stopp clusteret når du ikke bruker det:

```
        az aks stop -g $RG -n aks-lab      # stopper nodene (og kontrollplanet)
        az aks start -g $RG -n aks-lab
```

Alternativer som ofte er bedre for små team: Azure Container Apps (serverless containere basert på Kubernetes-teknologi, uten at du ser Kubernetes; skalerer til null) og **App Service for containere**. Velg AKS når du trenger full Kubernetes.

Oppsummering av 7.2

- **VNet** er ditt private nettverk; **subnett** deler det; **NSG** filtrerer trafikk etter prioritet.
- **VM** = IaaS med full kontroll. Husk *deallocate* for å slutte å betale for compute.

- **Load Balancer** (lag 4) og **Application Gateway** (lag 7, WAF) fordeler trafikk; **Front Door** er global.
- **Azure DNS** hoster offentlige og private soner; private soner er nødvendige for private endepunkter.
- **App Service** (PaaS for web) og **Functions** (serverless) gir mye mindre drift enn VM-er.
- **ACR** lagrer container-images; **AKS** kjører dem med Kubernetes.

Opprydding:

```
az group delete -n rg-lab-nett --yes --no-wait
```

7.3 Lagring, data, sikkerhet og drift

```
RG=rg-lab-data
LOC=norwayeast
az group create -n $RG -l $LOC --tags miljø=lab eier=student
```

Azure Storage Accounts

Hva den gjør. En lagringskonto er en **beholder for Azures lagringstjenester**, med ett felles navn, ett endepunkt-sett og felles innstillinger for sikkerhet og redundans. En konto kan inneholde:

Tjeneste	Hva	Endepunkt
Blob Storage	Objekter/filer (bilder, video, backup, logger, data)	`https://<konto>.blob.core.windows.net`
Azure Files	Delte filområder (SMB/NFS)	`https://<konto>.file.core.windows.net`
Queue Storage	Enkle meldingskøer	`...queue.core.windows.net`
Table Storage	Enkel NoSQL nøkkel–verdi	`...table.core.windows.net`

Hvorfor. Nesten alle løsninger trenger et sted å lagre filer billig og trygt – og mange Azure-tjenester (Functions, diagnostikklogger, Terraform-state) bruker lagringskontoer under panseret.

Viktige valg ved opprettelse:

1. **Navn** – 3–24 tegn, kun små bokstaver og tall, **globalt unikt** (det blir en del av en URL).
2. **Ytelse** – *Standard* (generell bruk) eller *Premium* (lav forsinkelse, SSD).
3. **Kontotype** – *StorageV2 (general purpose v2)* er standard.
4. **Redundans** – hvor mange kopier, og hvor:

```
LRS   3 kopier i ett datasenter                          ┌DC1┐
ZRS   3 kopier fordelt på 3 tilgjengelighetssoner         ┌Z1┐ ┌Z2┐ ┌Z3┐
GRS   LRS i primærregion + LRS i paret region (asynkront) ┌DC1┐ ⟹ ┌DC (region 2)┐
GZRS  ZRS i primærregion + LRS i paret region
RA-GRS / RA-GZRS: som GRS/GZRS, men du kan LESE fra sekundærregionen hele tiden
```

Redundans	Beskytter mot	Relativ pris
LRS	Disk-/rackfeil	Lavest
ZRS	Tap av et datasenter (sone)	Høyere
GRS / RA-GRS	Tap av en hel region	Høyere
GZRS / RA-GZRS	Sone- og regiontap	Høyest

> **Merk:** Geo-redundans krever en paret region. Norway East er paret med Norway West. Sjekk på Microsoft Learn hvilke redundansvalg som er tilgjengelige i regionen du bruker.

Opprett en sikker lagringskonto:

```
ST=stlab$RANDOM          # tilfeldig suffiks for unikt navn
az storage account create -n $ST -g $RG -l $LOC \
  --sku Standard_LRS --kind StorageV2 \
  --min-tls-version TLS1_2 \
  --allow-blob-public-access false \
  --https-only true
```

Tilgang til data – fire metoder:

Metode	Hva	Anbefaling
Entra ID + RBAC	Innlogging med identitet og data-roller (*Storage Blob Data Reader/Contributor*)	**Best** – bruk dette, særlig med managed identity
SAS-token (*Shared Access Signature*)	URL med tidsbegrenset, avgrenset tilgang	Bra for midlertidig deling; foretrekk *user delegation SAS*
Tilgangsnøkler (*account keys*)	To hovednøkler med full tilgang til alt	Unngå; kan slås av (`--allow-shared-key-access false`)
Anonym offentlig lesetilgang	Alle kan lese	Kun for helt offentlig innhold; av som standard

Nettverk: Lagringskontoer kan begrenses med brannmur (bare valgte VNets/IP-er) eller nås kun via **Private Endpoint**.

Datasikkerhet: Kryptert i ro som standard (Microsoft-administrerte nøkler, eventuelt kundeadministrerte i Key Vault). **Soft delete** for blobs og containere, **versjonering**, og **immutable storage**

(WORM – kan ikke endres eller slettes i en periode) beskytter mot sletting og løsepengevirus.

Kostnad. Betal for: lagret mengde (per GB/måned, avhengig av nivå og redundans), operasjoner (per 10 000 transaksjoner), datauthenting fra kalde nivåer, og utgående trafikk.

Blob Storage

Hva den gjør. Objektlagring for **ustrukturerte data** i ubegrensede mengder. Organisert som:

```
Lagringskonto (stlab12345)
└── Container (bilder)                    ← som en mappe på toppnivå
     ├── blob: logo.png                   ← filen
     ├── blob: 2026/10/kvittering-001.pdf ← «/» i navnet gir virtuelle mapper
     └── ...
```

Blob-typer: *Block blobs* (vanlige filer – de fleste tilfeller), *Append blobs* (logger som det bare legges til i), *Page blobs* (virtuelle disker).

Tilgangsnivåer (*access tiers*):

Nivå	For data som …	Lagring	Lesing
Hot	brukes ofte	Dyrest	Billigst
Cool	brukes sjelden (min. 30 dager)	Billigere	Dyrere
Cold	brukes svært sjelden (min. 90 dager)	Enda billigere	Enda dyrere
Archive	nesten aldri (min. 180 dager)	Billigst	Dyrest; må *rehydreres* (timer) før lesing

Livssyklusregler (*lifecycle management*) flytter eller sletter data automatisk:

```json
{
  "rules": [{
    "name": "flytt-gamle-logger",
    "enabled": true,
    "type": "Lifecycle",
    "definition": {
      "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["logger/"] },
      "actions": {
        "baseBlob": {
          "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
          "tierToArchive": { "daysAfterModificationGreaterThan": 180 },
          "delete":        { "daysAfterModificationGreaterThan": 365 }
        }
      }
    }
  }]
}
```

```
        az storage account management-policy create --account-name $ST -g $RG --policy
@policy.json
```

Last opp og ned med Entra-innlogging:

```
        # Gi deg selv data-tilgang (kontrollplan-tilgang som Owner er IKKE nok for --auth-
mode login)
        ME=$(az ad signed-in-user show --query id -o tsv)
        az role assignment create --assignee-object-id $ME --assignee-principal-type User
\
          --role "Storage Blob Data Contributor" \
          --scope $(az storage account show -n $ST -g $RG --query id -o tsv)

        az storage container create --account-name $ST -n bilder --auth-mode login
        echo "hei" > hei.txt
        az storage blob upload --account-name $ST -c bilder -f hei.txt -n hei.txt --auth-
mode login
        az storage blob list --account-name $ST -c bilder --auth-mode login -o table
        az storage blob download --account-name $ST -c bilder -n hei.txt -f ned.txt --
auth-mode login
```

AzCopy er et kommandolinjeverktøy for å kopiere store datamengder raskt (`azcopy copy ./mappe "https://$ST.blob.core.windows.net/bilder" --recursive`). **Azure Storage Explorer** er et gratis grafisk program.

Statisk nettside. Blob Storage kan hoste en statisk nettside (HTML/CSS/JS) direkte fra containeren `$web`:

```
        az storage blob service-properties update --account-name $ST --static-website \
          --index-document index.html --404-document 404.html
        az storage blob upload-batch --account-name $ST -s ./site -d '$web' --auth-mode
login
        az storage account show -n $ST -g $RG --query "primaryEndpoints.web" -o tsv
```

(For moderne frontend-apper med API og CI/CD er **Azure Static Web Apps** et godt alternativ, med en gratis plan.)

Bedriftseksempel. En nettavis lagrer millioner av bilder i Blob Storage (Hot), serverer dem via Azure Front Door (CDN), og flytter bilder eldre enn ett år til Cool med livssyklusregler.

Øvelse. Lag lagringskontoen, last opp en fil med Entra-innlogging, lag en *user delegation SAS* med 1 times gyldighet (`az storage blob generate-sas --as-user --auth-mode login --permissions r --expiry <tid> --full-uri ...`), og åpne URL-en i nettleseren.

Azure Files

Hva den gjør. Fullt administrerte **filområder** (*file shares*) som kan monteres over **SMB** (Windows, Linux, macOS) eller **NFS** (Linux, krever Premium). Som en nettverksdisk på kontoret – i skyen.

Hvorfor. «Lift and shift» av apper som forventer en delt mappe, delte konfigurasjonsfiler mellom flere VM-er, erstatning for lokale filservere, og vedvarende lagring for containere.

Azure File Sync synkroniserer lokale Windows-filservere med Azure Files, med *cloud tiering* (sjeldent brukte filer ligger bare i skyen).

```
        az storage share-rm create -g $RG --storage-account $ST -n delt --quota 100   # 100 GiB
```

Monter fra Linux (SMB): portalen gir deg et ferdig skript under *File share → Connect*. Merk at SMB bruker **port 445**, som mange internettleverandører og bedriftsnett blokkerer utgående – fra hjemmenett kan monteringen derfor feile; fra en Azure-VM virker det.

Sikkerhet og kostnad. Identitetsbasert tilgang med AD DS / Entra Kerberos for SMB, Private Endpoint, snapshots og Azure Backup. Betal per GiB (Standard: per brukt mengde; Premium: per *provisjonert* størrelse) pluss transaksjoner. Sjekk den nyere **provisioned v2**-faktureringsmodellen på Learn.

Azure SQL Database

Hva den gjør. En **fullt administrert relasjonsdatabase** (PaaS) basert på Microsoft SQL Server-motoren. Microsoft håndterer patching, backup, høy tilgjengelighet og oppgraderinger.

Familien Azure SQL:

Tjeneste	Modell	Bruk
Azure SQL Database	PaaS, én database (eller elastic pool)	Nye skyapplikasjoner
Azure SQL Managed Instance	PaaS, nesten 100 % kompatibel med SQL Server	Flytting av eksisterende SQL Server-løsninger
SQL Server på Azure VM	IaaS	Full kontroll, spesielle behov

(For PostgreSQL og MySQL finnes tilsvarende *Azure Database for PostgreSQL / MySQL – Flexible Server*.)

Bestanddeler:

```
Logisk server (sql-lab-123.database.windows.net)  ← innlogging, brannmur, Entra-admin
├── Database: appdb    (egen størrelse/pris)
└── Database: rapportdb
```

Kjøpsmodeller:

- **vCore** (anbefalt): velg antall vCores og nivå: *General Purpose*, *Business Critical* (raskere, flere replikaer), *Hyperscale* (svært stor, rask skalering). **Serverless** compute skalerer automatisk og kan pause når databasen ikke brukes.

- **DTU**: enklere pakker (Basic, Standard, Premium) med samlet ytelsesmål.

Gratis tilbud. Azure SQL Database har et gratistilbud (en månedlig kvote av vCore-sekunder og lagring for et begrenset antall databaser per abonnement) – sjekk gjeldende vilkår på Microsoft Learn («Azure SQL Database free offer»).

Opprett:

```
SQLSRV=sql-lab-$RANDOM
MY_UPN=$(az ad signed-in-user show --query userPrincipalName -o tsv)
MY_OID=$(az ad signed-in-user show --query id -o tsv)

# Logisk server med KUN Entra-autentisering (ingen SQL-passord) – anbefalt
az sql server create -g $RG -n $SQLSRV -l $LOC \
  --enable-ad-only-auth \
  --external-admin-principal-type User \
  --external-admin-name "$MY_UPN" --external-admin-sid "$MY_OID"

# Database med gratistilbudet (pauser når kvoten er brukt opp)
az sql db create -g $RG -s $SQLSRV -n appdb \
  --edition GeneralPurpose --compute-model Serverless --family Gen5 --capacity 2 \
  --use-free-limit --free-limit-exhaustion-behavior AutoPause

# Tillat din egen IP gjennom serverbrannmuren
MYIP=$(curl -s https://api.ipify.org)
az sql server firewall-rule create -g $RG -s $SQLSRV -n min-ip \
  --start-ip-address $MYIP --end-ip-address $MYIP
```

Koble til med **Query editor** i portalen, Azure Data Studio-etterfølgeren **VS Code med mssql-utvidelsen**, eller `sqlcmd`. Lag en tabell:

```
CREATE TABLE kunder (
    id INT IDENTITY(1,1) PRIMARY KEY,   -- automatisk økende ID
    navn NVARCHAR(100) NOT NULL,
    epost NVARCHAR(200) UNIQUE
);
INSERT INTO kunder (navn, epost) VALUES (N'Kari', N'kari@example.com');
SELECT * FROM kunder;
```

La appen logge inn med managed identity (ingen passord): i databasen, kjørt som Entra-admin:

```
CREATE USER [app-lab-123] FROM EXTERNAL PROVIDER;   -- navnet på web-appen/identiteten
ALTER ROLE db_datareader ADD MEMBER [app-lab-123];
ALTER ROLE db_datawriter ADD MEMBER [app-lab-123];
```

Høy tilgjengelighet og gjenoppretting:

- Innebygd HA i regionen; **zone redundancy** som valg.
- **Automatisk backup** med **point-in-time restore** (PITR) – gjenopprett til et hvilket som helst tidspunkt innen oppbevaringsperioden (standard 7 dager, kan økes til 35), og **long-term retention** (LTR) i opptil 10 år.

- **Active geo-replication** og **failover groups** for DR til en annen region.

Sikkerhet: Entra-autentisering, brannmur eller (bedre) **Private Endpoint** og *Deny public network access*, **TDE** (Transparent Data Encryption, på som standard), Always Encrypted for spesielt sensitive kolonner, **Microsoft Defender for SQL**, auditing til Log Analytics.

> **Vanlig feil:** Å slå på «Allow Azure services and resources to access this server». Det tillater trafikk fra **alle** Azure-kunder sine ressurser, ikke bare dine. Bruk Private Endpoint eller VNet-regler i stedet.

Bedriftseksempel. En SaaS-leverandør har én database per kunde i en **elastic pool**, slik at mange små databaser deler kapasitet og kostnad.

Øvelse. Opprett server og database som over, lag tabellen, og gjør en point-in-time restore til en ny database (`az sql db restore ... --time <ISO-tid> --dest-name appdb-restore`). Hvor lang tid tok det?

Azure Key Vault

Hva den gjør. Et sikkert **hvelv** for:

- **Secrets** – passord, tilkoblingsstrenger, API-nøkler.
- **Keys** – kryptografiske nøkler (for kryptering og signering), eventuelt i HSM.
- **Certificates** – TLS-sertifikater, med automatisk fornyelse fra støttede utstedere.

Hvorfor. Hemmeligheter skal ikke ligge i kode, konfigurasjonsfiler eller Git. Key Vault gir sentral lagring, tilgangskontroll, **revisjonslogg** (hvem leste hvilken hemmelighet når), versjonering og rotasjon.

Tilgangsmodell. Bruk **Azure RBAC** (standard for nye hvelv) med roller som:

Rolle	Kan
Key Vault Administrator	Alt på dataplanet
Key Vault Secrets Officer	Administrere hemmeligheter
Key Vault Secrets User	**Lese** hemmeligheter (det appene trenger)
Key Vault Crypto User	Bruke nøkler
Key Vault Certificates Officer	Administrere sertifikater

(Den eldre modellen *access policies* finnes fortsatt, men Microsoft anbefaler RBAC.)

Beskyttelse mot sletting:

- **Soft delete** (alltid på): slettede hvelv og hemmeligheter kan gjenopprettes i oppbevaringsperioden (7–90 dager).
- **Purge protection**: hindrer permanent sletting i oppbevaringsperioden – selv for administratorer. Anbefalt i produksjon.

Opprett og bruk:

```
        KV=kv-lab-$RANDOM          # 3–24 tegn, globalt unikt
        az keyvault create -n $KV -g $RG -l $LOC --enable-rbac-authorization true

        # Gi deg selv rett til å administrere hemmeligheter
        az role assignment create --assignee-object-id $(az ad signed-in-user show --query
id -o tsv) \
          --assignee-principal-type User --role "Key Vault Secrets Officer" \
          --scope $(az keyvault show -n $KV --query id -o tsv)

        az keyvault secret set --vault-name $KV -n DbPassword --value 'Eksempel-Ikke-Ekte-
123!'
        az keyvault secret show --vault-name $KV -n DbPassword --query value -o tsv
        az keyvault secret list --vault-name $KV -o table
```

Fra Python med managed identity / din innlogging:

```
        from azure.identity import DefaultAzureCredential
        from azure.keyvault.secrets import SecretClient   # pip install azure-keyvault-
secrets

        client = SecretClient(vault_url="https://kv-lab-12345.vault.azure.net",
credential=DefaultAzureCredential())
        passord = client.get_secret("DbPassword").value     # ALDRI print/logg verdien i
ekte kode
```

Sammenheng. App Service og Functions (Key Vault-referanser), AKS (Secrets Store CSI Driver), Azure Pipelines (variable groups koblet til Key Vault), GitHub Actions (hent hemmeligheter etter OIDC-innlogging), Terraform (`azurerm_key_vault_secret`), VM-diskkryptering og TLS-sertifikater til Application Gateway.

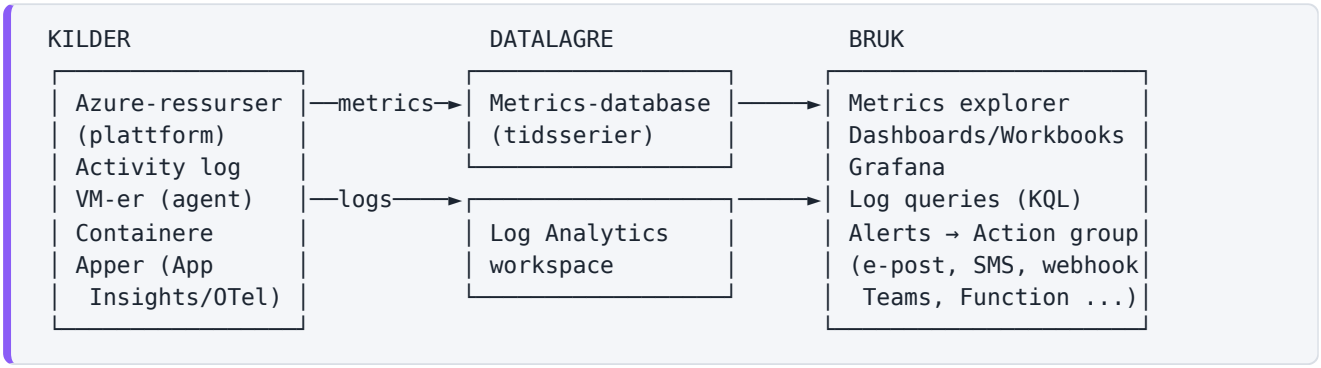
Kostnad. Lav: per 10 000 operasjoner, pluss per sertifikatfornyelse og høyere pris for HSM-nøkler. Det finnes også *Managed HSM* for strenge krav (dyrt).

> **Vanlig feil:** Å kalle Key Vault for hver forespørsel i en app med mye trafikk. Key Vault har **throttling-grenser**. Hent hemmeligheten ved oppstart og cache den (eller bruk Key Vault-referanser i App Service, som cacher for deg).

Azure Monitor

Hva den gjør. Azure Monitor er Azures samlede plattform for **overvåking**: den samler inn, lagrer, analyserer og reagerer på **metrikker** og **logger** fra Azure-ressurser, applikasjoner, VM-er og

containere.



Datatyper:

- **Metrikker** – tall over tid (CPU %, forespørsler/sek, ledig disk). Lette, raske, samles inn automatisk for de fleste ressurser.
- **Logger** – hendelser med detaljer (feilmeldinger, innlogginger, HTTP-forespørsler). Lagres i Log Analytics og søkes med **KQL**.
- **Activity log** – hvem gjorde hva på kontrollplanet (opprettet, slettet, endret ressurser). Samles automatisk, oppbevares i 90 dager.
- **Traces** – sporing av forespørsler gjennom flere tjenester (kapittel 13).

Viktige komponenter:

- **Diagnostic settings** – sender ressurslogger og metrikker til Log Analytics, lagring eller Event Hubs. **Ressurslogger samles ikke inn før du lager en diagnostic setting.**
- **Azure Monitor Agent (AMA)** + **Data Collection Rules (DCR)** – samler logger og ytelsesdata fra VM-er. (Den gamle Log Analytics-agenten ble pensjonert i 2024.)
- **Application Insights** – overvåking av applikasjoner (forespørsler, feil, avhengigheter, ytelse), basert på OpenTelemetry.
- **Container Insights** og **Managed Prometheus** / **Azure Managed Grafana** – for AKS.
- **Alerts** + **Action groups** – varsler når noe skjer, og bestemmer hvem som får beskjed og hva som skjer.
- **Workbooks** og **dashboards** – visualisering.

Lag et CPU-varsel på en VM:

```
        # Action group: hvem skal varsles
        az monitor action-group create -g $RG -n ag-student --short-name agstud \
          --action email student student@example.com

        VM_ID=$(az vm show -g rg-lab-nett -n vm-web01 --query id -o tsv)   # en VM du har
laget
        az monitor metrics alert create -g $RG -n cpu-hoy \
          --scopes "$VM_ID" \
          --condition "avg Percentage CPU > 80" \
          --window-size 5m --evaluation-frequency 1m \
          --action ag-student \
          --description "CPU over 80 % i 5 minutter"
```

Test varselet ved å stresse VM-en: `sudo apt install -y stress-ng && stress-ng --cpu 1 --time-out 600`.

Sikkerhet og kostnad. Plattformmetrikker og activity log er gratis. **Logginnsamling til Log Analytics koster per GB** – den største kostnadsdriveren i overvåking. Varsler koster litt per regel. Samle det du faktisk bruker.

Log Analytics og KQL

Hva den gjør. Et **Log Analytics workspace** er databasen der Azure Monitor lagrer logger. Du spør i den med **KQL** (*Kusto Query Language*).

```
        az monitor log-analytics workspace create -g $RG -n log-lab --retention-time 30

        # Send logger og metrikker fra Key Vault til workspace
        WS_ID=$(az monitor log-analytics workspace show -g $RG -n log-lab --query id -o
tsv)
        az monitor diagnostic-settings create --name diag-kv \
          --resource $(az keyvault show -n $KV --query id -o tsv) \
          --workspace "$WS_ID" \
          --logs '[{"categoryGroup":"allLogs","enabled":true}]' \
          --metrics '[{"category":"AllMetrics","enabled":true}]'
```

KQL – grunnleggende. En spørring starter med en **tabell**, og data sendes gjennom operatorer med `|` (som pipes i Linux):

```
// Siste 50 hendelser i activity log siste døgn
AzureActivity
| where TimeGenerated > ago(1d)
| project TimeGenerated, Caller, OperationNameValue, ActivityStatusValue, ResourceGroup
| order by TimeGenerated desc
| take 50
```

```
// Hvem har lest hemmeligheter i Key Vault?
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.KEYVAULT" and OperationName == "SecretGet"
| summarize antall = count() by CallerIPAddress, identity_claim_upn_s
```

```
// Gjennomsnittlig CPU per VM i 5-minutters intervaller (krever Azure Monitor Agent / VM
insights)
Perf
| where ObjectName == "Processor" and CounterName == "% Processor Time"
| summarize snitt_cpu = avg(CounterValue) by Computer, bin(TimeGenerated, 5m)
| render timechart
```

```
// Antall HTTP 5xx-feil per time fra App Service
AppServiceHTTPLogs
| where ScStatus >= 500
| summarize feil = count() by bin(TimeGenerated, 1h)
| render columnchart
```

KQL-operator	Gjør	SQL-ekvivalent
`where`	Filtrer rader	`WHERE`
`project`	Velg kolonner	`SELECT kol1, kol2`
`extend`	Lag ny kolonne	`SELECT ..., uttrykk AS ny`
`summarize ... by`	Aggreger	`GROUP BY`
`order by` / `sort by`	Sorter	`ORDER BY`
`take` / `limit`	Begrens antall	`TOP` / `LIMIT`
`join`	Koble tabeller	`JOIN`
`render`	Lag diagram	–
`ago(1h)`, `bin()`	Tid relativt til nå, gruppering i tidsintervaller	–

Log alerts kjører en KQL-spørring jevnlig og varsler hvis resultatet oppfyller en betingelse (f.eks. «mer enn 10 mislykkede innlogginger på 5 minutter»).

Sikkerhet og kostnad. Styr tilgang med RBAC (*Log Analytics Reader*). Kostnad per innsamlet GB og for oppbevaring utover inkludert periode; ulike **tabellplaner** (Analytics, Basic, Auxiliary) gir lavere pris for logger du sjelden spør i. Sett en **daglig grense** (*daily cap*) i labmiljøer.

Øvelse. Send Key Vault-logger til workspace, les hemmeligheten fem ganger med `az keyvault secret show`, og finn hendelsene med KQL-spørringen over. (Det kan ta noen minutter før data dukker opp.)

Azure Backup

Hva den gjør. En administrert **backup-tjeneste** for Azure-VM-er, Azure Files, SQL Server og SAP HANA på VM-er, Azure Database for PostgreSQL, blobs, AKS, disker og lokale servere.

Hvelv:

- **Recovery Services vault** – VM-er, Azure Files, SQL/SAP HANA i VM, lokale servere (MARS-agent).
- **Backup vault** – nyere arbeidslaster som Azure Disks, Blobs, PostgreSQL, AKS.

Begreper:

- **Backup policy** – hvor ofte (daglig/ukentlig, eller flere ganger daglig med *Enhanced policy*) og hvor lenge backuper beholdes.
- **Restore point** – et punkt du kan gjenopprette til.
- **Soft delete** – slettede backuper beholdes i en periode (beskytter mot ondsinnet sletting).
- **Immutable vault** og **Multi-user authorization** – ekstra beskyttelse mot at en angriper med admin-tilgang sletter backupene.
- **Cross Region Restore** – gjenopprett i den parede regionen (krever GRS på hvelvet).
- **Azure Site Recovery (ASR)** – noe annet enn backup: **replikerer** VM-er kontinuerlig til en annen region for **katastrofegjenoppretting** med lav RPO/RTO.

Ta backup av en VM:

```
        az backup vault create -g $RG -n rsv-lab -l $LOC
        az backup protection enable-for-vm -g $RG --vault-name rsv-lab \
          --vm $(az vm show -g rg-lab-nett -n vm-web01 --query id -o tsv) \
          --policy-name DefaultPolicy

        # Ta en backup nå (i stedet for å vente på planen)
        az backup protection backup-now -g $RG --vault-name rsv-lab \
          --container-name vm-web01 --item-name vm-web01 --backup-management-type
AzureIaasVM
        az backup job list -g $RG --vault-name rsv-lab -o table
```

Gjenoppretting kan lage en ny VM, erstatte disker, eller hente enkeltfiler (*file recovery*, monterer restore pointet som en disk).

Bedriftseksempel. En regnskapsbedrift tar daglig backup av alle VM-er med 30 dagers oppbevaring, månedlig med 12 måneder, og årlig med 10 år (lovkrav). Hvelvet er *immutable* slik at løsepengevirus ikke kan slette backupene.

Kostnad. Per beskyttet instans (etter størrelse) pluss lagring av backupdata (LRS/ZRS/GRS). **Merk:** Du kan ikke slette et Recovery Services vault som har beskyttede elementer eller soft-deleted backuper – stopp beskyttelsen og slett backupdata først. Det kan gjøre opprydding i lab tregere; se *«Delete a Recovery Services vault»* på Microsoft Learn.

> **Husk:** Test gjenoppretting jevnlig. En backup du aldri har gjenopprettet, er en antakelse – ikke en plan.

Oppsummering av 7.3

- **Storage account** samler Blob, Files, Queue og Table; velg redundans (LRS/ZRS/GRS/GZRS) etter behov.
- **Blob** for objekter med nivåer Hot/Cool/Cold/Archive og livssyklusregler; **Files** for delte mapper over SMB/NFS.
- **Azure SQL Database** er en administrert SQL-database med automatisk backup og PITR; bruk Entra-autentisering og Private Endpoint.
- **Key Vault** lagrer hemmeligheter, nøkler og sertifikater; tilgang med RBAC og managed identity.
- **Azure Monitor** samler metrikker og logger; **Log Analytics** + **KQL** for analyse; **alerts** + **action groups** for varsling.
- **Azure Backup** beskytter data; **Site Recovery** gir DR for hele VM-er.

Opprydding:

```
        # Backup-hvelv må tømmes først:
        az backup protection disable -g $RG --vault-name rsv-lab --container-name vm-web01
\
          --item-name vm-web01 --backup-management-type AzureIaasVM --delete-backup-data
true --yes
        az group delete -n rg-lab-data --yes --no-wait
        # Key Vault blir "soft-deleted" – fjern helt hvis du vil gjenbruke navnet:
        az keyvault purge -n $KV
```

7.4 Azure DevOps, Azure Pipelines og Bicep

Azure DevOps

Hva den gjør. Azure DevOps er Microsofts pakke av utviklingsverktøy (SaaS, `dev.azure.com`) for hele livssyklusen:

Tjeneste	Hva	GitHub-ekvivalent
Azure Boards	Arbeidselementer, backlog, sprinter, kanban	Issues + Projects
Azure Repos	Git-repoer med pull requests og branch policies	Repositories
Azure Pipelines	CI/CD	GitHub Actions
Azure Test Plans	Manuell og utforskende testing	(tredjepart)
Azure Artifacts	Pakkefeeder (NuGet, npm, Maven, Python, Universal)	GitHub Packages

Hvorfor. Mange bedrifter – spesielt i Microsoft-tunge miljøer, offentlig sektor og konsulentbransjen i Norge – har brukt Azure DevOps i mange år. Du vil sannsynligvis møte både Azure DevOps og GitHub i arbeidslivet. Microsoft investerer mest i GitHub fremover, men Azure DevOps er fullt støttet og mye brukt.

Struktur: *Organisasjon* (`dev.azure.com/<org>`) → *Prosjekter* → Boards/Repos/Pipelines/… i hvert prosjekt.

Kom i gang:

1. Gå til `dev.azure.com`, logg inn med Microsoft-/skolekontoen, opprett en organisasjon og et prosjekt (`devops-lab`, *Private*, Git).
2. *Repos* → importer eller push et repo (`git remote add origin https://dev.azure.com/<org>/devops-lab/_git/devops-lab`).
3. *Boards* → lag noen *User Stories* og *Tasks*, og en sprint.
4. Koble commits og PR-er til arbeidselementer med `AB#123` i commit-meldingen (eller `#123` i Azure Repos).

Kostnad. Gratis for et lite antall brukere med *Basic*-tilgang (og ubegrenset antall *Stakeholder*-brukere), pluss en gratis kvote av Microsoft-hostede pipeline-jobber. **Merk:** nye organisasjoner må ofte **søke om** gratis parallelljobb for Microsoft-hostede agenter (et skjema Microsoft lenker til i dokumentasjonen); inntil da kan pipelines stå i kø. Alternativet er en **selvhostet agent** (f.eks. på din egen PC eller en VM), som er gratis å bruke. Sjekk «Azure DevOps pricing» for gjeldende vilkår.

Azure Pipelines

Hva den gjør. CI/CD-tjenesten i Azure DevOps. Pipelines defineres i **YAML** (`azure-pipelines.yml`) i repoet. Kan bygge kode fra Azure Repos, GitHub og andre, og deploye til Azure, andre skyer eller egne servere.

Struktur:

```
Pipeline
├── trigger / pr            (når skal den kjøre)
├── variables
└── stages                  (f.eks. Build → Test → DeployDev → DeployProd)
     └── jobs               (kjører på en agent; parallelle som standard)
          └── steps         (task eller script)
```

Agenter. *Microsoft-hosted* (ny, ren VM for hver jobb: `ubuntu-latest`, `windows-latest`) eller *self-hosted* (din egen maskin, f.eks. inne i et privat nettverk).

Service connections gir pipelinen tilgang til Azure. Anbefalt type: **Azure Resource Manager med workload identity federation** – ingen hemmelighet lagres; Azure DevOps får kortlevde tokens via OIDC.

Environments representerer deploymål (`dev` , `prod`) med **godkjenninger og sjekker** (*approvals and checks*) – f.eks. at en person må godkjenne før produksjon.

Eksempel – bygg, test og deploy en Python-app til App Service:

```yaml
# azure-pipelines.yml
trigger:
  branches:
    include: [main]          # kjør ved push til main
pr:
  branches:
    include: [main]          # og ved pull requests mot main (gjelder GitHub/Bitbucket-repoer;
                             # for Azure Repos brukes branch policies i stedet)

variables:
  pythonVersion: '3.12'
  azureSubscription: 'sc-azure-lab'      # navnet på service connection
  webAppName: 'app-lab-12345'

stages:
- stage: Build
  jobs:
  - job: BuildAndTest
    pool:
      vmImage: ubuntu-latest
    steps:
    - task: UsePythonVersion@0
      inputs:
        versionSpec: $(pythonVersion)
    - script: |
        pip install -r requirements.txt
        pip install pytest
        pytest -q --junitxml=test-results.xml
      displayName: 'Installer og test'
    - task: PublishTestResults@2           # vis testresultater i Azure DevOps
      condition: always()
      inputs:
        testResultsFiles: test-results.xml
    - task: ArchiveFiles@2                 # pakk appen som zip
      inputs:
        rootFolderOrFile: '$(System.DefaultWorkingDirectory)'
        includeRootFolder: false
        archiveFile: '$(Build.ArtifactStagingDirectory)/app.zip'
    - publish: '$(Build.ArtifactStagingDirectory)/app.zip'   # lagre som pipeline-artefakt
      artifact: drop

- stage: DeployProd
  dependsOn: Build
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
  - deployment: Deploy
    environment: prod                      # godkjenninger settes opp på environment i Azure DevOps
    pool:
      vmImage: ubuntu-latest
    strategy:
      runOnce:
        deploy:
          steps:
          - task: AzureWebApp@1
            inputs:
```

```
                azureSubscription: $(azureSubscription)
                appType: webAppLinux
                appName: $(webAppName)
                package: '$(Pipeline.Workspace)/drop/app.zip'
```

Variabler og hemmeligheter. Hemmelige variabler settes i pipelinen eller i **variable groups** (*Library*), som kan kobles direkte til Key Vault. Hemmeligheter maskeres (`***`) i loggene.

Maler (*templates*) lar deg gjenbruke stages, jobs og steps på tvers av mange pipelines – nøkkelen til *Platform Engineering* i Azure DevOps.

Classic vs. YAML. Eldre «classic» pipelines lages i et grafisk grensesnitt (build- og release-pipelines). YAML anbefales: pipelinen versjonskontrolleres sammen med koden.

Bedriftseksempel. En bank har sentrale YAML-maler som alle team må bruke. Malene inkluderer sikkerhetsskanning, testing og godkjenning før produksjon, slik at alle team automatisk følger bankens krav.

Øvelse. Importer Flask-appen fra kapittel 7.2 til Azure Repos, lag en service connection (workload identity federation) til ressursgruppen, og kjør pipelinen over. Legg til en godkjenning på environment `prod`.

Bicep – Azures eget IaC-språk

Hva den gjør. Bicep er et **deklarativt språk** for å beskrive Azure-ressurser. Du beskriver **hva** du vil ha, og Azure Resource Manager finner ut **hvordan**. Bicep kompileres til ARM-maler (JSON), men er mye kortere og lettere å lese.

Hvorfor. Infrastruktur som kode (kapittel 11) gir repeterbarhet, versjonskontroll, kodegjennomgang og automatisering. Bicep er førstevalget hvis du bare bruker Azure: ingen state-fil å administrere (Azure *er* tilstanden), støtte for nye ressurstyper fra dag én, og gratis.

Bicep vs. ARM JSON – samme lagringskonto:

```
ARM JSON (utdrag, ~25 linjer)                    Bicep (8 linjer)
{                                                resource st 'Microsoft.Storage/
  "$schema": "...",                                storageAccounts@2023-05-01' = {
  "resources": [{                                  name: stName
    "type": "Microsoft.Storage/storageAccounts",    location: location
    "apiVersion": "2023-05-01",                     sku: { name: 'Standard_LRS' }
    "name": "[parameters('stName')]",              kind: 'StorageV2'
    "location": "[parameters('location')]",      }
    ...
```

Installer: Azure CLI installerer Bicep automatisk (`az bicep install` / `az bicep upgrade`). Bruk VS Code med **Bicep-utvidelsen** for autofullføring og validering.

Et komplett eksempel – `main.bicep` som lager en sikker lagringskonto, en App Service Plan og en web-app med managed identity:

```
// main.bicep
@description('Miljø: dev eller prod')
@allowed(['dev', 'prod'])
param miljo string = 'dev'

@description('Region for ressursene')
param location string = resourceGroup().location     // standard: samme region som
ressursgruppen

@description('Unikt suffiks for globale navn')
param suffiks string = uniqueString(resourceGroup().id)   // deterministisk «tilfeldig»
streng

var tags = {
  miljo: miljo
  eier: 'student'
  administrertAv: 'bicep'
}
var appSkuName = miljo == 'prod' ? 'P0v3' : 'F1'     // betingelse: større plan i prod

resource st 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: 'st${suffiks}'
  location: location
  tags: tags
  sku: { name: miljo == 'prod' ? 'Standard_ZRS' : 'Standard_LRS' }
  kind: 'StorageV2'
  properties: {
    minimumTlsVersion: 'TLS1_2'
    allowBlobPublicAccess: false
    supportsHttpsTrafficOnly: true
  }
}

resource plan 'Microsoft.Web/serverfarms@2023-12-01' = {
  name: 'plan-${miljo}-${suffiks}'
  location: location
  tags: tags
  sku: { name: appSkuName }
  kind: 'linux'
  properties: { reserved: true }                      // reserved: true = Linux-plan
}

resource app 'Microsoft.Web/sites@2023-12-01' = {
  name: 'app-${miljo}-${suffiks}'
  location: location
  tags: tags
  identity: { type: 'SystemAssigned' }                // managed identity
  properties: {
    serverFarmId: plan.id                             // referanse → Bicep ordner
rekkefølgen selv
    httpsOnly: true
    siteConfig: {
      linuxFxVersion: 'PYTHON|3.12'
      minTlsVersion: '1.2'
      appSettings: [
        { name: 'APP_ENV', value: miljo }
        { name: 'STORAGE_ACCOUNT', value: st.name }
      ]
    }
```

```
  }
}

output appUrl string = 'https://${app.properties.defaultHostName}'
output appPrincipalId string = app.identity.principalId
```

> **Merk:** Tallet etter `@` (f.eks. `@2023-05-01`) er **API-versjonen** til ressurstypen. Bicep-utvidelsen i VS Code foreslår nyere versjoner, og Microsoft Learn («Azure resource reference» / «Bicep resource definitions») viser alle egenskaper per versjon.

Deploy:

```
az group create -n rg-bicep-lab -l norwayeast

# Hva VIL skje? (som terraform plan)
az deployment group what-if -g rg-bicep-lab -f main.bicep -p miljo=dev

# Gjør det
az deployment group create -g rg-bicep-lab -f main.bicep -p miljo=dev

# Vis outputs
az deployment group show -g rg-bicep-lab -n main --query properties.outputs
```

Parameterfiler (`.bicepparam`):

```
// dev.bicepparam
using './main.bicep'
param miljo = 'dev'
```

```
az deployment group create -g rg-bicep-lab -f main.bicep -p dev.bicepparam
```

Moduler. Del opp i gjenbrukbare filer:

```
module nett 'modules/network.bicep' = {
  name: 'nettverk'
  params: {
    vnetName: 'vnet-${miljo}'
    addressPrefix: '10.0.0.0/16'
  }
}
// bruk output fra modulen: nett.outputs.subnetId
```

Azure Verified Modules (AVM) er Microsofts offisielle, vedlikeholdte bibliotek av Bicep- og Terraform-moduler (offentlig *Bicep registry*: `br/public:avm/res/...`). Bruk dem i stedet for å skrive alt selv.

Deployment-modi og scope:

- **Incremental** (standard): legger til/oppdaterer ressurser i malen, **rører ikke** andre ressurser i gruppen.
- **Complete**: sletter ressurser i gruppen som *ikke* er i malen. Farlig – bruk heller **deployment stacks** (`az stack group create`), som håndterer livssyklus og kan låse administrerte ressurser mot endring utenfra.
- Scope: `az deployment group` (ressursgruppe), `sub` (abonnement – f.eks. for å lage ressursgrupper og policyer), `mg` (management group), `tenant`.

Bicep vs. Terraform – kort:

	Bicep	Terraform
Skyer	Kun Azure	Mange (Azure, AWS, GCP, GitHub, Kubernetes …)
Tilstand	Ingen state-fil – Azure er kilden	State-fil som må lagres og beskyttes
Nye Azure-funksjoner	Tilgjengelig umiddelbart	Når provideren oppdateres (som regel raskt)
Forhåndsvisning	`what-if`	`plan`
Utbredelse i jobbannonser	Vanlig i Azure-miljøer	Svært vanlig, også multi-cloud

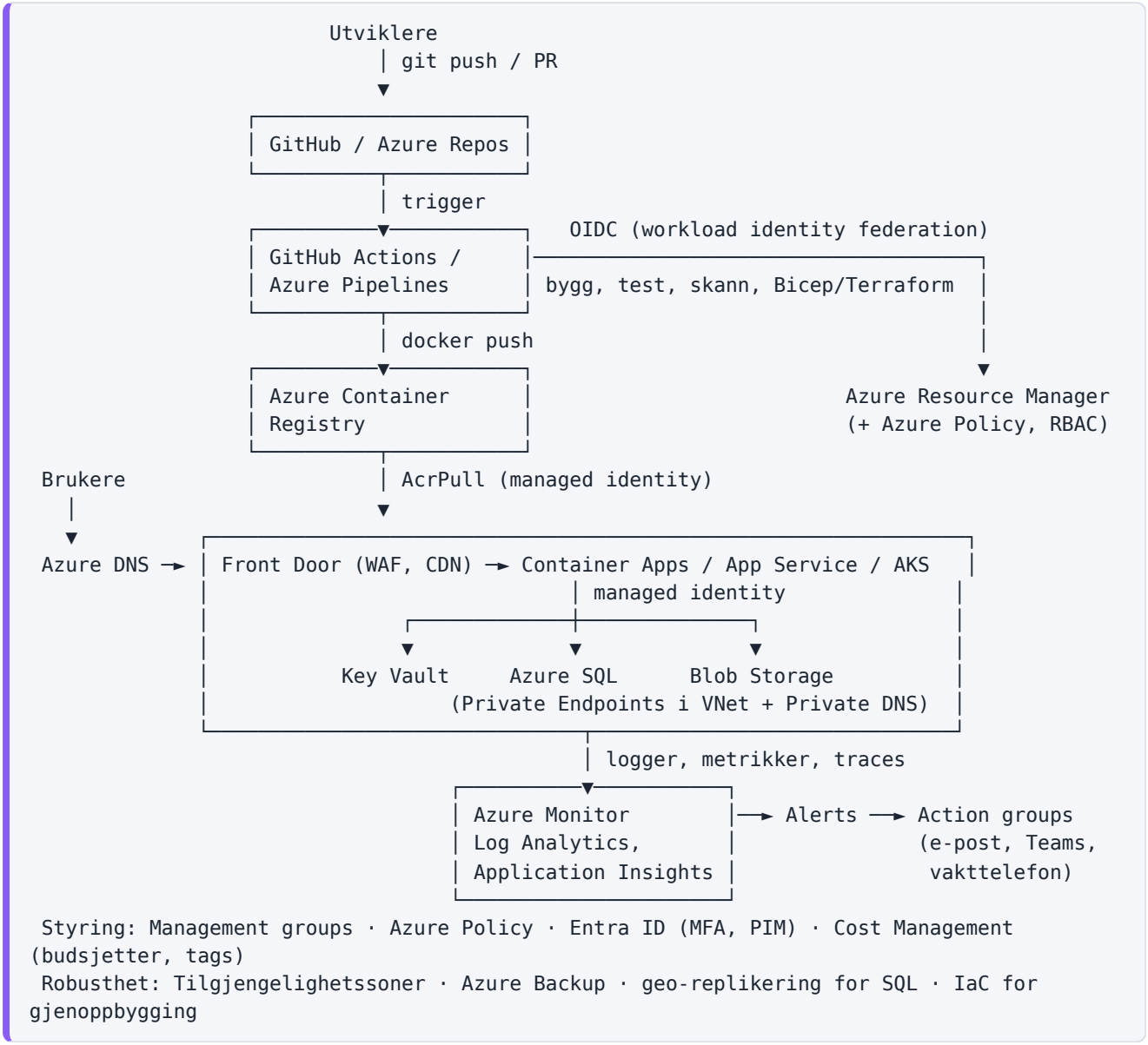
Lær begge på grunnleggende nivå; Terraform får et eget kapittel (11).

Sikkerhet og kostnad. Bicep er gratis. Ikke legg hemmeligheter i parameterfiler i Git; bruk `@secure()`-parametere og hent verdier fra Key Vault (`getSecret()` i `.bicepparam` eller i modulkall). Kjør deploy fra en pipeline med workload identity federation, ikke fra laptoper.

Øvelse. Deploy `main.bicep` med `miljo=dev`, sjekk at web-appen svarer, kjør `what-if` med en endring (legg til en ny tag), og deploy igjen. Slett ressursgruppen.

Hvordan alt henger sammen – referansearkitektur

Her er en realistisk, moderne arkitektur for en nettbutikk som binder sammen tjenestene i del 7:



Les diagrammet som en historie:

1. Utvikleren pusher kode. Pipelinen bygger, tester og skanner.
2. Pipelinen logger inn i Azure **uten hemmeligheter** (OIDC) og deployer infrastruktur med Bicep/Terraform – ARM sjekker RBAC og Policy.
3. Image legges i ACR; appen henter det med managed identity.
4. Brukerne går via DNS og Front Door (med WAF) til appen.
5. Appen bruker managed identity mot Key Vault, SQL og Storage, alt via private endepunkter.
6. Alt logges til Azure Monitor; varsler går til de som har vakt.
7. Styring, kostnadskontroll og backup ligger rundt alt.

Prosjekt 10 i kapittel 15 bygger en forenklet versjon av denne arkitekturen.

Del 7 – kapitteloppsummering

- Azure styres gjennom ARM; alt kan gjøres med portal, CLI, PowerShell, SDK og IaC.
- Hierarkiet (management group → subscription → resource group) styrer arv av RBAC og Policy.
- Identitet (Entra ID), tilgang (RBAC) og regler (Policy) er fundamentet for sikkerhet og styring.
- Managed identities og Key Vault fjerner hemmeligheter fra kode.
- Nettverk: VNet, subnett, NSG, Load Balancer, Application Gateway, DNS, Private Endpoints.
- Compute: VM (IaaS), App Service (PaaS), Functions (serverless), Container Apps/AKS (containere), ACR (images).
- Data: Storage (Blob/Files), Azure SQL Database.
- Drift: Azure Monitor, Log Analytics/KQL, alerts, Backup.
- Leveranse: Azure DevOps/Pipelines eller GitHub Actions, med Bicep eller Terraform.
- Kostnad: budsjetter, tags, riktig størrelse, deallokering og opprydding.

Del 7 – quiz

1. Hva er forskjellen på et abonnement og en ressursgruppe?
2. En bruker har Reader på abonnementet og Contributor på én ressursgruppe. Hva kan brukeren gjøre i den ressursgruppen?
3. Du vil hindre at noen lager ressurser utenfor Norway East. RBAC eller Policy?
4. Hvordan bør en web-app autentisere seg mot Key Vault?
5. En NSG har en regel som tillater port 22 på prioritet 200 og en som avviser port 22 på prioritet 150. Hva skjer?
6. Hvilken lastbalanserer bruker du for URL-basert ruting og WAF?
7. Hva er forskjellen på *Stopped* og *Stopped (deallocated)* for en VM?
8. Hvilken redundans beskytter en lagringskonto mot at et helt datasenter (sone) feiler, men ikke en hel region?
9. Hva er point-in-time restore i Azure SQL Database?
10. Hvorfor ser du ingen ressurslogger fra Key Vault i Log Analytics, selv om workspace finnes?
11. Hva gjør `az deployment group what-if`?
12. Hvilken RBAC-rolle trenger AKS for å hente images fra ACR?

Fasit

1. Et **abonnement** er en faktureringsenhet og tilgangsgrense som inneholder ressursgrupper. En **ressursgruppe** er en logisk mappe for ressurser med samme livssyklus.
2. Alt Contributor tillater (opprette, endre, slette ressurser), men ikke gi andre tilgang. Rettighetene er additive.
3. **Azure Policy** (f.eks. «Allowed locations» med effekt Deny).

4. Med **managed identity** og RBAC-rollen *Key Vault Secrets User* på hvelvet – uten lagrede hemmeligheter.
5. Port 22 **avvises** – lavest prioritetstall (150) evalueres først.
6. **Application Gateway** (regionalt) eller **Azure Front Door** (globalt).
7. *Stopped* (fra innsiden av OS-et) beholder maskinvaren og faktureres fortsatt for compute. *Deallocated* frigjør maskinvaren, og compute faktureres ikke (disker faktureres fortsatt).
8. **ZRS** (zone-redundant storage).
9. Gjenoppretting av databasen til et vilkårlig tidspunkt innenfor oppbevaringsperioden, basert på automatiske backuper.
10. Fordi ressurslogger krever en **diagnostic setting** som sender dem til workspace.
11. Viser hvilke endringer en Bicep/ARM-deploy vil gjøre – uten å gjøre dem.
12. **AcrPull** (gis automatisk med `az aks create/update --attach-acr`).

7.5 AZ-900: Microsoft Azure Fundamentals

Om sertifiseringen

Hva er det? AZ-900 er Microsofts grunnleggende Azure-sertifisering. Den tester at du forstår **skykonsepter**, **Azure-arkitektur og -tjenester**, og **styring og administrasjon** i Azure. Den er **ikke** praktisk-teknisk – du blir ikke bedt om å skrive kommandoer – men den gir et solid ordforråd og et kart over Azure.

Hvorfor ta den? Den er en fin første milepæl, den viser arbeidsgivere at du tar Azure på alvor, og den er et godt springbrett til AZ-104. Mange norske arbeidsgivere setter pris på den hos juniorkandidater og studenter.

Praktisk informasjon (sjekk alltid den offisielle siden på Microsoft Learn, «Microsoft Certified: Azure Fundamentals», før du melder deg opp – innhold, pris og format endres):

Nivå	Fundamentals (grunnleggende)
Format	Flervalg, «ja/nei»-utsagn, dra-og-slipp, case-lignende oppgaver
Bestått	700 av 1000 poeng
Gyldighet	Fundamentals-sertifiseringer utløper ikke
Forberedelse	Gratis læringsstier på Microsoft Learn + gratis *Practice Assessment*
Rabatt	Studentrabatt og gratis vouchere via Microsoft-arrangementer (f.eks. *Virtual Training Days*) forekommer – sjekk Microsoft Learn

Sertifiseringsmål (domener)

Fordelingen var ved siste revisjon omtrent slik (se *study guide* på Microsoft Learn for gjeldende vekting):

Domene	Vekt (ca.)	Kapitler i boken
Describe cloud concepts	25–30 %	6
Describe Azure architecture and services	35–40 %	6, 7.1–7.3
Describe Azure management and governance	30–35 %	7.1, 7.3, 7.4

Detaljert pensumliste og hvor i boken du finner det:

1. **Sky-konsepter**
   - Definere cloud computing, shared responsibility model → 6
   - Public, private, hybrid cloud; forbruksbasert modell; prismodeller → 6
   - Fordeler: høy tilgjengelighet, skalerbarhet, pålitelighet, forutsigbarhet, sikkerhet, styring, administrerbarhet → 6
   - IaaS, PaaS, SaaS og når de passer → 6
2. **Azure-arkitektur og tjenester**
   - Regioner, regionpar, *sovereign regions*, tilgjengelighetssoner, datasentre → 6
   - Ressurser, ressursgrupper, abonnementer, management groups → 7.1
   - Compute: VM, VM Scale Sets, availability sets, Azure Virtual Desktop, containere, Functions, App Service → 7.2
   - Nettverk: VNet, subnett, peering, Azure DNS, VPN Gateway, ExpressRoute, offentlige og private endepunkter → 2, 7.2
   - Lagring: lagringstjenester, nivåer, redundans, kontotyper, filflytting (AzCopy, Storage Explorer, File Sync), migrering (Azure Migrate, Data Box) → 7.3
   - Identitet, tilgang og sikkerhet: Entra ID, Entra Domain Services, autentisering (SSO, MFA, passordløs), B2B/B2C (External ID), Conditional Access, RBAC, Zero Trust, defense in depth, Microsoft Defender for Cloud → 7.1, 13
3. **Administrasjon og styring**
   - Kostnadsfaktorer, Pricing Calculator, Cost Management, tags → 6, 7.1
   - Microsoft Purview, Azure Policy, resource locks → 7.1
   - Verktøy: portal, Cloud Shell, CLI, PowerShell, Azure Arc, IaC/ARM/Bicep → 7.1, 7.4
   - Overvåking: Azure Advisor, Service Health, Azure Monitor (Log Analytics, alerts, Application Insights) → 7.3

Tjenester AZ-900 ofte stiller spørsmål om som ikke er dekket i detalj ellers

- **Azure Virtual Desktop** – Windows-skrivebord og -apper levert fra Azure til brukere hvor som helst.
- **Azure Container Instances (ACI)** – kjør én container raskt uten å administrere servere eller Kubernetes.
- **VPN Gateway vs. ExpressRoute** – kryptert tunnel over internett vs. privat dedikert forbindelse.
- **Azure Migrate** – verktøy for å kartlegge og flytte servere/databaser til Azure. **Azure Data Box** – fysisk enhet for å sende store datamengder til Azure.
- **Microsoft Purview** – styring av data: kartlegging, klassifisering, compliance.
- **Azure Service Health** – status for Azure-tjenester og planlagt vedlikehold *som påvirker deg*. (Azure Status-siden viser global status.)
- **Azure Advisor** – personlige anbefalinger om kostnad, sikkerhet, pålitelighet, ytelse og drift.
- **Microsoft Defender for Cloud** – sikkerhetsstilling (*secure score*), anbefalinger og trusselbeskyttelse.
- **Service Trust Portal** – Microsofts compliance-rapporter (ISO, SOC osv.).
- **Sovereign regions** – f.eks. Azure Government (USA) og Azure i Kina (driftet av en lokal partner).
- **Azure Arc** – administrer servere, Kubernetes og databaser *utenfor* Azure som om de var i Azure.
- **Defense in depth** – sikkerhet i lag: fysisk → identitet → perimeter → nettverk → compute → applikasjon → data.
- **Zero Trust** – «stol aldri, verifiser alltid»; anta brudd; minste privilegium.

Studieplan for AZ-900 (4 uker, ca. 5 timer per uke)

Uke	Les	Gjør
1	Kapittel 6	Microsoft Learn-stien «Describe cloud concepts». Lag ordkort over begrepene.
2	Kapittel 7.1–7.2	Learn: «Describe Azure architecture and services» (del 1). Opprett en ressursgruppe, et VNet og en VM – og slett dem.
3	Kapittel 7.3–7.4	Learn: resten av arkitektur-stien + «Describe Azure management and governance». Lag et budsjett, en policy og en lås.
4	Dette kapitlet	Ta prøveeksamenen under og Microsofts gratis *Practice Assessment* til du jevnt over får 80 %+. Repeter svake domener.

Øvingsoppgaver (praktiske)

1. Lag en ressursgruppe med tags, og bruk *Cost analysis* til å filtrere på tagen.
2. Tildel den innebygde policyen «Allowed locations» og test at den blokkerer en annen region.
3. Lag en `CanNotDelete`-lås og forsøk å slette ressursgruppen.
4. Bruk Pricing Calculator til å sammenligne én VM pay-as-you-go med 1 års reservasjon.
5. Åpne *Azure Advisor* og *Service Health* i portalen, og beskriv hva du ser.
6. Lag en lagringskonto og finn hvor du endrer redundans og tilgangsnivå.

Prøveeksamen AZ-900 (30 spørsmål)

Sett av 45 minutter. Skriv ned svarene før du ser i fasiten.

1. Hvilken egenskap ved skyen beskriver at du betaler bare for ressursene du bruker?
   a. Elastisitet b) Forbruksbasert modell c) Høy tilgjengelighet d) Feiltoleranse
2. En bedrift vil flytte en eksisterende applikasjon til skyen med minst mulig endringer og full kontroll over OS. Hvilken modell?
   a. SaaS b) PaaS c) IaaS d) Serverless
3. Sant eller usant: I PaaS er kunden ansvarlig for å patche operativsystemet.
4. Hvilken modell gir **minst** administrasjonsansvar for kunden?
   a. IaaS b) PaaS c) SaaS d) On-premises
5. Hva er en tilgjengelighetssone?
   a. En gruppe regioner b) Fysisk adskilte lokasjoner i en region c) Et rack i et datasenter d) En virtuell gruppe VM-er
6. Hva gjør du for å beskytte en applikasjon mot at et helt **datasenter** feiler?
   a. Bruker et availability set b) Sprer VM-er over tilgjengelighetssoner c) Bruker en større VM d) Bruker en ressurslås
7. Hvilket utsagn om ressursgrupper er riktig?
   a. En ressurs kan være i flere ressursgrupper b) Ressursgrupper kan nøstes c) Sletting av en ressursgruppe sletter ressursene i den d) Ressurser må ligge i samme region som ressursgruppen
8. Hvilken tjeneste bruker du for å organisere flere abonnementer og bruke policyer på dem samlet?
   a. Ressursgrupper b) Management groups c) Tags d) Azure Arc
9. Hvilken tjeneste kjører kode som svar på hendelser uten at du administrerer servere?
   a. Azure Functions b) Azure Virtual Machines c) Azure Virtual Desktop d) Azure Batch
10. Hvilken tjeneste gir en privat, dedikert forbindelse mellom et lokalt datasenter og Azure som ikke går over internett?
    a. VPN Gateway b) ExpressRoute c) Azure DNS d) VNet peering
11. Hvilken lagringsredundans holder kopier i en sekundær region?

a. LRS b) ZRS c) GRS d) Premium

.2. Hvilket blob-nivå er billigst for lagring, men krever rehydrering før lesing?

a. Hot b) Cool c) Cold d) Archive

.3. Hva er Microsoft Entra ID?

a. En database b) En skybasert identitets- og tilgangstjeneste c) En brannmur d) Et overvåkingsverktøy

.4. Hva er multifaktorautentisering?

a. To passord b) Innlogging med to eller flere uavhengige faktorer c) Innlogging fra to enheter samtidig d) Et sertifikat

.5. Hvilken funksjon kan kreve MFA bare når en bruker logger inn fra en ukjent lokasjon?

a. Azure Policy b) Conditional Access c) RBAC d) Resource locks

.6. Hvilken modell bygger på prinsippet «stol aldri, verifiser alltid»?

a. Defense in depth b) Zero Trust c) Shared responsibility d) Least connections

.7. Hva gjør Azure RBAC?

a. Definerer hvilke ressurstyper som kan opprettes b) Styrer hvem som har tilgang til hva, på hvilket scope c) Krypterer data d) Overvåker kostnader

.8. Hva gjør Azure Policy?

a. Gir brukere tilgang b) Håndhever regler for ressurskonfigurasjon og rapporterer etterlevelse c) Tar backup d) Fordeler trafikk

.9. Hvilken lås hindrer både endring og sletting?

a. CanNotDelete b) ReadOnly c) Delete d) NoChange

20. Hvilket verktøy estimerer kostnaden for ressurser **før** du oppretter dem?

a. Cost Management b) Pricing Calculator c) Azure Advisor d) Service Health

21. Hvilken faktor påvirker **ikke** normalt prisen på en VM?

a. Region b) Størrelse c) Antall tags d) Operativsystemlisens

22. Hvilken tjeneste gir personlige anbefalinger for å redusere kostnader og forbedre sikkerhet?

a. Azure Advisor b) Azure Monitor c) Azure Arc d) Microsoft Purview

23. Hvilken tjeneste varsler om hendelser i Azure-plattformen som påvirker **dine** ressurser?

a. Azure Status b) Azure Service Health c) Activity log d) Defender for Cloud

24. Hvilket verktøy gir en nettleserbasert terminal med Azure CLI og PowerShell?

a. Azure Arc b) Cloud Shell c) Azure Portal Dashboard d) Bicep

25. Hvilken tjeneste lar deg administrere lokale servere og Kubernetes-clustere i andre skyer fra Azure?

a. Azure Arc b) Azure Migrate c) ExpressRoute d) Azure Stack Edge

26. Hva beskriver infrastruktur som kode best?

a. Å skrive applikasjonskode i skyen b) Å beskrive og opprette infrastruktur med deklarative filer c) Å bruke portalen d) Å kjøre kode på VM-er

27. Hva er tags brukt til?

a. Kryptering b) Organisering og kostnadsfordeling med metadata c) Nettverksruting d) Autentisering

28. Hvilken tjeneste samler inn og analyserer metrikker og logger fra Azure-ressurser?
    a. Azure Monitor b) Azure Advisor c) Microsoft Entra ID d) Azure Policy
29. Hvilken tjeneste er best for å kjøre én enkel container raskt uten å administrere servere eller Kubernetes?
    a. AKS b) Azure Container Instances c) Azure Virtual Desktop d) VM Scale Sets
30. I modellen for delt ansvar – hva er **alltid** kundens ansvar?
    a. Fysisk sikkerhet b) Hypervisoren c) Data og identiteter d) Nettverkskablingen i datasenteret

Fasit med forklaringer

1. **b** – Forbruksbasert modell (*consumption-based*): betal for bruk, ingen forhåndskostnad.
2. **c** – IaaS gir full OS-kontroll og passer «lift and shift».
3. **Usant** – i PaaS patcher leverandøren OS-et.
4. **c** – SaaS: kunden bare bruker og konfigurerer.
5. **b** – Fysisk adskilte lokasjoner (datasentre) med egen strøm, kjøling og nettverk i en region.
6. **b** – Soner beskytter mot datasenterfeil. Availability sets beskytter bare mot rack-/vertsfeil i ett datasenter.
7. **c** – Sletting av gruppen sletter innholdet. Ressurser kan ligge i andre regioner enn gruppen; grupper kan ikke nøstes.
8. **b** – Management groups.
9. **a** – Azure Functions (serverless, hendelsesdrevet).
10. **b** – ExpressRoute. VPN Gateway går kryptert *over* internett.
11. **c** – GRS (og GZRS, RA-GRS, RA-GZRS).
12. **d** – Archive.
13. **b** – Skybasert identitets- og tilgangstjeneste.
14. **b** – To eller flere uavhengige faktorer: noe du vet, har eller er.
15. **b** – Conditional Access.
16. **b** – Zero Trust.
17. **b** – RBAC styrer hvem som har hvilken tilgang på hvilket scope.
18. **b** – Policy håndhever og rapporterer regler for konfigurasjon.
19. **b** – ReadOnly.
20. **b** – Pricing Calculator. Cost Management analyserer faktisk forbruk.
21. **c** – Tags påvirker ikke prisen (men hjelper med å fordele den).
22. **a** – Azure Advisor.
23. **b** – Service Health er personlig tilpasset dine ressurser og regioner.
24. **b** – Cloud Shell.
25. **a** – Azure Arc.

26. **b** – Deklarative filer som beskriver infrastrukturen (ARM, Bicep, Terraform).
27. **b** – Metadata for organisering, automatisering og kostnadsrapportering.
28. **a** – Azure Monitor.
29. **b** – Azure Container Instances.
30. **c** – Data, kontoer/identiteter (og enheter) er alltid kundens ansvar.

Vurdering: 24+ riktige (80 %) betyr at du sannsynligvis er klar. Under 21: repeter domenet der du bommet mest, og ta Microsofts offisielle *Practice Assessment*.

7.6 AZ-104: Microsoft Azure Administrator

Om sertifiseringen

Hva er det? AZ-104 gir tittelen **Microsoft Certified: Azure Administrator Associate**. Den tester at du kan **implementere, administrere og overvåke** et Azure-miljø: identiteter, styring, lagring, compute, nettverk og overvåking. Den er betydelig mer praktisk og krevende enn AZ-900.

Hvorfor ta den? Den er en av de mest etterspurte Azure-sertifiseringene for drift-, sky- og DevOps-roller, og grunnlaget for videre sertifiseringer som *AZ-400 (DevOps Engineer Expert)* og *AZ-305 (Solutions Architect Expert)*.

Praktisk informasjon (verifiser på Microsoft Learn, «Microsoft Certified: Azure Administrator Associate», og les den nyeste *study guide* – pensum oppdateres jevnlig):

Nivå	Associate
Format	Flervalg, case-studier, dra-og-slipp, «ja/nei»-serier; kan inneholde interaktive oppgaver
Tid	Omtrent 100 minutter for selve eksamen
Bestått	700 av 1000 poeng
Gyldighet	1 år; fornyes gratis med en online fornyelsestest på Microsoft Learn
Hjelpemidler	Under eksamen har Associate-eksamener tilgang til Microsoft Learn (åpen bok i begrenset form) – sjekk gjeldende regler
Forkunnskaper	Anbefalt minst 6 måneders praktisk Azure-erfaring – lab-arbeid i denne boken hjelper

Sertifiseringsmål (domener)

Vektingen ved siste revisjon var omtrent slik:

Domene	Vekt (ca.)	Kapitler
Manage Azure identities and governance	20–25 %	7.1
Implement and manage storage	15–20 %	7.3
Deploy and manage Azure compute resources	20–25 %	7.2, 7.4, 9, 10
Implement and manage virtual networking	15–20 %	2, 7.2
Monitor and maintain Azure resources	10–15 %	7.3, 13

Viktige ferdigheter per domene:

1. **Identiteter og styring:** brukere og grupper (også masseopprettelse og gjester), lisenser, SSPR; RBAC-roller og tildelinger på riktig scope, tolke tilganger; Azure Policy; låser; tags; ressursgrupper, abonnementer og management groups; budsjetter, kostnadsvarsler, Advisor.
2. **Lagring:** brannmur og virtuelle nettverk på lagringskontoer, SAS (konto, tjeneste, user delegation) og *stored access policies*, tilgangsnøkler og rotasjon, identitetsbasert tilgang til Azure Files; redundans, objektreplikering, kryptering (inkl. kundeadministrerte nøkler); containere, nivåer, soft delete, snapshots, versjonering, livssyklus; AzCopy og Storage Explorer.
3. **Compute:** tolke og endre ARM-maler og Bicep-filer, eksportere maler; opprette og administrere VM-er (størrelser, disker, kryptering, flytte VM-er mellom grupper/abonnementer/regioner, availability sets og soner, VM Scale Sets); containere (ACR, Container Instances, Container Apps, størrelser og skalering); App Service (planer, skalering, TLS-sertifikater, egne domener, backup, nettverk, deployment slots).
4. **Nettverk:** VNets og subnett, peering, offentlige IP-adresser, brukerdefinerte ruter (UDR), feilsøking av tilkobling; NSG og ASG, effektive regler, Azure Bastion, service endpoints og private endpoints; Azure DNS; interne og offentlige lastbalanserere og feilsøking av dem.
5. **Overvåking og vedlikehold:** metrikker, logger, varsler, action groups, *alert processing rules*, Log Analytics-spørringer, VM/storage/network insights, Network Watcher og Connection Monitor; Recovery Services vault og Backup vault, backup-policyer, gjenoppretting, Azure Site Recovery, backup-rapporter og -varsler.

Studieplan for AZ-104 (8 uker, ca. 8–10 timer per uke)

Uke	Fokus	Lab (se under)
1	Identitet: Entra ID, brukere, grupper, RBAC	Lab 1
2	Styring: Policy, låser, tags, kostnader, management groups	Lab 2

Uke	Fokus	Lab (se under)
3	Lagring: kontoer, Blob, Files, SAS, redundans, livssyklus	Lab 3
4	VM-er, disker, tilgjengelighet, VMSS, ARM/Bicep	Lab 4
5	App Service, containere (ACR, ACI, Container Apps)	Lab 5
6	Nettverk: VNet, peering, NSG, UDR, DNS, private endpoints	Lab 6
7	Lastbalansering, Bastion, Network Watcher; Monitor og Backup	Lab 7 og 8
8	Repetisjon, prøveeksamen, Microsoft *Practice Assessment*, svake områder	–

Microsoft publiserer også offisielle laboppgaver for AZ-104 på GitHub (søk etter «MicrosoftLearning AZ-104-MicrosoftAzureAdministrator»). Bruk dem i tillegg.

Laboppgaver

Alle labbene bruker ressursgrupper som starter med `rg-az104-`. **Slett dem etter hver lab.**

Lab 1 – Identitet og RBAC. 1. Opprett gruppene `helpdesk` og `nettverksteam` og to testbrukere. 2. Gi `helpdesk` rollen *Virtual Machine Contributor* på `rg-az104-lab1`. 3. Logg inn som testbrukeren (privat vindu). Bekreft at brukeren kan starte/stoppe VM-er, men ikke endre VNet. 4. Bruk *Access control (IAM) → Check access* for å se effektive tilganger.

Lab 2 – Styring. 1. Lag management group `mg-az104` og flytt abonnementet inn (hvis du har rettigheter – ellers gjør resten på abonnements- eller ressursgruppenivå). 2. Tildel «Require a tag on resource groups» (tag `kostnadssted`) med effekt Deny. 3. Prøv å opprette en ressursgruppe uten tag (feiler) og med tag (virker). 4. Tildel «Inherit a tag from the resource group if missing», lag en ressurs, og kjør en *remediation task*. 5. Legg en `CanNotDelete`-lås på gruppen og prøv å slette den.

Lab 3 – Lagring. 1. Lag en StorageV2-konto med ZRS. Slå av offentlig blob-tilgang og nøkkelbasert tilgang. 2. Lag en container, last opp filer med Entra-innlogging. 3. Lag en *stored access policy* på containeren og en tjeneste-SAS som refererer til den. Opphev tilgangen ved å slette policyen. 4. Slå på soft delete og versjonering, slett en blob og gjenopprett den. 5. Lag en livssyklusregel som flytter blobs til Cool etter 30 dager. 6. Lag en filshare og monter den fra en Azure-VM.

Lab 4 – VM-er. 1. Deploy to VM-er i forskjellige soner med en Bicep-fil (fra kapittel 7.4 som utgangspunkt). 2. Legg til en datadisk, partisjoner, formater og monter den i Linux (`lsblk`, `sudo mkfs.ext4`, `/etc/fstab` med UUID). 3. Endre størrelsen på en VM. Hva skjer med VM-en under

endringen? 4. Lag en VM Scale Set med autoskalering basert på CPU (min 1, maks 3). 5. Eksporter en mal fra ressursgruppen (*Export template*) og les den.

Lab 5 – App Service og containere. 1. Lag en App Service Plan (S1 eller høyere for slots), en web-app, og en `staging`-slot. 2. Deploy versjon 1 til produksjon og versjon 2 til staging. Gjør en swap. 3. Konfigurer autoskalering for planen. 4. Bygg et image, push til ACR, og kjør det i Azure Container Instances (`az container create`) og i Container Apps (`az containerapp up`).

Lab 6 – Nettverk. 1. Lag `vnet-hub` (10.0.0.0/16) og `vnet-spoke` (10.1.0.0/16) og peer dem (begge veier). 2. Lag én VM i hver uten offentlig IP. Bruk Bastion og test `ping` / `curl` mellom dem. 3. Lag en NSG som bare tillater SSH fra hub til spoke. Verifiser med *IP flow verify*. 4. Lag en route table med en UDR som sender `0.0.0.0/0` til en *virtual appliance*-IP og se på *Effective routes*. 5. Lag en privat DNS-sone med automatisk registrering, og slå opp VM-navn fra den andre VM-en. 6. Lag en lagringskonto med Private Endpoint og verifiser at `nslookup <konto>.blob.core.windows.net` gir privat IP fra VM-en.

Lab 7 – Lastbalansering og feilsøking. 1. Sett to webservere bak en Standard Load Balancer med helsesjekk. 2. Stopp nginx på én – observer *backend health* og trafikkfordelingen. 3. Bruk *Network Watcher → Connection troubleshoot / Connection Monitor* mellom en VM og en ekstern adresse.

Lab 8 – Overvåking og backup. 1. Lag et Log Analytics workspace og send VM-logger dit med Azure Monitor Agent og en Data Collection Rule. 2. Lag et CPU-varsel med en action group. Utløs det med `stress-ng`. 3. Skriv en KQL-spørring som viser *Heartbeat* per VM siste time. 4. Ta backup av en VM, og gjenopprett én fil med *File recovery*.

Eksamenstips

- **Les spørsmålet nøye.** Ord som «minimize administrative effort», «least privilege» og «minimize cost» avgjør hvilket av flere *mulige* svar som er *riktig*.
- **Kjenn grensene:** 5 reserverte IP-er per subnett, NSG-prioritet 100–4096 (lavest først), peering er ikke transitiv, lagringskontonavn 3–24 små bokstaver/tall.
- **Kjenn forskjellene:** availability set vs. zone, LRS/ZRS/GRS/GZRS, Load Balancer vs. Application Gateway, service endpoint vs. private endpoint, Recovery Services vault vs. Backup vault, Entra-roller vs. Azure-roller.
- **Ja/nei-serier:** Samme scenario med ulike løsningsforslag – vurder hvert forslag isolert. Du kan ikke gå tilbake til disse.
- **Bruk tiden:** Case-studier tar tid. Marker vanskelige spørsmål og kom tilbake.

Prøveeksamen AZ-104 (30 spørsmål)

Sett av 60 minutter.

1. Du må gi en gruppe tilgang til å administrere VM-er i én ressursgruppe, men ikke nettverket eller lagringen. Hvilken rolle og hvilket scope?

2. En bruker har *Owner* på et abonnement. En ressursgruppe har en `ReadOnly`-lås. Kan brukeren legge til en tag på en VM i gruppen?
3. Du må sikre at alle nye ressurser har taggen `kostnadssted`, og at ressurser uten taggen **ikke kan opprettes**. Hva bruker du?
4. Ressursgruppen har taggen `prosjekt=x`. Får en ny VM i gruppen automatisk samme tag?
5. Du skal gi en ekstern partner lesetilgang til én blob i 24 timer uten å opprette en konto for dem. Hva er den sikreste metoden?
6. Du har utstedt flere SAS-tokens basert på en *stored access policy*. Hvordan opphever du alle samtidig?
7. En lagringskonto må tåle at en hel tilgjengelighetssone feiler, og data skal ikke forlate regionen. Hvilken redundans?
8. Data som sjelden leses, skal lagres billigst mulig, men må kunne leses innen millisekunder når det trengs. Hvilket nivå?
9. Du skal kopiere 5 TB fra en lokal server til Blob Storage over nettverket med et skriptbart kommandolinjeverktøy. Hvilket?
10. Du har en VM i et availability set. Hva beskytter dette mot?
11. Du må endre størrelsen på en VM til en størrelse som ikke er tilgjengelig i den nåværende maskinvareklyngen. Hva må du ofte gjøre først?
12. En VM viser *Stopped* i portalen etter at noen kjørte `shutdown` i OS-et. Betaler dere for compute?
13. Du vil at antall VM-instanser skal øke automatisk når gjennomsnittlig CPU er over 70 %. Hvilken tjeneste?
14. Du skal teste en ny versjon av en web-app i produksjonsmiljøet og bytte uten nedetid. Hvilken App Service-funksjon, og hvilket minste nivå kreves?
15. En app i App Service må nå en database i et VNet via privat IP. Hva konfigurerer du på App Service?
16. Hvilken tjeneste kjører en enkelt container raskest, uten orkestrering, med fakturering per sekund?
17. Du har VNet A (10.0.0.0/16) og VNet B (10.0.0.0/16). Kan du peere dem?
18. VNet A er peeret med B, og B er peeret med C. Kan A og C kommunisere direkte?
19. Hvor mange IP-adresser kan du bruke til VM-er i et subnett `10.1.5.0/28`?
20. En NSG på subnettet tillater port 443. En NSG på VM-ens NIC har ingen egendefinerte regler. Når trafikk på 443 fra internett frem til VM-en?
21. Du vil la en VM nå en lagringskonto via en privat IP i VNet-et, og blokkere offentlig tilgang til kontoen. Hva bruker du?
22. Hva er forskjellen på en service endpoint og en private endpoint?
23. Du må tvinge all utgående internettrafikk fra et subnett gjennom en brannmur-VM. Hva lager du?
24. Du trenger SSH-tilgang til VM-er uten offentlige IP-er og uten VPN. Hvilken tjeneste?
25. Du skal fordele HTTPS-trafikk basert på URL-sti og beskytte mot SQL-injeksjon. Hvilken tjeneste?

26. Helsesjekken til en Load Balancer feiler for alle VM-er, men nginx kjører. NSG-en har en regel *DenyAll* inn på prioritet 100. Hva er sannsynlig årsak?
27. Du vil analysere hvilke regler som tillot eller blokkerte en bestemt pakke til en VM. Hvilket verktøy?
28. Du skal få e-post og SMS når en VM har vært utilgjengelig i 5 minutter. Hvilke to komponenter i Azure Monitor trenger du?
29. Hvilket hvelv bruker du for å ta backup av Azure VM-er og Azure Files?
30. Du må kunne gjenopprette en hel applikasjon (flere VM-er) i en annen region med RPO på minutter. Backup eller Site Recovery?

Fasit med forklaringer

1. **Virtual Machine Contributor** på **ressursgruppen**. Minste privilegium: bare VM-er, bare der.
2. **Nei.** Låser gjelder alle brukere, også Owner. ReadOnly hindrer endringer (som tags). Fjern låsen først (hvis du har rettighet).
3. **Azure Policy** med effekt **Deny** (f.eks. «Require a tag and its value on resources»).
4. **Nei.** Tags arves ikke automatisk. Bruk Policy (*Inherit a tag from the resource group*) med Modify-effekt.
5. **User delegation SAS** (signert med Entra-legitimasjon) med kun lese-rettighet og 24 timers utløp. Den er sikrere enn en SAS signert med kontonøkkel.
6. **Endre eller slett den lagrede tilgangspolicyen** (*stored access policy*). Alternativt rotere kontonøkkelen som signerte dem – men det påvirker alt annet som bruker nøkkelen.
7. **ZRS.** (GRS/GZRS sender kopier til en annen region.)
8. **Cool** eller **Cold** (avhengig av hvor sjelden og minimum lagringstid). **Ikke Archive** – der må data rehydreres i timer.
9. **AzCopy.**
10. Maskinvare-/rack-feil (fault domains) og planlagt vedlikehold (update domains) **innenfor ett datasenter** – ikke tap av hele datasenteret.
11. **Deallokere (stoppe)** VM-en, slik at den kan flyttes til maskinvare som støtter størrelsen.
12. **Ja.** Den er ikke deallokert. Stopp/dealloker fra portalen eller med `az vm deallocate`.
13. **Virtual Machine Scale Sets** med autoskaleringsregel.
14. **Deployment slots** med **swap**. Krever **Standard** eller høyere.
15. **VNet integration** (regional), eventuelt sammen med en Private Endpoint og privat DNS for databasen.
16. **Azure Container Instances.**
17. **Nei.** Adresseområdene overlapper.
18. **Nei.** Peering er ikke transitiv. Peer A–C direkte, eller bruk hub-and-spoke med ruting gjennom en NVA/Azure Firewall eller Virtual WAN.
19. **11.** /28 gir 16 adresser, minus 5 reservert av Azure.

20. **Nei.** Når det finnes NSG både på subnett og NIC, må **begge** tillate trafikken. En NSG uten egendefinerte regler har bare standardreglene, og *DenyAllInBound* blokkerer trafikk fra internett. (Hadde NIC-en **ingen** NSG i det hele tatt, ville trafikken sluppet frem.) Dette er en klassisk eksamensfelle: les nøye om NIC-en har «en NSG uten regler» eller «ingen NSG».
21. **Private endpoint** for lagringskontoen, privat DNS-sone (`privatelink.blob.core.windows.net`), og *Public network access: Disabled*.
22. **Service endpoint**: trafikken fra subnettet går over Microsofts nett til tjenestens **offentlige** endepunkt, og tjenesten kan begrenses til subnettet. **Private endpoint**: tjenesten får en **privat IP** i VNet-et ditt; kan også nås fra peerede nett og lokalt via VPN/ExpressRoute.
23. En **route table** med en **UDR** `0.0.0.0/0 → Virtual appliance (brannmurens private IP)`, knyttet til subnettet.
24. **Azure Bastion.**
25. **Application Gateway med WAF** (eller Front Door med WAF for globale scenarier).
26. NSG-regelen på prioritet 100 **blokkerer helsesjekkene** fra `AzureLoadBalancer` (de tillates normalt av standardregelen på 65001, men en egendefinert DenyAll med lavere tall vinner). Legg til en regel som tillater service tag `AzureLoadBalancer` med lavere prioritetstall enn 100 – eller endre DenyAll-regelen.
27. **Network Watcher → IP flow verify** (og *Effective security rules*).
28. En **alert rule** (f.eks. metrikk *VM Availability* eller en log alert på Heartbeat) og en **action group** (e-post og SMS).
29. **Recovery Services vault.**
30. **Azure Site Recovery** – kontinuerlig replikering gir RPO på minutter og orkestrert failover til en annen region.

Vurdering: 24+ riktige: god form. 18–23: repeter domenene der du bommet og gjør labbene på nytt. Under 18: gå gjennom kapittel 7.1–7.3 og labbene systematisk før du prøver igjen.

Veien videre etter AZ-104



(Microsoft endrer sertifiseringsporteføljen jevnlig – noen sertifiseringer pensjoneres eller erstattes. Sjekk Microsoft Learn sin sertifiseringsoversikt og vedlegg C.)

Del 8 – DevOps Engineering

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva DevOps er, hvorfor det oppstod, og hva DevOps-kultur betyr,
- forklare Agile, Scrum og livssyklusen for programvareutvikling (SDLC),
- skille mellom Continuous Integration, Continuous Delivery og Continuous Deployment,
- beskrive deploy-strategier, monitoring, observability og hendelseshåndtering,
- forklare SRE, Platform Engineering og DevSecOps,
- beskrive en typisk arbeidsdag for en DevOps Engineer.

Hva er DevOps?

Hva er det? DevOps er en **kultur og en måte å jobbe på** der utvikling (*Development*) og drift (*Operations*) samarbeider tett gjennom hele livsløpet til en tjeneste – fra idé til produksjon og videre drift – med mye **automatisering** og raske **tilbakemeldinger**.

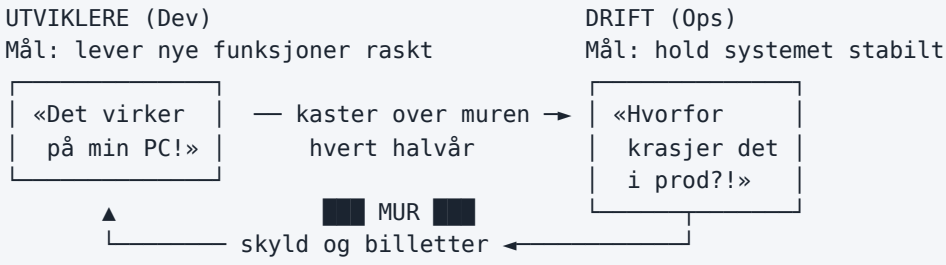
DevOps er **ikke**:

- bare et verktøy («vi har kjøpt Jenkins, så nå er vi DevOps»),
- bare en stillingstittel,
- et eget team som sitter mellom utviklere og drift og tar imot billetter (det er bare en ny mur).

> **Hverdagsanalogi:** Tenk på en restaurant der kokkene (utviklerne) lager retter og kaster dem over en vegg til servitørene (driften), som må finne ut hvordan de skal serveres. Når gjestene klager, skylder de på hverandre. I en DevOps-restaurant planlegger kokker og servitører menyen sammen, kokkene smaker på maten underveis, servitørene gir umiddelbar tilbakemelding fra gjestene, og kjøkkenet har maskiner som gjør de repeterende oppgavene.

Hvorfor oppstod DevOps?

Problemet: «muren» mellom Dev og Ops.



Før DevOps var det vanlig at:

- utviklere og drift hadde **motstridende mål** (endring vs. stabilitet),
- utgivelser skjedde **sjelden og var store** – derfor risikable og smertefulle («release-helg»),
- servere ble satt opp **manuelt**, ulikt fra miljø til miljø («snøfnugg-servere»),
- feil ble oppdaget **sent**, ofte av kundene,
- ingen eide helheten.

Bakgrunn. Rundt 2007–2009 begynte folk som Patrick Debois og Andrew Shafer å snakke om «agile infrastruktur». En kjent presentasjon fra Flickr i 2009, «10+ Deploys per Day: Dev and Ops Cooperation», viste at hyppige utgivelser og samarbeid ga **både** fart **og** stabilitet. Samme år ble den første *DevOpsDays*-konferansen arrangert i Belgia – og navnet var født. Samtidig gjorde skyen og API-er det mulig å automatisere infrastruktur.

Viktig innsikt: Fart og stabilitet er **ikke** motsetninger. Små, hyppige endringer er *lettere* å teste, *lettere* å forstå, og *lettere* å rulle tilbake enn store, sjeldne endringer.

DevOps-kultur

CALMS – fem søyler

Bokstav	Betyr	I praksis
Culture	Samarbeid, felles ansvar, tillit	«You build it, you run it» – teamet som lager tjenesten, drifter den også
Automation	Automatiser repeterende arbeid	CI/CD, IaC, automatiske tester
Lean	Små leveranser, fjern sløsing, flyt	Små PR-er, korte køer, WIP-grenser
Measurement	Mål for å forbedre	DORA-metrikker, overvåking
Sharing	Del kunnskap, verktøy og lærdom	Dokumentasjon, postmortems, interne plattformer

De tre veiene (*The Three Ways*)

Fra boken *The Phoenix Project* / *The DevOps Handbook*:

1. **Flyt** (*flow*) – få arbeid raskt og jevnt fra utvikler til kunde. Små batcher, automatisering, synliggjør arbeid.
2. **Tilbakemelding** (*feedback*) – få rask informasjon om problemer: tester, overvåking, varsler, kundeinnsikt.
3. **Kontinuerlig læring og eksperimentering** – lær av feil uten skyld, forbedre hele tiden.

Blameless culture – kultur uten skyld

Når noe går galt, spør man **«hvordan lot systemet dette skje?»**, ikke **«hvem gjorde feil?»**. Mennesker gjør feil; gode systemer gjør det vanskelig å gjøre feil og lett å oppdage og rette dem. Hvis folk blir straffet, skjuler de feil – og organisasjonen slutter å lære.

> **Fra arbeidslivet:** En utvikler slettet ved et uhell en produksjonsdatabase. I en skyld-kultur får hun sparken. I en DevOps-kultur spør teamet: hvorfor hadde en utvikler skrivetilgang til produksjon? Hvorfor fantes det ingen lås eller bekreftelse? Hvorfor tok gjenopprettingen fire timer? Tiltakene blir minste privilegium, ressurslåser, og testet backup – og hele organisasjonen blir tryggere.

Måling: DORA-metrikkene

Forskningsprogrammet **DORA** (*DevOps Research and Assessment*, nå del av Google Cloud) har i mange år studert tusenvis av organisasjoner. De fant fire nøkkeltall som skiller høytytende team fra andre:

Metrikk	Spørsmål	Høytytende team
Deployment frequency	Hvor ofte deployer vi til produksjon?	Ofte – flere ganger daglig, ved behov
Lead time for changes	Hvor lang tid fra commit til produksjon?	Kort – under en dag
Change failure rate	Hvor stor andel av deployene gir feil i produksjon?	Lav
Time to restore service (*failed deployment recovery time*)	Hvor raskt gjenoppretter vi etter en feil?	Raskt – under en time

De to første måler **fart**, de to siste **stabilitet**. DORA fant at de beste teamene er gode på **begge**. (De nyere rapportene har justert navn og lagt til *rework rate*; se dora.dev for gjeldende definisjoner.)

> **Husk:** I et intervju er DORA-metrikkene et sterkt svar på «Hvordan måler du om et DevOps-initiativ fungerer?»

Agile

Hva er det? Agile (smidig utvikling) er en tankegang for programvareutvikling basert på **Agile-manifestet** (2001). I stedet for å planlegge alt på forhånd (fossefall, *waterfall*), leverer man **små, fungerende biter ofte**, får tilbakemelding, og justerer.

Manifestets fire verdier:

- **Individer og samspill** fremfor prosesser og verktøy
- **Programvare som virker** fremfor omfattende dokumentasjon
- **Samarbeid med kunden** fremfor kontraktsforhandlinger
- **Å reagere på endringer** fremfor å følge en plan

(«Fremfor» betyr at tingene til høyre også har verdi, men tingene til venstre verdsettes mer.)

```
FOSSEFALL (waterfall)                    AGILE (iterativt)
Krav → Design → Kode → Test → Drift      ┌─► Planlegg ─► Bygg ─► Test ─► Lever ─┐
(måneder/år; kunden ser noe først        └──────────── tilbakemelding ◄──────────┘
 helt på slutten)                        (2-ukers sykluser; kunden ser noe hele tiden)
```

Agile og DevOps henger sammen: Agile gjorde *utviklingen* rask og iterativ. Men hvis koden bare blir deployet hvert halvår, hjelper det lite. DevOps utvider smidigheten **helt ut i produksjon**.

Scrum

Hva er det? Scrum er det mest brukte rammeverket for å jobbe smidig. Arbeidet deles inn i **sprinter** – faste perioder på 1–4 uker (oftest 2) – der teamet leverer en fungerende økning av produktet.

Roller:

Rolle	Ansvar
Product Owner	Bestemmer *hva* som er viktigst; eier og prioriterer produktbackloggen
Scrum Master	Hjelper teamet å følge Scrum og fjerner hindringer; en tjenende leder, ikke en sjef
Developers (utviklerteamet)	Alle som bygger produktet – også DevOps-, test- og designkompetanse

Artefakter: *Product backlog* (prioritert liste over alt som kan gjøres), *Sprint backlog* (det teamet forplikter seg til denne sprinten), *Increment* (fungerende resultat). **Definition of Done** sier når noe er ferdig (f.eks. «testet, kodegjennomgått, deployet til test, dokumentert»).

Hendelser:

```
┌──────────────────── SPRINT (f.eks. 2 uker) ────────────────────┐
│ Sprint Planning → Daily Scrum (15 min hver dag) → ...          │
│                   → Sprint Review → Sprint Retrospective       │
└────────────────────────────────────────────────────────────────┘
```

- **Sprint Planning** – hva skal vi levere, og hvordan?
- **Daily Scrum** – kort daglig synk: fremdrift mot sprintmålet, hindringer.

- **Sprint Review** – vis frem resultatet for interessenter, få tilbakemelding.
- **Sprint Retrospective** – hvordan jobbet vi? Hva skal vi forbedre?

Kanban er et alternativ: kontinuerlig flyt i stedet for sprinter, med en tavle (*To Do → In Progress → Review → Done*) og **begrensning av pågående arbeid** (*WIP limits*). Mange drifts- og plattformteam foretrekker Kanban, fordi uplanlagt arbeid (hendelser) passer dårlig inn i faste sprinter.

User stories beskriver behov fra brukerens perspektiv: «*Som kunde vil jeg få e-post når pakken er sendt, slik at jeg vet når den kommer.*» Med **akseptansekriterier**: «E-post sendes innen 5 minutter etter at status endres til *sendt*.»

SDLC – livssyklusen for programvareutvikling

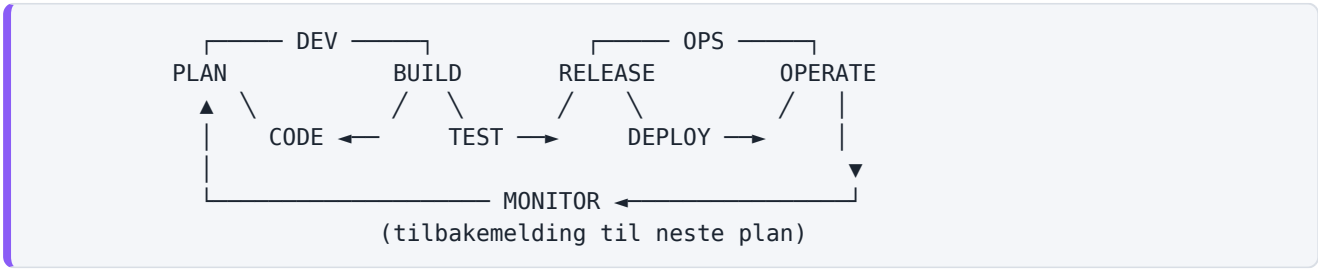
Hva er det? SDLC (*Software Development Life Cycle*) beskriver fasene programvare går gjennom:



I tradisjonell utvikling går man gjennom fasene én gang (fossefall). I Agile/DevOps går man gjennom alle fasene **kontinuerlig i små sløyfer** – for hver funksjon, hver dag.

DevOps-livssyklusen

DevOps-livssyklusen tegnes ofte som en **uendelighetssløyfe**:



Fase	Aktiviteter	Verktøy (eksempler)
Plan	Backlog, krav, prioritering	Azure Boards, Jira, GitHub Projects
Code	Skrive kode, kodegjennomgang	Git, GitHub, VS Code
Build	Kompilere, bygge artefakter/images	GitHub Actions, Azure Pipelines, Docker
Test	Enhetstester, integrasjonstester, sikkerhetsskanning	pytest, SonarQube, Trivy

Fase	Aktiviteter	Verktøy (eksempler)
Release	Versjonering, godkjenning	GitHub Releases, Azure Pipelines environments
Deploy	Rulle ut til miljøer	Actions, Pipelines, Argo CD, Helm
Operate	Drift, skalering, konfigurasjon	Kubernetes, Terraform, Ansible
Monitor	Metrikker, logger, varsler, brukeratferd	Azure Monitor, Prometheus, Grafana

CI/CD – kontinuerlig integrasjon, levering og deploy

Dette er hjertet av DevOps i praksis. Kapittel 12 viser hvordan du bygger det; her forklarer vi begrepene.

Continuous Integration (CI)

Hva er det? Utviklere **integrerer** (merger) koden sin i en felles hovedbranch **ofte** – minst daglig – og hver integrasjon **bygges og testes automatisk**.

Hvorfor? Hvis fem utviklere jobber hver for seg i tre uker og så prøver å slå sammen alt, får de «integrasjonshelvete»: konflikter og feil som er vanskelige å finne. Med CI oppdages problemer innen minutter, mens endringen fortsatt er liten og fersk i minnet.

Kjennetegn: én felles hovedbranch, automatisk bygg og test på hver push/PR, rask tilbakemelding (helst under 10 minutter), og en regel om at **rødt bygg fikses umiddelbart**.

Continuous Delivery (CD)

Hva er det? Hver endring som består testene, blir automatisk gjort **klar for produksjon** (bygget, testet, deployet til testmiljø). Selve produksjonsdeployen skjer med **et tastetrykk / en godkjenning**.

Continuous Deployment

Hva er det? Som Continuous Delivery, men **uten manuell godkjenning**: hver endring som består alle automatiske sjekker, går **rett i produksjon**.

```
                  CODE → BUILD → TEST → STAGING → PRODUKSJON
Continuous Integration  [  auto  ][ auto ]
Continuous Delivery     [  auto  ][ auto ][ auto  ] [ MANUELL godkjenning ]
Continuous Deployment   [  auto  ][ auto ][ auto  ] [ auto ]
```

> **Vanlig feil:** Å tro at «CD» alltid betyr Continuous Deployment. Mange organisasjoner (f.eks. banker med regulatoriske krav) har Continuous **Delivery** med manuell godkjenning til produksjon – og det er helt legitimt.

Automatisering

Prinsipp: Hvis du gjør noe **mer enn to ganger**, eller det er **kritisk at det gjøres likt hver gang**, bør det automatiseres.

Hva automatiseres typisk i DevOps:

- bygg, test, sikkerhetsskanning og deploy (CI/CD),
- opprettelse av infrastruktur (IaC – Terraform, Bicep),
- konfigurasjon av servere (Ansible, cloud-init),
- skalering (autoskalering),
- overvåking og varsling,
- rutineoppgaver: opprydding, sertifikatfornyelse, backup, rapporter,
- *self-service* for utviklere («lag et nytt miljø med ett klikk»).

Toil er Googles SRE-begrep for manuelt, repeterende, automatiserbart arbeid uten varig verdi. Målet er å redusere toil systematisk.

> **Vanlig feil:** Å automatisere en dårlig prosess. Forenkle prosessen først, automatiser etterpå.

Infrastruktur i DevOps

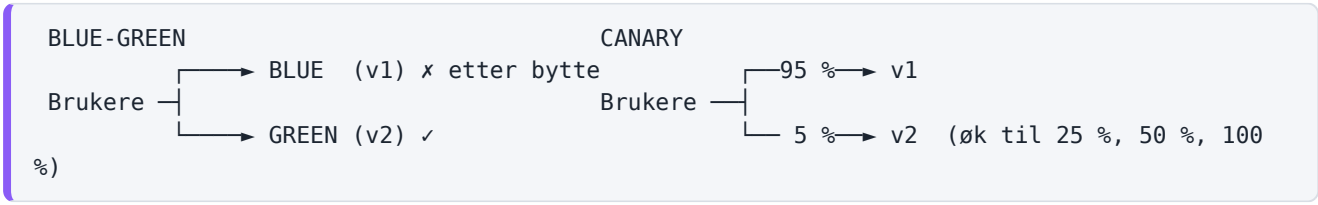
Moderne infrastruktur behandles som **programvare**:

- **Infrastructure as Code (IaC)** – infrastruktur beskrives i kode, versjonskontrolleres og gjennomgås (kapittel 11).
- **Immutable infrastructure** – servere oppdateres ikke på plass; man bygger en ny versjon og erstatter den gamle. Containere er et typisk eksempel.
- **Pets vs. cattle** – før hadde servere navn og ble pleid som kjæledyr («kjære gamle *zeus*»). Nå er servere som buskap: like, nummererte og byttbare. Feiler en, erstattes den automatisk.
- **Miljøparitet** – utvikling, test og produksjon skal være så like som mulig (samme image, samme IaC, ulik størrelse og konfigurasjon).

Deploy-strategier

Hvordan bytter du ut gammel versjon med ny, uten nedetid og med lav risiko?

Strategi	Hvordan	Fordel	Ulempe
Recreate	Stopp gammel, start ny	Enkel	Nedetid
Rolling	Bytt instanser én og én (eller noen om gangen)	Ingen nedetid, standard i Kubernetes	Gammel og ny kjører samtidig en stund; roll-back tar tid
Blue-green	Ny versjon (green) settes opp ved siden av gammel (blue); trafikken byttes på én gang	Rask, enkel rollback (bytt tilbake)	Dobbel kapasitet under bytte
Canary	Send en liten andel trafikk (f.eks. 5 %) til ny versjon, øk gradvis hvis alt ser bra ut	Lav risiko, ekte brukerdata	Krever god overvåking og trafikkstyring
Feature flags	Ny kode deployes «av», og slås på for utvalgte brukere uten ny deploy	Skiller *deploy* fra *release*	Flagg må ryddes opp i, ellers blir koden kompleks
A/B-testing	To varianter for å måle hvilken som fungerer best for brukerne	Forretningsinnsikt	Mer en produktteknikk enn en deploy-teknikk



Fra arbeidslivet: I Azure App Service gjør du blue-green med **deployment slots** og swap. I Kubernetes er rolling standard; canary gjøres med verktøy som Argo Rollouts, Flagger eller en service mesh. Azure Front Door og Container Apps kan dele trafikk mellom revisjoner.

Rollback – å gå tilbake til forrige versjon – må være planlagt, testet og helst automatisert. Husk at **databaseendringer** er det vanskeligste å rulle tilbake: gjør dem bakoverkompatible («expand and contract»: legg til ny kolonne → deploy kode som bruker begge → fjern gammel kolonne senere).

Monitoring og observability

Monitoring (overvåking) svarer på spørsmål du **visste** du måtte stille: Er serveren oppe? Er CPU over 90 %? Er feilraten over 1 %? Du definerer metrikker og terskler på forhånd.

Observability (observerbarhet) er evnen til å forstå **hva som skjer inne i systemet** ut fra dataene det sender ut – også for problemer du **ikke** forutså. «Hvorfor er utsjekken treg bare for brukere i Bergen som bruker Vipps?»

De tre søylene (mer i kapittel 13):

Søyle	Hva	Eksempel
Metrikker	Tall over tid	Forespørsler/sek, feilrate, responstid, CPU
Logger	Hendelser med detaljer	`ERROR betaling feilet ordre=123 kode=TIMEOUT`
Traces	En forespørsels vei gjennom mange tjenester	frontend → API → betaling → database, med tid i hvert ledd

De fire gylne signalene (Google SRE) for enhver tjeneste: **latency** (responstid), **traffic** (trafikk), **errors** (feil), **saturation** (metning/belastning).

Incident management – hendelseshåndtering

Hva er det? En strukturert måte å håndtere **hendelser** (*incidents*) – uplanlagte avbrudd eller forringelse av en tjeneste.

Livssyklus:

```
Oppdag → Varsle → Triager → Begrens/løs → Gjenopprett → Postmortem → Forbedre
(alert)  (vakt)    (alvorlighet) (mitigate)   (verifiser)    (læring)      (tiltak)
```

Nøkkelbegreper:

- **Alvorlighetsgrad** (*severity*): SEV1 (alle kunder rammet, kritisk) … SEV4 (liten påvirkning).
- **Vaktordning** (*on-call*): én person har ansvar for å svare på varsler utenom arbeidstid, ofte på rotasjon. Verktøy: PagerDuty, Opsgenie, Azure Monitor action groups.
- **Incident commander**: koordinerer håndteringen; andre fokuserer på teknisk feilsøking og kommunikasjon.
- **Mitigate først, root cause etterpå**: først få tjenesten opp igjen (rollback, failover, skalering), *deretter* finne grunnårsaken.
- **Runbooks / playbooks**: ferdige oppskrifter for kjente problemer («Hvis disken er full på DB-serveren, gjør …»).

- **MTTD / MTTR**: *Mean Time To Detect* og *Mean Time To Recover/Restore*.
- **Statusside** og kommunikasjon til kunder.

Postmortem (etteranalyse) – et dokument skrevet uten skyld etter en hendelse:

1. Sammendrag og påvirkning (hvem, hvor lenge, hvor mange).
2. Tidslinje.
3. Grunnårsak(er) og medvirkende faktorer («5 hvorfor»).
4. Hva gikk bra, hva gikk dårlig, hvor var vi heldige.
5. **Konkrete tiltak** med eier og frist.

SRE – Site Reliability Engineering

Hva er det? SRE er Googles tilnærming til drift, beskrevet som «hva som skjer når du ber en programvareingeniør designe et driftsteam». SRE kan sees som én **konkret implementasjon** av DevOps-prinsippene.

Kjernebegreper:

Begrep	Betyr	Eksempel
SLI (*Service Level Indicator*)	Et målt tall for tjenestekvalitet	Andel vellykkede forespørsler; andel forespørsler under 300 ms
SLO (*Service Level Objective*)	Internt **mål** for SLI-en	99,9 % vellykkede forespørsler over 30 dager
SLA (*Service Level Agreement*)	**Avtale** med kunden, med konsekvenser	99,5 %, ellers kreditt
Error budget	Hvor mye feil vi «har lov til»	100 % − 99,9 % = 0,1 % ≈ 43 minutter per 30 dager

Error budget i praksis: Så lenge budsjettet ikke er brukt opp, kan teamet slippe nye funksjoner raskt. Hvis budsjettet er brukt opp, prioriteres stabilitet foran nye funksjoner. Dette gjør konflikten mellom «fart» og «stabilitet» til en **datadrevet** beslutning.

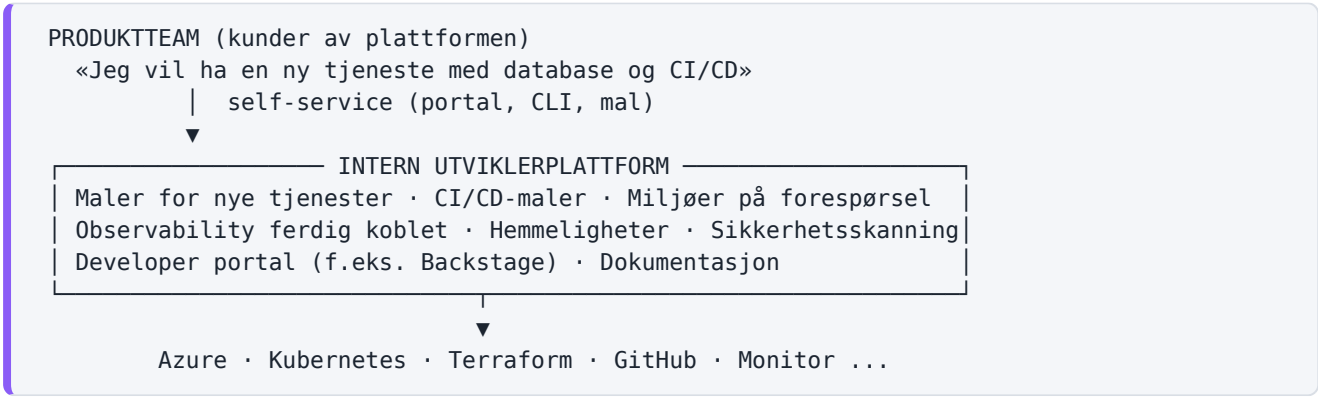
Andre SRE-prinsipper: maks ~50 % av tiden på toil (resten på automatisering og ingeniørarbeid), blameless postmortems, og at 100 % pålitelighet er feil mål (det er uendelig dyrt, og brukerne merker ikke forskjellen fra 99,99 %).

> **Husk: SLA ≤ SLO ≤ det systemet faktisk leverer.** Man lover kunden mindre enn man sikter mot internt, for å ha margin.

Platform Engineering

Hva er det? Platform Engineering er å bygge en **intern utviklerplattform** (*Internal Developer Platform, IDP*) – et sett med verktøy, maler og selvbetjente tjenester – slik at produktteamene kan levere raskt og trygt **uten** at hvert team må være eksperter på Kubernetes, nettverk, sikkerhet og CI/CD.

Hvorfor? «You build it, you run it» ga team mer ansvar – men også enorm **kognitiv belastning**. Plattformteamet tar på seg den felles kompleksiteten og tilbyr **«golden paths»** (asfalterte stier): anbefalte, ferdig oppsatte veier for vanlige oppgaver.



> **Fra arbeidslivet:** Mange norske selskaper har plattformteam (ofte kalt «Platform», «Developer Experience» eller «Cloud Platform»). En DevOps Engineer kan godt jobbe *i* et slikt team. Plattformen skal behandles som et **produkt**, med utviklerne som kunder.

DevSecOps

Hva er det? DevSecOps er å bygge **sikkerhet inn i hele DevOps-livssyklusen** i stedet for å legge den til på slutten. Ofte kalt **«shift left»** – flytt sikkerhet (og testing) til venstre, tidligere i prosessen, der feil er billigere å rette.



Teknikk	Hva den finner	Eksempel på verktøy
Secret scanning	Passord og nøkler i kode	GitHub secret scanning, gitleaks
SAST (statisk analyse)	Sårbarheter i egen kildekode	CodeQL, SonarQube, Semgrep
SCA (avhengighetsanalyse)	Kjente sårbarheter i biblioteker	Dependabot, `pip-audit`, `npm audit`

Teknikk	Hva den finner	Eksempel på verktøy
Container-skanning	Sårbarheter i images	Trivy, Microsoft Defender for Containers
IaC-skanning	Usikker infrastrukturkode	Checkov, tfsec/Trivy
DAST (dynamisk analyse)	Sårbarheter i kjørende app	OWASP ZAP
SBOM	Oversikt over alle komponenter	Syft, `docker sbom`

Mer i kapittel 13.

En typisk arbeidsdag for en DevOps Engineer

Ingen dager er like, men slik kan en dag se ut for en DevOps Engineer i et produktteam hos et norsk selskap som bruker Azure:

Tid	Aktivitet
08:30	Sjekker dashbordet i Grafana/Azure Monitor og varslingskanalen i Teams/Slack. Natten var rolig, bortsett fra et diskvarsel på en build-agent.
08:45	Rydder diskplassen på agenten og lager en oppgave om å automatisere opprydding av gamle Docker-images.
09:00	**Daily standup** med teamet (15 min). Status: Terraform-modul for nytt miljø er nesten ferdig.
09:15	Kodegjennomgang: en utviklers PR legger til en ny miljøvariabel. Foreslår at verdien hentes fra Key Vault i stedet for å stå i YAML.
10:00	Fokustid: skriver Terraform for et nytt testmiljø (VNet, Container Apps, PostgreSQL med private endpoint). Kjører `terraform plan` i pipelinen.
11:30	En utvikler spør hvorfor appen ikke når databasen i test. Feilsøker sammen: privat DNS-sone var ikke koblet til det nye VNet-et. Fikser i Terraform.
12:00	Lunsj.
12:30	Møte med sikkerhetsteamet om å innføre container-skanning i alle pipelines. Avtaler å lage en felles pipeline-mal.

Tid	Aktivitet
13:30	Oppdaterer CI-pipelinen: legger til Trivy-skanning som feiler bygget ved kritiske sårbarheter. Tester på en branch.
15:00	Deploy av ny versjon til produksjon (canary 10 % → 100 %). Følger med på feilrate og responstid. Alt grønt.
15:30	Skriver dokumentasjon / runbook for det nye miljøet.
16:00	Retro-oppfølging: kostnadsanalyse viser at et gammelt testmiljø har kostet penger i tre måneder uten bruk. Avtaler sletting med teamet, og lager en policy som krever `utløper`-tag.
16:30	Overleverer til kollegaen som har vakt i kveld.

Kjennetegn ved jobben: mye samarbeid og kommunikasjon, mye feilsøking, mye lesing av dokumentasjon og logger, en god del koding (YAML, HCL, Bash, Python), og et konstant fokus på å **gjøre andre mer produktive**.

Nøkkelegenskaper hos gode DevOps Engineers: nysgjerrighet, systematisk feilsøking, evne til å forklare tekniske ting enkelt, automatiseringsrefleks, sikkerhetsbevissthet, og ydmykhet – ingen kan alt.

Kapitteloppsummering

- DevOps er kultur + praksis + verktøy for raskt og trygt samarbeid mellom utvikling og drift.
- Det oppstod for å bryte muren mellom Dev og Ops; små, hyppige endringer gir både fart og stabilitet.
- CALMS og «de tre veiene» beskriver kulturen. DORA-metrikkene måler resultatet.
- Agile og Scrum organiserer arbeidet i korte iterasjoner; Kanban gir kontinuerlig flyt.
- CI = integrer og test ofte. Continuous Delivery = alltid klar for prod (manuell knapp). Continuous Deployment = automatisk til prod.
- Deploy-strategier: rolling, blue-green, canary, feature flags – alle med plan for rollback.
- Observability = metrikker, logger og traces. Hendelser håndteres strukturert og uten skyld.
- SRE bruker SLI/SLO/error budget. Platform Engineering bygger selvbetjente plattformer. DevSecOps bygger sikkerhet inn i alle steg.

Øvingsoppgaver

1. Forklar forskjellen på Continuous Delivery og Continuous Deployment til en medstudent på under ett minutt.
2. En tjeneste har SLO på 99,5 % over 30 dager. Hvor mange minutter error budget har den?
3. Skriv en blameless postmortem for et påfunnet scenario: «Nettbutikken var nede i 47 minutter fordi et TLS-sertifikat utløp.»
4. Lag et Kanban-brett i GitHub Projects eller Azure Boards for ditt neste prosjekt i kapittel 15.
5. Finn de fire DORA-metrikkene for ditt eget GitHub-prosjekt (så godt det lar seg gjøre): hvor ofte merger du til main, og hvor lang tid går det fra første commit til merge?

Quiz

1. Nevn de fem bokstavene i CALMS og hva de står for.
2. Hvorfor er små, hyppige deployer mindre risikable enn store, sjeldne?
3. Hva er de fire DORA-metrikkene?
4. Hva er forskjellen på Scrum Master og Product Owner?
5. Hva er forskjellen på en SLO og en SLA?
6. Hva er et error budget?
7. Hvilken deploy-strategi sender en liten andel av trafikken til ny versjon først?
8. Hva betyr «shift left»?
9. Hva er forskjellen på monitoring og observability?
10. Hva er hovedmålet med Platform Engineering?

Fasit

1. **C**ulture (samarbeid og felles ansvar), **A**utomation (automatisering), **L**ean (små leveranser, fjerne sløsing), **M**easurement (måling), **S**haring (deling av kunnskap).
2. Små endringer er lettere å teste, gjennomgå, forstå og rulle tilbake; når noe går galt, er det lett å finne hvilken endring som var årsaken.
3. Deployment frequency, lead time for changes, change failure rate, time to restore service (failed deployment recovery time).
4. **Product Owner** bestemmer *hva* som skal bygges og prioriterer backloggen. **Scrum Master** hjelper teamet med *prosessen* og fjerner hindringer.
5. **SLO** er et internt mål. **SLA** er en avtale med kunden med konsekvenser hvis den brytes.
6. Den mengden upålitelighet SLO-en tillater (100 % − SLO). Brukes til å balansere nye funksjoner mot stabilitet.
7. **Canary.**
8. Å flytte sikkerhet og testing tidligere i utviklingsprosessen, der feil er billigere å finne og rette.

9. **Monitoring** svarer på kjente spørsmål med forhåndsdefinerte metrikker og terskler. **Observability** er evnen til å forstå ukjente problemer ut fra systemets data (metrikker, logger, traces).
10. Å redusere kognitiv belastning for produktteam ved å tilby en intern, selvbetjent plattform med «golden paths», slik at de kan levere raskt og trygt.

Del 9 – Docker og containere

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva containere er, og hvordan de skiller seg fra virtuelle maskiner,
- forklare images, containere, lag (*layers*) og registre,
- skrive en god Dockerfile (inkludert multi-stage builds),
- kjøre flere containere sammen med Docker Compose,
- bruke volumes og Docker-nettverk,
- sikre og feilsøke containere,
- bygge, kjøre og publisere en komplett applikasjon.

Hva er containere?

Hva er det? Tenk deg at du har laget en applikasjon på PC-en din. Den fungerer perfekt hos deg, men når en kollega prøver den, fungerer den ikke – fordi PC-en deres har en annen Python-versjon, mangler et bibliotek, eller har andre innstillinger. «It works on my machine!»

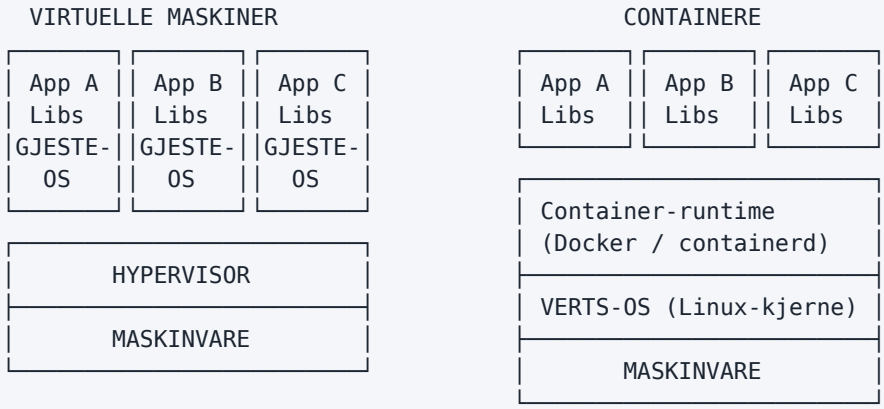
En **container** pakker applikasjonen **sammen med alt den trenger for å kjøre** – runtime (f.eks. Python), biblioteker, systemverktøy og konfigurasjon – i én standardisert enhet. Containeren kjører på samme måte på din laptop, kollegaens PC, i testmiljøet og i produksjon i Azure.

> **Hverdagsanalogi:** Skipscontaineren. Før 1950-tallet ble varer lastet løst på skip – sekker, tønner og kasser i alle former. Det var tregt, og ting ble ødelagt. Standardcontaineren endret alt: uansett hva som er inni, har den samme mål og festepunkter, og passer på alle skip, kraner, tog og lastebiler i verden. En Docker-container er det samme for programvare: uansett hva som er inni, kan den kjøres overalt der det finnes en container-runtime.

Hvorfor finnes det?

- **Konsistens** – samme pakke i alle miljøer.
- **Isolasjon** – apper med motstridende avhengigheter kan kjøre side om side.
- **Lettvekt og fart** – containere starter på sekunder (ofte under ett), og bruker lite ressurser.
- **Tetthet** – mange containere på samme server.
- **Grunnlag for moderne drift** – mikrotjenester, Kubernetes, CI/CD og skalering bygger på containere.

Containere vs. virtuelle maskiner



	Virtuell maskin	Container
Isolerer	Hele maskinen (eget OS og kjerne)	Prosesser (deler vertens kjerne)
Størrelse	Gigabyte	Megabyte (ofte 20–500 MB)
Oppstart	Minutter	Sekunder eller mindre
Isolasjonsstyrke	Sterk	God, men svakere (delt kjerne)
Typisk bruk	Ulike OS, sterk isolasjon, hele servere	Applikasjoner og mikrotjenester

> **Vanlig feil:** «Containere erstatter VM-er.» Ofte kjører containere **på** VM-er – for eksempel er nodene i AKS vanlige Azure-VM-er. De løser forskjellige problemer og brukes sammen.

Hvordan fungerer isolasjonen? Containere bruker funksjoner i Linux-kjernen:

- **Namespaces** – gir hver container sin egen «utsikt»: egne prosess-ID-er, eget nettverk, eget filsystem, eget vertsnavn. Prosessen tror den er alene.
- **cgroups** (*control groups*) – begrenser hvor mye CPU og minne en container kan bruke.
- **Union-filsystemer** (f.eks. overlayfs) – stabler lag av filer oppå hverandre (se «lag» under).

Docker Desktop på Windows og macOS kjører derfor en liten Linux-VM i bakgrunnen for å kjøre Linux-containere. (Det finnes også Windows-containere, men Linux-containere er det vanlige.)

Hva er Docker?

Docker er verktøyet som gjorde containere enkle å bruke (lansert 2013). Det består av:

- **Docker CLI** (`docker`) – kommandoene du skriver.

- **Docker Engine** (`dockerd`) – tjenesten som bygger og kjører containere (bruker **containerd** og **runc** under panseret).
- **Docker Desktop** – pakke for Windows/macOS med GUI, Engine og Linux-VM. Gratis for personlig bruk, utdanning og små bedrifter; større bedrifter trenger betalt abonnement – sjekk Dockers gjeldende lisensvilkår.
- **Docker Hub** – et offentlig register for images.

Containerformatet er standardisert av **OCI** (*Open Container Initiative*), så images bygget med Docker kan kjøres av andre verktøy (Podman, containerd, Kubernetes).

Installer:

- Windows/macOS: Docker Desktop fra docker.com.
- Linux (Ubuntu): følg «Install Docker Engine on Ubuntu» på docs.docker.com, og legg brukeren din i `docker`-gruppen: `sudo usermod -aG docker $USER` (logg ut og inn igjen).

```
$ docker version
$ docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

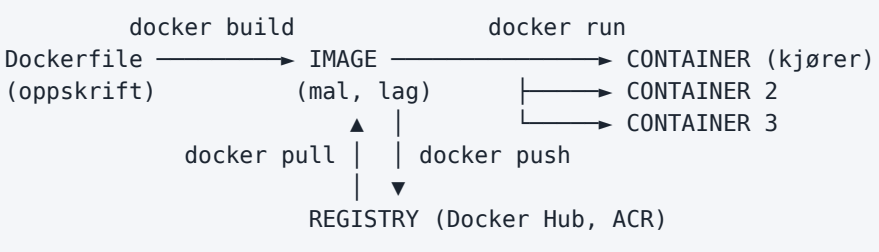
Sikkerhet: Medlemskap i `docker`-gruppen gir i praksis root-tilgang til maskinen. Gi det bare til betrodde brukere.

Images og containere

Image – en **skrivebeskyttet mal** med alt som trengs for å kjøre appen: filsystem, avhengigheter, standardkommando.

Container – en **kjørende instans** av et image, med et tynt skrivbart lag på toppen.

Hverdagsanalogi: Et image er en **oppskrift** (eller en kakeform). En container er **kaken** du har laget fra den. Du kan lage mange kaker fra samme oppskrift, og spiser du én kake, er oppskriften like hel. I programmeringsspråk: image = klasse, container = objekt.



Første container

```
        $ docker run -d -p 8080:80 --name web nginx:1.27
        # -d          = detached: kjør i bakgrunnen
        # -p 8080:80  = port 8080 på DIN maskin → port 80 i containeren
        # --name web  = gi containeren et navn
        # nginx:1.27  = image "nginx" med tag "1.27" (hentes fra Docker Hub hvis du ikke
har det)

        $ curl http://localhost:8080          # eller åpne i nettleseren
        <h1>Welcome to nginx!</h1>
```

Navn på images

```
acrlab123.azurecr.io / team / flaskapp : 1.4.2
└─────── register ──────┘ └─ repo-sti ──┘   └ tag ┘
Uten register: Docker Hub. Uten tag: «latest».
docker.io/library/nginx:1.27 = nginx:1.27
```

Vanlig feil: Å bruke `:latest` i produksjon. «latest» er bare en tag som kan peke på hva som helst, og endres uten forvarsel. Bruk **eksplisitte versjoner** (`python:3.12-slim` , `minapp:1.4.2` eller Git-commit-hash som tag). For maksimal sikkerhet kan du referere til **digest** (`nginx@sha256:...`), som aldri endres.

Viktige kommandoer

```
        # Containere
        $ docker ps                     # kjørende containere
        $ docker ps -a                  # alle, også stoppede
        $ docker logs web               # se loggene (stdout/stderr)
        $ docker logs -f web            # følg loggene live
        $ docker exec -it web sh        # åpne et shell INNE i containeren (-it =
interaktiv terminal)
        $ docker stop web               # stopp (SIGTERM, så SIGKILL etter 10 s)
        $ docker start web              # start igjen
        $ docker rm web                 # slett (må være stoppet, eller bruk -f)
        $ docker inspect web            # alle detaljer i JSON
        $ docker stats                  # live CPU/minne per container
        $ docker run --rm -it ubuntu:24.04 bash   # midlertidig container, slettes når du
avslutter

        # Images
        $ docker images                 # list lokale images
        $ docker pull python:3.12-slim  # hent et image
        $ docker rmi nginx:1.27         # slett et image
        $ docker image history nginx:1.27   # se lagene

        # Rydding
        $ docker system df              # hvor mye plass bruker Docker?
        $ docker system prune           # slett stoppede containere, ubrukte nettverk og
«dangling» images
```

Lag (*layers*) og cache

Et image består av **lag**. Hver instruksjon i en Dockerfile som endrer filsystemet, lager et nytt lag. Lagene er **delt** mellom images og **cachet**: hvis et lag ikke har endret seg, gjenbrukes det ved neste bygg.

```
┌──────────────────────────────┐  ← skrivbart container-lag (forsvinner med containeren)
├──────────────────────────────┤
│ COPY . .        (din kode)   │  ← endres ofte
│ RUN pip install ...          │  ← endres når requirements.txt endres
│ COPY requirements.txt .      │
│ python:3.12-slim (base)      │  ← endres sjelden, deles med andre images
└──────────────────────────────┘
```

> **Husk:** Det som endres **sjeldnest** skal stå **øverst** i Dockerfile, det som endres **oftest** nederst. Da gjenbrukes cachen, og byggene blir raske.

Dockerfile

Hva er det? En tekstfil med instruksjoner for å bygge et image.

En enkel Dockerfile for en Flask-app

Prosjektstruktur:

```
flaskapp/
├── app.py
├── requirements.txt
├── Dockerfile
└── .dockerignore
```

`app.py`:

```python
import os
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def hjem():
    return f"Hei fra container! Miljø: {os.environ.get('APP_ENV', 'ukjent')}"

@app.route("/health")
def health():
    return jsonify(status="ok")
```

`requirements.txt`:

```
flask==3.1.*
gunicorn==23.*
```

(Versjonsnumrene er eksempler; sjekk siste versjoner på PyPI.)

`Dockerfile`:

```
        # 1. Start fra et offisielt, slankt Python-image (fast versjon, ikke latest)
        FROM python:3.12-slim

        # 2. Python-innstillinger: ikke lag .pyc-filer, og skriv logger umiddelbart
        ENV PYTHONDONTWRITEBYTECODE=1 \
            PYTHONUNBUFFERED=1

        # 3. Arbeidsmappe inne i imaget (opprettes automatisk)
        WORKDIR /app

        # 4. Kopier BARE avhengighetslisten først – utnytter cache
        COPY requirements.txt .

        # 5. Installer avhengigheter (uten pip-cache for å holde imaget lite)
        RUN pip install --no-cache-dir -r requirements.txt

        # 6. Kopier resten av koden
        COPY . .

        # 7. Lag en vanlig bruker og bytt til den – ikke kjør som root
        RUN useradd --create-home --uid 10001 appuser
        USER appuser

        # 8. Dokumenter hvilken port appen lytter på
        EXPOSE 8000

        # 9. Kommandoen som starter appen. Lytt på 0.0.0.0 – IKKE 127.0.0.1!
        CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "app:app"]
```

`.dockerignore` (hva som **ikke** skal kopieres inn i imaget):

```
.git
.venv
__pycache__
*.pyc
.env
Dockerfile
.dockerignore
```

Bygg og kjør:

```
        $ docker build -t flaskapp:1.0 .          # -t = navn:tag, . = byggekontekst
(denne mappen)
        $ docker run -d -p 8000:8000 -e APP_ENV=lokal --name flask flaskapp:1.0
        $ curl localhost:8000
        Hei fra container! Miljø: lokal
```

Dockerfile-instruksjoner

Instruksjon	Gjør
`FROM`	Base-image (alltid først)
`WORKDIR`	Setter arbeidsmappe
`COPY`	Kopierer filer fra byggekonteksten inn i imaget
`ADD`	Som COPY, men kan også pakke ut arkiver og hente URL-er – bruk heller COPY
`RUN`	Kjører en kommando **under bygging** (installere pakker)
`ENV`	Setter miljøvariabler
`ARG`	Variabel som bare finnes under bygging (`docker build --build-arg`)
`EXPOSE`	Dokumenterer porten (åpner den ikke!)
`USER`	Hvilken bruker som kjører resten og containeren
`CMD`	Standardkommando **når containeren starter** (kan overstyres)
`ENTRYPOINT`	Fast hovedkommando; `CMD` blir da argumenter til den
`HEALTHCHECK`	Kommando Docker bruker for å sjekke om containeren er frisk
`LABEL`	Metadata (versjon, eier, kilde)

> **Vanlig feil:** `RUN` vs. `CMD` . `RUN` kjøres **når imaget bygges**. `CMD` kjøres **når containeren starter**. `RUN python app.py` vil prøve å starte appen under byggingen – og bygget henger.

> **Vanlig feil:** `EXPOSE 8000` publiserer ikke porten. Du må fortsatt bruke `-p 8000:8000` (eller tilsvarende i Compose/Kubernetes).

Multi-stage builds

Hva er det? En Dockerfile med **flere `FROM` -steg**. Du bygger i ett steg med alle byggeverktøy, og kopierer **bare resultatet** over i et lite, rent sluttimage.

Hvorfor? Mindre images (raskere, billigere), og færre verktøy i produksjonsimaget betyr **mindre angrepsflate**.

Eksempel med en Go-app (Go kompilerer til én fil):

```
        # --- Steg 1: bygg ---
        FROM golang:1.23 AS build
        WORKDIR /src
        COPY go.mod go.sum ./
        RUN go mod download
        COPY . .
        RUN CGO_ENABLED=0 go build -o /out/app .

        # --- Steg 2: kjør ---
        FROM gcr.io/distroless/static-debian12     # minimalt image uten shell og
pakkehåndterer
        COPY --from=build /out/app /app
        USER nonroot
        ENTRYPOINT ["/app"]
```

Byggeimaget er på hundrevis av MB; sluttimaget kan bli under 20 MB.

Eksempel med Python (bygg «wheels» i ett steg, installer i et rent steg):

```
        FROM python:3.12-slim AS build
        WORKDIR /app
        COPY requirements.txt .
        RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt

        FROM python:3.12-slim
        WORKDIR /app
        COPY --from=build /wheels /wheels
        RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels
        COPY . .
        RUN useradd --uid 10001 appuser
        USER appuser
        CMD ["gunicorn", "--bind", "0.0.0.0:8000", "app:app"]
```

Volumes – data som overlever containeren

Problemet. Containere er **flyktige** (*ephemeral*). Data skrevet i containerens filsystem forsvinner når containeren slettes. Det er bra for applikasjonskode, men katastrofalt for en database.

Løsningen. Lagre data **utenfor** containeren:

Type	Hva	Bruk
Named volume	Lagring administrert av Docker	Databaser, varige data (anbefalt)
Bind mount	En mappe fra vertsmaskinen monteres inn	Lokal utvikling (endre kode uten å bygge på nytt)
tmpfs	I minnet, forsvinner ved stopp	Midlertidige, sensitive data

```
        # Named volume for PostgreSQL
        $ docker volume create pgdata
        $ docker run -d --name db \
            -e POSTGRES_PASSWORD=bytt-meg \
            -v pgdata:/var/lib/postgresql/data \
            postgres:16
        $ docker rm -f db              # slett containeren ...
        $ docker run -d --name db -e POSTGRES_PASSWORD=bytt-meg -v
pgdata:/var/lib/postgresql/data postgres:16
        # ... dataene er der fortsatt!

        # Bind mount for utvikling
        $ docker run -d -p 8000:8000 -v "$(pwd)":/app flaskapp:1.0

        $ docker volume ls
        $ docker volume inspect pgdata
        $ docker volume rm pgdata       # slett (data er borte!)
```

> **Fra arbeidslivet:** I produksjon kjører man sjelden databaser i containere man drifter selv; man bruker ofte en administrert database (Azure SQL, Azure Database for PostgreSQL). Containere er best for **tilstandsløse** (*stateless*) apper.

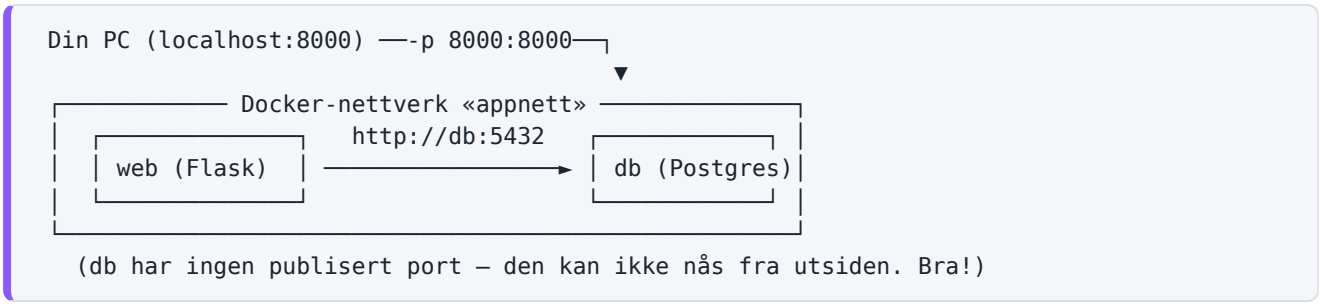
Nettverk i Docker

Docker lager virtuelle nettverk for containere.

Driver	Hva
`bridge`	Standard. Privat nettverk på verten; containere når hverandre, verten når dem via publiserte porter
`host`	Containeren deler vertens nettverk direkte (ingen isolasjon)
`none`	Ingen nettverk
`overlay`	Nettverk over flere verter (Swarm)

Viktig regel: På et **brukerdefinert** bridge-nettverk kan containere nå hverandre **med navn** via Dockers innebygde DNS. På standardnettverket `bridge` kan de ikke det.

```
        $ docker network create appnett
        $ docker run -d --name db --network appnett -e POSTGRES_PASSWORD=bytt-meg
postgres:16
        $ docker run -it --rm --network appnett postgres:16 psql -h db -U postgres
        #                                                       ▲ «db» er
containernavnet = DNS-navn
```



> **Vanlig feil:** Inne i en container betyr `localhost` **containeren selv**, ikke din PC og ikke en annen container. Bruk tjenestenavnet (`db`) for å nå andre containere.

Docker Compose

Hva er det? Et verktøy for å definere og kjøre **flere containere sammen** med én YAML-fil (`compose.yaml`) og én kommando.

Hvorfor? En ekte app består ofte av web + database + cache + kanskje en kø. Å starte alle med lange `docker run` -kommandoer er tungvint og feilutsatt.

`compose.yaml` :

```yaml
services:
  web:
    build: .                                # bygg fra Dockerfile i denne mappen
    ports:
      - "8000:8000"                         # vert:container
    environment:
      APP_ENV: utvikling
      DATABASE_URL: postgresql://app:${DB_PASSWORD}@db:5432/appdb   # ${} hentes
fra .env-fil
    depends_on:
      db:
        condition: service_healthy          # vent til databasen er frisk
    restart: unless-stopped

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: appdb
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d appdb"]
      interval: 5s
      retries: 10

  cache:
    image: redis:7

volumes:
  pgdata:                                   # named volume
```

`.env` (sjekkes **ikke** inn i Git):

```
DB_PASSWORD=lokalt-utviklingspassord
```

```
$ docker compose up -d           # bygg (ved behov) og start alt i bakgrunnen
$ docker compose ps              # status
$ docker compose logs -f web     # logger for én tjeneste
$ docker compose exec db psql -U app appdb   # kjør kommando i en tjeneste
$ docker compose up -d --build   # bygg på nytt etter kodeendringer
$ docker compose down            # stopp og fjern containere og nettverk
$ docker compose down -v         # ... og slett volumes (data!)
```

Compose lager automatisk et eget nettverk for prosjektet, så tjenestene når hverandre på navn (`db`, `cache`).

> **Merk:** Den moderne kommandoen er `docker compose` (med mellomrom, Compose v2 som Docker-plugin). Eldre guider bruker `docker-compose` (bindestrek) og en `version:`-linje øverst i filen – begge deler er utdaterte.

> **Fra arbeidslivet:** Compose brukes mest til **lokal utvikling og testing** (og i CI for integrasjonstester). I produksjon brukes typisk Kubernetes, Azure Container Apps eller App Service.

Registre (*registries*)

Et register lagrer og distribuerer images.

Register	Type
Docker Hub	Offentlig (og private repoer); har grenser for nedlastinger for anonyme/gratisbrukere
GitHub Container Registry (`ghcr.io`)	Knyttet til GitHub-repoer
Azure Container Registry (ACR)	Privat, i Azure (kapittel 7.2)
Amazon ECR, Google Artifact Registry	De andre skyene

Publisere til Docker Hub:

```
$ docker login                                    # brukernavn + access token
$ docker tag flaskapp:1.0 <brukernavn>/flaskapp:1.0
$ docker push <brukernavn>/flaskapp:1.0
```

Publisere til ACR:

```
$ az acr login -n acrlab123
$ docker tag flaskapp:1.0 acrlab123.azurecr.io/flaskapp:1.0
$ docker push acrlab123.azurecr.io/flaskapp:1.0
```

Bygg for riktig arkitektur. Bygger du på en Mac med Apple Silicon (ARM), får du et ARM-image som ikke kjører på vanlige x86-servere i skyen. Bruk:

```
$ docker buildx build --platform linux/amd64 -t acrlab123.azurecr.io/flaskapp:1.0
--push .
```

Container-sikkerhet

Praksis	Hvorfor
Kjør som ikke-root (`USER`)	Begrenser skaden hvis appen blir kompromittert
Små, offisielle base-images (slim, alpine, distroless)	Færre pakker = færre sårbarheter

Praksis	Hvorfor
Faste versjoner (tag eller digest)	Forutsigbarhet, ingen overraskelser
Skann images (Trivy, Docker Scout, Defender for Containers)	Finn kjente sårbarheter (CVE-er)
Ingen hemmeligheter i images	Alle som har imaget kan lese alle lag – også slettede filer i tidligere lag
`.dockerignore`	Hindrer at `.env`, `.git` og nøkler havner i imaget
Oppdater base-images jevnlig	Sikkerhetsoppdateringer
Begrens ressurser (`--memory`, `--cpus`)	Hindrer at én container tar ned verten
Read-only filsystem (`--read-only`) og dropp capabilities (`--cap-drop ALL`)	Herding
Signer images (Notation, Cosign) og lag SBOM	Forsyningskjedesikkerhet

```
        # Skann et image med Trivy (kan kjøres som container)
        $ docker run --rm -v /var/run/docker.sock:/var/run/docker.sock
aquasec/trivy:latest image flaskapp:1.0
```

(Her er `latest` for selve skanneverktøyet akseptabelt i en lab; i en pipeline bør også verktøyversjonen låses.)

> **Vanlig feil:** `ENV DB_PASSWORD=hemmelig` i Dockerfile, eller `COPY .env .`. Hemmeligheten ligger da i imaget for alltid, og alle med tilgang til registeret kan lese den. Hemmeligheter gis **når containeren kjører** (miljøvariabler fra Key Vault, Kubernetes Secrets, osv.), og byggehemmeligheter med `RUN --mount=type=secret`.

> **Vanlig feil:** Å montere `/var/run/docker.sock` inn i en app-container. Det gir containeren full kontroll over Docker – og dermed verten.

Feilsøking av containere

Metode:

```
1. Kjører den?           docker ps -a           (Exited? Restarting?)
2. Hvorfor stoppet den?  docker logs <navn>     (feilmelding fra appen)
                         docker inspect <navn> --format '{{.State.ExitCode}}'
3. Gå inn og se          docker exec -it <navn> sh
4. Nettverk/porter       docker port <navn> ; curl fra innsiden ; lytter appen på 0.0.0.0?
5. Ressurser             docker stats           (minne fullt? → OOMKilled)
6. Bygg-problemer        docker build --progress=plain --no-cache .
```

Vanlige feil og løsninger:

Symptom	Sannsynlig årsak	Løsning
Containeren stopper umiddelbart (`Exited (0)`)	Hovedprosessen avsluttet (f.eks. CMD kjører et skript som er ferdig)	Containeren lever bare så lenge hovedprosessen kjører; start en prosess som blir kjørende i forgrunnen
`Exited (1)`	Appen krasjet	`docker logs`
`Exited (137)`	Drept med SIGKILL, ofte **OOMKilled** (tom for minne)	`docker inspect` → `OOMKilled: true`; øk minne eller fiks lekkasje
Ingen svar på `localhost:8000`	Port ikke publisert, eller appen lytter på `127.0.0.1`	`-p 8000:8000`, og bind til `0.0.0.0`
`port is already allocated`	Porten brukes allerede på verten	Velg en annen vertsport (`-p 8001:8000`) eller stopp den andre
`exec format error`	Feil CPU-arkitektur (ARM vs. AMD64)	`docker buildx build --platform linux/amd64`
`permission denied` på filer	Appen kjører som ikke-root, men filene eies av root	`COPY --chown=appuser:appuser` eller `chown` i Dockerfile
Kan ikke nå `db`	Containerne er ikke på samme brukerdefinerte nettverk	Bruk Compose eller `--network`
Endringer i koden vises ikke	Du bruker et gammelt image	`docker compose up --build`
`toomanyrequests` fra Docker Hub	Nedlastingsgrense	Logg inn, eller bruk et eget register (ACR) som speil

```
# Nyttig: start et feilende image med et shell i stedet for vanlig kommando
$ docker run -it --rm --entrypoint sh flaskapp:1.0
```

Prosjekt: bygg, kjør og publiser en applikasjon

Mål: Containeriser Flask-appen, kjør den med PostgreSQL via Compose, skann den, og publiser imaget til GitHub Container Registry automatisk med GitHub Actions. Tid: 2–3 timer. Kostnad: gratis.

Steg 1 – appen med database

Utvid `app.py` til å telle besøk i PostgreSQL:

```
        import os
        import psycopg                                  # pip: psycopg[binary]
        from flask import Flask, jsonify

        app = Flask(__name__)
        DB_URL = os.environ["DATABASE_URL"]             # krasj tidlig hvis den mangler

        def db():
            return psycopg.connect(DB_URL, autocommit=True)

        with db() as conn:                              # lag tabellen ved oppstart
            conn.execute("CREATE TABLE IF NOT EXISTS besok (id SERIAL PRIMARY KEY, tid
TIMESTAMPTZ DEFAULT now())")

        @app.route("/")
        def hjem():
            with db() as conn:
                conn.execute("INSERT INTO besok DEFAULT VALUES")
                antall = conn.execute("SELECT count(*) FROM besok").fetchone()[0]
            return f"Hei! Denne siden har hatt {antall} besøk."

        @app.route("/health")
        def health():
            return jsonify(status="ok")
```

`requirements.txt`:

```
flask==3.1.*
gunicorn==23.*
psycopg[binary]==3.2.*
```

Steg 2 – Dockerfile, .dockerignore og compose.yaml

Bruk Dockerfile og `.dockerignore` fra tidligere i kapitlet, og `compose.yaml` fra Compose-seksjonen (du kan fjerne `cache`). Legg til en `HEALTHCHECK` i Dockerfile:

```
        HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"
```

Steg 3 – kjør

```
        $ echo "DB_PASSWORD=lokalt-passord" > .env
        $ docker compose up -d --build
        $ curl localhost:8000          # «Denne siden har hatt 1 besøk.»
        $ curl localhost:8000          # 2 ...
        $ docker compose down && docker compose up -d
        $ curl localhost:8000          # 3 – dataene overlevde takket være volumet
```

Steg 4 – skann

```
        $ docker build -t flaskapp:test .
        $ docker run --rm -v /var/run/docker.sock:/var/run/docker.sock
aquasec/trivy:latest image --severity HIGH,CRITICAL flaskapp:test
```

Steg 5 – publiser automatisk med GitHub Actions

`.github/workflows/docker.yml`:

```
        name: Docker
        on:
          push:
            branches: [main]

        permissions:
          contents: read
          packages: write                          # tillat push til ghcr.io

        jobs:
          build-push:
            runs-on: ubuntu-latest
            steps:
              - uses: actions/checkout@v4
              - uses: docker/login-action@v3
                with:
                  registry: ghcr.io
                  username: ${{ github.actor }}
                  password: ${{ secrets.GITHUB_TOKEN }}     # automatisk token, ingen egen
hemmelighet
              - uses: docker/build-push-action@v6
                with:
                  push: true
                  tags: |
                    ghcr.io/${{ github.repository }}:${{ github.sha }}
                    ghcr.io/${{ github.repository }}:latest
```

(Repo-navnet må være med små bokstaver for ghcr.io. Har du store bokstaver i brukernavnet, sett taggen manuelt med små bokstaver.)

Steg 6 – verifiser

Push til `main`. Under **Packages** på GitHub-profilen din ser du imaget. Hent og kjør det et annet sted:

```
        $ docker run -d -p 8000:8000 -e DATABASE_URL=... ghcr.io/<bruker>/<repo>:<sha>
```

Forventet resultat, feilsøking og opprydding

- **Resultat:** Et repo med Dockerfile, compose.yaml, workflow, og et publisert image i ghcr.io.

- **Feilsøking:** Workflow feiler med `denied` → sjekk `permissions: packages: write`. Appen krasjer med `KeyError: 'DATABASE_URL'` → miljøvariabelen mangler.
- **Opprydding:** `docker compose down -v`, `docker system prune`. Pakker på ghcr.io kan slettes under *Package settings*.
- **På CV/GitHub:** «Containeriserte en Python-webapp med multi-stage Dockerfile, Docker Compose for lokal utvikling med PostgreSQL, sårbarhetsskanning med Trivy og automatisk publisering til GitHub Container Registry med GitHub Actions.»

Kapitteloppsummering

- En container pakker app + avhengigheter, og deler vertens kjerne. Lettere og raskere enn VM-er, men svakere isolasjon.
- Image = mal (lag, skrivebeskyttet). Container = kjørende instans. Registre lagrer images.
- Dockerfile: sjeldent endrede ting øverst for cache, `.dockerignore`, ikke-root, faste versjoner, multi-stage for små images.
- Volumes gjør data varige; brukerdefinerte nettverk lar containere finne hverandre på navn.
- Compose kjører flere containere sammen – ideelt for lokal utvikling.
- Sikkerhet: ingen hemmeligheter i images, skann, minimalt image, ikke-root.
- Feilsøking: `ps -a`, `logs`, `inspect`, `exec`, `stats`.

Øvingsoppgaver

1. Kjør `docker run -it --rm alpine:3.20 sh`, lag en fil, avslutt, og start på nytt. Finnes filen? Hvorfor ikke?
2. Bygg Flask-imaget to ganger etter hverandre. Hvor lang tid tar andre bygg? Endre `app.py` og bygg igjen – hvilke lag gjenbrukes?
3. Sammenlign størrelsen på `python:3.12`, `python:3.12-slim` og `python:3.12-alpine` med `docker images`.
4. Gjør Python-imaget ditt om til multi-stage og mål størrelsesforskjellen.
5. Fremprovoser `OOMKilled`: `docker run --memory=50m python:3.12-slim python -c "a = 'x' * 10**9"` og finn årsaken med `docker inspect`.
6. Legg Redis til i compose-filen og bruk den som teller i stedet for PostgreSQL.

Quiz

1. Hva er forskjellen på et image og en container?
2. Hva deler containere som VM-er ikke deler?
3. Hvorfor kopierer man `requirements.txt` før resten av koden i Dockerfile?

4. Hva er forskjellen på `RUN` og `CMD`?
5. Åpner `EXPOSE 80` porten slik at du kan nå den fra din PC?
6. Hvorfor bør du ikke bruke `:latest` i produksjon?
7. Hva skjer med data skrevet i en containers filsystem når containeren slettes?
8. Hvordan når en container en annen container i Compose?
9. Hva betyr exit-kode 137?
10. Du har slettet en fil med passord i et senere lag (`RUN rm .env`). Er passordet borte fra imaget?

Fasit

1. **Image**: skrivebeskyttet mal med app og avhengigheter. **Container**: en kjørende instans av et image, med et skrivbart lag.
2. **Operativsystemets kjerne** (Linux-kjernen på verten).
3. For å utnytte lag-cachen: avhengighetene installeres bare på nytt når `requirements.txt` endres, ikke ved hver kodeendring.
4. `RUN` kjøres **når imaget bygges**. `CMD` er kommandoen som kjøres **når containeren starter**.
5. **Nei.** `EXPOSE` er bare dokumentasjon. Du må publisere porten med `-p` (eller `ports:` i Compose).
6. Fordi taggen kan peke på en ny versjon når som helst, slik at du ikke vet hva som kjører, og rollback blir vanskelig.
7. Den forsvinner – med mindre den er lagret i et volume eller en bind mount.
8. Med **tjenestenavnet** som vertsnavn (f.eks. `db:5432`), via Docker sin innebygde DNS på Compose-nettverket.
9. Prosessen ble drept med SIGKILL (128 + 9), ofte fordi containeren gikk tom for minne (OOMKilled).
10. **Nei.** Filen finnes fortsatt i laget der den ble kopiert inn, og kan hentes ut. Bytt passordet og bygg imaget uten å kopiere filen inn.

Del 10 – Kubernetes

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hvorfor Kubernetes finnes og hvilke problemer det løser,
- beskrive arkitekturen: cluster, kontrollplan og noder,
- jobbe med pods, deployments, services, ingress, ConfigMaps, Secrets og namespaces,
- forstå persistent lagring og autoskalering,
- bruke Helm, og forstå grunnleggende nettverk, sikkerhet og overvåking i Kubernetes,
- deploye en applikasjon til Azure Kubernetes Service (AKS).

Hvorfor finnes Kubernetes?

Problemet. Docker gjør det enkelt å kjøre **én** container på **én** maskin. Men i produksjon har du kanskje 50 mikrotjenester, hver med 3–20 kopier, fordelt på 30 servere. Hvem bestemmer:

- hvilken server hver container skal kjøre på?
- hva som skjer når en container krasjer, eller en hel server dør?
- hvordan du oppdaterer til ny versjon uten nedetid?
- hvordan containere finner hverandre når de flytter rundt?
- hvordan du skalerer når trafikken øker?

Hva er det? Kubernetes (ofte forkortet **K8s** – K, åtte bokstaver, s) er en **container-orkestreringsplattform**. Den kjører containere på et cluster av maskiner og sørger hele tiden for at virkeligheten matcher det du har beskrevet.

Kubernetes ble opprinnelig utviklet av Google (inspirert av deres interne system *Borg*), gjort åpen kildekode i 2014, og er nå et prosjekt under **Cloud Native Computing Foundation (CNCF)**.

> **Hverdagsanalogi:** Kubernetes er **dirigenten i et orkester**. Du gir dirigenten noten («vi skal ha 3 fioliner, 2 cello og 1 tromme som spiller dette stykket»). Dirigenten plasserer musikerne, passer på at alle spiller, og hvis en fiolinist besvimer, henter dirigenten straks en ny fra reservebenken. Du forteller **hva** du vil ha – dirigenten sørger for **hvordan**.

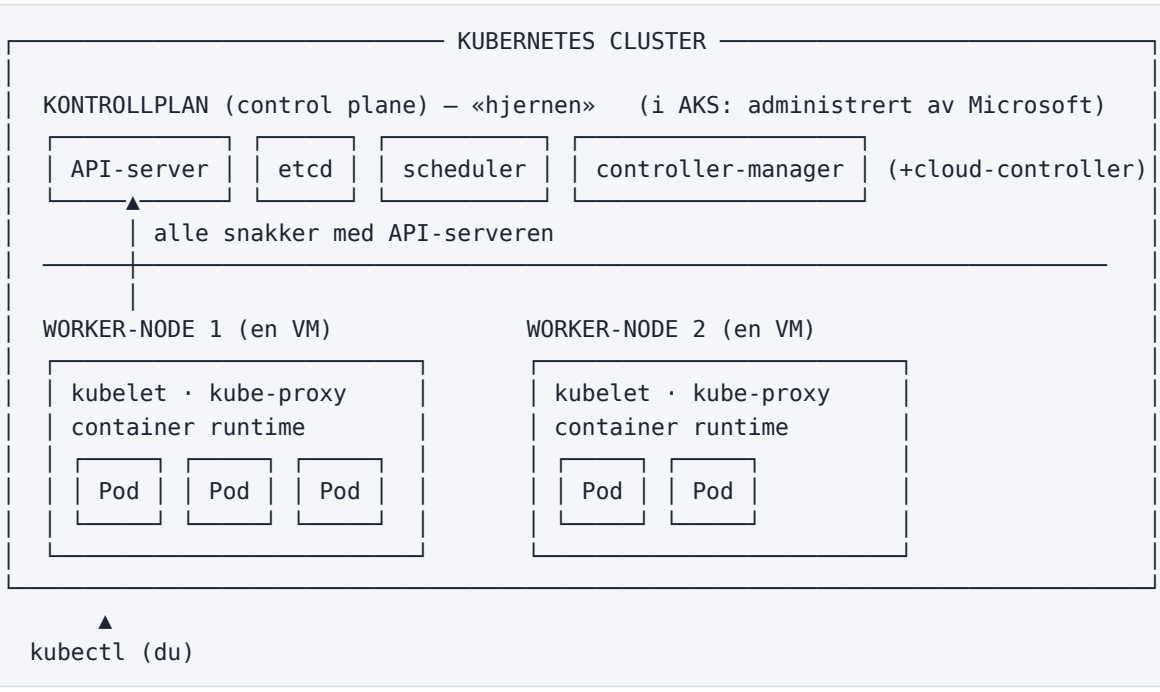
Det deklarative prinsippet – ønsket tilstand

Dette er den viktigste ideen i Kubernetes:



Du sier ikke «start en container på server 4». Du sier «det skal alltid være 3». Kubernetes' **kontrollere** jobber kontinuerlig for å få den faktiske tilstanden til å matche den ønskede. Dette kalles **reconciliation** (avstemming) og gjør systemet **selvreparerende** (*self-healing*).

Arkitekturen: cluster og noder



Kontrollplanet:

Komponent	Oppgave
kube-apiserver	Inngangsdøren. Alle kommandoer (fra deg, `kubectl`, og fra andre komponenter) går gjennom API-serveren
etcd	Nøkkel–verdi-database som lagrer **hele** clusterets tilstand. Det viktigste å ta backup av i selvdriftede clustere
kube-scheduler	Bestemmer **hvilken node** nye pods skal kjøre på, basert på ressurser og regler

Komponent	Oppgave
kube-controller-manager	Kjører kontrollerne som sørger for ønsket tilstand (f.eks. at riktig antall pods kjører)
cloud-controller-manager	Snakker med skyleverandøren (lager lastbalanserere, disker i Azure)

Hver node:

Komponent	Oppgave
kubelet	Agenten på noden. Får beskjed om hvilke pods som skal kjøre, og sørger for at containerne kjører og er friske
container runtime	Kjører containerne (typisk **containerd**)
kube-proxy	Setter opp nettverksregler slik at Services fungerer (noen nettverksløsninger erstatter den)

Hvordan en deploy faktisk skjer:

1. Du kjører `kubectl apply -f deployment.yaml`.
2. API-serveren validerer og lagrer ønsket tilstand i etcd.
3. Deployment-kontrolleren ser at det trengs pods, og lager pod-objekter.
4. Scheduleren ser pods uten node, og tildeler dem noder.
5. Kubelet på hver node ser «sine» nye pods, ber container-runtime hente image og starte containerne.
6. Kubelet rapporterer status tilbake. Kontrollerne fortsetter å passe på.

kubectl og første møte

kubectl er kommandolinjeverktøyet for Kubernetes. Det leser tilkoblingsinformasjon fra `~/.kube/config` (*kubeconfig*).

Et lokalt cluster å øve på (gratis):

- **Docker Desktop** – slå på Kubernetes i innstillingene.
- **kind** (*Kubernetes in Docker*) – `kind create cluster`.
- **minikube** – `minikube start`.
- **k3d/k3s** – lettvektsvarianter.

```
$ kubectl version                    # klient- og serverversjon
$ kubectl cluster-info
$ kubectl get nodes                  # list noder
NAME                 STATUS   ROLES           AGE   VERSION
kind-control-plane   Ready    control-plane   2m    v1.3x.x

$ kubectl get pods -A                # pods i ALLE namespaces (systemkomponenter)
$ kubectl api-resources              # alle ressurstyper clusteret kjenner
$ kubectl explain deployment.spec    # dokumentasjon for et felt – svært nyttig!
```

Kommandomønsteret: `kubectl <verb> <ressurstype> <navn> [flagg]`

Verb	Gjør
`get`	List ressurser (`-o wide` , `-o yaml` , `-w` = følg med)
`describe`	Detaljer og **Events** – første sted å se ved feil
`apply -f`	Opprett/oppdater fra YAML (deklarativt – foretrukket)
`delete`	Slett
`logs`	Logger fra en container (`-f` følg, `--previous` forrige krasjede)
`exec -it`	Kjør kommando i en container
`port-forward`	Videresend en lokal port til en pod/service
`rollout`	Status, historikk og rollback for deployments
`scale`	Endre antall replikaer

Pods

Hva er det? En **pod** er den minste enheten i Kubernetes: **én eller flere containere** som kjører sammen på samme node, deler **IP-adresse** og kan dele **lagring**.

Hvorfor ikke bare containere? Noen ganger trenger to prosesser å være tett koblet: f.eks. en app-container og en hjelpecontainer (*sidecar*) som sender logger eller håndterer nettverkstrafikk. De må alltid kjøre sammen og snakke via `localhost` .

> **Husk:** I de aller fleste tilfeller er **én container per pod**. Tenk «pod ≈ én instans av appen din».

```yaml
# pod.yaml – en enkel pod (i praksis lager du sjelden pods direkte)
apiVersion: v1              # hvilken API-versjon ressurstypen tilhører
kind: Pod                   # ressurstype
metadata:
  name: hei                 # navn
  labels:
    app: hei                # merkelapper – brukes til å velge ut ressurser
spec:                       # ønsket tilstand
  containers:
    - name: web
      image: nginx:1.27
      ports:
        - containerPort: 80
```

```
$ kubectl apply -f pod.yaml
$ kubectl get pods -o wide            # se hvilken node og IP poden fikk
$ kubectl describe pod hei
$ kubectl logs hei
$ kubectl exec -it hei -- sh          # -- skiller kubectl-flagg fra kommandoen
$ kubectl port-forward pod/hei 8080:80   # nå på http://localhost:8080
$ kubectl delete pod hei
```

Pods er dødelige. En pod som dør, blir *ikke* gjenopplivet – den blir erstattet av en **ny** pod med ny IP. Derfor lager vi nesten aldri pods direkte, men bruker **Deployments**.

Livssyklus og status: `Pending` (venter på node/image) → `Running` → `Succeeded` / `Failed`. Vanlige feilstatuser: `ImagePullBackOff`, `CrashLoopBackOff`, `OOMKilled` (se feilsøking).

Deployments

Hva er det? En Deployment beskriver **ønsket tilstand for en tilstandsløs app**: hvilket image, hvor mange kopier (*replicas*), og hvordan oppdateringer skal rulles ut. Den lager og administrerer et **ReplicaSet**, som igjen sørger for riktig antall pods.

```
Deployment (web)  ──► ReplicaSet (web-7d9f, image v2) ──► Pod, Pod, Pod
                  └─► ReplicaSet (web-5c8a, image v1) ──► (0 pods – beholdt for rollback)
```

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3                          # ønsket antall pods
  selector:
    matchLabels:
      app: web                         # hvilke pods denne deploymenten eier
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0                # aldri færre enn 3 tilgjengelige under
oppdatering
      maxSurge: 1                      # maks 1 ekstra pod under oppdatering
  template:                            # mal for podene
    metadata:
      labels:
        app: web                       # MÅ matche selector
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports:
            - containerPort: 80
          resources:
            requests:                  # det poden er garantert (brukes av
scheduleren)
              cpu: 100m                # 100 millicores = 0,1 CPU-kjerne
              memory: 128Mi
            limits:                    # maks den får bruke
              memory: 256Mi            # over dette → OOMKilled
          readinessProbe:              # er poden klar til å ta trafikk?
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:               # lever den fortsatt? hvis ikke → restart
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 10
            periodSeconds: 10
```

```
$ kubectl apply -f deployment.yaml
$ kubectl get deployments,replicasets,pods
$ kubectl delete pod <en-av-web-podene>     # se at en ny pod straks blir laget!
$ kubectl scale deployment web --replicas=5

# Rullerende oppdatering til ny versjon
$ kubectl set image deployment/web web=nginx:1.27-alpine
$ kubectl rollout status deployment/web
$ kubectl rollout history deployment/web
$ kubectl rollout undo deployment/web       # rollback til forrige versjon
```

(I et GitOps-/CI-oppsett endrer man heller image-taggen i YAML-filen og kjører `apply` – da er Git sannheten.)

Probes – helsesjekker

Probe	Spørsmål	Hvis den feiler
readinessProbe	Er poden klar til å ta imot trafikk?	Poden fjernes fra Service-ens endepunkter (får ikke trafikk), men startes ikke på nytt
livenessProbe	Lever poden, eller har den hengt seg opp?	Containeren **restartes**
startupProbe	Har en treg app rukket å starte?	Liveness/readiness venter til denne lykkes

> **Vanlig feil:** En for aggressiv livenessProbe (kort tid, sjekker databasen) får pods til å restarte i en løkke når databasen er treg – og gjør et lite problem til et stort. Liveness skal sjekke at *prosessen selv* lever, ikke avhengighetene.

Requests og limits

- **Requests** – det poden reserverer. Scheduleren plasserer bare poden på en node med nok ledige requests.
- **Limits** – taket. Overskrides **minne**-limit → containeren drepes (**OOMKilled**). Overskrides **CPU**-limit → containeren strupes (*throttling*), ikke drepes.
- Enheter: CPU i kjerner eller millicores (`500m` = en halv kjerne). Minne i `Mi` / `Gi`.

> **Husk:** Sett alltid **requests** (ellers kan scheduleren ikke planlegge fornuftig, og autoskalering virker ikke) og **minne-limits**.

Andre arbeidslast-typer

Ressurs	Bruk
Deployment	Tilstandsløse apper (web, API) – det vanligste
StatefulSet	Apper som trenger stabil identitet og egen lagring per pod (databaser, Kafka): `db-0`, `db-1` …
DaemonSet	Én pod på **hver** node (loggagenter, overvåkingsagenter)
Job	Kjør en oppgave til den er ferdig (migrering, batch)
CronJob	Jobb på tidsplan (samme cron-syntaks som Linux)

Services

Problemet. Pods kommer og går, og får ny IP hver gang. Hvordan skal andre finne dem?

Hva er det? En **Service** gir et sett med pods én **stabil IP-adresse og ett DNS-navn**, og lastbalanserer trafikk mellom dem. Den finner podene med en **label selector**.



Typer:

Type	Tilgjengelig fra	Bruk
ClusterIP (standard)	Bare inne i clusteret	Intern kommunikasjon mellom tjenester
NodePort	En port (30000–32767) på hver node	Testing, sjelden i produksjon
LoadBalancer	Ekstern IP via skyens lastbalanserer	I AKS lages en Azure Load Balancer med offentlig (eller intern) IP automatisk
ExternalName	DNS-alias til noe utenfor clusteret	Peke på en ekstern database

```yaml
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP
  selector:
    app: web              # send trafikk til pods med denne labelen
  ports:
    - port: 80            # porten Service-en lytter på
      targetPort: 80      # porten containeren lytter på
```

```
$ kubectl apply -f service.yaml
$ kubectl get svc
$ kubectl get endpointslices -l kubernetes.io/service-name=web   # hvilke pod-IP-er står bak?
$ kubectl run test --rm -it --image=busybox:1.36 -- wget -qO- http://web    # test fra innsiden
$ kubectl port-forward svc/web 8080:80                                       # test fra din PC
```

DNS i clusteret: En Service `web` i namespace `butikk` kan nås som `web` (fra samme namespace) eller `web.butikk` / `web.butikk.svc.cluster.local` (fra andre namespaces). CoreDNS sørger for dette.

> **Vanlig feil:** Service-en har ingen endepunkter. Nesten alltid fordi **selector ikke matcher** labels på podene (skrivefeil, `app: Web` vs. `app: web`), eller fordi podene ikke er *Ready*. Sjekk med `kubectl get pods --show-labels` og `kubectl describe svc web`.

Ingress og Gateway API

Problemet. Med `type: LoadBalancer` får **hver** tjeneste sin egen offentlige IP og lastbalanserer. Med 20 tjenester blir det dyrt og uoversiktlig, og du får ikke HTTP-funksjoner som ruting på sti og TLS.

Hva er det? Ingress er et Kubernetes-objekt med **HTTP-rutingregler**: hvilke vertsnavn og stier som skal til hvilke Services. Selve trafikken håndteres av en **Ingress-kontroller** (en reverse proxy som kjører i clusteret).

```
Internett ──► (én offentlig IP) ──► INGRESS-KONTROLLER
                                    │  butikk.no/        → Service frontend
                                    │  butikk.no/api     → Service api
                                    │  admin.butikk.no   → Service admin
```

```yaml
        # ingress.yaml
        apiVersion: networking.k8s.io/v1
        kind: Ingress
        metadata:
          name: butikk
        spec:
          ingressClassName: webapprouting.kubernetes.azure.com   # hvilken kontroller
(her: AKS app routing)
          tls:
            - hosts: [butikk.example.com]
              secretName: butikk-tls               # TLS-sertifikat lagret som Secret
          rules:
            - host: butikk.example.com
              http:
                paths:
                  - path: /api
                    pathType: Prefix
                    backend:
                      service:
                        name: api
                        port:
                          number: 80
                  - path: /
                    pathType: Prefix
                    backend:
                      service:
                        name: frontend
                        port:
                          number: 80
```

Ingress-kontrollere og utviklingen fremover:

- Kubernetes-prosjektet har kunngjort at det populære community-prosjektet **ingress-nginx** pensjoneres (med begrenset vedlikehold frem til en sluttdato i 2026). Mange miljøer bruker det fortsatt, men nye oppsett bør velge et aktivt vedlikeholdt alternativ. Sjekk kubernetes.io-bloggen for detaljer.
- **Gateway API** er Kubernetes' moderne etterfølger til Ingress: mer uttrykksfull (trafikkdeling for canary, header-basert ruting), med roller for plattformteam (*Gateway*) og appteam (*HTTPRoute*). Støttes av mange kontrollere (Envoy Gateway, Istio, Cilium, Traefik, NGINX Gateway Fabric m.fl.).
- I **AKS** finnes **application routing add-on** (administrert NGINX-basert kontroller, med Microsofts egne planer for overgang – sjekk Learn) og **Application Gateway for Containers** (Azure-administrert lag 7-lastbalansering som støtter Gateway API).

Et enkelt eksempel med Gateway API:

```yaml
        apiVersion: gateway.networking.k8s.io/v1
        kind: HTTPRoute
        metadata:
          name: butikk
        spec:
          parentRefs:
            - name: hoved-gateway          # en Gateway som plattformteamet har satt opp
          hostnames: ["butikk.example.com"]
          rules:
            - matches:
                - path: { type: PathPrefix, value: /api }
              backendRefs:
                - name: api
                  port: 80
                  weight: 90               # 90 % til stabil versjon
                - name: api-canary
                  port: 80
                  weight: 10               # 10 % til ny versjon (canary)
```

> **Husk:** Lær **begge** konseptene. Du vil møte Ingress i nesten alle eksisterende clustere, og Gateway API i nye.

TLS-sertifikater håndteres ofte automatisk med **cert-manager**, som henter sertifikater fra Let's Encrypt og lagrer dem som Secrets.

ConfigMaps og Secrets

Prinsipp: Samme image i alle miljøer – **konfigurasjonen** settes utenfra (jf. Twelve-Factor, kapittel 3).

ConfigMap – ikke-sensitiv konfigurasjon (loggnivå, URL-er, feature-flagg, konfigurasjonsfiler).

Secret – sensitiv informasjon (passord, tokens, nøkler).

```yaml
        apiVersion: v1
        kind: ConfigMap
        metadata:
          name: web-config
        data:
          APP_ENV: produksjon
          LOG_LEVEL: info
        ---                                # --- skiller flere objekter i samme fil
        apiVersion: v1
        kind: Secret
        metadata:
          name: web-secret
        type: Opaque
        stringData:                        # stringData: skriv i klartekst, lagres
base64-kodet
          DB_PASSWORD: ikke-sjekk-meg-inn-i-git
```

Bruk dem i en pod:

```
          spec:
            containers:
              - name: web
                image: minapp:1.0
                envFrom:
                  - configMapRef:
                      name: web-config          # alle nøkler blir miljøvariabler
                env:
                  - name: DB_PASSWORD
                    valueFrom:
                      secretKeyRef:
                        name: web-secret
                        key: DB_PASSWORD
                volumeMounts:                   # eller monter som filer
                  - name: config
                    mountPath: /etc/minapp
            volumes:
              - name: config
                configMap:
                  name: web-config
```

```
        $ kubectl create secret generic web-secret --from-literal=DB_PASSWORD='...'   #
uten å skrive YAML-fil
        $ kubectl get secret web-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
```

Vanlig feil: Å tro at Secrets er kryptert. Som standard er de bare **base64-kodet** – det er *koding*, ikke kryptering; alle som kan lese Secret-objektet kan lese verdien. Beskytt dem med: - **RBAC** – begrens hvem som kan lese Secrets, - **kryptering i etcd** (AKS krypterer etcd; KMS-integrasjon med Key Vault er mulig), - **ekstern hemmelighetslagring**: Azure Key Vault med **Secrets Store CSI Driver**, eller External Secrets Operator, - **aldri** sjekk inn Secret-YAML med ekte verdier i Git (bruk Sealed Secrets/SOPS hvis du må).

Merk: Pods som leser miljøvariabler fra en ConfigMap får **ikke** nye verdier automatisk når ConfigMap-en endres – podene må startes på nytt (`kubectl rollout restart deployment/web`). Monterte filer oppdateres etter en stund.

Namespaces

Hva er det? Namespaces deler et cluster i **logiske områder**. Navn må være unike innenfor et namespace, men kan gjentas i andre.

Hvorfor? Skille team, applikasjoner eller miljøer; sette **ressurskvoter** (*ResourceQuota*) og standardgrenser (*LimitRange*); og gi tilgang med RBAC per namespace.

```
        $ kubectl get namespaces
        NAME              STATUS
        default           Active     ← der ting havner hvis du ikke sier noe
        kube-system       Active     ← systemkomponenter (ikke rør!)
        kube-public, kube-node-lease ...

        $ kubectl create namespace butikk
        $ kubectl apply -f deployment.yaml -n butikk
        $ kubectl get pods -n butikk
        $ kubectl config set-context --current --namespace=butikk   # sett standard
namespace
```

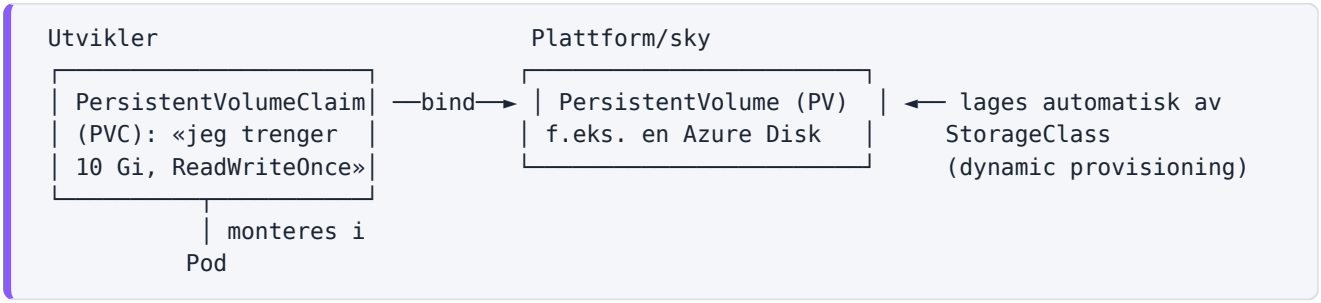
> **Vanlig feil:** «Podene mine er borte!» – de ligger i et annet namespace. Bruk `-n <namespace>` eller `-A`.

> **Merk:** Namespaces er **ikke** en sterk sikkerhetsgrense alene. Pods i ulike namespaces kan som standard snakke fritt med hverandre over nettverket – bruk NetworkPolicies.

Persistent lagring

Problemet. Containere er flyktige (kapittel 9). En database i Kubernetes trenger lagring som overlever at poden flyttes til en annen node.

Begrepene:



- **PersistentVolume (PV)** – en lagringsressurs i clusteret (en Azure Disk, en Azure Files-share).
- **PersistentVolumeClaim (PVC)** – en **forespørsel** om lagring fra en app («10 Gi, lese/skrive fra én node»).
- **StorageClass** – en «oppskrift» for å lage PV-er automatisk. AKS har innebygde klasser som `managed-csi` (Azure Disk) og `azurefile-csi` (Azure Files).

Tilgangsmoduser: `ReadWriteOnce` (én node om gangen – Azure Disk), `ReadWriteMany` (mange noder – Azure Files), `ReadOnlyMany`.

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: managed-csi
  resources:
    requests:
      storage: 10Gi
---
# I pod-spesifikasjonen:
#   volumes:
#     - name: data
#       persistentVolumeClaim:
#         claimName: data
#   containers[].volumeMounts:
#     - name: data
#       mountPath: /var/lib/postgresql/data
```

Reclaim policy bestemmer hva som skjer med disken når PVC-en slettes: `Delete` (disken slettes – standard for dynamiske) eller `Retain` (beholdes).

> **Kostnad:** Hver PVC med Azure Disk er en ekte, fakturert disk. Slett PVC-er (og sjekk etter «foreldreløse» disker i ressursgruppen `MC_...`) når du rydder opp.

Autoskalering

Kubernetes kan skalere på tre nivåer:

Hva	Skalerer	Basert på
HPA (*Horizontal Pod Autoscaler*)	Antall **pods**	CPU, minne eller egne metrikker
VPA (*Vertical Pod Autoscaler*)	Requests/limits per pod	Faktisk forbruk over tid
Cluster Autoscaler / **Node Auto-Provisioning**	Antall **noder**	Pods som ikke får plass (Pending) / ubrukte noder
KEDA (*event-driven*)	Pods, også til null	Kølengde, meldinger, HTTP-trafikk, cron … (AKS-add-on finnes)

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70        # mål: 70 % av CPU-request i snitt
```

```
$ kubectl get hpa -w      # følg med mens du genererer last
```

HPA krever **metrics-server** (installert i AKS) og at podene har **CPU-requests**.

```
Trafikk ↑ → CPU ↑ → HPA lager flere pods → noen blir Pending (ikke plass)
          → Cluster Autoscaler legger til en node → pods kjører → CPU ↓
```

Helm – pakkehåndtereren for Kubernetes

Problemet. En ekte app består av mange YAML-filer (Deployment, Service, Ingress, ConfigMap, HPA …) som er nesten like for dev, test og prod, men med ulike verdier.

Hva er det? Helm pakker Kubernetes-ressurser i **charts**: maler med variabler, pluss en `values.yaml` med standardverdier. En installert chart kalles en **release**.

> **Hverdagsanalogi:** Helm er `apt` for Kubernetes. Og en chart er en oppskrift med tomme felt («antall personer: ___»), der `values.yaml` fyller inn tallene.

```
minapp/                     # en Helm chart
├── Chart.yaml              # navn, versjon, beskrivelse
├── values.yaml             # standardverdier
└── templates/
    ├── deployment.yaml     # maler med {{ .Values.... }}
    ├── service.yaml
    └── _helpers.tpl
```

`templates/deployment.yaml` (utdrag):

```
        spec:
          replicas: {{ .Values.replicaCount }}
          template:
            spec:
              containers:
                - name: {{ .Chart.Name }}
                  image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

`values.yaml`:

```
        replicaCount: 2
        image:
          repository: acrlab123.azurecr.io/flaskapp
          tag: "1.0"
```

```
        $ helm create minapp                                  # lag en ferdig chart-mal å
starte fra
        $ helm install web ./minapp -n butikk                 # installer
        $ helm upgrade web ./minapp --set image.tag=1.1       # oppgrader med ny verdi
        $ helm upgrade --install web ./minapp -f values-prod.yaml   # installer eller
oppgrader (vanlig i CI)
        $ helm list -n butikk
        $ helm history web -n butikk
        $ helm rollback web 1 -n butikk                       # tilbake til revisjon 1
        $ helm template web ./minapp                          # vis ferdig YAML uten å
installere

        # Installer andres charts, f.eks. Prometheus-stacken:
        $ helm repo add prometheus-community https://prometheus-community.github.io/helm-
charts
        $ helm repo update
        $ helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring
--create-namespace
```

Kustomize er et alternativ (innebygd i `kubectl apply -k`) som bruker *overlegg* (*overlays*) i stedet for maler: en felles base + små endringer per miljø.

Kubernetes-nettverk

Grunnreglene i Kubernetes-nettverksmodellen:

1. Hver pod får **sin egen IP-adresse**.
2. Alle pods kan nå alle andre pods **uten NAT** (med mindre du begrenser det).
3. Agenter på en node kan nå alle pods på den noden.

Dette implementeres av en **CNI-plugin** (*Container Network Interface*). I AKS: **Azure CNI** (pods får IP fra VNet-et), **Azure CNI Overlay** (pods får IP fra et eget, privat område – sparer VNet-adresser; anbefalt for de fleste nye clustere), og **Azure CNI Powered by Cilium** (eBPF-basert, med avanserte nettverkspolicyer og observerbarhet).

Trafikkflyt:

```
Ekstern bruker → Azure LB / App Gateway → Ingress/Gateway → Service → Pod
Pod A → «http://api» (CoreDNS slår opp) → Service api (ClusterIP) → Pod B
```

NetworkPolicy – brannmurregler mellom pods. Uten policyer er **alt tillatt**.

```yaml
# Tillat bare trafikk til databasen fra pods med app=api
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-tillat-kun-api
  namespace: butikk
spec:
  podSelector:
    matchLabels:
      app: db                # policyen gjelder db-podene
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: api
      ports:
        - protocol: TCP
          port: 5432
```

NetworkPolicy krever at CNI-en støtter det (Azure NPM, Calico eller Cilium i AKS).

Service mesh (Istio, Linkerd, Cilium) legger til mTLS mellom tjenester, avansert trafikkstyring og observability – avansert tema, men nyttig å kjenne navnet på.

Kubernetes-sikkerhet

Område	Tiltak
Tilgang til API-et	Kubernetes **RBAC**: `Role` / `ClusterRole` (hva) + `RoleBinding` / `ClusterRoleBinding` (hvem). I AKS: Entra ID-innlogging og gjerne Azure RBAC for Kubernetes
Pod-identitet	**ServiceAccounts** for pods; i AKS **Workload Identity** for å nå Azure-tjenester uten hemmeligheter
Pod-sikkerhet	`securityContext`: `runAsNonRoot`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, drop capabilities. **Pod Security Admission** (nivåene *privileged*, *baseline*, *restricted*) per namespace
Nettverk	NetworkPolicies, private clustere, begrens API-servertilgang (authorized IP ranges / private API)

Område	Tiltak
Images	Bare fra godkjente registre (Azure Policy / Kyverno / Gatekeeper), skanning, signering
Hemmeligheter	Key Vault + CSI-driver, RBAC på Secrets
Oppdatering	Hold Kubernetes-versjon og node-images oppdatert (AKS auto-upgrade-kanaler)
Overvåking	Audit-logger, Microsoft Defender for Containers

```yaml
# Herdet container
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
```

```yaml
# RBAC: «utviklere» kan lese pods og logger i namespace butikk
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-leser
  namespace: butikk
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: utviklere-leser-pods
  namespace: butikk
subjects:
  - kind: Group
    name: "<objekt-ID til Entra-gruppen>"     # i AKS med Entra-integrasjon
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-leser
  apiGroup: rbac.authorization.k8s.io
```

```
$ kubectl auth can-i delete pods -n butikk --as=kari@firma.no    # test tilganger
```

Overvåking av Kubernetes

Hva vil du vite? Er nodene friske? Hvor mye CPU/minne brukes? Restarter pods? Feiler deployer? Hva sier applikasjonsloggene?

Grunnleggende kommandoer:

```
        $ kubectl top nodes                    # CPU/minne per node (krever metrics-
server)
        $ kubectl top pods -A --sort-by=memory
        $ kubectl get events -A --sort-by=.lastTimestamp   # hva har skjedd i clusteret?
        $ kubectl logs -l app=web --tail=50 -f  # logger fra alle pods med label
```

Plattformer:

- **Prometheus** (metrikker) + **Grafana** (dashboards) + **Alertmanager** – de facto standard i Kubernetes-verdenen (kapittel 13). Installeres ofte med Helm-charten `kube-prometheus-stack`.
- **I AKS: Azure Monitor managed service for Prometheus**, **Azure Managed Grafana**, og **Container Insights** (logger til Log Analytics). Slås på med `az aks update --enable-azure-monitor-metrics` og tilsvarende for logger.
- **Logger:** Containere skal logge til **stdout/stderr**; en agent (DaemonSet) samler dem.

Azure Kubernetes Service i praksis

Se også kapittel 7.2. Her er en komplett flyt fra image til offentlig app i AKS.

```
        RG=rg-aks-lab; LOC=norwayeast; ACR=acrlab$RANDOM; AKS=aks-lab
        az group create -n $RG -l $LOC

        # 1. Register og image
        az acr create -g $RG -n $ACR --sku Basic
        az acr build -r $ACR -t flaskapp:1.0 ./flaskapp            # eller docker build +
push

        # 2. Cluster med tilgang til ACR og app routing (administrert ingress)
        az aks create -g $RG -n $AKS --tier free --node-count 2 --node-vm-size
Standard_B2s \
          --attach-acr $ACR --enable-app-routing --generate-ssh-keys
        az aks get-credentials -g $RG -n $AKS

        # 3. Deploy
        kubectl create namespace demo
        kubectl apply -n demo -f k8s/                              # deployment.yaml,
service.yaml, ingress.yaml
        kubectl get pods,svc,ingress -n demo
```

`k8s/deployment.yaml` bruker `image: <ACR>.azurecr.io/flaskapp:1.0`, `k8s/service.yaml` er en ClusterIP-service, og `k8s/ingress.yaml` bruker `ingressClassName:`

`webapprouting.kubernetes.azure.com`. Etter et par minutter viser `kubectl get ingress -n demo` en offentlig IP-adresse.

Nyttige AKS-operasjoner:

```
        az aks nodepool add -g $RG --cluster-name $AKS -n userpool --node-count 1 --mode
User   # ny node pool
        az aks update -g $RG -n $AKS --enable-cluster-autoscaler --min-count 1 --max-count
3
        az aks get-upgrades -g $RG -n $AKS -o table          # tilgjengelige Kubernetes-
versjoner
        az aks upgrade -g $RG -n $AKS --kubernetes-version <versjon>
        az aks stop -g $RG -n $AKS                           # spar penger når du ikke
bruker clusteret
```

AKS-spesifikke ting å kjenne til:

- AKS lager en **egen ressursgruppe** (`MC_<rg>_<cluster>_<region>`) for nodene, diskene og last-balansereren. Ikke endre ting der manuelt.
- **Workload Identity** + Key Vault CSI-driveren for hemmeligheter.
- **Azure Policy-add-on** for å håndheve regler i clusteret.
- **Kubernetes-versjoner** støttes i en begrenset periode; planlegg jevnlige oppgraderinger (eller bruk auto-upgrade-kanaler).

> **Kostnad:** Nodene er VM-er som faktureres per sekund. To `Standard_B2s` -noder som står på i en måned kan koste mer enn hele studentkreditten tåler over tid. **Stopp eller slett clusteret samme dag.** Offentlige IP-er og disker koster også.

Feilsøking i Kubernetes

Gullregelen: `kubectl describe` og **Events** forteller deg som regel hva som er galt.

Status / symptom	Betyr	Sjekk
`Pending`	Poden får ikke node	`describe pod` → «Insufficient cpu/memory», node selector, PVC som ikke binder
`ImagePullBackOff` / `ErrImagePull`	Kan ikke hente image	Feil navn/tag, privat register uten tilgang (AcrPull?), Docker Hub-grense
`CrashLoopBackOff`	Containeren starter og krasjer gjentatte ganger	`kubectl logs <pod> --previous`, manglende miljøvariabel, feil kommando
`OOMKilled`	Overskred minne-limit	Øk limit eller fiks minnelekkasje

Status / symptom	Betyr	Sjekk
`Running`, men `0/1 READY`	readinessProbe feiler	Feil sti/port i proben, appen ikke klar
Service svarer ikke	Ingen endepunkter	Selector vs. labels, `targetPort`, pods ikke Ready
Ingress gir 404/502/503	Feil regel, service eller backend	`describe ingress`, kontrollerens logger, test service med port-forward
`CreateContainerConfigError`	Mangler ConfigMap/Secret som refereres	`describe pod`

Systematisk:

```
        kubectl get pods -n demo                          # 1. status
        kubectl describe pod <pod> -n demo                # 2. events nederst
        kubectl logs <pod> -n demo [--previous]           # 3. app-logger
        kubectl get svc,endpointslices -n demo            # 4. nettverk: har service
endepunkter?
        kubectl port-forward svc/web 8080:80 -n demo      # 5. test service direkte
        kubectl get events -n demo --sort-by=.lastTimestamp
```

Kapitteloppsummering

- Kubernetes orkestrerer containere over mange maskiner og holder **ønsket tilstand** med kontroll-løkker.
- Kontrollplan (API-server, etcd, scheduler, controller-manager) + noder (kubelet, runtime, kube-proxy).
- **Pod** = minste enhet. **Deployment** = ønsket antall pods og rullerende oppdateringer. Bruk probes og requests/limits.
- **Service** = stabil adresse og lastbalansering for pods. **Ingress/Gateway API** = HTTP-ruting inn i clusteret.
- **ConfigMap** for konfigurasjon, **Secret** for hemmeligheter (base64, ikke kryptert – beskytt dem).
- **Namespaces** organiserer; **PVC/StorageClass** gir varig lagring; **HPA/Cluster Autoscaler** skalerer.
- **Helm** pakker og versjonerer applikasjoner.
- Sikkerhet: RBAC, Workload Identity, securityContext, NetworkPolicies, oppdateringer.
- AKS = administrert kontrollplan; du betaler for nodene.

Øvingsoppgaver

1. Lag et lokalt cluster med kind eller Docker Desktop. Deploy nginx med 3 replikaer og en ClusterIP-service. Slett en pod og se hva som skjer.
2. Gjør en rullerende oppdatering til et image-tag som **ikke finnes**. Hva skjer med de gamle podene? Rull tilbake.
3. Lag en ConfigMap med `LOG_LEVEL` og bruk den i en deployment. Endre verdien – når ser podene den nye verdien?
4. Lag en HPA og generer last med `kubectl run load --rm -it --image=busybox:1.36 -- sh -c "while true; do wget -qO- http://web; done"`.
5. Skriv en NetworkPolicy som blokkerer all innkommende trafikk til et namespace, og en som åpner for én bestemt app.
6. Lag en Helm chart med `helm create`, og installer den med to ulike `values`-filer (dev og prod) i to namespaces.

Quiz

1. Hva betyr det at Kubernetes er deklarativt?
2. Hva lagres i etcd?
3. Hva er forskjellen på en pod og en deployment?
4. Hva skjer hvis en pod overskrider minne-limit? CPU-limit?
5. Hva er forskjellen på readinessProbe og livenessProbe?
6. Hvorfor trenger vi Services når pods har IP-adresser?
7. Hva er forskjellen på en Service av typen LoadBalancer og en Ingress?
8. Er Kubernetes Secrets kryptert som standard?
9. Hva er en PersistentVolumeClaim?
10. En pod står i `ImagePullBackOff` i AKS. Nevn to sannsynlige årsaker.
11. Hva er Helm, og hva er en release?
12. Hva gjør HPA, og hva krever den?

Fasit

1. Du beskriver **ønsket tilstand** (hva), og Kubernetes jobber kontinuerlig for at faktisk tilstand skal matche den (hvordan), i stedet for at du gir trinnvise kommandoer.
2. **Hele clusterets tilstand og konfigurasjon** – alle objekter (deployments, pods, secrets …).
3. En **pod** er én kjørende instans (en eller flere containere). En **deployment** beskriver ønsket antall pods og håndterer opprettelse, erstatning og rullerende oppdateringer av dem.
4. Minne: containeren **drepes (OOMKilled)** og restartes. CPU: containeren **strupes** (throttling), men drepes ikke.

5. **Readiness**: er poden klar for trafikk? Feiler → tas ut av Service, ikke restart. **Liveness**: lever poden? Feiler → containeren restartes.
6. Fordi pods er flyktige og får nye IP-er når de erstattes. En Service gir en **stabil** IP/DNS og lastbalanserer mellom de pods som til enhver tid finnes.
7. **LoadBalancer** gir én ekstern IP per tjeneste (lag 4). **Ingress** gir HTTP-ruting (vertsnavn/sti, TLS) til mange tjenester bak én inngang, via en Ingress-kontroller.
8. **Nei**, de er bare base64-kodet i API-et. Kryptering i etcd, RBAC og eksterne hvelv (Key Vault) må brukes for beskyttelse.
9. En forespørsel om lagring fra en app (størrelse og tilgangsmodus), som bindes til et PersistentVolume – ofte opprettet automatisk via en StorageClass.
10. Feil image-navn/tag; clusteret mangler tilgang til ACR (ikke `--attach-acr`/AcrPull); imaget er bygget for feil arkitektur; nedlastingsgrense i Docker Hub.
11. Helm er en pakkehåndterer for Kubernetes som bruker **charts** (maler + verdier). En **release** er en installert instans av en chart i et cluster, med versjonshistorikk.
12. HPA justerer **antall pods** automatisk basert på metrikker (f.eks. CPU). Den krever **metrics-server** og at podene har **resource requests**.

Del 11 – Infrastructure as Code

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva Infrastructure as Code (IaC) er og hvorfor det er standard i bransjen,
- bruke Terraform: providers, resources, variables, outputs, state, modules, plan og apply,
- sette opp remote state i Azure på en sikker måte,
- bygge Azure-infrastruktur automatisk med Terraform og Bicep,
- forklare configuration management med Ansible,
- forklare GitOps.

Hva er Infrastructure as Code?

Hva er det? Infrastructure as Code (IaC) betyr at du **beskriver infrastrukturen din i kodefiler** – nettverk, VM-er, databaser, tilganger – og lar et verktøy opprette og vedlikeholde den, i stedet for å klikke i en portal.

Hvorfor finnes det? Tenk deg at du har satt opp et produksjonsmiljø med 40 ressurser ved å klikke i Azure Portal. Nå skal du:

- lage et identisk testmiljø (husker du alle innstillingene?),
- forklare en kollega hva som er satt opp (finnes det dokumentasjon?),
- finne ut hvem som endret brannmurregelen i forrige uke, og hvorfor,
- gjenoppbygge alt etter at noen slettet ressursgruppen.

Med klikking er alt dette vanskelig og feilutsatt. Med IaC:

Fordel	Forklaring
Repeterbarhet	Samme kode gir samme infrastruktur – hver gang, i hvert miljø
Versjonskontroll	Infrastrukturen ligger i Git: historikk, hvem, hva, hvorfor
Kodegjennomgang	Endringer gjennomgås i pull requests før de rulles ut
Automatisering	Kjøres fra CI/CD-pipelines
Dokumentasjon	Koden *er* dokumentasjonen av hva som finnes
Gjenoppretting	Gjenoppbygg alt etter en katastrofe
Testing	Valider, lint, skann for sikkerhetsfeil før noe lages

> **Hverdagsanalogi:** Å klikke i portalen er som å lage mat uten oppskrift – det kan bli godt, men det blir aldri helt likt neste gang, og ingen andre kan gjenta det. IaC er **oppskriften**: presis, delbar og forbedrebar.

Deklarativ vs. imperativ

	Deklarativ	Imperativ
Du beskriver	**Hva** sluttresultatet skal være	**Hvordan** – stegene
Eksempel	«Det skal finnes et VNet med disse subnettene»	«Kjør `az network vnet create`, så `az network vnet subnet create` …»
Kjører du to ganger	Ingen endring (allerede riktig)	Kan feile eller lage duplikater
Verktøy	Terraform, Bicep, Kubernetes YAML	Bash-/Python-skript med CLI-kommandoer

Idempotens betyr at det å kjøre samme kode flere ganger gir samme resultat. Deklarative verktøy er idempotente av natur – en av hovedgrunnene til at de foretrekkes.

To kategorier IaC

```
 PROVISIONERING (lage infrastruktur)       KONFIGURASJONSSTYRING (konfigurere innsiden)
 Terraform, Bicep, Pulumi, CloudFormation  Ansible, Chef, Puppet, cloud-init
 «Lag VNet, VM, database, Key Vault»       «Installer nginx, sett opp brukere, kopier
config»
                    └──────────────── brukes ofte sammen ────────────────┘
```

Med containere og PaaS har behovet for konfigurasjonsstyring av servere blitt mindre (imaget *er* konfigurasjonen), men Ansible er fortsatt svært utbredt.

Terraform

Hva er det? Terraform er et deklarativt IaC-verktøy fra **HashiCorp**. Du skriver konfigurasjonen i **HCL** (*HashiCorp Configuration Language*), og Terraform snakker med API-ene til Azure, AWS, GCP, GitHub, Kubernetes og hundrevis av andre plattformer via **providers**.

> **Merk om lisens:** HashiCorp endret Terraforms lisens i 2023 til Business Source License (BSL), og HashiCorp er nå en del av IBM. Som følge av lisensendringen ble **OpenTofu** opprettet – en åpen kildekode-gren av Terraform under Linux Foundation, som er i stor grad kompatibel. Det du lærer her, gjelder i praksis for begge. Sjekk lisensvilkårene hvis du skal bygge kommersielle produkter *rundt* Terraform; vanlig bruk for å administrere egen infrastruktur er ikke berørt.

Installasjon

Følg «Install Terraform» på developer.hashicorp.com (pakker finnes for Windows, macOS og Linux). Terraform er også forhåndsinstallert i Azure Cloud Shell.

```
$ terraform version
Terraform v1.x.x
```

Arbeidsflyten

```
  skriv .tf-filer → terraform init → terraform plan → terraform apply → (endre) → plan
→ apply
                    (last ned         (vis hva som     (gjør endringene,
                     providers,        VIL skje)        oppdater state)
                     sett opp backend)
                                                          terraform destroy (slett alt)
```

Kommando	Gjør
`terraform init`	Initialiserer mappen: laster ned providers og moduler, setter opp state-backend
`terraform fmt`	Formaterer koden pent
`terraform validate`	Sjekker syntaks og intern konsistens
`terraform plan`	Sammenligner kode, state og virkelighet – viser hva som vil bli **lagt til (+), endret (~), slettet (-)**
`terraform apply`	Utfører planen (spør om bekreftelse)
`terraform destroy`	Sletter alt Terraform administrerer i denne konfigurasjonen
`terraform output`	Viser outputs
`terraform state list`	Lister ressurser i state
`terraform import` / `import`-blokk	Tar en eksisterende ressurs under Terraform-kontroll

Providers

Hva er det? En **provider** er en plugin som vet hvordan Terraform skal snakke med et bestemt API. For Azure bruker vi primært **azurerm** (Azure Resource Manager). Det finnes også **azuread** (Entra ID-objekter) og **azapi** (direkte mot ARM-API-et, for helt nye funksjoner).

```
# providers.tf
terraform {
  required_version = ">= 1.6"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"   # hvor provideren hentes fra (Terraform Registry)
      version = "~> 4.0"              # tillat 4.x, men ikke 5.0 (unngå brytende endringer)
    }
  }
}

provider "azurerm" {
  features {}                          # påkrevd blokk (kan være tom)
  subscription_id = var.subscription_id   # påkrevd i azurerm v4 (eller miljøvariabelen
ARM_SUBSCRIPTION_ID)
}
```

Autentisering mot Azure. Lokalt bruker provideren din `az login`-innlogging. I CI/CD bruker man en service principal eller managed identity, helst med **OIDC** (workload identity federation) via miljøvariabler som `ARM_CLIENT_ID`, `ARM_TENANT_ID`, `ARM_SUBSCRIPTION_ID` og `ARM_USE_OIDC=true` (kapittel 12).

Låsefilen `.terraform.lock.hcl` lagrer nøyaktig hvilken provider-versjon som ble valgt. **Sjekk den inn i Git**, slik at alle bruker samme versjon.

Resources

Hva er det? En **resource** er én infrastrukturkomponent Terraform skal administrere.

```
#        ressurstype            lokalt navn (brukes i Terraform-koden)
resource "azurerm_resource_group" "main" {
  name     = "rg-tf-lab"            # navnet i Azure
  location = "norwayeast"
  tags = {
    miljo = "lab"
    eier  = "student"
  }
}

resource "azurerm_virtual_network" "main" {
  name                = "vnet-tf-lab"
  location            = azurerm_resource_group.main.location   # REFERANSE til en annen
ressurs
  resource_group_name = azurerm_resource_group.main.name
  address_space       = ["10.0.0.0/16"]
}
```

Referanser og avhengigheter. `azurerm_resource_group.main.name` leses: «attributtet `name` fra ressursen av typen `azurerm_resource_group` med lokalt navn `main`». Fordi VNet-et refererer til ressursgruppen, **forstår Terraform at ressursgruppen må lages først**. Terraform bygger en **avhengighetsgraf** og lager uavhengige ressurser parallelt. (`depends_on` brukes bare når en avhengighet ikke kan uttrykkes med referanser.)

Data sources leser informasjon om ting som finnes fra før, uten å administrere dem:

```
data "azurerm_client_config" "current" {}          # info om hvem som kjører Terraform

data "azurerm_resource_group" "delt" {
  name = "rg-delte-tjenester"                      # en eksisterende gruppe noen andre eier
}
# bruk: data.azurerm_client_config.current.tenant_id
```

Dokumentasjonen for hver ressurstype finnes i **Terraform Registry** (registry.terraform.io → providers → hashicorp/azurerm). Den viser påkrevde og valgfrie argumenter og eksempler – bruk den hele tiden.

Variables

Hva er det? Input-variabler gjør koden **gjenbrukbar** – samme kode for dev og prod, med ulike verdier.

```
# variables.tf
variable "subscription_id" {
  type        = string
  description = "Azure subscription ID"
}

variable "location" {
  type        = string
  description = "Azure-region for alle ressurser"
  default     = "norwayeast"
}

variable "miljo" {
  type        = string
  description = "Miljø: dev eller prod"
  validation {
    condition     = contains(["dev", "prod"], var.miljo)
    error_message = "miljo må være 'dev' eller 'prod'."
  }
}

variable "subnets" {
  type = map(string)                    # en map: navn → CIDR
  default = {
    web  = "10.0.1.0/24"
    data = "10.0.2.0/24"
  }
}

variable "db_admin_password" {
  type      = string
  sensitive = true                      # skjules i plan/apply-utskrift
}
```

Typer: `string`, `number`, `bool`, `list(...)`, `map(...)`, `set(...)`, `object({...})`.

Hvordan sette verdier (senere vinner over tidligere):

1. `default` i variabelen,
2. `terraform.tfvars` eller `*.auto.tfvars` (lastes automatisk),
3. `-var-file=prod.tfvars`,
4. miljøvariabler `TF_VAR_navn` (f.eks. `export TF_VAR_db_admin_password=...`),
5. `-var="miljo=prod"` på kommandolinjen.

```
# dev.tfvars
miljo    = "dev"
location = "norwayeast"
```

Locals er hjelpeverdier som beregnes i koden:

```
locals {
  prefix = "tf-${var.miljo}"
  tags = {
    miljo          = var.miljo
    administrertAv = "terraform"
  }
}
# bruk: local.prefix, local.tags
```

> **Vanlig feil:** Å legge hemmeligheter i `terraform.tfvars` og sjekke den inn i Git. Bruk miljøvariabler (`TF_VAR_...`) fra et hemmelighetshvelv i pipelinen, eller enda bedre: la Terraform generere passordet og lagre det i Key Vault (eksempel senere).

Outputs

Hva er det? Verdier Terraform skriver ut etter `apply`, og som andre konfigurasjoner eller skript kan bruke.

```
# outputs.tf
output "resource_group_name" {
  value = azurerm_resource_group.main.name
}

output "web_app_url" {
  value       = "https://${azurerm_linux_web_app.main.default_hostname}"
  description = "URL til web-appen"
}
```

```
$ terraform output
$ terraform output -raw web_app_url      # bare verdien – nyttig i skript
```

Løkker og betingelser

```
# Lag ett subnett per element i var.subnets
resource "azurerm_subnet" "main" {
  for_each             = var.subnets               # web, data
  name                 = "snet-${each.key}"        # each.key = "web" / "data"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = [each.value]              # each.value = CIDR
}
# referanse: azurerm_subnet.main["web"].id

# Betingelse: lag Application Insights bare i prod
resource "azurerm_application_insights" "main" {
  count               = var.miljo == "prod" ? 1 : 0
  name                = "appi-${local.prefix}"
  location            = var.location
  resource_group_name = azurerm_resource_group.main.name
  workspace_id        = azurerm_log_analytics_workspace.main.id
  application_type    = "web"
}
```

Husk: Foretrekk `for_each` fremfor `count` for lister av ting. Med `count` identifiseres ressurser med indeks (0, 1, 2); fjerner du element nummer 0, «flytter» alle de andre seg, og Terraform vil slette og gjenskape dem. Med `for_each` identifiseres de med nøkkel.

State-filen

Hva er det? Terraform lagrer det den vet om infrastrukturen i en **state-fil** (`terraform.tfstate`): en kobling mellom ressursene i koden og de ekte ressursene i Azure (med ID-er og attributter).



Hvorfor trengs state?

- Koble kode til ekte ressurser (hvilket VNet i Azure er `azurerm_virtual_network.main`?).
- Vite hva som skal **slettes** når du fjerner noe fra koden.
- Ytelse og avhengigheter.

Viktig om state:

- State kan inneholde **hemmeligheter i klartekst** (passord, nøkler). Behandle den som sensitiv.
- **Aldri** sjekk inn `terraform.tfstate` i Git (legg i `.gitignore`).
- **Aldri** rediger state manuelt; bruk `terraform state mv/rm`, `moved`-blokker og `import`-blokker.
- **To personer som kjører apply samtidig** mot samme state kan ødelegge den → du trenger **lå-sing**.

Drift (*configuration drift*) er når virkeligheten avviker fra koden – f.eks. noen endret en innstilling i portalen. `terraform plan` avslører det, og `apply` setter det tilbake.

> **Vanlig feil:** «Jeg slettet ressursen i portalen, og nå klager Terraform.» Eller: «Jeg endret noe i portalen, og Terraform overskrev det.» Når et miljø er administrert av Terraform, gjør **alle** endringer gjennom koden.

Remote state

Problemet. Lokal state-fil på din laptop fungerer ikke for et team eller en pipeline: andre har den ikke, den kan bli borte, og det er ingen låsing.

Løsningen. Lagre state i en **remote backend**. I Azure: en **Blob Storage-container**, som støtter **låsing** via blob leases.

1. Lag lagringskontoen for state (én gang, f.eks. med CLI):

```
        RG=rg-tfstate; LOC=norwayeast; ST=sttfstate$RANDOM
        az group create -n $RG -l $LOC
        az storage account create -n $ST -g $RG -l $LOC --sku Standard_ZRS \
          --min-tls-version TLS1_2 --allow-blob-public-access false
        az storage account blob-service-properties update --account-name $ST -g $RG \
          --enable-versioning true --enable-delete-retention true --delete-retention-days
30
        az storage container create -n tfstate --account-name $ST --auth-mode login

        # Gi deg selv (og senere pipeline-identiteten) data-tilgang
        az role assignment create --assignee-object-id $(az ad signed-in-user show --query
id -o tsv) \
          --assignee-principal-type User --role "Storage Blob Data Contributor" \
          --scope $(az storage account show -n $ST -g $RG --query id -o tsv)
        echo $ST
```

2. Konfigurer backend:

```
# backend.tf
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-tfstate"
    storage_account_name = "sttfstate12345"     # navnet fra over
    container_name       = "tfstate"
    key                  = "lab/dev.terraform.tfstate"   # én state-fil per miljø/prosjekt
    use_azuread_auth     = true                  # Entra-innlogging i stedet for
kontonøkkel
  }
}
```

```
        $ terraform init           # flytter/kobler state til Azure (spør om å migrere
eksisterende state)
```

Nå er state delt, låst under `apply`, versjonert og sikret med RBAC.

> **Husk:** Ressursene for remote state (lagringskontoen) lages som regel **utenfor** den Terraform-konfigurasjonen som bruker dem – ellers får du et høna-og-egget-problem. Beskytt kontoen med en `CanNotDelete`-lås.

Modules

Hva er det? En **modul** er en mappe med Terraform-filer som kan **gjenbrukes** – som en funksjon i programmering. All Terraform-kode er egentlig en modul (*root module*); moduler du kaller fra den kalles *child modules*.

Hvorfor? Unngå copy-paste, standardiser (f.eks. «slik lager vi alltid en sikker lagringskonto hos oss»), og skjul kompleksitet.

```
infra/
├── modules/
│   └── webapp/                 # en gjenbrukbar modul
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
└── envs/
    ├── dev/
    │   ├── main.tf             # kaller modulen med dev-verdier
    │   ├── backend.tf
    │   └── dev.tfvars
    └── prod/
        ├── main.tf
        ├── backend.tf
        └── prod.tfvars
```

```
# envs/dev/main.tf
module "webapp" {
  source   = "../../modules/webapp"     # lokal sti; kan også være Git-URL eller Registry
  name     = "butikk-dev"
  location = "norwayeast"
  sku_name = "F1"
}

output "url" {
  value = module.webapp.url             # output fra modulen
}
```

Offentlige moduler: Terraform Registry har tusenvis av moduler, inkludert **Azure Verified Modules (AVM)** fra Microsoft (`Azure/avm-res-...`). Les koden og lås versjonen (`version = "x.y.z"`) før du bruker andres moduler.

Komplett eksempel: Azure-webapp med Key Vault, overvåking og managed identity

Denne konfigurasjonen bygger en realistisk liten plattform automatisk. Filene ligger i én mappe.

```
# main.tf
locals {
  prefix = "${var.prosjekt}-${var.miljo}"
  tags = {
    prosjekt       = var.prosjekt
    miljo          = var.miljo
    administrertAv = "terraform"
  }
}

data "azurerm_client_config" "current" {}

resource "random_string" "suffix" {          # krever provideren hashicorp/random
  length  = 5
  upper   = false
  special = false
}

resource "azurerm_resource_group" "main" {
  name     = "rg-${local.prefix}"
  location = var.location
  tags     = local.tags
}

# --- Overvåking ---
resource "azurerm_log_analytics_workspace" "main" {
  name                = "log-${local.prefix}"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  sku                 = "PerGB2018"
  retention_in_days   = 30
  tags                = local.tags
}

resource "azurerm_application_insights" "main" {
  name                = "appi-${local.prefix}"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  workspace_id        = azurerm_log_analytics_workspace.main.id
  application_type    = "web"
  tags                = local.tags
}

# --- Key Vault med RBAC ---
resource "azurerm_key_vault" "main" {
  name                        = "kv-${var.prosjekt}-${random_string.suffix.result}"
  location                    = azurerm_resource_group.main.location
  resource_group_name         = azurerm_resource_group.main.name
  tenant_id                   = data.azurerm_client_config.current.tenant_id
  sku_name                    = "standard"
  enable_rbac_authorization   = true          # nyere azurerm-versjoner:
rbac_authorization_enabled
  purge_protection_enabled    = false         # true i produksjon
  soft_delete_retention_days = 7
  tags                        = local.tags
}

# Den som kjører Terraform må kunne skrive hemmeligheter
resource "azurerm_role_assignment" "kv_terraform" {
```

```
  scope                = azurerm_key_vault.main.id
  role_definition_name = "Key Vault Secrets Officer"
  principal_id         = data.azurerm_client_config.current.object_id
}

resource "random_password" "api_key" {
  length  = 32
  special = false
}

resource "azurerm_key_vault_secret" "api_key" {
  name         = "ApiKey"
  value        = random_password.api_key.result
  key_vault_id = azurerm_key_vault.main.id
  depends_on   = [azurerm_role_assignment.kv_terraform]   # vent på tilgangen
}

# --- App Service ---
resource "azurerm_service_plan" "main" {
  name                = "plan-${local.prefix}"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  os_type             = "Linux"
  sku_name            = var.app_sku
  tags                = local.tags
}

resource "azurerm_linux_web_app" "main" {
  name                = "app-${local.prefix}-${random_string.suffix.result}"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  service_plan_id     = azurerm_service_plan.main.id
  https_only          = true
  tags                = local.tags

  identity {
    type = "SystemAssigned"                  # managed identity
  }

  site_config {
    always_on           = var.app_sku == "F1" ? false : true   # Free støtter ikke
always_on
    minimum_tls_version = "1.2"
    application_stack {
      python_version = "3.12"
    }
  }

  app_settings = {
    APP_ENV                                 = var.miljo
    APPLICATIONINSIGHTS_CONNECTION_STRING =
azurerm_application_insights.main.connection_string
    API_KEY =
"@Microsoft.KeyVault(VaultName=${azurerm_key_vault.main.name};SecretName=${azurerm_key_vaul
t_secret.api_key.name})"
  }
}

# Appens identitet får lese hemmeligheter
```

```
resource "azurerm_role_assignment" "kv_app" {
  scope                = azurerm_key_vault.main.id
  role_definition_name = "Key Vault Secrets User"
  principal_id         = azurerm_linux_web_app.main.identity[0].principal_id
}
```

```
# variables.tf
variable "subscription_id" { type = string }
variable "prosjekt" {
  type    = string
  default = "butikk"
}
variable "miljo" {
  type    = string
  default = "dev"
}
variable "location" {
  type    = string
  default = "norwayeast"
}
variable "app_sku" {
  type    = string
  default = "F1"
}
```

```
# providers.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 4.0" }
    random  = { source = "hashicorp/random",  version = "~> 3.6" }
  }
}
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

```
# outputs.tf
output "web_app_url" {
  value = "https://${azurerm_linux_web_app.main.default_hostname}"
}
output "key_vault_name" {
  value = azurerm_key_vault.main.name
}
```

Merk: Argumentnavn i azurerm endres av og til mellom hovedversjoner (f.eks. er `enable_rbac_authorization` på Key Vault i ferd med å erstattes av `rbac_authorization_enabled`). Hvis `terraform validate` klager på et argument, sjekk dokumentasjonen for akkurat din provider-versjon i Terraform Registry.

Kjør:

```
$ export TF_VAR_subscription_id=$(az account show --query id -o tsv)
$ terraform init
$ terraform fmt && terraform validate
$ terraform plan -out=tfplan          # lagre planen
Plan: 12 to add, 0 to change, 0 to destroy.
$ terraform apply tfplan              # utfør nøyaktig den lagrede planen
$ terraform output web_app_url
$ terraform destroy                   # rydd opp når du er ferdig
```

Les en plan:

```
  # azurerm_linux_web_app.main will be updated in-place
  ~ resource "azurerm_linux_web_app" "main" {
      ~ app_settings = {
          ~ "APP_ENV" = "dev" -> "test"
        }
    }
  # azurerm_subnet.main["data"] must be replaced
-/+ resource "azurerm_subnet" "main" {
      ~ address_prefixes = ["10.0.2.0/24"] -> ["10.0.3.0/24"]   # forces replacement
    }
```

`+` lages, `~` endres på stedet, `-` slettes, `-/+` **slettes og lages på nytt** (pass på – kan gi nedetid eller datatap!).

> **Husk: Les alltid planen** før du kjører `apply`. «Forces replacement» på en database er et rødt flagg.

Beste praksis for Terraform

- Én state per miljø og logisk komponent (ikke én gigantisk state for alt).
- Remote state med låsing, versjonering og begrenset tilgang.
- Lås provider- og modulversjoner; sjekk inn `.terraform.lock.hcl`.
- `terraform fmt`, `validate`, og sikkerhetsskanning (Checkov, Trivy) i CI.
- `plan` på pull request (vis planen som kommentar), `apply` etter merge – fra pipeline, ikke fra laptop.
- `lifecycle { prevent_destroy = true }` på kritiske ressurser.
- Ikke hardkod hemmeligheter; bruk `sensitive = true`.
- `.gitignore`: `.terraform/`, `*.tfstate*`, `*.tfvars` med hemmeligheter, `tfplan`.

Bicep – kort repetisjon

Bicep ble gjennomgått i kapittel 7.4. Kort sagt: Azure-spesifikt, ingen state-fil, deploy med `az deployment group create`, forhåndsvis med `what-if`. Her er den samme ressursgruppen + VNet som Terraform-eksemplet, for sammenligning:

```
// main.bicep (deployes på abonnementsnivå for å kunne lage ressursgruppen)
targetScope = 'subscription'

param location string = 'norwayeast'

resource rg 'Microsoft.Resources/resourceGroups@2024-03-01' = {
  name: 'rg-bicep-lab'
  location: location
}

module nett 'network.bicep' = {
  name: 'nett'
  scope: rg
  params: { location: location }
}
```

```
// network.bicep
param location string

resource vnet 'Microsoft.Network/virtualNetworks@2024-01-01' = {
  name: 'vnet-bicep-lab'
  location: location
  properties: {
    addressSpace: { addressPrefixes: ['10.0.0.0/16'] }
    subnets: [
      { name: 'snet-web', properties: { addressPrefix: '10.0.1.0/24' } }
      { name: 'snet-data', properties: { addressPrefix: '10.0.2.0/24' } }
    ]
  }
}
output vnetId string = vnet.id
```

```
        $ az deployment sub create -l norwayeast -f main.bicep
```

Valget mellom Terraform og Bicep handler ofte om hva organisasjonen allerede bruker. Multi-cloud eller mange ikke-Azure-ressurser (GitHub, Datadog, Cloudflare) → Terraform. Bare Azure og ønske om null state-håndtering → Bicep.

Ansible og configuration management

Hva er det? Configuration management sørger for at **innsiden** av servere er konfigurert riktig og likt: pakker, filer, brukere, tjenester. **Ansible** (Red Hat) er det mest brukte verktøyet.

Hvorfor Ansible?

- **Agentløst** – kobler til servere over **SSH** (eller WinRM for Windows); ingenting må installeres på målmaskinene utover Python.
- **YAML** – lett å lese.
- **Idempotent** – moduler sjekker tilstanden før de endrer noe.

Begreper:

Begrep	Betyr
Control node	Maskinen du kjører Ansible fra
Managed nodes	Serverne som konfigureres
Inventory	Liste over servere (statisk fil, eller dynamisk fra Azure)
Module	En enhet som gjør én ting (`apt`, `copy`, `template`, `service`, `user`)
Task	Et kall til en modul
Playbook	YAML-fil med en eller flere *plays* (hvilke verter + hvilke tasks)
Role	Gjenbrukbar pakke av tasks, maler og variabler
Handler	Task som bare kjøres når noe er endret (f.eks. re-start nginx)

Inventory (`inventory.ini`):

```
[web]
vm-web01 ansible_host=10.0.1.4
vm-web02 ansible_host=10.0.1.5

[web:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

Playbook (`webserver.yml`):

```
        - name: Konfigurer webservere
          hosts: web                      # gruppen fra inventory
          become: true                    # bruk sudo
          vars:
            sidetittel: "Hei fra Ansible"

          tasks:
            - name: Installer nginx
              ansible.builtin.apt:
                name: nginx
                state: present            # deklarativt: «skal være installert»
                update_cache: true

            - name: Legg ut forsiden fra mal
              ansible.builtin.template:
                src: templates/index.html.j2     # Jinja2-mal med {{ sidetittel }} og {{
inventory_hostname }}
                dest: /var/www/html/index.html
                mode: "0644"
              notify: Restart nginx               # kjør handleren hvis filen endres

            - name: Sørg for at nginx kjører og starter ved oppstart
              ansible.builtin.service:
                name: nginx
                state: started
                enabled: true

          handlers:
            - name: Restart nginx
              ansible.builtin.service:
                name: nginx
                state: restarted
```

`templates/index.html.j2`:

```
        <h1>{{ sidetittel }}</h1>
        <p>Server: {{ inventory_hostname }}</p>
```

```
        $ pip install ansible                                    # eller: sudo apt install
ansible
        $ ansible -i inventory.ini web -m ping                   # test tilkobling
        $ ansible-playbook -i inventory.ini webserver.yml --check --diff   # tørrkjøring:
hva VILLE endret seg?
        $ ansible-playbook -i inventory.ini webserver.yml        # kjør
        $ ansible-playbook -i inventory.ini webserver.yml        # kjør igjen → "changed=0"
(idempotent!)
```

Hemmeligheter i Ansible krypteres med **Ansible Vault** (`ansible-vault encrypt secrets.yml`).

Terraform + Ansible sammen: Terraform lager VM-ene og skriver ut IP-adressene; Ansible konfigurerer dem. (Alternativt brukes **dynamic inventory** med Azure-pluginen `azure.azcollection.azure_rm`, som henter VM-er basert på tags.)

GitOps

Hva er det? GitOps er en arbeidsmåte der **Git er den eneste sannhetskilden** for ønsket tilstand til infrastruktur og applikasjoner, og en **agent kontinuerlig synkroniserer** miljøet mot det som står i Git.

Fire prinsipper (OpenGitOps):

1. **Deklarativt** – ønsket tilstand er beskrevet deklarativt.
2. **Versjonert og uforanderlig** – lagret i Git med full historikk.
3. **Hentes automatisk** (*pulled*) – agenter henter ønsket tilstand selv.
4. **Kontinuerlig avstemt** – agenter retter avvik automatisk.

Push vs. pull:

```
 PUSH (klassisk CI/CD)                          PULL (GitOps)
 Git ─► Pipeline ──kubectl apply──► Cluster   Git ◄──overvåker── Agent (Argo CD/Flux) I
clusteret
        (pipelinen trenger tilgang                              └─► gjør clusteret likt Git
         til clusteret)                         (ingen ekstern tilgang til clusteret
nødvendig)
```

Hvorfor GitOps?

- Alle endringer går via pull request → gjennomgang og historikk.
- **Rollback = `git revert`.**
- Drift rettes automatisk (noen endrer noe manuelt → agenten setter det tilbake).
- Sikrere: CI-systemet trenger ikke administratortilgang til clusteret.

Verktøy: Argo CD og **Flux** (begge CNCF-prosjekter). AKS har en **GitOps-utvidelse basert på Flux v2**.

Typisk oppsett:

```
 app-repo (kode)                              config-repo (Kubernetes-manifester / Helm
values)
   │ push                                       ▲
   ▼                                            │ 3. CI oppdaterer image-tag (PR eller
commit)
 CI: test, bygg image ─► push til ACR ──────────┘
                                                │ 4. Argo CD/Flux ser endringen
                                                ▼
                                    AKS-cluster synkroniseres → ny versjon kjører
```

Et Argo CD `Application`-objekt (forenklet):

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: butikk
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<org>/butikk-config.git
    targetRevision: main
    path: envs/prod                      # mappen med manifester for prod
  destination:
    server: https://kubernetes.default.svc
    namespace: butikk
  syncPolicy:
    automated:
      prune: true                        # slett ressurser som er fjernet fra Git
      selfHeal: true                     # rett manuelle endringer automatisk
```

> **Fra arbeidslivet:** GitOps brukes mest for **Kubernetes**. For Azure-infrastruktur (Terraform/Bicep) er det vanligste mønsteret «PR → plan som kommentar → merge → apply fra pipeline», som deler mange av GitOps-fordelene.

Kapitteloppsummering

- IaC beskriver infrastruktur i kode: repeterbart, versjonert, gjennomgått og automatisert.
- Deklarative verktøy (Terraform, Bicep) er idempotente; imperative skript er det ikke nødvendigvis.
- Terraform: providers snakker med API-er; resources beskriver infrastruktur; variables/locals/outputs gjør koden fleksibel; `init → plan → apply`.
- State kobler kode til virkelighet; lagre den remote (Azure Blob) med låsing, og behandle den som sensitiv.
- Moduler gir gjenbruk og standardisering.
- Bicep er Azures eget, state-løse alternativ.
- Ansible konfigurerer servere agentløst over SSH med idempotente playbooks.
- GitOps: Git er sannheten, agenter (Argo CD/Flux) synkroniserer kontinuerlig.

Øvingsoppgaver

1. Skriv Terraform som lager en ressursgruppe, et VNet med tre subnett (bruk `for_each`) og en NSG knyttet til web-subnettet. Kjør `plan`, `apply`, endre en tag, `plan` igjen, og `destroy`.
2. Sett opp remote state i Azure Blob og flytt state dit med `terraform init -migrate-state`.
3. Gjør VNet-koden om til en modul og kall den to ganger (dev og test) med ulike adresseområder.

4. Endre en tag manuelt i portalen, og se hvordan `terraform plan` oppdager driften.
5. Skriv en Ansible-playbook som oppretter en bruker `deploy`, legger inn din SSH-nøkkel, og installerer `htop` og `git`. Kjør den to ganger og bekreft idempotens.
6. Skriv samme ressursgruppe + VNet i Bicep og sammenlign med Terraform-versjonen.

Quiz

1. Hva betyr idempotens, og hvorfor er det viktig i IaC?
2. Hva er forskjellen på provisionering og configuration management?
3. Hva gjør `terraform plan`?
4. Hvorfor skal du ikke sjekke inn `terraform.tfstate` i Git?
5. Hva oppnår du med remote state i Azure Blob Storage?
6. Hva betyr `-/+` i en Terraform-plan?
7. Hvorfor er `for_each` ofte bedre enn `count`?
8. Hva er en Terraform-modul?
9. Hvorfor trenger ikke Bicep en state-fil?
10. Hva betyr at Ansible er agentløst?
11. Hva er forskjellen på push- og pull-basert deploy i GitOps-sammenheng?
12. En kollega endret en NSG-regel i portalen. Hva skjer ved neste `terraform apply`?

Fasit

1. At det å kjøre samme kode flere ganger gir samme resultat. Det gjør det trygt å kjøre IaC ofte og automatisk, uten duplikater eller utilsiktede endringer.
2. **Provisionering** lager infrastrukturen (VM, nettverk, database). **Configuration management** konfigurerer innsiden av serverne (pakker, filer, tjenester).
3. Sammenligner koden med state og den faktiske infrastrukturen, og viser hvilke endringer `apply` vil gjøre – uten å gjøre dem.
4. Den kan inneholde hemmeligheter i klartekst, og den må deles og låses på en trygg måte, noe Git ikke gir.
5. Delt tilgang for team og pipelines, låsing mot samtidige endringer, versjonering/backup og tilgangskontroll.
6. Ressursen vil bli **slettet og laget på nytt** (replacement) – kan gi nedetid eller datatap.
7. Med `for_each` identifiseres ressurser med stabile nøkler. Med `count` identifiseres de med indeks, så fjerning av ett element kan føre til at andre ressurser slettes og lages på nytt.
8. En gjenbrukbar samling Terraform-filer med input-variabler og outputs, som kalles fra annen Terraform-kode.
9. Fordi Azure Resource Manager selv er kilden til sannhet om hva som finnes; Bicep sammenligner direkte mot Azure ved deploy.

.0. Det krever ingen egen agent på serverne; Ansible kobler til over SSH (eller WinRM) og kjører modulene der.

.1. **Push**: en pipeline utenfor clusteret sender endringer inn (trenger tilgang til clusteret). **Pull**: en agent i clusteret henter ønsket tilstand fra Git og synkroniserer selv.

.2. Terraform oppdager avviket og **setter regelen tilbake** til det som står i koden (hvis regelen er administrert av Terraform). Endringen bør derfor gjøres i koden.

Del 12 – CI/CD og automatisering

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- designe en CI/CD-pipeline fra kode til produksjon,
- bruke GitHub Actions i dybden, og kjenne Azure Pipelines og Jenkins,
- forklare build-, test- og deploy-pipelines, artefakter og hemmeligheter,
- koble GitHub Actions til Azure uten lagrede hemmeligheter (OIDC),
- implementere rollback, blue-green og canary deployment,
- bygge en komplett pipeline som deployer en container til Azure.

Hva er en pipeline?

Hva er det? En **pipeline** er en automatisert kjede av steg som tar koden fra en commit til en kjørende tjeneste: hent kode → bygg → test → skann → pakk → deploy → verifiser.

> **Hverdagsanalogi:** Et samlebånd i en bilfabrikk. Hver stasjon gjør én ting (sveise, lakkere, kvalitetskontroll). Feiler kvalitetskontrollen, stopper båndet før bilen når kunden. Ingen bygger biler for hånd én og én – det er for tregt og for ujevnt.



Prinsipper for gode pipelines:

1. **Rask** – CI bør gi svar på minutter. Trege pipelines blir ignorert eller omgått.
2. **Pålitelig** – ingen «flaky» tester som feiler tilfeldig.
3. **Bygg én gang, deploy mange ganger** – samme artefakt (f.eks. samme image) går gjennom dev, test og prod. Aldri bygg på nytt for prod.
4. **Alt i kode** – pipelinen ligger i repoet (YAML) og gjennomgås som annen kode.
5. **Sikker** – minste privilegium, ingen hemmeligheter i klartekst, låste versjoner av tredjepartskomponenter.
6. **Sporbar** – du vet hvilken commit som kjører i hvilket miljø.

Testing i pipelinen

Testene er det som gjør automatisk deploy **trygt**.



Testtype	Tester	Når i pipelinen
Linting / formatering	Kodestil og åpenbare feil (ruff, shellcheck, yamllint, hadolint)	Først – sekunder
Enhetstester	Enkeltfunksjoner isolert	CI på hver push/PR
Integrasjonstester	Komponenter sammen (app + database i Docker Compose)	CI
Sikkerhetsskanning	SAST, avhengigheter, image, IaC, hemmeligheter	CI
Smoke tests	«Lever det i det hele tatt?» (`/health` svarer 200)	Rett etter hver deploy
E2E-tester	Hele brukerflyter i nettleser	Mot staging
Last-/ytelsestester	Tåler systemet forventet trafikk? (k6, JMeter, Azure Load Testing)	Periodisk / før store lanseringer

Kodedekning (*code coverage*) måler hvor mye av koden testene kjører. Nyttig som indikator, men 100 % dekning betyr ikke feilfri kode.

GitHub Actions i dybden

Grunnleggende ble vist i kapittel 5. Nå går vi dypere.

Byggesteinene

```
.github/workflows/ci.yml
└── workflow           (utløses av events: push, pull_request, schedule, workflow_dispatch …)
    └── jobs           (kjører parallelt som standard; needs: gir rekkefølge)
        ├── runs-on    (runner: ubuntu-latest, windows-latest, self-hosted …)
        └── steps      (uses: en action  |  run: et skall-kommando)
```

Begrep	Forklaring
Event / trigger	Hva som starter workflowen: `push`, `pull_request`, `schedule` (cron), `workflow_dispatch` (manuelt), `release`, `workflow_call` (gjenbruk)
Job	En gruppe steg på samme runner. Jobber er isolerte (egne maskiner)
Step	Én handling: en `run`-kommando eller en `uses`-action
Action	Gjenbrukbar komponent fra Marketplace eller eget repo (`actions/checkout`, `azure/login`)
Runner	Maskinen som kjører jobben. GitHub-hosted (ny VM hver gang) eller self-hosted
Artifact	Filer som lagres fra en jobb og kan lastes ned eller brukes av andre jobber
Environment	Deploymål (`staging`, `production`) med beskyttelsesregler, godkjenninger og egne hemmeligheter
Secret / variable	Konfigurasjon på repo-, environment- eller organisasjonsnivå
Context / expression	`${{ github.sha }}`, `${{ secrets.X }}`, `${{ needs.build.outputs.tag }}`

Eksempel: en profesjonell CI-workflow

```yaml
name: CI

on:
  pull_request:
  push:
    branches: [main]

# Minste privilegium: tokenet kan bare lese repoet som standard
permissions:
  contents: read

# Avbryt eldre kjøringer på samme branch når en ny kommer
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install ruff
      - run: ruff check .                # linting
      - run: ruff format --check .       # formatering

  test:
    runs-on: ubuntu-latest
    needs: lint                          # vent på lint
    strategy:
      matrix:
        python-version: ["3.11", "3.12"]  # kjør testene på to versjoner parallelt
    services:
      postgres:                          # database-container for
integrasjonstester
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready --health-interval 5s --health-retries 10
    env:
      DATABASE_URL: postgresql://postgres:test@localhost:5432/postgres
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip                     # cache pip-pakker mellom kjøringer
      - run: pip install -r requirements.txt pytest pytest-cov
      - run: pytest --cov=. --cov-report=xml -q
      - uses: actions/upload-artifact@v4  # lagre dekningsrapporten
        with:
```

```
          name: coverage-${{ matrix.python-version }}
          path: coverage.xml
```

Nøkkelpunkter:

- `needs:` lager rekkefølge mellom jobber.
- `strategy.matrix` kjører samme jobb med ulike parametere.
- `services:` gir deg containere (database, Redis) under jobben.
- `cache:` gjør pipelinen raskere.
- `permissions:` og `concurrency:` er tegn på en moden workflow.

Gjenbruk

- **Reusable workflows** (`on: workflow_call`) – en hel workflow som andre repoer kan kalle. Grunnlaget for felles pipeline-maler i en organisasjon.
- **Composite actions** – samle flere steg i én egen action.

```yaml
# I et annet repo:
jobs:
  ci:
    uses: min-org/pipeline-maler/.github/workflows/python-ci.yml@v1
    with:
      python-version: "3.12"
```

Sikkerhet i GitHub Actions

Risiko	Tiltak
For brede rettigheter på `GITHUB_TOKEN`	Sett `permissions:` eksplisitt, minst mulig
Kompromitterte tredjeparts-actions	Lås til **full commit-SHA** (`uses: noen/action@<40-tegns-sha>`), bruk Dependabot for å oppdatere
Hemmeligheter i logger	Bruk `secrets`, aldri `echo` dem; GitHub maskerer kjente hemmeligheter, men ikke avledede verdier
Lagrede sky-nøkler som lekker	**OIDC** til Azure – ingen lagrede nøkler (se under)
PR-er fra forks som kjører med hemmeligheter	Vær svært forsiktig med `pull_request_target`; hemmeligheter er ikke tilgjengelige for fork-PR-er med vanlig `pull_request`
Script injection	Ikke bruk `${{ github.event.pull_request.title }}` o.l. direkte i `run:` – send via `env:`
Ubeskyttet produksjonsdeploy	**Environments** med *required reviewers* og begrensning til `main`

Azure DevOps Pipelines

Azure Pipelines ble gjennomgått i kapittel 7.4. Sammenlignet med GitHub Actions:

	GitHub Actions	Azure Pipelines
Fil	`.github/workflows/*.yml`	`azure-pipelines.yml`
Struktur	workflow → jobs → steps	pipeline → stages → jobs → steps
Gjenbruk	Actions, reusable workflows	Tasks, templates
Miljøer/godkjenning	Environments	Environments med approvals and checks
Azure-tilgang	`azure/login` med OIDC	Service connection med workload identity federation
Hemmeligheter	Secrets (repo/env/org)	Secret variables, variable groups (koblet til Key Vault)
Styrke	Tett integrasjon med GitHub, stort Marketplace	Moden for store bedrifter, sterke stages og godkjenninger

Prinsippene er de samme; YAML-syntaksen er forskjellig.

Jenkins

Hva er det? Jenkins er en eldre, svært utbredt **åpen kildekode CI/CD-server** som du drifter selv. Mange bedrifter har Jenkins fra før, så du vil sannsynligvis møte det.

Kjennetegn:

- Selvhostet (VM, container eller Kubernetes) – du har ansvar for oppdatering og sikkerhet.
- Enormt plugin-økosystem.
- Pipelines defineres i en **Jenkinsfile** (Groovy-basert DSL) i repoet.
- *Controller* (styrer) og *agents* (kjører jobbene).

```
        // Jenkinsfile (deklarativ pipeline)
        pipeline {
            agent { docker { image 'python:3.12-slim' } }   // kjør stegene i en container

            environment {
                APP_ENV = 'ci'
            }

            stages {
                stage('Installer') {
                    steps {
                        sh 'pip install -r requirements.txt pytest'
                    }
                }
                stage('Test') {
                    steps {
                        sh 'pytest -q --junitxml=results.xml'
                    }
                    post {
                        always { junit 'results.xml' }        // vis testresultater i
Jenkins
                    }
                }
                stage('Deploy') {
                    when { branch 'main' }                     // bare fra main
                    steps {
                        input message: 'Deploy til produksjon?'   // manuell godkjenning
                        withCredentials([string(credentialsId: 'deploy-token', variable:
'TOKEN')]) {
                            sh './deploy.sh'                   // TOKEN er tilgjengelig
som miljøvariabel
                        }
                    }
                }
            }
            post {
                failure { echo 'Pipelinen feilet – varsle teamet her' }
            }
        }
```

> **Fra arbeidslivet:** Mange organisasjoner migrerer fra Jenkins til GitHub Actions eller Azure Pipelines for å slippe å drifte CI-serveren selv. Å kunne lese en Jenkinsfile og forklare hvordan den kan oversettes, er en nyttig ferdighet.

Artefakter

Hva er det? Et **artefakt** er det ferdige produktet av et bygg: et container-image, en zip-fil, en Python-wheel, en JAR-fil, en Helm-chart, en Terraform-plan.

Prinsipper:

- **Bygg én gang** – samme artefakt promoteres gjennom alle miljøer.

- **Uforanderlig og versjonert** – tag med commit-SHA og/eller semantisk versjon (`1.4.2`). Aldri overskriv en eksisterende versjon.
- **Lagres i et register** – ACR/ghcr.io for images, Azure Artifacts/GitHub Packages for pakker.
- **Sporbar** – kunne svare på «hvilken commit kjører i produksjon nå?».
- **Signert og med SBOM** – for forsyningskjedesikkerhet (Cosign/Notation, `actions/attest-build-provenance`).

```
commit a1b2c3d → bygg → image  minapp:a1b2c3d → dev ✓ → staging ✓ → prod
                               (samme image hele veien – bare konfigurasjonen endres)
```

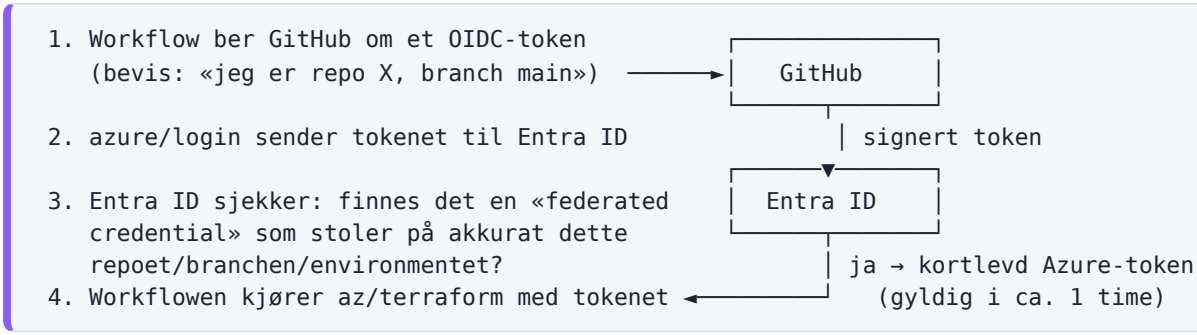
Hemmeligheter i pipelines

Regel nummer én: Hemmeligheter står **aldri** i koden eller i YAML-filen.

Nivå	Løsning
OK	Pipeline-hemmeligheter (GitHub Secrets, Azure DevOps secret variables)
Bedre	Hemmeligheter i Key Vault, hentet av pipelinen ved kjøring
Best	**Ingen hemmelighet i det hele tatt**: OIDC / workload identity federation for skytilgang, managed identity for appen

OIDC fra GitHub Actions til Azure

Hvordan det fungerer:



Ingen klienthemmelighet lagres noe sted. Tokenet gjelder bare for det repoet og den branchen/det miljøet du har godkjent.

Oppsett (én gang):

```
        # 1. App-registrering og service principal
        APP_ID=$(az ad app create --display-name "gh-minapp-deploy" --query appId -o tsv)
        az ad sp create --id $APP_ID

        # 2. Federated credential: stol på GitHub-environmentet «production» i repoet ditt
        az ad app federated-credential create --id $APP_ID --parameters '{
          "name": "gh-production",
          "issuer": "https://token.actions.githubusercontent.com",
          "subject": "repo:<github-bruker>/<repo>:environment:production",
          "audiences": ["api://AzureADTokenExchange"]
        }'

        # 3. Gi tilgang – minst mulig: Contributor på én ressursgruppe
        az role assignment create --assignee $APP_ID --role Contributor \
          --scope $(az group show -n rg-minapp-prod --query id -o tsv)

        # 4. Verdier som skal inn i GitHub (som variables eller secrets – de er ikke
hemmelige i seg selv)
        echo "AZURE_CLIENT_ID=$APP_ID"
        echo "AZURE_TENANT_ID=$(az account show --query tenantId -o tsv)"
        echo "AZURE_SUBSCRIPTION_ID=$(az account show --query id -o tsv)"
```

(Alternativt kan du bruke en **user-assigned managed identity** med federated credential i stedet for en app-registrering.)

I workflowen:

```
        permissions:
          id-token: write          # NØDVENDIG for å be om OIDC-token
          contents: read

        jobs:
          deploy:
            runs-on: ubuntu-latest
            environment: production               # må matche "subject" i federated
credential
            steps:
              - uses: azure/login@v2
                with:
                  client-id: ${{ vars.AZURE_CLIENT_ID }}
                  tenant-id: ${{ vars.AZURE_TENANT_ID }}
                  subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
              - run: az group show -n rg-minapp-prod -o table
```

Vanlig feil: `AADSTS70021: No matching federated identity record found` . **Subject** i federated credential matcher ikke det GitHub sender. Bruker jobben `environment: production` , er subject `repo:<eier>/<repo>:environment:production` . Uten environment på en push til main er det `repo:<eier>/<repo>:ref:refs/heads/main` . Store/små bokstaver må stemme.

Automated deployment

Når CI er grønn, skal deploy skje **automatisk og likt hver gang**:

- **Til staging/test:** automatisk etter merge til `main`.
- **Til produksjon:** automatisk (Continuous Deployment) eller etter godkjenning (Continuous Delivery), via et **environment** med beskyttelsesregler.
- **Etter hver deploy:** automatiske **smoke tests**, og overvåking av feilrate og responstid.
- **Ved feil:** automatisk eller ett-klikks **rollback**.

```
PR ──► CI (lint, test, skann) ──► merge til main
                                        │
                                        ▼
                  Bygg image (tag = commit-SHA) ──► push til ACR
                                        │
                                        ▼
                Deploy til STAGING ──► smoke test ──► E2E-tester
                                        │
                       [ godkjenning i environment «production» ]
                                        │
                                        ▼
                Deploy til PROD (canary 10 % → 50 % → 100 %)
                                        │
                       overvåk feilrate/latency ──► ✗ → automatisk rollback
```

Rollback

Hva er det? Å gå tilbake til forrige fungerende versjon når en deploy gir problemer.

Strategier:

Strategi	Hvordan	Fart
Redeploy forrige artefakt	Kjør deploy-jobben med forrige image-tag	Minutter
Bytt tilbake (blue-green)	Swap slots / flytt trafikk tilbake	Sekunder
Revisjonsbasert	`kubectl rollout undo`, `helm rollback`, Container Apps: trafikk til forrige revisjon	Sekunder–minutter
Feature flag av	Slå av funksjonen uten deploy	Sekunder
Roll forward	Rett feilen raskt og deploy en ny versjon	Avhenger av feilen
GitOps	`git revert` → agenten synkroniserer	Minutter

> **Husk:** En rollback-plan som aldri er testet, fungerer sjelden når det gjelder. Øv på rollback i staging. Og husk databasemigreringer (kapittel 8): skriv dem bakoverkompatible, slik at forrige appversjon fortsatt virker etter migreringen.

Blue-green deployment

Hva er det? To identiske produksjonsmiljøer: **blue** (nåværende versjon, får all trafikk) og **green** (ny versjon). Du deployer og tester green, og **bytter all trafikk på én gang**. Rollback = bytt tilbake.

```
 FØR BYTTE                          ETTER BYTTE                    ROLLBACK
 Brukere ──► BLUE v1 ✓              Brukere ──► GREEN v2 ✓         Brukere ──► BLUE v1 ✓
             GREEN v2 (testes)                  BLUE v1 (venter)               GREEN v2
(feilsøkes)
```

I Azure App Service med deployment slots:

```yaml
              - name: Deploy til staging-slot
                uses: azure/webapps-deploy@v3
                with:
                  app-name: app-minapp-prod
                  slot-name: staging
                  images: acrminapp.azurecr.io/minapp:${{ github.sha }}

              - name: Smoke test staging
                run: curl --fail --retry 10 --retry-delay 6 https://app-minapp-prod-
staging.azurewebsites.net/health

              - name: Swap staging → production
                run: az webapp deployment slot swap -g rg-minapp-prod -n app-minapp-prod -
-slot staging --target-slot production
```

(Det eksakte vertsnavnet for slots kan variere – bruk `az webapp deployment slot list` for å se det.)

Fordeler: nesten null nedetid, øyeblikkelig rollback. **Ulemper:** dobbel kapasitet, og delt database må være kompatibel med begge versjoner.

Canary deployment

Hva er det? Den nye versjonen får først **en liten andel** av trafikken (f.eks. 5–10 %). Ser metrikkene gode ut, øker du gradvis til 100 %. Ser de dårlige ut, flytter du trafikken tilbake – og bare en liten andel brukere ble berørt.

Navnet kommer fra kanarifuglene gruvearbeidere tok med seg: fuglen reagerte på giftig gass før menneskene.

I Azure Container Apps med revisjoner:

```
        # Container-appen kjører i "multiple revisions"-modus
        az containerapp revision set-mode -n ca-minapp -g rg-minapp-prod --mode multiple

        # Deploy ny versjon som ny revisjon (får 0 % trafikk først hvis vi styrer vektene
eksplisitt)
        az containerapp update -n ca-minapp -g rg-minapp-prod \
          --image acrminapp.azurecr.io/minapp:v2 --revision-suffix v2

        az containerapp revision list -n ca-minapp -g rg-minapp-prod -o table

        # 10 % til ny versjon, 90 % til gammel
        az containerapp ingress traffic set -n ca-minapp -g rg-minapp-prod \
          --revision-weight ca-minapp--v1=90 ca-minapp--v2=10

        # Alt ser bra ut → 100 % til ny
        az containerapp ingress traffic set -n ca-minapp -g rg-minapp-prod --revision-
weight ca-minapp--v2=100

        # Problemer → tilbake
        az containerapp ingress traffic set -n ca-minapp -g rg-minapp-prod --revision-
weight ca-minapp--v1=100
```

Andre steder: Kubernetes med Gateway API-vekter (kapittel 10), Argo Rollouts eller Flagger (automatisk analyse og utrulling), Azure Front Door, App Service (*Testing in production* – prosentvis trafikk til en slot).

Automatisert canary-analyse: Verktøy som Argo Rollouts sammenligner feilrate og latency mellom canary og stabil versjon, og ruller tilbake **automatisk** hvis canary er dårligere. Det er målet i modne miljøer.

	Blue-green	Canary
Trafikkbytte	Alt på én gang	Gradvis
Risiko ved feil	Alle brukere (kort tid)	Få brukere
Krav til overvåking	Moderat	Høyt – må kunne sammenligne versjoner
Kapasitet	Dobbel under bytte	Litt ekstra

Komplett CI/CD-pipeline: fra kode til produksjon

Nå setter vi alt sammen. Vi deployer Flask-appen fra kapittel 9 som container til **Azure Container Apps**, med:

- lint og test på hver PR,
- bygg og push av image til ACR med commit-SHA som tag,
- deploy til **staging** automatisk etter merge,

- smoke test,
- deploy til **produksjon** etter manuell godkjenning, som **canary** (20 % → 100 %),
- OIDC – ingen lagrede Azure-hemmeligheter.

Arkitektur

```
 GitHub repo ──PR──► CI (lint, test, Trivy)
      │ merge
      ▼
 GitHub Actions ──OIDC──► Entra ID ──token──► Azure
      │ docker push                              │
      ▼                                          ▼
 ACR (acrminapp) ◄──AcrPull (managed identity)── Container Apps-miljø (cae-minapp)
                                                  ├── ca-minapp-staging
                                                  └── ca-minapp-prod  (revisjoner: v-old 80
% / v-new 20 %)
                                                         │
                                                  Log Analytics / Azure Monitor
```

Engangsoppsett i Azure

```
        RG=rg-minapp; LOC=norwayeast; ACR=acrminapp$RANDOM
        az group create -n $RG -l $LOC
        az acr create -g $RG -n $ACR --sku Basic

        # Container Apps-miljø (lager også Log Analytics automatisk hvis du ikke oppgir
et)
        az containerapp env create -n cae-minapp -g $RG -l $LOC

        # Bygg et første image slik at appene kan opprettes
        az acr build -r $ACR -t minapp:init .

        for APP in ca-minapp-staging ca-minapp-prod; do
          az containerapp create -n $APP -g $RG --environment cae-minapp \
            --image $ACR.azurecr.io/minapp:init \
            --registry-server $ACR.azurecr.io --registry-identity system \
            --target-port 8000 --ingress external \
            --min-replicas 0 --max-replicas 3 \
            --revisions-mode multiple
        done
        # --registry-identity system: appens managed identity får AcrPull og henter images
uten passord
        # --min-replicas 0: skalerer til null når ingen bruker den (billig i lab)
```

Sett deretter opp OIDC som beskrevet tidligere i kapitlet, med **to** federated credentials (subject for environment `staging` og `production`) og rollen **Contributor** på `rg-minapp` samt **AcrPush** på registeret.

I GitHub-repoet:

- *Settings* → *Environments*: lag `staging` og `production`. På `production`: **Required reviewers** (deg selv) og **Deployment branches: main**.

- *Settings → Secrets and variables → Actions → Variables*: `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, `ACR_NAME`, `RESOURCE_GROUP`.

Workflowen

`.github/workflows/cicd.yml`:

```yaml
name: CI/CD

on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

env:
  IMAGE: ${{ vars.ACR_NAME }}.azurecr.io/minapp:${{ github.sha }}

jobs:
  # ---------- CI: kjører på alle PR-er og på main ----------
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements.txt pytest ruff
      - run: ruff check .
      - run: pytest -q

  scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Bygg image lokalt for skanning
        run: docker build -t minapp:scan .
      - name: Skann med Trivy (feil ved kritiske sårbarheter)
        uses: aquasecurity/trivy-action@0.28.0    # sjekk siste versjon; lås
gjerne til SHA
        with:
          image-ref: minapp:scan
          severity: CRITICAL
          exit-code: "1"
          ignore-unfixed: true

  # ---------- Bygg og push (bare main) ----------
  build:
    if: github.ref == 'refs/heads/main'
    needs: [test, scan]
    runs-on: ubuntu-latest
    environment: staging                        # OIDC-subject: environment:staging
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - run: az acr login -n ${{ vars.ACR_NAME }}
```

```
              - run: |
                  docker build -t "$IMAGE" .
                  docker push "$IMAGE"

          # ---------- Deploy til staging ----------
          deploy-staging:
            needs: build
            runs-on: ubuntu-latest
            environment: staging
            permissions:
              id-token: write
              contents: read
            steps:
              - uses: azure/login@v2
                with:
                  client-id: ${{ vars.AZURE_CLIENT_ID }}
                  tenant-id: ${{ vars.AZURE_TENANT_ID }}
                  subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
              - name: Oppdater staging og send all trafikk til ny revisjon
                run: |
                  SUFFIX=$(echo ${{ github.sha }} | cut -c1-7)
                  az containerapp update -n ca-minapp-staging -g ${{ vars.RESOURCE_GROUP
}} \
                    --image "$IMAGE" --revision-suffix "s$SUFFIX"
                  az containerapp ingress traffic set -n ca-minapp-staging -g ${{
vars.RESOURCE_GROUP }} \
                    --revision-weight latest=100
              - name: Smoke test
                run: |
                  FQDN=$(az containerapp show -n ca-minapp-staging -g ${{
vars.RESOURCE_GROUP }} \
                         --query properties.configuration.ingress.fqdn -o tsv)
                  curl --fail --silent --show-error --retry 10 --retry-delay 6 --retry-
all-errors "https://$FQDN/health"

          # ---------- Deploy til produksjon som canary ----------
          deploy-prod:
            needs: deploy-staging
            runs-on: ubuntu-latest
            environment: production                    # krever godkjenning
            permissions:
              id-token: write
              contents: read
            steps:
              - uses: azure/login@v2
                with:
                  client-id: ${{ vars.AZURE_CLIENT_ID }}
                  tenant-id: ${{ vars.AZURE_TENANT_ID }}
                  subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
              - name: Finn nåværende produksjonsrevisjon
                id: current
                run: |
                  OLD=$(az containerapp ingress traffic show -n ca-minapp-prod -g ${{
vars.RESOURCE_GROUP }} \
                        --query "[?weight==\`100\`].revisionName | [0]" -o tsv)
                  echo "old=$OLD" >> "$GITHUB_OUTPUT"
              - name: Ny revisjon med 20 % trafikk
                id: canary
                run: |
```

```
                  SUFFIX=p$(echo ${{ github.sha }} | cut -c1-7)
                  az containerapp update -n ca-minapp-prod -g ${{ vars.RESOURCE_GROUP }} \
                    --image "$IMAGE" --revision-suffix "$SUFFIX"
                  NEW="ca-minapp-prod--$SUFFIX"
                  echo "new=$NEW" >> "$GITHUB_OUTPUT"
                  if [ -n "${{ steps.current.outputs.old }}" ]; then
                    az containerapp ingress traffic set -n ca-minapp-prod -g ${{
vars.RESOURCE_GROUP }} \
                      --revision-weight "${{ steps.current.outputs.old }}=80" "$NEW=20"
                  fi
              - name: Verifiser canary
                run: |
                  FQDN=$(az containerapp show -n ca-minapp-prod -g ${{ vars.RESOURCE_GROUP
}} \
                         --query properties.configuration.ingress.fqdn -o tsv)
                  for i in $(seq 1 30); do
                    curl --fail --silent --output /dev/null "https://$FQDN/health" || {
echo "Feil ved forsøk $i"; exit 1; }
                    sleep 2
                  done
                  # I et ekte miljø: spør Azure Monitor/Application Insights om feilrate
for ny revisjon her.
              - name: Fullfør utrulling (100 %)
                run: |
                  az containerapp ingress traffic set -n ca-minapp-prod -g ${{
vars.RESOURCE_GROUP }} \
                    --revision-weight "${{ steps.canary.outputs.new }}=100"
              - name: Rollback ved feil
                if: failure() && steps.current.outputs.old != ''
                run: |
                  az containerapp ingress traffic set -n ca-minapp-prod -g ${{
vars.RESOURCE_GROUP }} \
                    --revision-weight "${{ steps.current.outputs.old }}=100"
```

Hva skjer, steg for steg:

1. **PR:** `test` og `scan` kjører. PR-en kan ikke merges før de er grønne (branch ruleset).
2. **Merge til main:** `build` logger inn med OIDC, bygger og pusher `minapp:<sha>`.
3. `deploy-staging` lager en ny revisjon i staging og smoke-tester den.
4. `deploy-prod` venter på **godkjenning** i environment `production`.
5. Ny revisjon får **20 %** trafikk, verifiseres, og får så **100 %**.
6. Feiler noe, flyttes trafikken **automatisk tilbake** til forrige revisjon.

> **Merk:** Første gang (når prod bare har `init`-revisjonen) kan logikken for «gammel revisjon» trenge en manuell start. Sjekk revisjonene med `az containerapp revision list`. Gamle, inaktive revisjoner kan deaktiveres med `az containerapp revision deactivate`.

Forventet resultat

- En grønn pipeline med fem jobber, synlig under *Actions*.
- To kjørende Container Apps; produksjon viser ny versjon etter godkjenning.

- Full sporbarhet: image-tag = commit-SHA.

Feilsøking

Feil	Løsning
`AADSTS70021` ved login	Subject i federated credential matcher ikke environment/branch
`unauthorized` ved `docker push`	Identiteten mangler **AcrPush**
Container App starter ikke / henger i «Activating»	`az containerapp logs show -n <app> -g <rg> --follow`; sjekk `--target-port` (må matche porten appen lytter på)
Smoke test feiler	Appen lytter på 127.0.0.1, `/health` mangler, eller kald start (øk retries)
Trivy feiler bygget	Oppdater base-image eller avhengigheter; bruk `.trivyignore` kun med begrunnelse

Kostnad og opprydding

Container Apps på *Consumption* med `min-replicas 0` har en månedlig gratiskvote og koster lite i lab; ACR Basic har fast dagspris; Log Analytics koster per GB. Rydd opp:

```
az group delete -n rg-minapp --yes --no-wait
az ad app delete --id <APP_ID>      # fjern app-registreringen for OIDC
```

På GitHub og CV

Lag en README med arkitekturdiagram, pipeline-badge og en kort forklaring av OIDC, canary og rollback. CV-linje: *«Bygget en CI/CD-pipeline i GitHub Actions med testing, sårbarhetsskanning, OIDC-autentisering mot Azure (ingen lagrede hemmeligheter), deploy til Azure Container Apps med godkjenning, canary-utrulling og automatisk rollback.»*

Kapitteloppsummering

- En pipeline automatiserer veien fra commit til produksjon; den skal være rask, pålitelig, sikker og sporbar.
- Testpyramiden: mange enhetstester, færre integrasjonstester, få E2E-tester. Smoke tests etter hver deploy.
- GitHub Actions: workflows → jobs → steps, med matrix, services, cache, environments og reusable workflows.
- Azure Pipelines og Jenkins bygger på de samme prinsippene.

- Bygg artefaktet én gang, versjoner det med commit-SHA, og promoter det gjennom miljøene.
- Hemmeligheter: best er ingen – bruk OIDC til Azure og managed identity for appen.
- Rollback må være planlagt og testet; blue-green bytter alt på én gang, canary gradvis.

Øvingsoppgaver

1. Legg til en matrix i CI-workflowen som tester på både `ubuntu-latest` og `windows-latest`.
2. Lag en reusable workflow for Python-CI og kall den fra to forskjellige repoer.
3. Sett opp OIDC fra GitHub Actions til Azure og kjør `az group list` i en workflow – uten noen hemmeligheter.
4. Implementer blue-green med App Service deployment slots (krever Standard-plan – husk kostnad og opprydding).
5. Lag en workflow med `on: schedule` som kjører uptime-skriptet fra kapittel 4 hver time og lager en GitHub Issue hvis noe feiler.
6. Oversett Jenkinsfile-eksemplet i dette kapitlet til GitHub Actions.

Quiz

1. Hva betyr «bygg én gang, deploy mange ganger»?
2. Hvorfor bør tester ligge tidlig i pipelinen?
3. Hva gjør `needs:` og `strategy.matrix` i GitHub Actions?
4. Hvorfor bør tredjeparts actions låses til en full commit-SHA?
5. Hva er fordelen med OIDC sammenlignet med en lagret klienthemmelighet?
6. Hvilken permission må workflowen ha for å bruke OIDC?
7. Hva er et environment i GitHub Actions, og hvorfor bruke det?
8. Hva er forskjellen på blue-green og canary?
9. Hva er en smoke test?
10. Nevn tre måter å gjøre rollback på.

Fasit

1. Artefaktet (f.eks. container-imaget) bygges én gang i CI, og nøyaktig samme artefakt deployes til alle miljøer. Bare konfigurasjonen endres. Det sikrer at det som ble testet, er det som kjører i produksjon.
2. For å gi rask tilbakemelding og stoppe feil før man bruker tid og ressurser på bygging og deploy.
3. `needs:` gjør at en jobb venter på andre jobber. `strategy.matrix` kjører samme jobb flere ganger med ulike parametere (f.eks. versjoner eller OS).
4. En tag kan flyttes til ny (potensielt ondsinnet) kode; en commit-SHA er uforanderlig.

5. Ingen langlivet hemmelighet som kan lekke eller må roteres; tokenet er kortlevd og bundet til et bestemt repo/branch/environment.
6. `id-token: write`.
7. Et deploymål (f.eks. production) med beskyttelsesregler: godkjenninger, begrensning til bestemte brancher, og egne hemmeligheter/variabler. Gir kontroll og sporbarhet for deploy.
8. **Blue-green** bytter all trafikk fra gammel til ny versjon på én gang (med mulighet til å bytte tilbake). **Canary** sender gradvis økende andel trafikk til ny versjon.
9. En rask, enkel test rett etter deploy som sjekker at tjenesten lever og svarer (f.eks. `GET /health` gir 200).
10. Redeploy forrige artefakt, bytt tilbake slot/trafikk (blue-green), `kubectl rollout undo` / `helm rollback`, flytt trafikk til forrige revisjon, slå av feature flag, `git revert` i GitOps.

Del 13 – Cloud security og monitoring

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare IAM, minste privilegium og Zero Trust,
- håndtere hemmeligheter og kryptering riktig,
- sikre nettverk og skanne etter sårbarheter,
- planlegge backup og katastrofegjenoppretting,
- forklare og bruke logger, metrikker og traces (OpenTelemetry),
- sette opp Prometheus og Grafana, og bruke Azure Monitor,
- håndtere hendelser og feilsøke systematisk.

> **Hvorfor dette kapitlet er viktig:** De fleste alvorlige hendelser i skyen skyldes ikke avanserte angrep, men **grunnleggende feil**: lekkede nøkler, for brede tilganger, åpne lagringskontoer, manglende MFA, manglende oppdateringer og manglende overvåking. Det betyr også at du kan forhindre det meste med god grunnhygiene.

Del A – Sikkerhet

CIA-triaden

All informasjonssikkerhet handler om tre mål:

Mål	Betyr	Trussel	Tiltak
Confidentiality (konfidensialitet)	Bare de som skal, kan lese data	Datalekkasje	Tilgangskontroll, kryptering
Integrity (integritet)	Data endres ikke uautorisert	Manipulering	Signering, hashing, tilgangskontroll, logging
Availability (tilgjengelighet)	Systemet er tilgjengelig når det trengs	DDoS, løsepengevirus, feil	Redundans, backup, DDoS-beskyttelse

Identity and Access Management (IAM)

Hva er det? IAM handler om å styre **hvem** (identitet) som får gjøre **hva** (tilgang), på **hvilke ressurser**, og **under hvilke betingelser**.

```
AUTENTISERING (AuthN)          AUTORISASJON (AuthZ)
«Hvem er du?»                  «Hva får du gjøre?»
passord + MFA, sertifikat,     RBAC-roller, policyer,
managed identity, OIDC-token   betingelser (Conditional Access)
```

Typer identiteter:

Type	Eksempler	Beste praksis
Menneskelige	Ansatte, konsulenter, gjester	MFA (helst phishing-resistent), SSO, Conditional Access, PIM for admin-roller
Arbeidslast-identiteter (*workload identities*)	Apper, pipelines, skript	Managed identity, workload identity federation (OIDC) – **ikke** passord/nøkler

I Azure (kapittel 7.1): Entra ID for identitet, Azure RBAC for tilgang til ressurser, Conditional Access for betingelser, PIM for midlertidig admin-tilgang, managed identities for apper.

Livssyklus: *Joiner – Mover – Leaver*. Når noen slutter, skal **all** tilgang fjernes umiddelbart. Når noen bytter rolle, skal gammel tilgang fjernes (ellers «samler» folk på seg tilganger over tid – *privilege creep*). Gjennomfør jevnlige **tilgangsgjennomganger** (*access reviews*).

Least privilege – minste privilegium

Hva er det? Hver identitet skal ha **akkurat** den tilgangen den trenger, på **minst mulig** scope, i **kortest mulig** tid.

> **Hverdagsanalogi:** Et hotell gir deg nøkkelkort til **ditt** rom i **dine** netter – ikke en universalnøkkel til hele hotellet for alltid. Rengjøringspersonalet har tilgang til flere rom, men bare i arbeidstiden.

I praksis:

- *Reader* som standard; *Contributor* bare der det trengs; *Owner* nesten aldri.
- Gi tilgang til **grupper**, ikke enkeltpersoner.
- Gi på **ressursgruppe**-nivå, ikke abonnement, når det holder.
- Bruk **tjenestespesifikke roller** (*Key Vault Secrets User*, *AcrPull*, *Storage Blob Data Reader*) i stedet for brede roller.
- **Just-in-time**: PIM for administratorroller; JIT VM-tilgang.
- Separate kontoer for administrasjon og hverdagsbruk i modne organisasjoner.
- Ingen mennesker med skrivetilgang til produksjon i det daglige – endringer går via pipeline.

> **Vanlig feil:** Å gi en CI/CD-pipeline *Owner* på hele abonnementet «for å slippe tilgangsproblemer». Blir pipelinen kompromittert, har angriperen alt.

Secrets management – håndtering av hemmeligheter

Hva er hemmeligheter? Passord, API-nøkler, tilkoblingsstrenger, private nøkler, sertifikater, tokens.

Hierarkiet – fra verst til best:

```
✗ Hardkodet i koden / i Git
✗ I konfigurasjonsfiler på disk
~ I miljøvariabler (bedre, men må komme fra et sted og roteres)
✓ I et hemmelighetshvelv (Azure Key Vault), hentet ved kjøring
✓✓ Ingen hemmelighet: managed identity / workload identity federation
```

Regler:

1. Aldri hemmeligheter i Git – bruk `.gitignore`, pre-commit-hooks (gitleaks) og GitHub secret scanning med *push protection*.
2. Sentral lagring (Key Vault) med RBAC og revisjonslogg.
3. **Rotasjon** – bytt hemmeligheter jevnlig og automatisk; Key Vault kan varsle før utløp.
4. **Kortlevde legitimasjoner** fremfor langlivede.
5. Ikke logg hemmeligheter.
6. Ved lekkasje: **roter først**, undersøk etterpå.

Kryptering

Kryptering i ro (*at rest*) – data lagret på disk er kryptert. I Azure er dette **på som standard** for Storage, Managed Disks, SQL (TDE) og de fleste tjenester, med Microsoft-administrerte nøkler.

Kryptering under overføring (*in transit*) – data som sendes over nettverket er kryptert med **TLS**. Krev TLS 1.2 eller nyere og HTTPS overalt, også internt.

Kryptering i bruk (*in use*) – data krypteres også mens det behandles i minnet (**confidential computing**, f.eks. Azure confidential VMs). Avansert.

Nøkkelhåndtering:

Modell	Hvem kontrollerer nøkkelen	Bruk
Plattformadministrerte nøkler (PMK)	Microsoft	Standard, null arbeid
Kundeadministrerte nøkler (CMK)	Du, i Key Vault / Managed HSM	Regulatoriske krav, mulighet til å «trekke tilbake» tilgang

Modell	Hvem kontrollerer nøkkelen	Bruk
Kundelevert nøkkel per forespørsel	Du, ved hver forespørsel	Spesielle behov

Begreper du bør kjenne:

- **Symmetrisk kryptering** – samme nøkkel krypterer og dekrypterer (AES). Rask; brukes for store datamengder.
- **Asymmetrisk kryptering** – nøkkelpar (RSA, ECC); brukes til nøkkelutveksling og signaturer (kapittel 2).
- **Hashing** – enveisfunksjon (SHA-256). Brukes til integritetssjekk og passordlagring (med salt og trege algoritmer som bcrypt/Argon2).
- **Digital signatur** – bevis på hvem som laget noe og at det ikke er endret (f.eks. signerte container-images).

> **Vanlig feil:** Å «kryptere» med base64. Base64 er **koding**, ikke kryptering – alle kan dekode det.

Nettverkssikkerhet

Defense in depth – flere lag med beskyttelse, slik at ett brudd ikke gir full tilgang:



Tiltak i Azure:

Tiltak	Tjeneste
Segmentering	VNets, subnett, NSG, ASG
Sentral brannmur og utgående kontroll	Azure Firewall, NAT Gateway, UDR
Webbeskyttelse	WAF på Application Gateway / Front Door

Tiltak	Tjeneste
DDoS-beskyttelse	Grunnleggende er inkludert; **Azure DDoS Protection** for utvidet beskyttelse
Ingen offentlig eksponering av PaaS	**Private Endpoints** + *public network access disabled*
Sikker admin-tilgang	Azure Bastion, JIT, VPN – aldri åpen SSH/RDP
Synlighet	NSG/VNet flow logs, Network Watcher

Zero Trust

Hva er det? En sikkerhetsmodell som går ut fra at **ingenting er automatisk til å stole på** – verken brukere, enheter eller nettverk – selv om de er «innenfor». Den gamle modellen var «borgen med vollgrav»: stol på alt innenfor brannmuren. Problemet: når en angriper først er inne (via phishing, en sårbar server), kan de bevege seg fritt.

De tre prinsippene (Microsofts formulering):

1. **Verify explicitly** – autentiser og autoriser alltid basert på alle tilgjengelige signaler: identitet, lokasjon, enhetens helse, tjeneste, dataklassifisering, avvik.
2. **Use least privilege access** – JIT og JEA (*just enough access*), risikobaserte policyer, databeskyttelse.
3. **Assume breach** – anta at angriperen allerede er inne: segmenter for å begrense skadeomfang (*blast radius*), krypter ende-til-ende, overvåk og analyser for å oppdage trusler.

> **Hverdagsanalogi:** En flyplass. Du viser pass ved innsjekk, går gjennom sikkerhetskontroll, viser boardingkort ved gaten – og selv om du er «inne på flyplassen», kommer du ikke inn i cockpiten eller bagasjehallen. Hver dør sjekker på nytt.

Sårbarhetsskanning (*vulnerability scanning*)

Hva er det? Automatisk leting etter **kjente sårbarheter** (registrert som **CVE**-er, *Common Vulnerabilities and Exposures*, med alvorlighetsgrad etter **CVSS**-skala) og feilkonfigurasjoner.

Hva skannes	Verktøy (eksempler)	Når
Egen kildekode (SAST)	CodeQL (GitHub code scanning), SonarQube, Semgrep	Hver PR
Avhengigheter (SCA)	Dependabot, `pip-audit`, `npm audit`, Trivy	Hver PR + daglig
Hemmeligheter	GitHub secret scanning, gitleaks	Hver push

Hva skannes	Verktøy (eksempler)	Når
Container-images	Trivy, Grype, Docker Scout, Defender for Containers	Ved bygg + kontinuerlig i registeret
IaC	Checkov, Trivy, tfsec, PSRule for Azure	Hver PR
Kjørende webapp (DAST)	OWASP ZAP	Mot staging
Sky-konfigurasjon (CSPM)	**Microsoft Defender for Cloud** (secure score, anbefalinger)	Kontinuerlig
VM-er / OS	Defender for Servers, Azure Update Manager	Kontinuerlig

```
        # Eksempler
        $ pip-audit                                  # sårbarheter i Python-avhengigheter
        $ trivy fs .                                 # skann prosjektmappen
(avhengigheter, IaC, hemmeligheter)
        $ trivy image minapp:1.0 --severity HIGH,CRITICAL
        $ checkov -d infra/                          # skann Terraform/Bicep for
feilkonfigurasjon
```

Prioritering: Du kan ikke fikse alt med en gang. Prioriter etter alvorlighet, om sårbarheten faktisk kan utnyttes (er pakken i bruk? er tjenesten eksponert?), og om den utnyttes aktivt i verden (f.eks. CISA KEV-listen).

OWASP Top 10 er listen over de vanligste sårbarhetene i webapplikasjoner (f.eks. *Broken Access Control*, *Injection*, *Security Misconfiguration*). Les den – den kommer ofte opp i intervjuer.

Forsyningskjedesikkerhet (*supply chain security*): angrep via avhengigheter, byggeverktøy eller CI-systemer. Tiltak: lås versjoner, bruk interne speil av pakkeregistre, generer SBOM, signer artefakter, og følg rammeverk som **SLSA**.

Backup og disaster recovery

Begrepene RPO og RTO ble forklart i kapittel 6, og Azure Backup og Site Recovery i kapittel 7.3. Her er prinsippene.

3-2-1-regelen:

- **3** kopier av dataene,
- på **2** forskjellige typer lagring/medier,
- **1** kopi utenfor hovedlokasjonen (annen region).

Moderne tillegg: **1** kopi som er **uforanderlig eller frakoblet** (*immutable/air-gapped*) – beskyttelse mot løsepengevirus som prøver å slette backupene – og **0** feil ved test av gjenoppretting (**3-2-1-1-0**).

Hva må beskyttes?

Komponent	Hvordan
Databaser	Automatisk backup + PITR, geo-replikering
Filer og blobs	Soft delete, versjonering, immutable storage, Azure Backup
VM-er	Azure Backup, Site Recovery
Kubernetes	Konfigurasjonen ligger i Git (GitOps); persistente volumer med AKS Backup / Velero
Infrastrukturen	**IaC i Git** – kan gjenoppbygges
Hemmeligheter og nøkler	Key Vault med soft delete og purge protection
Konfigurasjon og kode	Git (med backup av Git-plattformen)

DR-plan – dokumenter og **øv**:

1. Hva er kritisk, og hva er RPO/RTO for hver tjeneste?
2. Hvem bestemmer at vi aktiverer DR?
3. Steg-for-steg-runbook for failover og tilbakeføring (*failback*).
4. Avhengigheter (DNS, identitet, tredjeparter).
5. **Regelmessige øvelser** – minst årlig, gjerne oftere.

> **Husk:** Backup handler om **gjenoppretting**. Mål suksess på om du faktisk klarte å gjenopprette innen RTO, ikke på om backupjobben var grønn.

Del B – Monitoring og observability

Hvorfor overvåke?

- **Oppdage** problemer før kundene gjør det.
- **Feilsøke** raskt når noe skjer.
- **Forstå** ytelse, kapasitet og kostnader.
- **Måle** mot SLO-er (kapittel 8).
- **Sikkerhet** – oppdage angrep og misbruk.

Logging

Hva er det? Logger er **tidsstemplede hendelser** med detaljer: «kl. 10:15:03 feilet betaling for ordre 123 med timeout mot Vipps».

Gode logger er:

- **Strukturerte** (JSON), slik at de kan søkes og filtreres på felt:


```
{"tid":"2026-10-30T10:15:03Z","nivå":"ERROR","tjeneste":"betaling","ordre_id":123,
"feil":"timeout","varighet_ms":5003,"trace_id":"4bf92f3577b34da6a3ce929d0e0e4736"}
```


- **På riktig nivå** (DEBUG/INFO/WARNING/ERROR – kapittel 4).
- **Med kontekst**: request-ID / trace-ID, bruker-ID (ikke personopplysninger), versjon.
- **Uten hemmeligheter og unødvendige personopplysninger** (GDPR!).
- **Sentralisert**: samlet på ett sted (Log Analytics, Elasticsearch/OpenSearch, Grafana Loki, Splunk).
- **Med oppbevaringstid** tilpasset behov og lovkrav (og kostnad).

Typer logger i skyen:

Type	Eksempel	I Azure
Applikasjonslogger	Appens egne meldinger	Application Insights, container-logger
Plattformlogger	Ressurslogger fra tjenester	Diagnostic settings → Log Analytics
Revisjonslogger (*audit*)	Hvem gjorde hva i kontrollplanet	Activity log, Entra-innloggingslogger
Sikkerhetslogger	Innlogginger, avvik	Microsoft Sentinel (SIEM)

Metrics – metrikker

Hva er det? Tall målt over tid (tidsserier): CPU-bruk, antall forespørsler per sekund, responstid, feilrate, kølengde.

Hvorfor? Metrikker er billige å lagre, raske å spørre i, og perfekte for **dashboards og varsler**. Logger forteller *hva* som skjedde i detalj; metrikker forteller *hvor mye* og *hvor ofte*.

Metrikktyper (Prometheus-terminologi):

Type	Beskrivelse	Eksempel
Counter	Øker bare (nullstilles ved restart)	`http_requests_total`
Gauge	Kan gå opp og ned	`memory_usage_bytes`, `queue_length`
Histogram	Fordeling i «bøtter»	`http_request_duration_seconds` (gir persentiler)

Type	Beskrivelse	Eksempel
Summary	Ferdigberegnede persentiler	Brukes sjeldnere

Rammeverk for hva du skal måle:

- **De fire gylne signalene** (kapittel 8): latency, traffic, errors, saturation.
- **RED** for tjenester: **R**ate (forespørsler/s), **E**rrors (feil/s), **D**uration (responstid).
- **USE** for ressurser: **U**tilization (utnyttelse), **S**aturation (kø/metning), **E**rrors.

> **Husk:** Mål **persentiler**, ikke bare gjennomsnitt. Hvis snittet er 200 ms men **p99** er 4 sekunder, har 1 av 100 brukere en elendig opplevelse – og gjennomsnittet skjuler det.

Tracing – sporing

Hva er det? En **trace** følger **én forespørsel** gjennom alle tjenestene den berører, med tidsbruk i hvert ledd (**spans**).

```
 Trace 4bf92f35...  (totalt 820 ms)
 ├─ frontend   GET /kasse                    ████████████████  820 ms
 │  ├─ api     POST /ordre                   ██████████████    700 ms
 │  │  ├─ db   INSERT ordre                  ██                 60 ms
 │  │  └─ betaling  POST /betal                ████████████    580 ms  ← flaskehalsen!
 │  │       └─ ekstern Vipps-API                ██████████    540 ms
```

Hvorfor? I mikrotjenestearkitekturer er det umulig å finne ut hvorfor noe er tregt bare ved å se på logger fra hver tjeneste for seg. Tracing viser hele kjeden.

Hvordan? Hver forespørsel får en **trace-ID** som sendes videre i HTTP-headere (standarden **W3C Trace Context**, headeren `traceparent`) til neste tjeneste. Logger som inkluderer trace-ID-en, kan kobles til tracen.

OpenTelemetry

Hva er det? OpenTelemetry (OTel) er en åpen, leverandøruavhengig standard og et sett med SDK-er og verktøy for å **samle inn traces, metrikker og logger** fra applikasjoner. Et CNCF-prosjekt. Instrumenter koden én gang med OTel, og send dataene dit du vil (Azure Monitor, Prometheus, Grafana, Jaeger, Datadog …).

```
App (OTel SDK) ──OTLP──► OpenTelemetry Collector ──► Azure Monitor / Application Insights
                                                  ├──► Prometheus (metrikker)
                                                  └──► Jaeger / Tempo (traces)
```

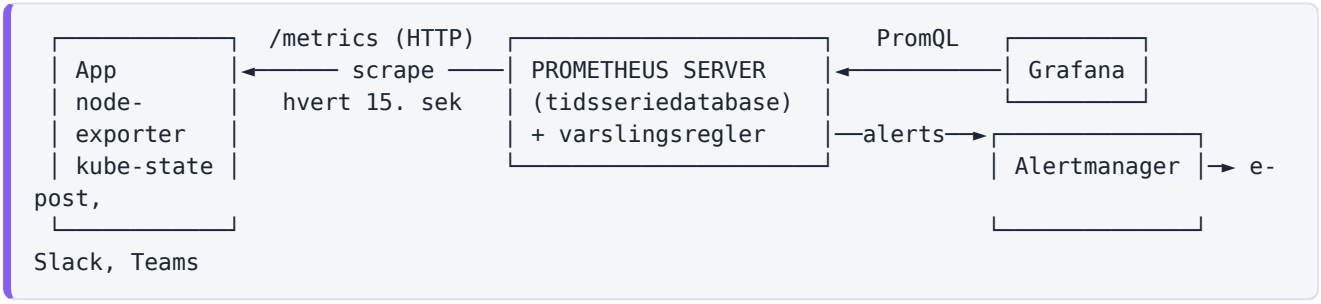
Python med Azure Monitor:

```
        # pip install azure-monitor-opentelemetry flask
        from azure.monitor.opentelemetry import configure_azure_monitor
        configure_azure_monitor()          # leser APPLICATIONINSIGHTS_CONNECTION_STRING
fra miljøet
                                           # og instrumenterer automatisk Flask, requests,
logging m.m.
        from flask import Flask
        app = Flask(__name__)
```

Prometheus

Hva er det? Prometheus er et åpen kildekode-system for **innsamling og lagring av metrikker** og **varsling**, og de facto standard i Kubernetes-verdenen (CNCF-prosjekt).

Hvordan fungerer det?



- Prometheus **henter** (*pull/scrape*) metrikker fra et HTTP-endepunkt, typisk `/metrics`, på hvert mål.
- **Exporters** gjør metrikker tilgjengelige for ting som ikke har det selv: **node_exporter** (Linux-servere), **kube-state-metrics** (Kubernetes-objekter), blackbox_exporter (sjekker URL-er utenfra).
- **PromQL** er spørrespråket.
- **Alertmanager** grupperer, demper og sender varsler.

Eksponer metrikker fra en Python-app:

```
        # pip install prometheus-client flask
        from flask import Flask
        from prometheus_client import Counter, Histogram, make_wsgi_app
        from werkzeug.middleware.dispatcher import DispatcherMiddleware

        app = Flask(__name__)
        FORESPØRSLER = Counter("app_requests_total", "Antall forespørsler", ["endepunkt",
"status"])
        VARIGHET = Histogram("app_request_duration_seconds", "Responstid", ["endepunkt"])

        @app.route("/")
        def hjem():
            with VARIGHET.labels("/").time():                # mål tiden
                FORESPØRSLER.labels("/", "200").inc()        # tell forespørselen
                return "Hei!"

        # Legg /metrics til appen
        app.wsgi_app = DispatcherMiddleware(app.wsgi_app, {"/metrics": make_wsgi_app()})
```

```
$ curl localhost:8000/metrics
# HELP app_requests_total Antall forespørsler
# TYPE app_requests_total counter
app_requests_total{endepunkt="/",status="200"} 42.0
app_request_duration_seconds_bucket{endepunkt="/",le="0.005"} 40.0
...
```

PromQL-eksempler:

```
# Forespørsler per sekund siste 5 minutter
rate(app_requests_total[5m])

# Feilrate i prosent
100 * sum(rate(app_requests_total{status=~"5.."}[5m])) / sum(rate(app_requests_total[5m]))

# 95-persentil responstid
histogram_quantile(0.95, sum(rate(app_request_duration_seconds_bucket[5m])) by (le))

# Ledig diskplass i prosent (node_exporter)
100 * node_filesystem_avail_bytes{mountpoint="/"} /
node_filesystem_size_bytes{mountpoint="/"}
```

Varslingsregel:

```
        groups:
          - name: app
            rules:
              - alert: HøyFeilrate
                expr: |
                  sum(rate(app_requests_total{status=~"5.."}[5m]))
                    / sum(rate(app_requests_total[5m])) > 0.05
                for: 10m                          # må vare i 10 minutter før varsel
                labels:
                  severity: critical
                annotations:
                  summary: "Feilraten er over 5 % i 10 minutter"
                  runbook: "https://wiki.firma.no/runbooks/hoy-feilrate"
```

Kjør Prometheus og Grafana lokalt med Docker Compose:

```
        # compose.yaml
        services:
          prometheus:
            image: prom/prometheus:v3.0.0          # sjekk siste versjon
            volumes:
              - ./prometheus.yml:/etc/prometheus/prometheus.yml
            ports: ["9090:9090"]
          node-exporter:
            image: prom/node-exporter:v1.8.2
          grafana:
            image: grafana/grafana:11.3.0
            ports: ["3000:3000"]                   # logg inn admin/admin første gang og
bytt passord
```

```
        # prometheus.yml
        global:
          scrape_interval: 15s
        scrape_configs:
          - job_name: prometheus
            static_configs: [{ targets: ["localhost:9090"] }]
          - job_name: node
            static_configs: [{ targets: ["node-exporter:9100"] }]
```

I Kubernetes installeres alt med Helm-charten **kube-prometheus-stack** (kapittel 10). I Azure finnes **Azure Monitor managed service for Prometheus** – Microsoft drifter lagringen, og du bruker PromQL og Grafana som vanlig.

Grafana

Hva er det? Grafana er et åpen kildekode-verktøy for **visualisering og dashboards**. Det kobler seg til mange datakilder: Prometheus, Azure Monitor, Log Analytics, Loki (logger), Tempo (traces), Elasticsearch, SQL-databaser.

Bruk:

1. *Connections* → *Data sources* → *Add* → Prometheus → URL `http://prometheus:9090`.

2. *Dashboards* → *New* → *Add visualization* → skriv en PromQL-spørring.
3. Eller *Import* et ferdig dashboard fra grafana.com (f.eks. «Node Exporter Full»).

Gode dashboards:

- Ett dashboard per tjeneste med **RED**-metrikker øverst.
- Fra overblikk (er alt grønt?) til detaljer (hvilken pod/endepunkt?).
- Merk deployer på tidsaksen (*annotations*) – «feilene startet rett etter deploy kl. 14:02».
- Ikke for mange paneler; hvert panel skal svare på et spørsmål.

Azure Managed Grafana er en administrert Grafana-tjeneste i Azure, integrert med Entra ID og Azure Monitor.

Azure Monitor – oppsummert

Azure Monitor (kapittel 7.3) er Azures samlede plattform:

Behov	Azure Monitor-komponent
Plattformmetrikker	Metrics (automatisk)
Logger	Log Analytics workspace + KQL
Applikasjonsovervåking	**Application Insights** (OpenTelemetry-basert): forespørsler, avhengigheter, unntak, *Application Map*, distribuert tracing
VM-er	Azure Monitor Agent + VM insights
Kubernetes	Container Insights + managed Prometheus + Managed Grafana
Syntetiske tester	Application Insights *availability tests* (sjekker URL-en din fra flere steder i verden)
Varsler	Alert rules + action groups (+ alert processing rules for demping)
Sikkerhet	**Microsoft Sentinel** (SIEM/SOAR) bygget på Log Analytics

Varslingsfilosofi – unngå *alert fatigue* (folk slutter å reagere fordi det er for mange varsler):

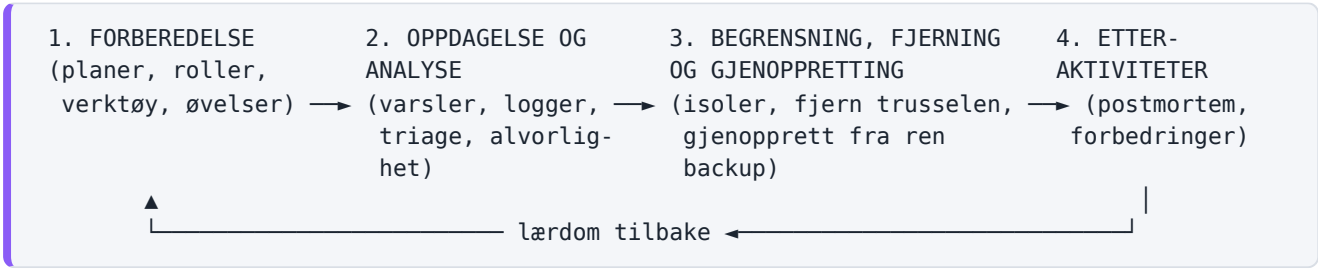
- Varsle på **symptomer** brukerne merker (høy feilrate, treg responstid, SLO i fare), ikke på hver høy CPU-topp.
- Hvert varsel skal være **handlingsbart** og ha en **runbook**.
- Skill mellom **page** (vekk noen nå – kritisk) og **ticket** (fiks i arbeidstiden).
- Gå gjennom og rydd i varsler jevnlig.

Del C – Hendelseshåndtering og feilsøking

Incident response

Sikkerhetshendelser og **driftshendelser** håndteres etter samme grunnmønster, men sikkerhetshendelser krever ekstra hensyn (bevis, varsling til myndigheter).

NIST-modellen for hendelseshåndtering:



Eksempel – lekket lagringskontonøkkel funnet i et offentlig GitHub-repo:

1. **Oppdag:** GitHub secret scanning varsler.
2. **Begrens umiddelbart:** roter nøkkelen (`az storage account keys renew`), og vurder å slå av nøkkelbasert tilgang helt.
3. **Analyser:** les lagringskontoens logger i Log Analytics – har noen brukt nøkkelen? Hva ble lest/endret?
4. **Fjern:** fjern hemmeligheten fra koden; skriv om Git-historikken hvis nødvendig.
5. **Gjenopprett:** gjenopprett eventuelt endrede data fra versjonering/backup.
6. **Varsle:** hvis personopplysninger kan være berørt – GDPR krever som hovedregel varsling til **Datatilsynet innen 72 timer**. Følg virksomhetens rutiner.
7. **Lær:** innfør push protection, managed identity i stedet for nøkler, og Azure Policy som krever *allowSharedKeyAccess = false*.

Under en hendelse:

- Utnevn en **incident commander**.
- Ha én kommunikasjonskanal og en løpende logg med tidsstempler.
- Prioriter: **stopp blødningen** (mitigate) før du finner grunnårsaken.
- Ikke slett bevis ved sikkerhetshendelser (ta snapshot av disker før du gjenoppbygger).
- Kommuniser jevnlig til interessenter og kunder.

Troubleshooting – en generell metode

Den viktigste ferdigheten til en DevOps Engineer. Bruk **den vitenskapelige metoden**:

```
 1. Definer problemet presist   «/kasse gir 502 for ca. 30 % av forespørslene siden 14:05»
 2. Samle data                  dashboards, logger, traces, siste endringer (deploy?
konfig?)
 3. Lag hypoteser               «én av podene har feil konfigurasjon etter deploy»
 4. Test én hypotese om gangen  sjekk podene; sammenlign; port-forward til hver
 5. Fiks (mitigate først)       rollback til forrige versjon → feilen forsvinner
 6. Verifiser                   feilrate tilbake til normalt
 7. Finn grunnårsak og dokumenter → postmortem, tiltak, oppdater runbook
```

Spørsmål som ofte løser saken:

- **Hva er endret?** Deploy, konfigurasjon, sertifikat, avhengighet, trafikk, data? (Over halvparten av hendelser skyldes endringer.)
- **Hva er felles** for det som feiler? (Én region, én node, én versjon, én kundetype?)
- **Hvor i kjeden** stopper det? (DNS → nettverk → lastbalanserer → app → database – kapittel 2.)
- **Hva sier dataene**, ikke hva du *tror*?

Verktøykasse per lag:

Lag	Verktøy
DNS	`dig`, `nslookup`
Nettverk/port	`curl -v`, `nc -zv`, Network Watcher, NSG flow logs
TLS	`openssl s_client`
Lastbalanserer/gateway	Backend health, gateway-logger
Kubernetes	`kubectl describe/logs/get events`, `kubectl top`
Container	`docker logs/inspect/exec`
VM/OS	`systemctl`, `journalctl`, `top`, `df -h`, `free -h`
Applikasjon	Application Insights, traces, logger
Database	Query performance insights, langsomme spørringer, låser
Azure-plattform	Service Health, Activity log, Resource health

> **Vanlig feil:** Å endre mange ting samtidig under feilsøking. Da vet du ikke hva som løste problemet – eller hva som skapte et nytt. Én endring om gangen, og noter alt.

Kapitteloppsummering

- Sikkerhet bygger på CIA-triaden, defense in depth og Zero Trust (verify explicitly, least privilege, assume breach).
- IAM: sterk autentisering (MFA), minste privilegium, managed identities og OIDC for arbeidslaster.

- Hemmeligheter i Key Vault – eller enda bedre: ingen hemmeligheter. Krypter i ro og under overføring.
- Skann kode, avhengigheter, images, IaC og sky-konfigurasjon kontinuerlig.
- Backup etter 3-2-1(-1-0), og test gjenoppretting.
- Observability: logger (hendelser), metrikker (tall over tid), traces (forespørselens reise), samlet med OpenTelemetry.
- Prometheus samler metrikker med pull og PromQL; Grafana visualiserer; Azure Monitor er Azures samlede plattform.
- Hendelser håndteres strukturert; feilsøking gjøres med hypoteser og én endring om gangen.

Øvingsoppgaver

1. Gå gjennom ditt eget Azure-abonnement: hvem har Owner? Har alle MFA? Finnes det lagringskontoer med offentlig tilgang? (Se Microsoft Defender for Cloud → Recommendations.)
2. Kjør `trivy fs .` og `pip-audit` på et av prosjektene dine, og fiks minst én funnet sårbarhet.
3. Installer gitleaks som pre-commit-hook, og test at den stopper en commit med en falsk nøkkel.
4. Kjør Prometheus, node-exporter og Grafana med Compose. Importer «Node Exporter Full»-dashboardet.
5. Legg til Prometheus-metrikker i Flask-appen og lag et Grafana-dashboard med RED-metrikker.
6. Skriv en DR-plan (1–2 sider) for prosjekt 10 i kapittel 15: RPO, RTO, backup, failover-steg.
7. Lag en Application Insights *availability test* som sjekker appen din hvert 5. minutt, med varsel på e-post.

Quiz

1. Hva står C, I og A for i CIA-triaden?
2. Hva er de tre prinsippene i Zero Trust?
3. Hva er forskjellen på autentisering og autorisasjon?
4. Hvorfor er managed identity bedre enn en lagret tilkoblingsstreng?
5. Hva er forskjellen på kryptering i ro og under overføring?
6. Hva er en CVE?
7. Hva er 3-2-1-regelen?
8. Hva er forskjellen på logger, metrikker og traces?
9. Hvorfor bør du se på p99 og ikke bare gjennomsnittlig responstid?
10. Bruker Prometheus push eller pull for å samle metrikker?
11. Hva er OpenTelemetry?
12. Hva er det første du gjør når du oppdager at en hemmelighet har lekket?

Fasit

1. **Confidentiality** (konfidensialitet), **Integrity** (integritet), **Availability** (tilgjengelighet).
2. **Verify explicitly**, **use least privilege access**, **assume breach**.
3. **Autentisering**: bekrefte hvem du er. **Autorisasjon**: bestemme hva du får gjøre.
4. Det finnes ingen hemmelighet som kan lekke, glemmes i kode eller må roteres; Azure utsteder kortlevde tokens automatisk, og tilgangen styres med RBAC.
5. **I ro**: data lagret på disk er kryptert. **Under overføring**: data som sendes over nettverket er kryptert (TLS).
6. *Common Vulnerabilities and Exposures* – en offentlig registrert, kjent sårbarhet med en unik ID (f.eks. CVE-2021-44228).
7. Tre kopier av dataene, på to forskjellige medier/lagringstyper, hvorav én utenfor hovedlokasjonen.
8. **Logger**: detaljerte hendelser. **Metrikker**: tall målt over tid. **Traces**: en forespørsels vei gjennom flere tjenester med tidsbruk per ledd.
9. Fordi gjennomsnittet skjuler de tregeste forespørslene; p99 viser opplevelsen til de 1 % verst stilte brukerne.
10. **Pull** – Prometheus scraper `/metrics`-endepunkter.
11. En åpen, leverandøruavhengig standard med SDK-er og verktøy for å samle inn traces, metrikker og logger fra applikasjoner.
12. **Roter (bytt ut) hemmeligheten umiddelbart**, så den lekkede verdien blir ubrukelig. Deretter undersøk misbruk og fjern den fra koden/historikken.

Del 14 – AI Infrastructure og MLOps

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva AI-infrastruktur og MLOps er, og hvordan de bygger på DevOps,
- forklare forskjellen på CPU og GPU, og trening vs. inferens,
- beskrive hvordan modeller deployes, serveres, versjoneres og overvåkes,
- kjenne Azure Machine Learning og Azure AI-tjenestene,
- forstå Kubernetes og infrastruktur for store språkmodeller (LLM-er),
- vurdere sikkerhet og kostnader ved AI-løsninger,
- planlegge veien fra DevOps til MLOps / AI Infrastructure.

Hva er AI-infrastruktur?

Hva er det? AI-infrastruktur er all maskinvare, programvare og plattform som trengs for å **trene, kjøre (servere), skalere og overvåke** AI-modeller: GPU-er, lagring for store datamengder, raske nettverk, Kubernetes-clustere, modellregistre, pipelines og overvåking.

Hvorfor finnes det som eget felt? AI-arbeidslaster er annerledes enn vanlige webapper:

	Vanlig webapp	**AI-arbeidslast**
Maskinvare	CPU, litt RAM	Ofte **GPU** med mye GPU-minne
Data	Megabyte–gigabyte	Gigabyte–petabyte (treningsdata, modellvekter)
Artefakt	Container-image (100 MB)	Modell (fra MB til hundrevis av GB) + container
Responstid	Millisekunder	Fra millisekunder til sekunder (LLM-er strømmer svar)
Kostnad	Moderat	Kan være svært høy (GPU-timer)
Feil	Krasj, feilkode	Også «stille feil»: modellen sva-rer, men **dårligere**
Kapasitet	Lett å få	GPU-er kan være mangelvare – kvoter og tilgjengelighet per region

> **Hverdagsanalogi:** En vanlig webapp er en kafé: en kaffemaskin og en kasse. En AI-plattform er et industrikjøkken med spesialovner (GPU-er), et kjølelager av råvarer (data), oppskrifter som stadig forbedres (modeller), kvalitetskontroll av hver rett (overvåking), og en strømregning som må følges tett (kostnad).

Hva er MLOps?

Hva er det? MLOps (*Machine Learning Operations*) er DevOps-prinsipper anvendt på maskinlæring: automatisering, versjonering, testing, CI/CD, overvåking og samarbeid – men utvidet til å håndtere **data** og **modeller** i tillegg til kode.

Hvorfor? En kjent observasjon fra industrien er at bare en liten del av et produksjonssystem for ML er selve modellkoden. Resten er datainnsamling, datavalidering, features, infrastruktur, servering, overvåking og konfigurasjon. Mange ML-prosjekter stopper i en Jupyter-notebook på en dataforskers laptop og kommer aldri i produksjon. MLOps løser det.

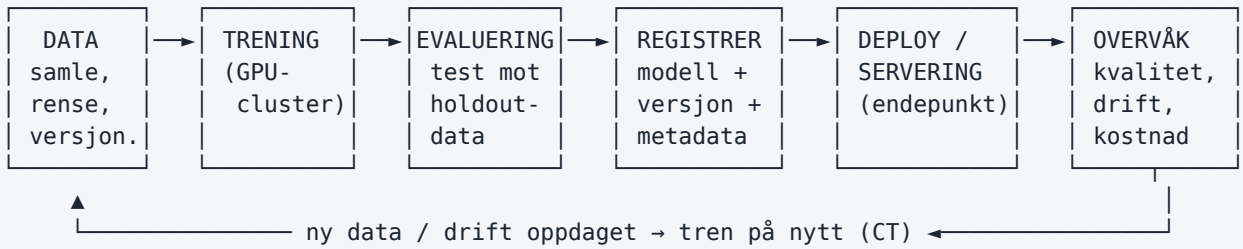
DevOps vs. MLOps:

	DevOps	**MLOps**
Versjonerer	Kode	Kode **+ data + modeller + eksperimenter**
Testing	Enhetstester, integrasjonstester	+ datavalidering, modellevaluering (nøyaktighet, rettferdighet)
Bygg-artefakt	Image, pakke	Trent modell (+ image for servering)
Pipeline	CI/CD	CI/CD **+ CT (Continuous Training)**
Overvåking	Oppetid, latency, feil	+ **modellkvalitet, data drift, concept drift**
Hvorfor ytelsen faller	Feil i kode/infrastruktur	Også fordi **verden endrer seg** (dataene ser annerledes ut enn treningsdataene)

LLMOps er MLOps for store språkmodeller: prompt-versjonering, evaluering av svar, RAG-pipelines, guardrails, kostnad per token.

Hvordan kjøres AI-modeller i Cloud?

Livssyklusen til en modell



Trening vs. inferens

	Trening (*training*)	**Inferens** (*inference*)
Hva	Modellen lærer fra data	Modellen brukes til å gi svar/prediksjoner
Når	Sjeldent (timer–uker per kjøring)	Hele tiden, for hver forespørsel
Ressurser	Mange kraftige GPU-er, mye data	Fra CPU (små modeller) til GPU (store modeller)
Mønster	Batch-jobb	Tjeneste (API) eller batch
Kostnadsprofil	Store, sjeldne utgifter	Løpende utgift som skalerer med bruk

Fine-tuning er å videretrene en eksisterende modell på egne data – mye billigere enn å trene fra bunnen. Mange bedrifter trener aldri egne store modeller, men bruker ferdige modeller via API, eventuelt med fine-tuning eller **RAG** (se under).

Tre måter å kjøre AI i skyen

```
1. FERDIGE AI-TJENESTER (SaaS/API)      «Kall et API»
   Azure AI-tjenester, modeller i Microsoft Foundry (f.eks. OpenAI-modeller)
   → minst drift, betal per kall/token

2. ADMINISTRERT ML-PLATTFORM (PaaS)     «Tren og deploy med plattformens verktøy»
   Azure Machine Learning: compute clusters, managed online endpoints
   → middels drift

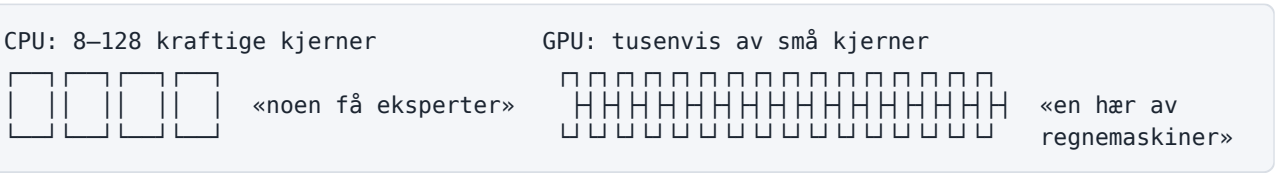
3. EGEN INFRASTRUKTUR (IaaS/containere) «Full kontroll»
   GPU-VM-er, AKS med GPU-noder, egne serveringsrammeverk (vLLM, Triton)
   → mest drift, mest kontroll
```

Samme prinsipp som i kapittel 6: velg det høyeste abstraksjonsnivået som dekker behovet.

CPU vs. GPU

CPU – få, kraftige og fleksible kjerner. God på variert logikk og sekvensielle oppgaver.

GPU – tusenvis av enklere kjerner som gjør **samme regneoperasjon på mye data samtidig** (parallellisering). Nevrale nettverk består i stor grad av matrisemultiplikasjoner – akkurat det GPU-er er laget for.



> **Hverdagsanalogi:** Du skal rette 10 000 enkle gangestykker. En CPU er fem matteprofessorer – de kan løse hva som helst, men må ta stykkene etter hverandre. En GPU er 5 000 elever som hver tar to stykker samtidig. For enkle, like oppgaver i stor skala vinner elevene.

Viktige egenskaper ved GPU-er for AI:

Egenskap	Hvorfor det betyr noe
GPU-minne (VRAM)	Modellen må (helst) få plass i GPU-minnet. Ofte den viktigste begrensningen
Regnekraft (FLOPS, ved ulike presisjoner som FP16/BF16/FP8)	Hastighet på trening og inferens
Minnebåndbredde	Hvor raskt data flyttes – avgjørende for LLM-inferens
Kobling mellom GPU-er (NVLink, InfiniBand)	Store modeller fordeles over mange GPU-er som må snakke raskt sammen

Tommelfingerregel for minne: En modell med *N* milliarder parametere i 16-bits presisjon trenger omtrent **2 × N GB** bare for vektene (en 7B-modell ≈ 14 GB), pluss ekstra til mellomlagring under kjøring (*KV-cache*). **Kvantisering** (8-bit, 4-bit) reduserer minnebehovet betydelig, med noe tap av kvalitet.

GPU-er i Azure. GPU-VM-er finnes i **N-familien**: **NC**-serien (generell GPU-beregning, trening og inferens), **ND**-serien (store treningsjobber med mange tett koblede GPU-er), **NV**-serien (visualisering og virtuelle skrivebord). Tilgjengelige GPU-typer (f.eks. NVIDIA T4, A100, H100 og nyere, og AMD Instinct) varierer per region og endres raskt – sjekk *«GPU-optimized virtual machine sizes»* på Microsoft Learn.

> **Kostnad:** GPU-VM-er er blant de **dyreste** ressursene i skyen – fra noen få til mange titalls dollar per time per VM for de største. De krever også ofte at du **ber om kvoteøkning** før du kan lage dem, og student-/gratisabonnementer har vanligvis ingen GPU-kvote. Øv på CPU og små modeller, eller bruk API-baserte tjenester med lave volumer.

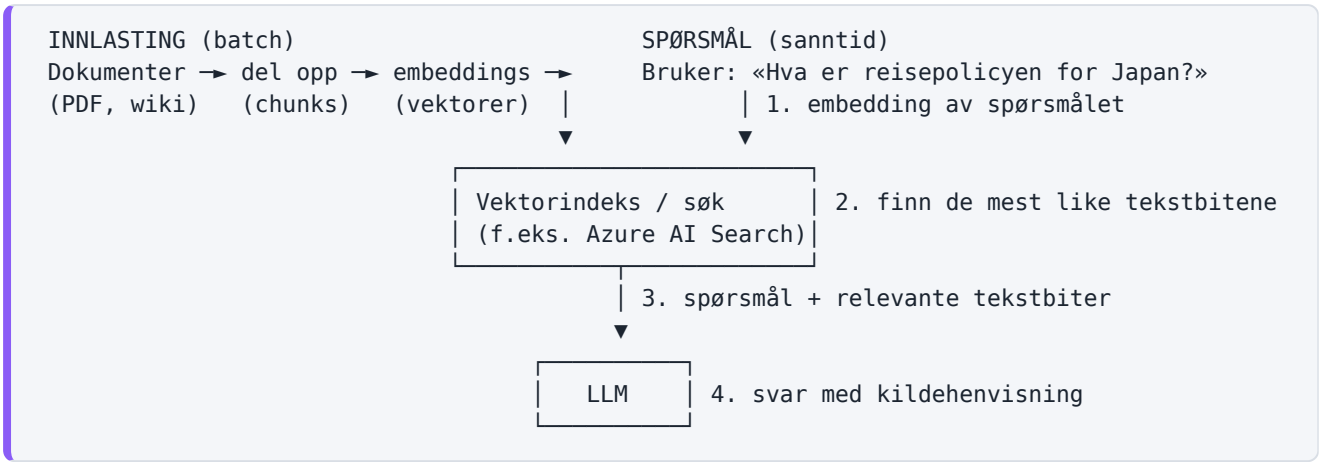
Andre akseleratorer: Google TPU, AWS Trainium/Inferentia, Microsoft Maia – spesialbrikker for AI.

AI-arbeidslaster (*AI workloads*)

Type	Eksempel	Infrastrukturbehov
Klassisk ML	Kredittrisiko, prisprediksjon, svindeldeteksjon (scikit-learn, XGBoost)	Ofte CPU er nok
Datasyn (*computer vision*)	Bildeklassifisering, kvalitetskontroll i fabrikk	GPU for trening, ofte GPU/edge for inferens
Tale og lyd	Transkribering, talesyntese	GPU eller ferdige tjenester
Generativ AI / LLM	Chatboter, oppsummering, kodeassistenter	Store GPU-er, eller API
RAG (*Retrieval-Augmented Generation*)	Chatbot som svarer basert på bedriftens dokumenter	LLM + vektordatabase/søk + datapipeline
Agenter	AI som bruker verktøy og API-er for å utføre oppgaver	LLM + verktøyintegrasjoner + streng sikkerhet
Batch-inferens	Klassifiser 10 millioner dokumenter over natten	Skalerbare jobber, spot-VM-er

RAG – forklart

Hva er det? En LLM vet bare det den ble trent på. **RAG** henter først relevante tekstbiter fra **dine** dokumenter og legger dem ved spørsmålet, slik at modellen kan svare basert på dem.



Infrastrukturen bak RAG er klassisk DevOps: datapipelines (innlasting og oppdatering), søkeindeks, API-er, hemmeligheter, tilgangskontroll (brukeren skal bare få svar fra dokumenter hen har tilgang til!), overvåking og kostnadsstyring.

Model deployment og model serving

Model deployment – å gjøre en trent modell tilgjengelig i et miljø. **Model serving** – selve tjenesten som tar imot forespørsler og returnerer prediksjoner.

Serveringsmønstre

Mønster	Beskrivelse	Eksempel
Sanntid (*online*)	API som svarer på hver forespørsel	«Er denne transaksjonen svindel?» på 50 ms
Batch	Kjør modellen på store datamengder periodisk	Nattlig scoring av alle kunder
Strømming	Prediksjoner på hendelser fra en kø	Sensordata fra IoT
Edge	Modellen kjører på enheten	Kamera i fabrikk, mobiltelefon

En enkel modell som API

```
        # serve.py – en scikit-learn-modell som REST-API med FastAPI
        # pip install fastapi uvicorn scikit-learn joblib
        import joblib
        from fastapi import FastAPI
        from pydantic import BaseModel

        app = FastAPI()
        modell = joblib.load("modell.joblib")         # lastes én gang ved oppstart, ikke
per forespørsel
        MODELL_VERSJON = "1.3.0"

        class Input(BaseModel):                         # validering av input
            areal: float
            rom: int
            byggeår: int

        @app.get("/health")
        def health():
            return {"status": "ok", "modell_versjon": MODELL_VERSJON}

        @app.post("/predict")
        def predict(x: Input):
            pris = modell.predict([[x.areal, x.rom, x.byggeår]])[0]
            return {"estimert_pris": round(float(pris)), "modell_versjon": MODELL_VERSJON}
```

```
        FROM python:3.12-slim
        WORKDIR /app
        COPY requirements.txt .
        RUN pip install --no-cache-dir -r requirements.txt
        COPY serve.py modell.joblib ./
        RUN useradd --uid 10001 app
        USER app
        CMD ["uvicorn", "serve:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dette er nøyaktig det du lærte i kapittel 9 og 12 – et container-image som deployes med CI/CD. Forskjellen er at **modellen** er en ekstra artefakt som må versjoneres og spores.

Serveringsrammeverk

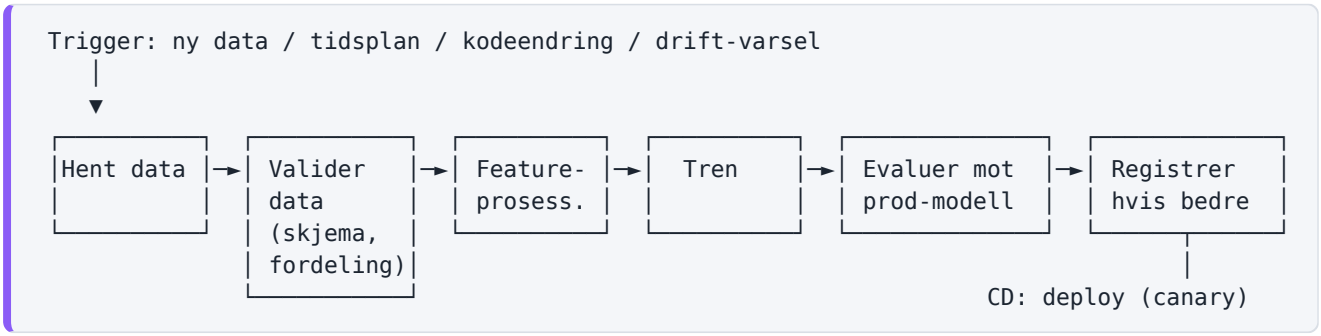
Verktøy	Bruk
FastAPI / Flask	Enkle modeller pakket som API
vLLM	Høyytelses servering av LLM-er (effektiv minnehåndtering, batching)
NVIDIA Triton Inference Server	Mange modelltyper og rammeverk, GPU-optimalisert
KServe	Standardisert modellservering på Kubernetes, med autoskalering

Verktøy	Bruk
Ollama / llama.cpp	Kjøre små LLM-er lokalt (også på CPU) – fint for læring
ONNX Runtime	Kjør modeller fra mange rammeverk effektivt

Utrullingsstrategier fra kapittel 8 og 12 gjelder også for modeller: canary (5 % av trafikken til ny modell), **shadow deployment** (ny modell får kopi av trafikken, men svarene brukes ikke – du sammenligner bare), og A/B-testing av modeller.

ML-pipelines

Hva er det? Automatiserte arbeidsflyter for hele ML-livssyklusen – som CI/CD, men med data- og treningssteg.



Verktøy: Azure Machine Learning pipelines, Kubeflow Pipelines, MLflow, Apache Airflow, Prefect, og vanlige CI/CD-verktøy (GitHub Actions, Azure Pipelines) som orkestrerer det hele.

Viktig prinsipp: Pipelinen skal være **reproduserbar** – samme kode + samme data + samme konfigurasjon skal gi samme modell. Derfor versjoneres data, kode, miljø (container/conda) og hyperparametere.

Model versioning – versjonering av modeller

Hva må versjoneres?

Hva	Hvordan
Kode	Git
Data	Datasett-versjoner (Azure ML data assets, DVC, Delta Lake/tidsstempel i lagring)
Modell	**Modellregister** (Azure ML registry, MLflow Model Registry)
Miljø	Container-image / conda-miljø

Hva	Hvordan
Eksperimenter	Parametere og metrikker per treningskjøring (MLflow tracking)
For LLM-apper	Prompter, systeminstruksjoner, modellversjon, evalueringsdatasett

Modellregisteret er for modeller det et container-register er for images: hver modellversjon lagres med metadata – hvilke data og hvilken kode den ble trent med, evalueringsresultater, hvem som godkjente den, og hvilken versjon som er i produksjon.

Lineage (avstamming): Kunne svare på «hvilke data og hvilken kode laget modellen som ga dette svaret i produksjon 14. mars?». Viktig for feilsøking, revisjon og regulering (f.eks. **EUs KI-forordning / AI Act** for høyrisikosystemer).

```
        # MLflow – logg en treningskjøring og registrer modellen
        import mlflow
        import mlflow.sklearn
        from sklearn.ensemble import RandomForestRegressor

        with mlflow.start_run():
            params = {"n_estimators": 200, "max_depth": 8}
            modell = RandomForestRegressor(**params).fit(X_train, y_train)
            mae = abs(modell.predict(X_test) - y_test).mean()

            mlflow.log_params(params)                       # hyperparametere
            mlflow.log_metric("mae", mae)                   # evaluering
            mlflow.sklearn.log_model(modell, "modell",
                                     registered_model_name="boligpris")   # registrer ny
versjon
```

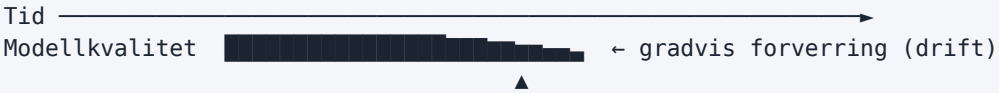
(Azure Machine Learning workspaces støtter MLflow som sporings-API.)

Model monitoring – overvåking av modeller

En modell kan være **teknisk frisk** (200 OK, rask) men **gi dårlige svar**. Derfor overvåkes flere lag:

Nivå	Hva	Eksempel
Drift/infrastruktur	Latency, feilrate, GPU-utnyttelse, minne, kø	Som kapittel 13
Data drift	Input-dataene endrer seg sammenlignet med treningsdataene	Gjennomsnittlig boligareal i forespørslene har økt 30 %
Prediction drift	Fordelingen av modellens svar endrer seg	Plutselig flagges 15 % av transaksjonene som svindel i stedet for 1 %

Nivå	Hva	Eksempel
Concept drift	Sammenhengen mellom input og riktig svar endrer seg	Svindlere endrer metoder; renten endrer boligpriser
Modellkvalitet	Nøyaktighet mot fasit når fasiten blir kjent	Faktisk salgspris vs. estimert
For LLM-er	Relevans, korrekthet (*grounded-ness*), skadelig innhold, token-forbruk, kostnad per forespørsel, brukerfeedback	Andel svar med «tommel ned»



I Azure: Azure Machine Learning har modellovervåking (data drift, prediction drift, datakvalitet); Application Insights og Azure Monitor for teknisk drift; Microsoft Foundry har evaluering og overvåking av generative AI-apper.

Azure Machine Learning

Hva den gjør. Azure Machine Learning (Azure ML) er Azures **plattform for hele ML-livssyklusen**: data, eksperimenter, trening, modellregister, deploy og overvåking.

Bestanddeler:

```
 Azure ML workspace
 ├── Data assets / datastores        (koblet til Blob Storage / Data Lake)
 ├── Compute
 │    ├── Compute instance           (personlig utviklings-VM med Jupyter/VS Code)
 │    ├── Compute cluster            (autoskalerende CPU/GPU-cluster for trening, kan
skalere til 0)
 │    ├── Serverless compute         (bare oppgi størrelse, Azure håndterer resten)
 │    └── Kubernetes compute         (koble til et AKS-cluster)
 ├── Environments                    (container-/conda-miljøer)
 ├── Jobs og Pipelines               (trening, batch)
 ├── Experiments / MLflow tracking
 ├── Models (modellregister) og Registries (deles mellom workspaces)
 ├── Endpoints
 │    ├── Managed online endpoints   (sanntids-API, Azure drifter infrastrukturen; blue-
green med trafikkdeling)
 │    └── Batch endpoints
 └── Monitoring
 Avhengigheter: Storage Account, Key Vault, Container Registry, Application Insights
```

Eksempel – deploy en registrert modell til et managed online endpoint med CLI (ml-utvidelsen):

```
        az extension add -n ml

        # endpoint.yml
        # $schema:
https://azuremlschemas.azureedge.net/latest/managedOnlineEndpoint.schema.json
        # name: boligpris-ep
        # auth_mode: key

        # deployment-blue.yml
        # $schema:
https://azuremlschemas.azureedge.net/latest/managedOnlineDeployment.schema.json
        # name: blue
        # endpoint_name: boligpris-ep
        # model: azureml:boligpris:3          # modell «boligpris» versjon 3 fra
registeret
        # instance_type: Standard_DS3_v2
        # instance_count: 1

        az ml online-endpoint create -f endpoint.yml -g rg-ml -w mlw-lab
        az ml online-deployment create -f deployment-blue.yml -g rg-ml -w mlw-lab --all-
traffic

        # Ny modellversjon som "green" med 10 % trafikk (canary)
        az ml online-deployment create -f deployment-green.yml -g rg-ml -w mlw-lab
        az ml online-endpoint update -n boligpris-ep -g rg-ml -w mlw-lab --traffic
"blue=90 green=10"
```

(Modeller i MLflow-format kan deployes uten egen scoring-kode; andre modeller trenger et scoring-skript og et miljø. Se Microsoft Learn: «Deploy and score a machine learning model by using an online endpoint».)

Sikkerhet: Managed virtual network / private endpoints for workspace, managed identity for compute, RBAC-roller som *AzureML Data Scientist*, nøkkel- eller Entra-autentisering på endepunkter.

Kostnad: Workspace i seg selv koster lite, men du betaler for **compute** (instanser som står på!, clustere, endepunkter – endepunkter kjører hele tiden), og avhengighetene (lagring, ACR, Key Vault, Application Insights). **Slå av compute instances** når du ikke bruker dem – sett *idle shutdown*.

Azure AI-tjenester og Microsoft Foundry

Ferdige AI-tjenester (tidligere kalt *Cognitive Services*, nå del av **Azure AI services / Foundry Tools**) gir AI via API uten at du trener modeller selv:

Tjeneste	Gjør
Azure OpenAI / Foundry Models	Tilgang til store språkmodeller (bl.a. OpenAI-modeller) og mange andre modeller fra en modellkatalog, med Azures sikkerhet, nettverk og compliance
Azure AI Search	Søk og vektorsøk – ofte kjernen i RAG

Tjeneste	Gjør
Azure AI Speech	Tale til tekst, tekst til tale, oversettelse
Azure AI Language	Sentimentanalyse, nøkkelfraser, gjenkjenning av personopplysninger (PII)
Azure AI Vision	Bildeanalyse, OCR
Azure AI Document Intelligence	Hente strukturerte data fra fakturaer, skjemaer, kvitteringer
Azure AI Translator	Oversettelse
Azure AI Content Safety	Oppdage skadelig innhold og prompt-angrep i input/output

Microsoft Foundry (tidligere *Azure AI Foundry*, og før det *Azure AI Studio*) er Microsofts samlede plattform for å bygge generative AI-apper og agenter: modellkatalog, prompt-utvikling, evaluering, agenter, innholdssikkerhet og overvåking. Navn og struktur har endret seg flere ganger – sjekk Microsoft Learn for gjeldende begreper.

Kalle en språkmodell fra Python (eksempel med OpenAI-biblioteket mot en Azure-deployment og Entra-autentisering):

```
        # pip install openai azure-identity
        from azure.identity import DefaultAzureCredential, get_bearer_token_provider
        from openai import AzureOpenAI

        token_provider = get_bearer_token_provider(
            DefaultAzureCredential(), "https://cognitiveservices.azure.com/.default"
        )
        client = AzureOpenAI(
            azure_endpoint="https://<din-ressurs>.openai.azure.com/",
            azure_ad_token_provider=token_provider,        # managed identity / az login – ingen API-nøkkel
            api_version="<api-versjon fra Microsoft Learn>",
        )

        svar = client.chat.completions.create(
            model="<navnet på din deployment>",             # deploymentnavnet du valgte i Azure
            messages=[
                {"role": "system", "content": "Du er en hjelpsom DevOps-mentor. Svar kort på norsk."},
                {"role": "user", "content": "Forklar forskjellen på en container og en VM."},
            ],
        )
        print(svar.choices[0].message.content)
        print(svar.usage)                                    # antall tokens = grunnlaget for kostnaden
```

> **Merk:** SDK-er, API-versjoner og anbefalte klienter for Azure OpenAI/Foundry endres ofte (Microsoft har bl.a. introdusert et nytt «v1»-API som gjør at man ikke trenger å angi `api_version` på samme måte). Følg den nyeste *quickstart* på Microsoft Learn når du skriver kode.

Driftsbegreper for LLM-tjenester:

- **Tokens** – tekst måles i tokens (biter av ord). Både input og output koster.
- **Kvoter og rate limits** – målt i tokens per minutt (TPM) og forespørsler per minutt (RPM) per deployment og region. Får du `429 Too Many Requests`, er grensen nådd → retry med backoff (kapittel 4), eller fordel lasten.
- **Deploymenttyper** – *Standard/pay-as-you-go* (betal per token, delt kapasitet), *Global*/*Data Zone* (rutes mellom regioner innen et område – vurder datakrav), og **provisioned throughput** (reservert kapasitet med forutsigbar ytelse og fast pris).
- **API-gateway foran AI** – Azure API Management kan lastbalansere mellom flere AI-deployments, håndheve token-grenser per team og logge forbruk («AI gateway»-mønsteret).

Kubernetes for AI

Kubernetes er mye brukt for AI fordi det gir felles plattform for både vanlige tjenester og GPU-arbeidslaster.

Byggesteiner:

- **GPU node pools** – egne node pools med GPU-VM-er, som kan skalere til null når de ikke brukes.
- **NVIDIA device plugin / GPU Operator** – gjør GPU-er synlige for Kubernetes, slik at pods kan be om dem.
- **Taints og tolerations** – sørger for at bare GPU-arbeidslaster havner på (dyre) GPU-noder.
- **Jobber og køsystemer** – Kubernetes Jobs, **Kueue**, Volcano for batch- og treningsjobber.
- **KServe / vLLM / Triton** for servering; **Ray** og **Kubeflow** for distribuert trening og pipelines.
- **KEDA** – skaler inferenspods etter kølengde eller forespørsler.
- I AKS: **KAITO** (*Kubernetes AI Toolchain Operator*) – en add-on som automatiserer utrulling av åpne modeller med riktige GPU-noder.

```yaml
        # Pod som ber om én GPU
        apiVersion: v1
        kind: Pod
        metadata:
          name: gpu-test
        spec:
          restartPolicy: Never
          tolerations:
            - key: "sku"
              operator: "Equal"
              value: "gpu"
              effect: "NoSchedule"              # tåler GPU-nodenes taint
          containers:
            - name: cuda
              image: nvidia/cuda:12.4.1-base-ubuntu22.04
              command: ["nvidia-smi"]           # viser GPU-ene containeren ser
              resources:
                limits:
                  nvidia.com/gpu: 1             # be om én GPU (krever device plugin /
GPU Operator)
```

```bash
        # GPU node pool i AKS (krever GPU-kvote i regionen!)
        az aks nodepool add -g rg-ai -n gpupool --cluster-name aks-ai \
          --node-vm-size <GPU-VM-størrelse> --node-count 0 \
          --enable-cluster-autoscaler --min-count 0 --max-count 2 \
          --node-taints sku=gpu:NoSchedule
```

Utfordringer: store images og modellfiler (tregt å laste – bruk cache, forhåndsnedlasting, eller montert lagring), GPU-deling (MIG, time-slicing), og at GPU-er er dyre når de står ledige.

Infrastruktur for LLM-er

Hva skal til for å kjøre en LLM selv?

```
 Klient ─► API Gateway (auth, rate limit, logging, token-telling)
              │
              ▼
        Inferenstjeneste (vLLM / Triton / KServe) på GPU-noder
              │  modellvekter lastet i GPU-minne
              │  KV-cache for samtalekontekst
              │  continuous batching (flere forespørsler samtidig)
              ▼
        Svar strømmes token for token (Server-Sent Events)
 Rundt: modellager (Blob), autoskalering (KEDA), overvåking (tokens/s, time-to-first-
token),
        guardrails (Content Safety), kostnadsrapportering per team
```

Nøkkelmetrikker for LLM-servering:

Metrikk	Betyr
TTFT (*time to first token*)	Hvor lang tid før brukeren ser første ord
Tokens per sekund (per bruker og totalt)	Hastighet og kapasitet
Samtidige forespørsler	Hvor mange brukere samtidig
GPU-utnyttelse og GPU-minne	Om du utnytter (dyre) ressurser
Kostnad per 1000 tokens / per forespørsel	Økonomi

Kjøpe (API) eller bygge (egen hosting)?

	Administrert API (Azure OpenAI/Foundry)	Egen hosting (åpne modeller på GPU)
Oppstart	Minutter	Dager–uker
Drift	Minimal	Betydelig (GPU-er, servering, oppdateringer)
Kostnadsmodell	Per token – billig ved lavt volum	Per GPU-time – kan lønne seg ved høyt, jevnt volum
Kontroll	Begrenset til tjenestens muligheter	Full (modellvalg, fine-tuning, data)
Datahåndtering	Avtalebasert; velg region/data zone	Data blir i ditt miljø
Skalering	Kvoter	Begrenset av GPU-tilgang

For de fleste bedrifter er det riktig å **starte med administrerte API-er** og vurdere egen hosting når volum, krav eller kostnader tilsier det.

Scaling AI applications – skalering

- **Horisontal skalering** av inferens: flere replikaer bak lastbalanserer, autoskalert på kø eller samtidighet (ikke bare CPU – GPU-arbeidslaster er ofte begrenset av GPU-minne).
- **Batching**: samle flere forespørsler i én GPU-kjøring for høyere gjennomstrømning.
- **Caching**: cache svar på identiske forespørsler; **semantisk cache** for lignende spørsmål.
- **Modellvalg**: bruk den **minste modellen som er god nok**. En liten modell kan håndtere enkle oppgaver; send bare vanskelige oppgaver til den store (*routing*).
- **Asynkron behandling**: lange jobber via kø (Service Bus) + workers, ikke synkrone HTTP-kall.
- **Flere regioner/deployments** bak en gateway for å omgå kvoter og øke tilgjengelighet.
- **Skaler til null** for sjelden brukte modeller (men husk oppstartstid – store modeller tar tid å laste).

AI-sikkerhet

AI-systemer har **alle** de vanlige sikkerhetsutfordringene (kapittel 13) **pluss** nye:

Trussel	Beskrivelse	Tiltak
Prompt injection	Brukeren (eller et dokument/en nettside modellen leser) gir instruksjoner som overstyrer systemet: «Ignorer tidligere instruksjoner og ...»	Behandle all input som upålitelig, Content Safety / prompt shields, begrens hva modellen *kan* gjøre, menneskelig godkjenning for viktige handlinger
Datalekkasje	Modellen avslører sensitiv informasjon fra kontekst eller treningsdata	Tilgangskontroll i RAG (sikkerhetstrimming), ikke legg hemmeligheter i prompter, PII-filtrering
Overprivilegerte agenter	En AI-agent med verktøy kan gjøre skade hvis den lures	Minste privilegium for agentens identitet, sandkasse, logging, godkjenning
Usikker output-håndtering	Modellens svar brukes direkte i SQL, shell eller HTML	Valider og «escape» output som all annen upålitelig input
Modell- og datatyveri / forgiftning	Noen stjeler modellvekter eller manipulerer treningsdata	Tilgangskontroll, integritetssjekk, signering, kontroll med datakilder
Forsyningskjede	Ondsinnede modeller eller pakker fra offentlige kilder	Bruk kjente kilder, skann, foretrekk trygge formater (f.eks. safetensors fremfor pickle)
Denial of wallet	Angriper sender store mengder dyre forespørsler	Rate limiting, token-grenser per bruker, budsjettvarsler

OWASP Top 10 for LLM Applications er en god sjekkliste. Microsoft har også rammeverk for **ansvarlig AI** (*Responsible AI*): rettferdighet, pålitelighet og sikkerhet, personvern, inkludering, åpenhet og ansvarlighet. I EU stiller **AI Act** krav til visse AI-systemer – relevant også i Norge via EØS.

Kostnader ved AI-infrastruktur

Hva koster?

Kostnadsdriver	Eksempel
GPU-compute	Trening og selvhostet inferens – den desidert største posten
Tokens	Input + output for API-baserte modeller; lange prompter og RAG-kontekst koster

Kostnadsdriver	Eksempel
Ledig kapasitet	GPU-er/endepunkter som står på uten trafikk
Lagring	Datasett, modellversjoner, checkpoints, logger
Datatrafikk	Flytting av store datasett mellom regioner
Søk/vektorindekser	AI Search-nivåer, embeddings
Overvåking	Logging av prompter og svar kan bli store volumer

Optimalisering:

1. **Velg riktig modell** – minste modell som løser oppgaven; test billigere modeller først.
2. **Kort ned prompter** og begrens output-lengde; cache gjenbrukbar kontekst der tjenesten støtter det.
3. **Batch** ikke-hastende jobber (flere tjenester har rabatterte batch-API-er).
4. **Skaler GPU-noder til null**; bruk **spot**-VM-er for avbrytbar trening med checkpointing.
5. **Provisioned throughput** kun når volumet er høyt og forutsigbart.
6. **Mål kostnad per forespørsel/per bruker** og tagg alt per team/prosjekt (FinOps for AI).
7. **Budsjetter og varsler** – særlig for eksperimenter.

> **Kostnad:** Et klassisk studentuhell er en compute instance i Azure ML eller en GPU-VM som står på over helgen. Sett **auto-shutdown** og **idle shutdown** første gang du lager noe.

Fra DevOps Engineer til MLOps / AI Infrastructure Engineer

Hva du allerede har fra denne boken:

```
DevOps-kompetanse                  Brukes i MLOps / AI Infra som …
─────────────────                  ──────────────────────────────
Linux, nettverk, sky               Grunnlaget for alle AI-plattformer
Docker                             Pakking av modeller og treningsmiljøer
Kubernetes                         GPU-clustere, KServe, KAITO, Kubeflow
Terraform/Bicep                    Reproduserbare AI-miljøer
CI/CD                              ML-pipelines, CT, modell-deploy med canary
Monitoring                         Modellovervåking, token- og GPU-metrikker
Sikkerhet                          AI-sikkerhet, hemmeligheter, private endepunkter
Kostnadsstyring                    FinOps for GPU og tokens
```

Hva du må lære i tillegg:

1. **ML-grunnlag** – hva trening, validering, overtilpasning, evalueringsmetrikker og typiske algoritmer er. (Du trenger ikke være dataforsker, men du må snakke språket.)
2. **Python for data** – pandas, NumPy, scikit-learn; grunnleggende PyTorch.

3. **ML-verktøy** – MLflow, Azure Machine Learning, Jupyter.
4. **GPU-infrastruktur** – CUDA-grunnlag, GPU-er i Kubernetes, `nvidia-smi`.
5. **LLM-økosystemet** – API-er, RAG, vektorsøk, evaluering, vLLM, guardrails.
6. **Data engineering-grunnlag** – datapipelines, formater (Parquet), datakvalitet.

Foreslått læringsvei (6–12 måneder etter DevOps-grunnlaget):

Steg	Innhold	Prosjekt
1	ML-grunnlag (f.eks. Microsoft Learn, fast.ai, Andrew Ngs kurs)	Tren en scikit-learn-modell og spor den med MLflow
2	Modellservering	FastAPI-modell i Docker med CI/CD til Container Apps (prosjekt 11)
3	Azure ML	Trening på compute cluster, registrering, managed online endpoint med blue-green
4	LLM-apper	RAG-chatbot med Azure AI Search og en Foundry-modell, med evaluering
5	Kubernetes for AI	Kjør en liten åpen LLM med vLLM/Ollama på Kubernetes (CPU lokalt, ev. GPU i sky)
6	Overvåking	Drift- og kvalitetsmonitorering av modellen; kostnadsdashboard

Relevante sertifiseringer: *AI-900* (grunnleggende AI på Azure), *AI-102* / etterfølgere (Azure AI Engineer), *DP-100* (Azure Data Scientist – Azure ML), *CKA* (Kubernetes). Microsoft fornyer AI-sertifiseringene ofte – sjekk Microsoft Learn.

Rolletitler du vil se: MLOps Engineer, ML Platform Engineer, AI Infrastructure Engineer, LLMOps Engineer, AI Platform Engineer, ML Engineer (ofte mer modellnært).

> **Husk:** Bedrifter trenger langt flere som kan **drifte AI pålitelig, sikkert og kostnadseffektivt** enn som kan finne opp nye modeller. En solid DevOps-bakgrunn er et svært godt utgangspunkt.

Kapitteloppsummering

- AI-infrastruktur er plattformen for å trene, servere og overvåke modeller; MLOps er DevOps for ML (kode + data + modeller).

- Trening er sjeldent og GPU-tungt; inferens skjer hele tiden og bestemmer løpende kostnad.
- GPU-er er laget for parallelle matriseoperasjoner; GPU-minne er ofte den viktigste begrensningen.
- Modeller serveres som API-er (sanntid), batch, strømming eller på edge; deploy med canary/shadow.
- ML-pipelines automatiserer data → trening → evaluering → registrering → deploy; alt versjoneres.
- Overvåk både teknisk helse og modellkvalitet (data drift, concept drift).
- Azure ML er plattformen for egen ML; Azure AI-tjenester og Microsoft Foundry gir ferdige modeller og verktøy for generativ AI.
- Kubernetes med GPU-noder, KServe/vLLM og KEDA er vanlig for AI i skala.
- AI-sikkerhet: prompt injection, datalekkasje, overprivilegerte agenter, «denial of wallet».
- Kostnad styres med modellvalg, skalering til null, batching, caching og budsjetter.

Øvingsoppgaver

1. Installer Ollama lokalt, kjør en liten åpen modell, og kall den via HTTP-API-et med `curl`.
2. Tren en enkel scikit-learn-modell (f.eks. på et åpent boligprisdatasett), lagre den med `joblib`, og server den med FastAPI-eksemplet i en Docker-container.
3. Legg til MLflow-sporing og sammenlign tre treningskjøringer med ulike hyperparametere i MLflow UI (`mlflow ui`).
4. Regn ut omtrentlig GPU-minne for vektene til en 8B- og en 70B-modell i 16-bit og 4-bit.
5. Skriv en trusselmodell (1 side) for en RAG-chatbot som svarer på spørsmål om HR-dokumenter. Hvilke trusler fra tabellen gjelder, og hvilke tiltak velger du?
6. Lag et kostnadsestimat i Azure Pricing Calculator for en chatbot med 1 000 brukere som stiller 10 spørsmål om dagen (anta et gjennomsnittlig tokenantall per spørsmål og svar).

Quiz

1. Hva er forskjellen på DevOps og MLOps?
2. Hva er forskjellen på trening og inferens?
3. Hvorfor egner GPU-er seg bedre enn CPU-er for nevrale nettverk?
4. Hva er data drift?
5. Hva er et modellregister?
6. Hva er RAG, og hvorfor brukes det?
7. Hva er prompt injection?
8. Hva er TTFT?
9. Nevn tre måter å redusere kostnadene for en LLM-applikasjon.

10. Hvorfor er en god DevOps-bakgrunn verdifull for AI Infrastructure-roller?

Fasit

1. MLOps anvender DevOps-prinsippene på ML, men versjonerer og tester i tillegg **data og modeller**, har **kontinuerlig trening** og overvåker **modellkvalitet** (drift), ikke bare teknisk helse.
2. **Trening**: modellen lærer fra data (sjelden, ressurskrevende). **Inferens**: den trente modellen brukes til å gi svar (kontinuerlig, per forespørsel).
3. Nevrale nettverk består i stor grad av matriseoperasjoner som kan utføres parallelt, og GPU-er har tusenvis av kjerner laget for nettopp slik parallell beregning, pluss høy minnebåndbredde.
4. At input-dataene i produksjon endrer seg sammenlignet med dataene modellen ble trent på, slik at modellens ytelse kan forverres.
5. Et sentralt lager for modellversjoner med metadata (data, kode, metrikker, status), brukt til sporbarhet, godkjenning og deploy.
6. *Retrieval-Augmented Generation*: relevante tekstbiter fra egne dokumenter hentes og legges ved spørsmålet til LLM-en, slik at den kan svare basert på oppdatert, bedriftsspesifikk informasjon uten å trenes på nytt.
7. Et angrep der input (fra bruker eller fra innhold modellen leser) inneholder instruksjoner som prøver å overstyre systemets instruksjoner eller få modellen til å gjøre noe uønsket.
8. *Time to first token* – tiden fra forespørselen sendes til brukeren mottar første del av svaret.
9. For eksempel: bruke en mindre modell, korte ned prompter og begrense output, caching, batching av ikke-hastende jobber, skalere GPU-ressurser til null, rate limiting per bruker.
10. Fordi AI i produksjon i stor grad er et infrastrukturproblem: containere, Kubernetes, IaC, CI/CD, overvåking, sikkerhet og kostnadsstyring – akkurat DevOps-ferdighetene.

Del 15 – Virkelige prosjekter

Prosjektene er der kunnskapen blir til **ferdigheter** – og der du bygger **porteføljen** arbeidsgivere vil se. De har økende vanskelighetsgrad og bygger på hverandre.

#	Prosjekt	Nivå	Kapitler	Tid	Kostnad
1	Sett opp og herd en Linux-server	★☆☆☆☆	2, 3	4–6 t	Gratis (lokal VM/WSL)
2	Python-automatisering: Azure-kostnadsjeger	★★☆☆☆	4, 7.1	4–6 t	Gratis
3	Azure-VM med kode: CLI-skript + cloud-init	★★☆☆☆	7.2	3–4 t	Øre–kroner
4	Profesjonelt GitHub-repo og portefølje	★★☆☆☆	5	3–4 t	Gratis
5	Containeriser en app med Docker	★★★☆☆	9	4–6 t	Gratis
6	Publiser en nettside på Azure med CI/CD	★★★☆☆	5, 7, 12	3–5 t	Gratis (Free-plan)
7	CI/CD-pipeline til Azure App Service	★★★☆☆	12	6–8 t	Gratis–lav
8	Azure-infrastruktur med Terraform og pipeline	★★★★☆	11, 12	8–12 t	Lav
9	Deploy en app på Kubernetes med Helm	★★★★☆	10	8–12 t	Gratis lokalt / moderat i AKS
10	Komplett Cloud/DevOps-plattform	★★★★★	Alle	25–40 t	Lav–moderat

#	Prosjekt	Nivå	Kapitler	Tid	Kostnad
11	MLOps: modell-API med CI/CD og overvåking	★★★★☆	14	10–15 t	Lav
12	Observability og hendelses-øvelse (game day)	★★★★☆	8, 13	8–10 t	Gratis lokalt

Generelle regler for alle sky-prosjekter:

1. Én ressursgruppe per prosjekt, med tags `prosjekt`, `eier`, `utløper`.
2. Budsjettvarsel på abonnementet (kapittel 7.1).
3. **Opprydding samme dag** – hvert prosjekt har en oppryddingsseksjon.
4. Alt i Git; ingen hemmeligheter i repoet.
5. Skriv README underveis, ikke etterpå.

Mal for README i hvert prosjekt:

```
# <Prosjektnavn>
Kort beskrivelse (2–3 setninger): hva, hvorfor.

## Arkitektur
(diagram – bilde eller tekstdiagram)

## Teknologier
Azure (...), Terraform, Docker, GitHub Actions ...

## Slik kjører du det
Steg for steg.

## Hva jeg lærte
3–5 punkter – også hva som gikk galt og hvordan jeg løste det.

## Videre arbeid
Hva ville jeg gjort med mer tid?
```

> **Husk:** Arbeidsgivere leser **«Hva jeg lærte»** og ser etter hvordan du feilsøkte. Et prosjekt med ærlig beskrivelse av problemer du løste, er mer imponerende enn et som later som alt gikk knirkefritt.

Prosjekt 1 – Sett opp og herd en Linux-server

Læringsmål: Installere og sikre en Linux-server, administrere brukere og tjenester, sette opp en webserver, brannmur, automatiske oppdateringer og backup med cron.

Forutsetninger: Kapittel 2 og 3.

Arkitektur:

```
Din PC ──SSH (nøkkel, port 22)──► Ubuntu Server 24.04 LTS (VM i VirtualBox/UTM eller WSL)
                                  ├── bruker «admin» (sudo), ingen root-innlogging
                                  ├── ufw: tillat 22, 80
                                  ├── nginx → /var/www/prosjekt1
                                  ├── fail2ban
                                  ├── unattended-upgrades
                                  └── cron: backup.sh kl 02:30 → /var/backups
```

Verktøy: VirtualBox eller UTM (eller WSL), Ubuntu Server 24.04 LTS ISO, terminal med SSH.

Tid: 4–6 timer. **Kostnad:** Gratis.

Steg for steg:

1. **Installer Ubuntu Server** i en VM (2 vCPU, 2 GB RAM, 20 GB disk). Velg «Install OpenSSH server» under installasjonen. Sett nettverket til *Bridged* (eller bruk port forwarding 2222→22 med NAT).
2. **Oppdater systemet:**

   ```
   sudo apt update && sudo apt upgrade -y
   ```

3. **Lag en administrasjonsbruker og SSH-nøkkel** (fra din PC):

   ```
   # på serveren
   sudo adduser admin
   sudo usermod -aG sudo admin
   # på din PC
   ssh-keygen -t ed25519 -C "prosjekt1"
   ssh-copy-id admin@<server-ip>
   ssh admin@<server-ip>          # test at nøkkelinnlogging virker!
   ```

4. **Herd SSH** – lag `/etc/ssh/sshd_config.d/99-herding.conf`:

   ```
   PermitRootLogin no
   PasswordAuthentication no
   KbdInteractiveAuthentication no
   ```

   ```
   sudo sshd -t && sudo systemctl restart ssh
   ```

   Hold den gamle SSH-økten åpen og test i et nytt vindu.

5. **Brannmur:**

```
        sudo ufw allow OpenSSH
        sudo ufw allow 80/tcp
        sudo ufw enable
        sudo ufw status verbose
```

6. **Fail2ban og automatiske sikkerhetsoppdateringer:**

```
        sudo apt install -y fail2ban unattended-upgrades
        sudo systemctl enable --now fail2ban
        sudo dpkg-reconfigure -plow unattended-upgrades    # svar «Yes»
        sudo fail2ban-client status sshd
```

7. **Webserver:**

```
        sudo apt install -y nginx
        sudo mkdir -p /var/www/prosjekt1
        echo "<h1>Prosjekt 1 – $(hostname)</h1>" | sudo tee
/var/www/prosjekt1/index.html
        sudo chown -R www-data:www-data /var/www/prosjekt1
```

`/etc/nginx/sites-available/prosjekt1`:

```
server {
    listen 80 default_server;
    root /var/www/prosjekt1;
    index index.html;
    access_log /var/log/nginx/prosjekt1.access.log;
}
```

```
        sudo ln -s /etc/nginx/sites-available/prosjekt1 /etc/nginx/sites-enabled/
        sudo rm /etc/nginx/sites-enabled/default
        sudo nginx -t && sudo systemctl reload nginx
        curl http://localhost
```

8. **Backup-skript med cron** – bruk `backup.sh` fra kapittel 3 (med `KILDE=/var/www/prosjekt1`), legg det i `/usr/local/bin/backup.sh`, `sudo chmod 755`, og legg inn i roots crontab (`sudo crontab -e`):

```
30 2 * * * /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
```

9. **Helsesjekkskript** `/usr/local/bin/helse.sh` som skriver disk, minne, load og status for nginx/ssh/fail2ban til en logg – kjøres hvert 15. minutt.

.0. **Dokumenter** alt i en README, inkludert kommandoene, i et Git-repo.

Forventet resultat: En server du bare kan nå med SSH-nøkkel, som serverer en nettside, blokkerer brute-force-forsøk, oppdaterer seg selv, og tar daglig backup.

Vanlige feil og feilsøking:

Problem	Løsning
Låst ute etter SSH-herding	Logg inn via VM-konsollen, rett konfigurasjonen. (Derfor: test i nytt vindu før du lukker det gamle.)
`ufw enable` kuttet SSH	Du glemte `ufw allow OpenSSH`. Bruk konsollen.
nginx gir 403	Feil eier/rettigheter på `/var/www/prosjekt1`
Cron-jobben kjører ikke	Bruk fulle stier, sjekk `/var/log/backup.log` og `grep CRON /var/log/syslog`

Opprydding: Ta et snapshot av VM-en (nyttig senere), eller slett den. Ingen sky-kostnader.

Videreutvikling: Automatiser hele oppsettet med en Ansible-playbook (kapittel 11) – kjør den mot en ny, tom VM og se at resultatet blir identisk.

På GitHub og CV: Repo `linux-server-hardening` med skript, konfigurasjonsfiler og README. *«Installerte og herdet Ubuntu Server: nøkkelbasert SSH, ufw, fail2ban, automatiske sikkerhetsoppdateringer, nginx og automatisert backup med cron og Bash.»*

Prosjekt 2 – Python-automatisering: Azure-kostnadsjeger

Læringsmål: Skrive et profesjonelt Python-verktøy som bruker Azure SDK, argumenter, logging, feilhåndtering og tester.

Forutsetninger: Kapittel 4 og 7.1. Azure-abonnement og `az login`.

Arkitektur:

```
 kostnadsjeger.py ──DefaultAzureCredential (az login / managed identity)──► Azure Resource Manager
   │  1. list alle ressursgrupper
   │  2. sjekk tags: eier, utløper
   │  3. rapport: mangler tag / utløpt
   │  4. (valgfritt, med --slett) slett utløpte grupper etter bekreftelse
   └─► rapport.json + loggutskrift + returkode (0 = ok, 1 = funn)
```

Verktøy: Python 3.10+, `azure-identity`, `azure-mgmt-resource`, `pytest`.

Tid: 4–6 timer. **Kostnad:** Gratis (leser bare metadata; sletting fjerner kostnader).

Steg for steg:

1. Lag prosjektet:

```
mkdir kostnadsjeger && cd kostnadsjeger
python3 -m venv .venv && source .venv/bin/activate
pip install azure-identity azure-mgmt-resource pytest
pip freeze > requirements.txt
```

2. Lag noen testgrupper:

```
        az group create -n rg-test-ok   -l norwayeast --tags eier=student utløper=2099-
12-31
        az group create -n rg-test-gml  -l norwayeast --tags eier=student utløper=2020-
01-01
        az group create -n rg-test-utag -l norwayeast
```

3. Skriv `kostnadsjeger.py`:

```python
        #!/usr/bin/env python3
        """Finn ressursgrupper som mangler eier-tag eller har utløpt."""
        import argparse
        import json
        import logging
        import os
        import sys
        from datetime import date

        from azure.identity import DefaultAzureCredential
        from azure.mgmt.resource import ResourceManagementClient

        log = logging.getLogger("kostnadsjeger")


        def vurder(navn: str, tags: dict | None, idag: date) -> list[str]:
            """Returner en liste med problemer for én ressursgruppe (tom liste = ok)."""
            tags = tags or {}
            problemer = []
            if "eier" not in tags:
                problemer.append("mangler tag 'eier'")
            utl = tags.get("utløper")
            if utl is None:
                problemer.append("mangler tag 'utløper'")
            else:
                try:
                    if date.fromisoformat(utl) < idag:
                        problemer.append(f"utløpt {utl}")
                except ValueError:
                    problemer.append(f"ugyldig dato '{utl}'")
            return problemer


        def main() -> int:
            p = argparse.ArgumentParser(description=__doc__)
            p.add_argument("--subscription",
default=os.environ.get("AZURE_SUBSCRIPTION_ID"))
            p.add_argument("--out", default="rapport.json")
            p.add_argument("--slett", action="store_true", help="Slett UTLØPTE grupper
(spør først)")
            p.add_argument("--ignorer", nargs="*", default=["NetworkWatcherRG"],
                           help="Grupper som aldri skal vurderes")
            args = p.parse_args()

            logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
            if not args.subscription:
                log.error("Mangler --subscription eller AZURE_SUBSCRIPTION_ID")
                return 2

            klient = ResourceManagementClient(DefaultAzureCredential(),
args.subscription)
            idag = date.today()
            funn = []
            for rg in klient.resource_groups.list():
                if rg.name in args.ignorer or rg.name.startswith("MC_"):   # AKS-
administrerte grupper
                    continue
                problemer = vurder(rg.name, rg.tags, idag)
                if problemer:
```

```
                    log.warning("%s: %s", rg.name, "; ".join(problemer))
                    funn.append({"gruppe": rg.name, "problemer": problemer})
                else:
                    log.info("%s: ok", rg.name)

            with open(args.out, "w", encoding="utf-8") as f:
                json.dump({"dato": idag.isoformat(), "funn": funn}, f, indent=2,
ensure_ascii=False)

            if args.slett:
                for f_ in funn:
                    if any(pr.startswith("utløpt") for pr in f_["problemer"]):
                        svar = input(f"Slette {f_['gruppe']}? Skriv navnet for å
bekrefte: ")
                        if svar == f_["gruppe"]:
                            klient.resource_groups.begin_delete(f_["gruppe"])
                            log.info("Sletting startet: %s", f_["gruppe"])
            return 1 if funn else 0


        if __name__ == "__main__":
            sys.exit(main())
```

4. Skriv tester for logikken (`test_kostnadsjeger.py`) – uten å snakke med Azure:

```
        from datetime import date
        from kostnadsjeger import vurder

        IDAG = date(2026, 10, 30)

        def test_ok():
            assert vurder("rg", {"eier": "kari", "utløper": "2027-01-01"}, IDAG) == []

        def test_mangler_alt():
            assert len(vurder("rg", None, IDAG)) == 2

        def test_utløpt():
            assert "utløpt 2020-01-01" in vurder("rg", {"eier": "x", "utløper": "2020-
01-01"}, IDAG)

        def test_ugyldig_dato():
            assert any("ugyldig" in p for p in vurder("rg", {"eier": "x", "utløper":
"snart"}, IDAG))
```

```
        pytest -q
        export AZURE_SUBSCRIPTION_ID=$(az account show --query id -o tsv)
        python kostnadsjeger.py
```

5. Legg til en GitHub Actions-workflow som kjører `pytest` og `ruff` på hver push.
6. **Bonus:** Kjør skriptet automatisk hver morgen som en Azure Function (timer trigger) med managed identity og *Reader*-rolle, eller som en planlagt GitHub Actions-workflow med OIDC.

Forventet resultat:

```
INFO rg-test-ok: ok
WARNING rg-test-gml: utløpt 2020-01-01
WARNING rg-test-utag: mangler tag 'eier'; mangler tag 'utløper'
```

Vanlige feil:

Problem	Løsning
`DefaultAzureCredential failed to retrieve a token`	Kjør `az login`; sjekk at riktig tenant/abonnement er valgt
`AuthorizationFailed`	Identiteten trenger minst *Reader* på abonnementet
Testene snakker med Azure	Hold logikk (`vurder`) adskilt fra I/O – da kan den testes isolert

Opprydding: `az group delete -n rg-test-ok --yes`, samme for de to andre.

På GitHub og CV: *«Utviklet et Python-verktøy som bruker Azure SDK og managed identity til å finne ressurser uten eier- og utløpstagger, med tester, logging og CI – for å redusere unødvendige skykostnader.»*

Prosjekt 3 – Azure-VM med kode: CLI-skript og cloud-init

Læringsmål: Opprette en komplett, sikker VM-løsning i Azure **utelukkende med kode**: VNet, subnett, NSG, VM med cloud-init, auto-shutdown, og overvåking – og rydde opp med én kommando.

Forutsetninger: Kapittel 2, 3 og 7.2.

Arkitektur:

```
rg-prosjekt3 (norwayeast)
└── vnet-p3 10.30.0.0/16
    └── snet-web 10.30.1.0/24 ── nsg-web: 80 fra Internet, 22 KUN fra din IP
        └── vm-p3 (Ubuntu 24.04, B1s/B2s) ── cloud-init: nginx + side
            ├── offentlig IP (Standard)
            ├── auto-shutdown 19:00 UTC
            └── CPU-varsel → action group (e-post)
```

Verktøy: Azure CLI (lokalt eller Cloud Shell), Bash.

Tid: 3–4 timer. **Kostnad:** En liten B-serie-VM i noen timer koster lite (sjekk Pricing Calculator); kan være dekket av gratiskontoens VM-timer. Disk og offentlig IP koster så lenge de finnes – **slett etterpå**.

Steg for steg:

1. Lag `cloud-init.yaml`:

```
        #cloud-config
        package_update: true
        package_upgrade: true
        packages: [nginx, unattended-upgrades]
        write_files:
          - path: /var/www/html/index.html
            content: |
              <h1>Prosjekt 3</h1><p>Laget med Azure CLI og cloud-init.</p>
        runcmd:
          - systemctl enable --now nginx
```

2. Lag `deploy.sh`:

```
        #!/usr/bin/env bash
        set -euo pipefail

        RG=rg-prosjekt3
        LOC=norwayeast
        VM=vm-p3
        MYIP=$(curl -s https://api.ipify.org)
        EPOST="${1:?Bruk: ./deploy.sh <e-post-for-varsler>}"

        az group create -n $RG -l $LOC --tags prosjekt=p3 eier=student utløper=$(date -d
'+7 days' +%F) -o none

        az network vnet create -g $RG -n vnet-p3 --address-prefix 10.30.0.0/16 \
          --subnet-name snet-web --subnet-prefixes 10.30.1.0/24 -o none

        az network nsg create -g $RG -n nsg-web -o none
        az network nsg rule create -g $RG --nsg-name nsg-web -n allow-http --priority
100 \
          --access Allow --protocol Tcp --direction Inbound \
          --source-address-prefixes Internet --destination-port-ranges 80 -o none
        az network nsg rule create -g $RG --nsg-name nsg-web -n allow-ssh-me --priority
110 \
          --access Allow --protocol Tcp --direction Inbound \
          --source-address-prefixes "$MYIP/32" --destination-port-ranges 22 -o none
        az network vnet subnet update -g $RG --vnet-name vnet-p3 -n snet-web --network-
security-group nsg-web -o none

        az vm create -g $RG -n $VM --image Ubuntu2404 --size Standard_B1s \
          --vnet-name vnet-p3 --subnet snet-web --nsg "" \
          --public-ip-sku Standard --admin-username azureuser --generate-ssh-keys \
          --custom-data cloud-init.yaml -o none

        az vm auto-shutdown -g $RG -n $VM --time 1900 -o none

        az monitor action-group create -g $RG -n ag-p3 --short-name agp3 --action email
meg "$EPOST" -o none
        az monitor metrics alert create -g $RG -n cpu-hoy-p3 \
          --scopes "$(az vm show -g $RG -n $VM --query id -o tsv)" \
          --condition "avg Percentage CPU > 80" --window-size 5m --evaluation-frequency
1m \
          --action ag-p3 -o none

        IP=$(az vm show -d -g $RG -n $VM --query publicIps -o tsv)
        echo "Ferdig! http://$IP   (ssh azureuser@$IP)"
```

3. Kjør: `chmod +x deploy.sh && ./deploy.sh deg@example.com`. Vent 1–3 minutter på at cloud-init blir ferdig, og åpne URL-en.
4. Verifiser sikkerheten: test SSH fra mobilnettet (annen IP) – skal **ikke** virke. Sjekk *Network Watcher → IP flow verify*.
5. Test varselet: `ssh azureuser@<ip> "sudo apt-get install -y stress-ng && stress-ng --cpu 1 --timeout 600"`.
6. Lag `destroy.sh` med `az group delete -n rg-prosjekt3 --yes --no-wait`.

Forventet resultat: En nettside på en VM laget helt fra kode, med SSH bare fra din IP, auto-shutdown og e-postvarsel ved høy CPU.

Vanlige feil:

Problem	Løsning
Siden svarer ikke rett etter opprettelse	cloud-init kjører fortsatt; sjekk `sudo cloud-init status --wait` og `/var/log/cloud-init-output.log`
`apt` får ikke kontakt med internett	Utgående tilgang – se merknaden om *default outbound access* i kapittel 7.2. VM-en har offentlig IP her, så det skal virke
`SkuNotAvailable` / kvotefeil	Velg annen størrelse (`Standard_B2s`, `Standard_B2ats_v2`) eller region
`date -d` virker ikke på macOS	Bruk `gdate` (coreutils) eller skriv datoen manuelt

Opprydding: `./destroy.sh`. Sjekk i portalen at ressursgruppen er borte.

På GitHub og CV: *«Automatiserte opprettelse av en sikker Linux-VM i Azure med Azure CLI og cloud-init: segmentert nettverk med NSG (SSH kun fra godkjent IP), automatisk avslåing og metrikkvarsler med Azure Monitor.»*

Prosjekt 4 – Profesjonelt GitHub-repo og portefølje

Læringsmål: Sette opp GitHub som en profesjonell utvikler: profil-README, porteføljerepo, branch rulesets, PR-maler, issues, Dependabot og CI.

Forutsetninger: Kapittel 5.

Arkitektur:

```
github.com/<deg>
├── <deg>/<deg>            ← profil-README (vises på profilsiden din)
└── devops-portefolje      ← samler alle prosjektene
    ├── README.md (oversikt + lenker + badges)
    ├── .github/
    │   ├── workflows/ci.yml          (lint av Markdown/YAML/shell)
    │   ├── pull_request_template.md
    │   ├── ISSUE_TEMPLATE/bug.md
    │   └── dependabot.yml
    ├── CODEOWNERS
    └── prosjekter/ (eller lenker til egne repoer per prosjekt)
```

Tid: 3–4 timer. **Kostnad:** Gratis.

Steg for steg:

1. **Profil-README:** Lag et repo med *nøyaktig samme navn som brukernavnet ditt*. README-en vises på profilen. Skriv kort: hvem du er, hva du lærer, teknologier, og lenker til 3–4 beste prosjekter. Hold det ryddig – ingen overdrevne animasjoner.
2. **Porteføljerepo:** `devops-portefolje` med README som har en tabell over prosjektene (navn, beskrivelse, teknologier, status, lenke).
3. **PR-mal** `.github/pull_request_template.md`:

   ```
   ## Hva
   ## Hvorfor
   ## Hvordan testet
   - [ ] CI er grønn
   - [ ] README oppdatert ved behov
   ```

4. **CI** `.github/workflows/ci.yml` som kjører `markdownlint`, `yamllint` og `shellcheck`.
5. **Dependabot** `.github/dependabot.yml`:

   ```
   version: 2
   updates:
     - package-ecosystem: "github-actions"
       directory: "/"
       schedule:
         interval: "weekly"
   ```

6. **Ruleset** på `main`: krev PR, krev grønn CI, blokker force push.
7. **Sikkerhet:** Slå på *secret scanning* og *push protection* (Settings → Code security).
8. **Arbeid som et team:** Lag 3 issues, en Project-tavle (Kanban), og løs issuene via brancher og PR-er med `Closes #n`.
9. **Lisens:** Legg til en LICENSE (f.eks. MIT) så andre vet hva de kan gjøre med koden.

Forventet resultat: En profil som ser profesjonell ut, og et repo som viser at du kan moderne Git-arbeidsflyt.

Vanlige feil: Profil-README vises ikke → repoet må hete nøyaktig det samme som brukernavnet og være *public*. Ruleset hindrer deg i å merge → statussjekkens navn må matche jobbnavnet i workflowen.

Opprydding: Ingen.

På CV: Lenke til GitHub-profilen øverst på CV-en, og til 2–3 utvalgte repoer.

Prosjekt 5 – Containeriser en app med Docker

Læringsmål: Containerisere en webapp med database, optimalisere imaget, skanne det og publisere det automatisk.

Forutsetninger: Kapittel 9.

Arkitektur: Se prosjektet i kapittel 9 (Flask + PostgreSQL med Compose, Trivy-skanning, publisering til ghcr.io med GitHub Actions).

Tid: 4–6 timer. **Kostnad:** Gratis.

Steg for steg: Følg «Prosjekt: bygg, kjør og publiser en applikasjon» i kapittel 9, og gjør i tillegg:

1. **Multi-stage Dockerfile** – mål størrelsen før og etter med `docker images`, og dokumenter det i README.
2. `HEALTHCHECK` i Dockerfile og `depends_on: condition: service_healthy` i Compose.
3. **Lint Dockerfile** med hadolint: `docker run --rm -i hadolint/hadolint < Dockerfile`.
4. **Integrasjonstest i CI:** start Compose i workflowen, kall `/` to ganger, og sjekk at telleren øker:

```
              - run: docker compose up -d --build --wait
                env:
                  DB_PASSWORD: ci-passord
              - run: |
                  curl -sf localhost:8000/ && curl -sf localhost:8000/ | grep -q "2
besøk"
              - if: always()
                run: docker compose logs && docker compose down -v
```

5. **Tagging:** både commit-SHA og, ved Git-tag `v*`, semantisk versjon.

Forventet resultat: En app som starter med én kommando, et lite og skannet image, og automatisk publisering.

Vanlige feil: Se feilsøkingstabellen i kapittel 9 (port, `0.0.0.0`, nettverk, arkitektur).

Opprydding: `docker compose down -v`, `docker system prune`.

På CV: Se kapittel 9.

Prosjekt 6 – Publiser en nettside på Azure med CI/CD

Læringsmål: Publisere en statisk nettside (f.eks. din egen CV/portefølje) på **Azure Static Web Apps** med automatisk deploy fra GitHub, forhåndsvisningsmiljøer for pull requests, og eventuelt eget domene.

Forutsetninger: Kapittel 5 og 7.

Arkitektur:

```
GitHub repo (index.html, style.css)
    │ push / PR
    ▼
GitHub Actions (workflow generert av Azure)
    │ deploy
    ▼
Azure Static Web Apps (Free) ──► global distribusjon, HTTPS automatisk
    ├── produksjon: https://<navn>.azurestaticapps.net   (fra main)
    └── PR-miljø:   https://<navn>-<pr>.<region>.azurestaticapps.net (slettes når PR lukkes)
```

Verktøy: GitHub, Azure CLI eller portalen, HTML/CSS (eller en statisk side-generator).

Tid: 3–5 timer. **Kostnad:** **Free**-planen er gratis (med begrensninger på størrelse og funksjoner – sjekk gjeldende vilkår). Eget domene kjøpes hos en registrar.

Steg for steg:

1. Lag repoet `cv-nettside` med `index.html` (din CV/portefølje), `style.css` og `404.html`.
2. Opprett Static Web App koblet til repoet. Enklest i portalen: *Static Web Apps* → *+ Create* → Plan *Free* → Source *GitHub* → velg repo og branch `main` → Build preset *Custom*, App location `/`, Output location tom. Eller med CLI:

   ```
   az group create -n rg-prosjekt6 -l westeurope
   az staticwebapp create -n swa-cv-<unik> -g rg-prosjekt6 -l westeurope \
     --source https://github.com/<deg>/cv-nettside --branch main \
     --app-location "/" --output-location "" --login-with-github
   ```

   (Static Web Apps er tilgjengelig i et utvalg regioner – innholdet distribueres globalt uansett.)
3. Azure legger automatisk inn en workflow i `.github/workflows/` og en deploy-token som GitHub Secret. Se jobben kjøre under *Actions*.
4. Åpne URL-en (`az staticwebapp show -n swa-cv-<unik> --query defaultHostname -o tsv`).

5. Lag en branch, endre noe, åpne en PR – legg merke til at det lages et **forhåndsvisningsmiljø** med egen URL i PR-kommentaren.
6. Legg til `staticwebapp.config.json` for sikkerhetsheadere:

```
{
  "globalHeaders": {
    "Content-Security-Policy": "default-src 'self'",
    "X-Content-Type-Options": "nosniff",
    "Referrer-Policy": "strict-origin-when-cross-origin"
  },
  "responseOverrides": { "404": { "rewrite": "/404.html" } }
}
```

7. **Valgfritt – eget domene:** *Custom domains* → *+ Add*, lag CNAME `www` → `<navn>.azurestaticapps.net` hos registraren (eller i Azure DNS). HTTPS-sertifikat ordnes automatisk.
8. Legg til en Lighthouse-sjekk i workflowen (ytelse/tilgjengelighet) som bonus.

Forventet resultat: Din egen nettside med HTTPS, automatisk deploy ved push, og forhåndsvisning av endringer.

Vanlige feil:

Problem	Løsning
Deploy feiler med «App Directory Location … could not be found»	`app_location` i workflowen peker feil
404 på undersider	Sjekk filnavn (store/små bokstaver) og `navigationFallback` for SPA-er
Domenet valideres ikke	DNS-endringen har ikke propagert ennå (TTL), eller feil posttype

Opprydding: `az group delete -n rg-prosjekt6 --yes` (eller behold den – Free-planen koster ikke).

På CV: Lenk til nettsiden! *«Publiserte personlig nettside på Azure Static Web Apps med CI/CD fra GitHub Actions, forhåndsvisningsmiljøer per pull request og sikkerhetsheadere.»*

Prosjekt 7 – CI/CD-pipeline til Azure App Service

Læringsmål: Bygge en komplett pipeline for en Python-API: lint, test, sikkerhetsskanning, build, deploy til staging og produksjon med godkjenning – med OIDC og managed identity.

Forutsetninger: Kapittel 4, 7.2 og 12.

Arkitektur:

```
PR ──► CI: ruff · pytest · pip-audit · CodeQL
merge ──► build: zip-artefakt (én gang)
      ──► deploy staging  (app-p7-staging, F1/B1)  ── smoke test /health
      ──► [godkjenning: environment «production»]
      ──► deploy prod     (app-p7-prod)            ── smoke test /health
Begge apper: managed identity → Key Vault (Key Vault-referanser i app settings)
             Application Insights (OpenTelemetry)
GitHub ──OIDC──► Entra ID (federated credentials for staging og production)
```

Verktøy: GitHub Actions, Azure CLI, Python (Flask/FastAPI), Key Vault, Application Insights.

Tid: 6–8 timer. **Kostnad:** To apper på **F1** er gratis (Free-planen har begrensninger, f.eks. ingen deployment slots og begrenset CPU-tid). B1 koster litt per time. Key Vault og Application Insights koster lite ved lav bruk.

Steg for steg:

1. **Appen:** En liten Flask-API med `/`, `/health` og `/api/hilsen?navn=`. Legg til `azure-monitor-opentelemetry` (kapittel 13) og tester med `pytest`.
2. **Infrastruktur (CLI eller Bicep fra kapittel 7.4):** ressursgruppe, App Service Plan (Linux), to webapper (`-staging`, `-prod`), Key Vault, Log Analytics + Application Insights. Slå på managed identity på begge appene, gi dem *Key Vault Secrets User*, og sett app setting `API_KEY=@Microsoft.KeyVault(VaultName=...;SecretName=ApiKey)`.
3. **OIDC:** App-registrering med to federated credentials (`environment:staging` og `environment:production`), rollen *Website Contributor* (eller *Contributor*) på ressursgruppen.
4. **GitHub:** environments `staging` og `production` (med required reviewer), variabler `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`.
5. **Workflow** `.github/workflows/cicd.yml`:

```yaml
name: CI/CD
on:
  pull_request:
  push:
    branches: [main]
permissions:
  contents: read

jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: pip install -r requirements.txt ruff pytest pip-audit
      - run: ruff check .
      - run: pytest -q
      - run: pip-audit -r requirements.txt

  build:
    if: github.ref == 'refs/heads/main'
    needs: ci
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: zip -r app.zip . -x '.git/*' '.github/*' 'tests/*'
      - uses: actions/upload-artifact@v4
        with: { name: app, path: app.zip }

  deploy-staging:
    needs: build
    runs-on: ubuntu-latest
    environment: staging
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/download-artifact@v4
        with: { name: app }
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - uses: azure/webapps-deploy@v3
        with:
          app-name: app-p7-staging
          package: app.zip
      - name: Smoke test
        run: curl --fail --retry 10 --retry-delay 10 --retry-all-errors
https://app-p7-staging.azurewebsites.net/health

  deploy-prod:
    needs: deploy-staging                    # bare etter vellykket staging
    runs-on: ubuntu-latest
    environment: production                  # krever godkjenning
    permissions:
      id-token: write
```

```
      contents: read
    steps:
      - uses: actions/download-artifact@v4
        with: { name: app }                  # SAMME artefakt som i staging
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - uses: azure/webapps-deploy@v3
        with:
          app-name: app-p7-prod
          package: app.zip
      - name: Smoke test
        run: curl --fail --retry 10 --retry-delay 10 --retry-all-errors
https://app-p7-prod.azurewebsites.net/health
```

(App Service bygger Python-appen ved deploy når app setting `SCM_DO_BUILD_DURING_DEPLOYMENT=true` er satt. Web-app-navn må være globalt unike – legg til et eget suffiks.)

6. **Branch ruleset:** krev at `ci` er grønn før merge.
7. **Test hele flyten:** lag en PR som bryter en test (rød), fiks den (grønn), merge, se staging deploye, godkjenn produksjon.
8. **Rollback-øvelse:** deploy en versjon som feiler smoke test i staging – bekreft at produksjon aldri ble rørt. Dokumenter hvordan du redeployer forrige artefakt (kjør workflowen på nytt fra en tidligere commit med *Re-run jobs*).

Forventet resultat: Hver merge går automatisk til staging; produksjon krever godkjenning; ingen hemmeligheter lagret i GitHub; telemetri i Application Insights.

Vanlige feil:

Problem	Løsning
Appen starter ikke («Application Error»)	`az webapp log tail`; sjekk oppstartskommando (`gunicorn --bind=0.0.0.0 app:app`) under *Configuration → Startup Command*
Key Vault-referansen viser «error» i portalen	Mangler RBAC-rolle, eller identiteten er ikke slått på
OIDC-feil	Subject må matche `environment:<navn>` nøyaktig

Opprydding: `az group delete -n rg-prosjekt7 --yes`, slett app-registreringen, `az keyvault purge` hvis du vil gjenbruke navnet.

På CV: *«Bygget CI/CD med GitHub Actions for Python-API: linting, tester, avhengighetsskanning, artefakt bygget én gang, OIDC-innlogging mot Azure, staging med smoke tests og godkjenningsstyrt produksjonsdeploy til Azure App Service, hemmeligheter i Key Vault via managed identity.»*

Prosjekt 8 – Azure-infrastruktur med Terraform og pipeline

Læringsmål: Bygge et realistisk Azure-miljø med Terraform-moduler, remote state, flere miljøer, og en pipeline med `plan` på PR og `apply` etter merge.

Forutsetninger: Kapittel 11 og 12.

Arkitektur:

```
infra-repo
├── modules/
│   ├── network/   (VNet, subnett for app og data, NSG-er)
│   ├── webapp/    (App Service Plan, Linux Web App med managed identity og VNet-
integrasjon)
│   ├── keyvault/  (Key Vault, RBAC, private endpoint)
│   └── monitoring/ (Log Analytics, Application Insights, diagnostic settings)
└── envs/
    ├── dev/  (backend: key = dev.tfstate)
    └── prod/ (backend: key = prod.tfstate)

PR ──► GitHub Actions: fmt · validate · Checkov/Trivy · plan (som PR-kommentar)
merge ──► apply dev automatisk ──► [godkjenning] ──► apply prod
State: Azure Blob (rg-tfstate), låsing, versjonering, RBAC
```

Tid: 8–12 timer. **Kostnad:** Lav hvis du bruker F1/B1 og sletter etterpå. Private endpoints faktureres per time – kan droppes i dev for å spare.

Steg for steg:

1. **Remote state** som i kapittel 11 (`rg-tfstate`, ZRS, versjonering, `CanNotDelete`-lås).
2. **Moduler:** Start med `network` og `monitoring`. Hver modul har `variables.tf`, `main.tf`, `outputs.tf` og en kort `README.md`. Bruk `for_each` for subnett.
3. **Miljøer:** `envs/dev/main.tf` kaller modulene med dev-verdier; `envs/prod` med prod-verdier (større SKU, `purge_protection_enabled = true`, ZRS).
4. **Lokal kjøring først:** `terraform init/plan/apply` i `envs/dev`. Verifiser i portalen.
5. **Pipeline** `.github/workflows/terraform.yml`:

```yaml
name: Terraform
on:
  pull_request:
    paths: ["infra/**"]
  push:
    branches: [main]
    paths: ["infra/**"]

permissions:
  id-token: write
  contents: read
  pull-requests: write          # for å kommentere planen på PR-en

env:
  ARM_CLIENT_ID: ${{ vars.AZURE_CLIENT_ID }}
  ARM_TENANT_ID: ${{ vars.AZURE_TENANT_ID }}
  ARM_SUBSCRIPTION_ID: ${{ vars.AZURE_SUBSCRIPTION_ID }}
  ARM_USE_OIDC: "true"          # Terraform azurerm bruker GitHub OIDC-token
  TF_VAR_subscription_id: ${{ vars.AZURE_SUBSCRIPTION_ID }}

jobs:
  plan:
    runs-on: ubuntu-latest
    environment: dev
    defaults:
      run:
        working-directory: infra/envs/dev
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform fmt -check -recursive ../..
      - run: terraform init -input=false
      - run: terraform validate
      - run: terraform plan -input=false -no-color -out=tfplan | tee plan.txt
      - name: Kommenter planen på PR
        if: github.event_name == 'pull_request'
        uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const plan = fs.readFileSync('infra/envs/dev/plan.txt',
'utf8').slice(-60000);
            await github.rest.issues.createComment({
              owner: context.repo.owner, repo: context.repo.repo,
              issue_number: context.issue.number,
              body: "### Terraform plan (dev)\n```\n" + plan + "\n```"
            });
      - uses: actions/upload-artifact@v4
        if: github.ref == 'refs/heads/main'
        with: { name: tfplan-dev, path: infra/envs/dev/tfplan }

  apply-dev:
    if: github.ref == 'refs/heads/main'
    needs: plan
    runs-on: ubuntu-latest
    environment: dev
    defaults:
      run:
        working-directory: infra/envs/dev
```

```
            steps:
              - uses: actions/checkout@v4
              - uses: hashicorp/setup-terraform@v3
              - uses: actions/download-artifact@v4
                with: { name: tfplan-dev, path: infra/envs/dev }
              - run: terraform init -input=false
              - run: terraform apply -input=false tfplan     # nøyaktig den planen som
ble vist
```

Lag tilsvarende `plan-prod` og `apply-prod` (med environment `prod` og required reviewer). Federated credentials trengs for `environment:dev` og `environment:prod`, og identiteten trenger *Contributor* på målabonnementet/-gruppene og *Storage Blob Data Contributor* på state-kontoen.

6. **Sikkerhetsskanning:** legg til `bridgecrewio/checkov-action` eller `aquasecurity/trivy-action` med `scan-type: config` på `infra/`.
7. **Drift-deteksjon:** en planlagt workflow (`schedule`) som kjører `terraform plan -detailed-exit-code` hver natt og lager en issue hvis exit-koden er 2 (endringer funnet).
8. **Dokumenter** arkitekturen og hvordan man legger til et nytt miljø.

Forventet resultat: Infrastrukturendringer går kun via PR, planen vises som kommentar, dev oppdateres automatisk, prod etter godkjenning, og drift oppdages automatisk.

Vanlige feil:

Problem	Løsning
`Error acquiring the state lock`	En annen kjøring holder låsen, eller en kjøring krasjet. Vent, eller `terraform force-unlock <id>` hvis du er sikker
`AuthorizationFailed` på state	Identiteten mangler *Storage Blob Data Contributor*
Planen i apply avviker fra PR-planen	Noe endret seg mellom plan og apply – derfor brukes lagret `tfplan`
Globale navn kolliderer	Bruk `random_string`-suffiks eller `uniqueString`

Opprydding: `terraform destroy` i `envs/prod` og `envs/dev` (eller via en manuell workflow), deretter `rg-tfstate` til slutt.

På CV: *«Designet modulær Terraform-kode for Azure (nettverk, App Service, Key Vault, overvåking) med remote state i Azure Storage, flere miljøer, og GitHub Actions-pipeline med plan på pull requests, sikkerhetsskanning, godkjenningsstyrt apply via OIDC og nattlig drift-deteksjon.»*

Prosjekt 9 – Deploy en app på Kubernetes med Helm

Læringsmål: Deploye en flerkomponent-app på Kubernetes med Helm: deployment, service, ingress, config, secrets, probes, ressursgrenser, HPA og NetworkPolicy – lokalt først, deretter (valgfritt) i AKS.

Forutsetninger: Kapittel 9 og 10.

Arkitektur:

```
kind-cluster (lokalt)  /  AKS (valgfritt)
└── namespace: butikk
     ├── Ingress / HTTPRoute  butikk.localhost → web
     ├── Deployment web (2–5 replikaer, HPA på CPU) ── Service web (ClusterIP)
     │     env fra ConfigMap + Secret, readiness/liveness, requests/limits
     ├── StatefulSet / Deployment postgres + PVC ── Service db
     └── NetworkPolicy: bare web → db:5432
Pakket som Helm chart «butikk» med values-dev.yaml og values-prod.yaml
```

Verktøy: Docker, kind, kubectl, Helm.

Tid: 8–12 timer. **Kostnad:** Gratis lokalt. I AKS: noder faktureres – stopp/slett samme dag.

Steg for steg:

1. **Cluster med ingress-støtte** (kind):

   ```yaml
   # kind-config.yaml
   kind: Cluster
   apiVersion: kind.x-k8s.io/v1alpha4
   nodes:
     - role: control-plane
       extraPortMappings:
         - containerPort: 80
           hostPort: 80
   ```

   ```
   kind create cluster --name butikk --config kind-config.yaml
   ```

   Installer en ingress- eller Gateway API-kontroller etter dokumentasjonen til kontrolleren du velger (kind-dokumentasjonen har en egen side om ingress). Se merknaden i kapittel 10 om ingress-nginx sin pensjonering, og velg et aktivt vedlikeholdt alternativ.

2. **Last imaget inn i kind** (slipper register lokalt):

   ```
   docker build -t butikk-web:1.0 .
   kind load docker-image butikk-web:1.0 --name butikk
   ```

3. **Skriv manifestene først som ren YAML** (deployment, service, configmap, secret, postgres med PVC, ingress) og test med `kubectl apply -f k8s/ -n butikk`. Bruk eksemplene i kapittel 10.

4. **Gjør dem om til en Helm chart:** `helm create butikk`, slett eksempelmalene du ikke trenger, flytt inn dine manifester og erstatt verdier med `{{ .Values... }}`.
5. **To miljøer:**

```
        helm upgrade --install butikk ./butikk -n butikk-dev  --create-namespace -f
values-dev.yaml
        helm upgrade --install butikk ./butikk -n butikk-prod --create-namespace -f
values-prod.yaml
```

6. **Robusthet:** Slett en web-pod og se at den erstattes. Sett en feil i readinessProbe og se at podene ikke får trafikk. Rull ut et image som ikke finnes og gjør `helm rollback`.
7. **HPA:** installer metrics-server i kind (kind trenger flagget `--kubelet-insecure-tls` – se metrics-server-dokumentasjonen), lag HPA, generer last og se skaleringen med `kubectl get hpa -w`.
8. **NetworkPolicy:** kind sin standard-CNI håndhever ikke NetworkPolicy; installer Calico eller Cilium i kind for å teste, eller test i AKS med nettverkspolicy slått på.
9. **CI:** en workflow som kjører `helm lint` og `kubeconform` (validerer manifester) på hver PR.
10. **Valgfritt – AKS:** push imaget til ACR, `az aks create ... --attach-acr --enable-app-routing`, sett `ingressClassName: webapprouting.kubernetes.azure.com` i `values-aks.yaml`, og installer chart-en. **Slett clusteret etterpå.**

Forventet resultat: `http://butikk.localhost` viser appen; data overlever pod-restart; HPA skalerer; rollback virker.

Vanlige feil: Se feilsøkingstabellen i kapittel 10. Spesielt: `ImagePullBackOff` i kind betyr ofte at du glemte `kind load docker-image`, eller at `imagePullPolicy: Always` får kind til å prøve et register – bruk `IfNotPresent` lokalt.

Opprydding: `kind delete cluster --name butikk`. I AKS: `az group delete`.

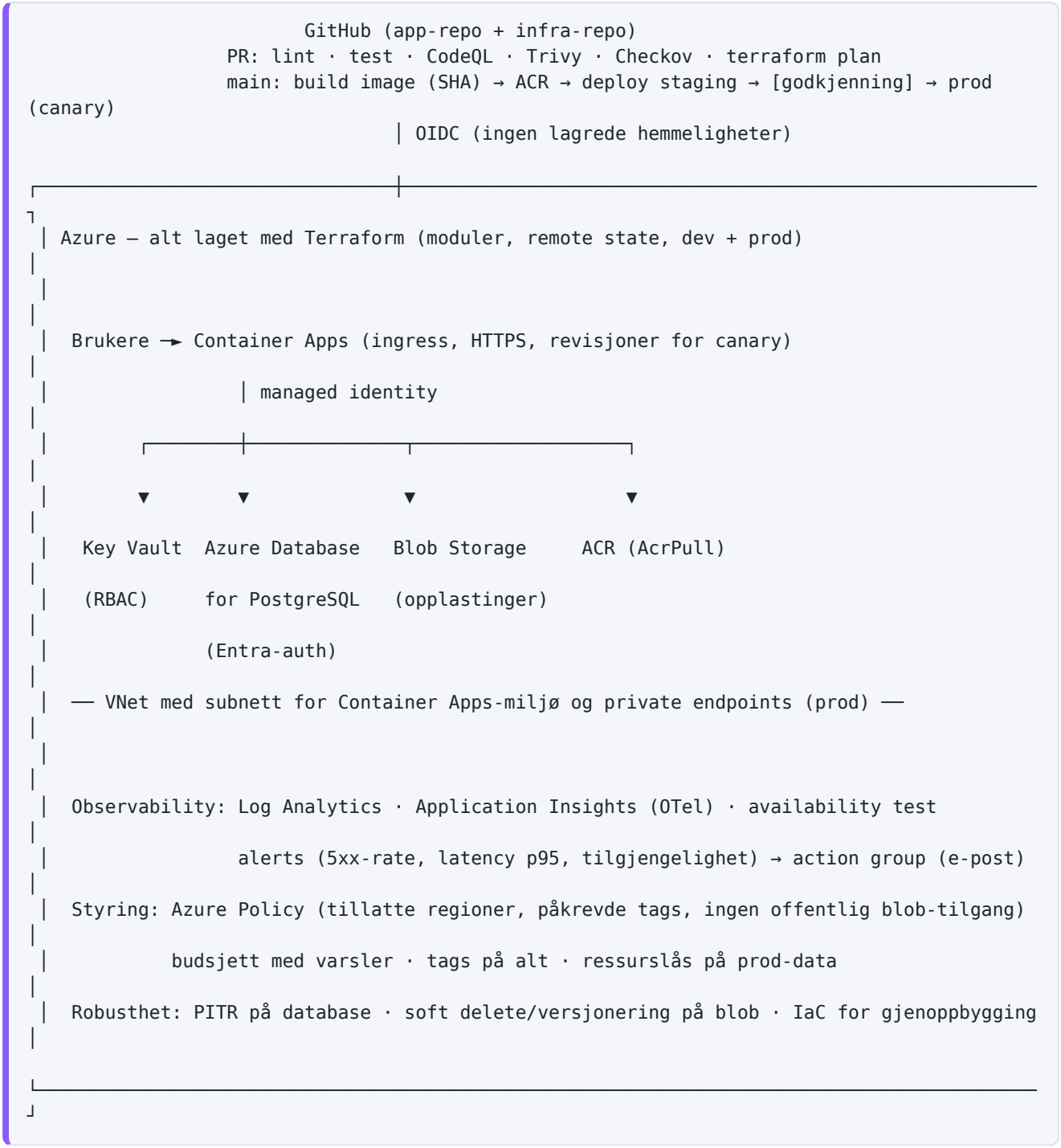
På CV: *«Pakket en webapp med PostgreSQL som Helm chart for Kubernetes med probes, ressursgrenser, HPA, persistent lagring, NetworkPolicy og separate verdier per miljø; validert i CI med helm lint og kubeconform; deployet til AKS.»*

Prosjekt 10 – Komplett Cloud/DevOps-plattform

Læringsmål: Kombinere alt: IaC, containere, CI/CD, sikkerhet, overvåking, kostnadsstyring og dokumentasjon i én helhetlig plattform – sluttprosjektet ditt.

Forutsetninger: Hele boken, særlig prosjekt 5, 7, 8 og 9.

Arkitektur (målbilde):



Verktøy: Alt fra boken.

Tid: 25–40 timer, gjerne fordelt over 3–4 uker. **Kostnad:** Lav–moderat. Hold dev og prod små (Container Apps med min-replicas 0, minste databasenivå, ingen private endpoints i dev). **Lag et kostnadsestimat i Pricing Calculator før du starter**, og slett alt når du har tatt opp demo-videoen.

Faser:

Fase 1 – Design (2–3 timer). - Skriv en kort **arkitekturbeslutningslogg** (ADR – *Architecture Decision Record*): hvorfor Container Apps og ikke AKS? Hvorfor PostgreSQL Flexible Server? Hvorfor Terraform? - Definer **SLO-er**: f.eks. 99,5 % vellykkede forespørsler og p95 < 500 ms over 30 dager. - Tegn arkitekturen.

Fase 2 – Grunnmur med IaC (6–10 timer). - Remote state, moduler for nettverk, overvåking, Key Vault, database, lagring, ACR og Container Apps. - Azure Policy-tildelinger og budsjett som Terraform-ressurser (`azurerm_resource_group_policy_assignment`, `azurerm_consumption_budget_resource_group`). - Tags på alt via `locals`.

Fase 3 – Applikasjonen (4–6 timer). - En enkel, men ekte app (f.eks. «lenkesamling», «treningslogg» eller «utstyrsutlån») med API, database og filopplasting til Blob. - Managed identity mot database (Entra-autentisering), Blob og Key Vault – **null passord i konfigurasjonen**. - OpenTelemetry for traces, metrikker og strukturerte logger.

Fase 4 – CI/CD (5–8 timer). - App-pipeline fra kapittel 12 (canary + automatisk rollback). - Infra-pipeline fra prosjekt 8. - Sikkerhetsskanning: CodeQL, Dependabot, Trivy (image og IaC), secret scanning med push protection.

Fase 5 – Observability og drift (4–6 timer). - Dashboard (Azure Workbook eller Managed Grafana) med RED-metrikker og SLO-status. - Varsler: 5xx-rate, p95-latency, availability test, budsjett. - **Runbooks** for de tre mest sannsynlige hendelsene (database utilgjengelig, høy feilrate etter deploy, kostnadsvarsel).

Fase 6 – Robusthet og sikkerhet (3–5 timer). - Test **point-in-time restore** av databasen til en ny server, og dokumenter tiden (din faktiske RTO). - Test **gjenoppbygging**: slett dev-miljøet og bygg det opp igjen fra Terraform. Hvor lang tid tok det? - Gå gjennom Defender for Cloud-anbefalingene og fiks det du kan.

Fase 7 – Dokumentasjon og presentasjon (3–4 timer). - README med arkitekturdiagram, ADR-er, SLO-er, hvordan kjøre det, kostnadsestimat, og «Hva jeg lærte». - En **3–5 minutters demovideo**: arkitektur, en PR gjennom pipelinen, canary, et varsel som utløses, og en rollback. - Rydd opp og skriv i README at miljøet er slettet for å spare kostnader (videoen viser at det fungerte).

Forventet resultat: En plattform du kan forklare fra ende til annen i et intervju – med kode, pipeline, overvåking, sikkerhet, kostnadskontroll og dokumentert gjenoppretting.

Vanlige feil:

- **For stort omfang.** Start med en «walking skeleton»: minste versjon som går hele veien fra commit til prod. Bygg ut deretter.
- **Manuelle endringer** i portalen som ikke kommer inn i Terraform.
- **Glemte kostnader**: databaser og private endpoints koster per time selv uten trafikk.

Feilsøking: Bruk metoden i kapittel 13 – og noter hvert problem du løser i en `TROUBLESHOOTING.md`. Den filen er gull i intervjuer.

Opprydding: `terraform destroy` for prod og dev, slett state-gruppen, slett app-registreringer, purge Key Vault, og sjekk *Cost Management* et par dager senere for å bekrefte at forbruket er null.

På GitHub og CV: Dette er porteføljens hovedprosjekt. *«Designet og bygget en komplett Azure-plattform med Terraform (moduler, remote state, flere miljøer), containerisert Python-applikasjon på Azure Container Apps, CI/CD med GitHub Actions (OIDC, sikkerhetsskanning, canary og auto-*

matisk rollback), passordløs tilgang med managed identity, observability med OpenTelemetry/Application Insights og SLO-baserte varsler, Azure Policy, budsjettstyring og testet gjenoppretting (PITR og full gjenoppbygging fra kode).»

Prosjekt 11 – MLOps: modell-API med CI/CD og overvåking

Læringsmål: Ta en ML-modell fra notebook til produksjon: reproduserbar trening, sporing med MLflow, modell som API i en container, CI/CD med evaluering som kvalitetsport, og overvåking av både drift og modellkvalitet.

Forutsetninger: Kapittel 4, 9, 12 og 14.

Arkitektur:

```
data/ (versjonert datasett, f.eks. åpent boligprisdatasett)
  |
train.py ──► MLflow tracking (parametere, MAE) ──► modell.joblib + metrics.json
  |
CI (GitHub Actions):
  1. test kode (pytest)
  2. tren modellen (liten nok til å kjøre i CI)
  3. KVALITETSPORT: MAE ≤ terskel OG ikke dårligere enn forrige prod-modell → ellers rødt
  4. bygg image (serve.py + modell) tag = SHA → ACR / ghcr.io
  5. deploy til Container Apps (canary 20 % → 100 %)
serve.py (FastAPI): /predict, /health, /metrics  ── logger input-statistikk for drift
Overvåking: Application Insights (latency, feil) + daglig drift-sjekk (jobb som
sammenligner
            fordelingen av input mot treningsdata og varsler ved avvik)
```

Verktøy: Python (pandas, scikit-learn, MLflow, FastAPI), Docker, GitHub Actions, Azure Container Apps (eller lokalt).

Tid: 10–15 timer. **Kostnad:** Lav (Container Apps med min-replicas 0). Kan gjøres helt lokalt.

Steg for steg:

1. **Velg et åpent, lite datasett** (f.eks. et tabulært boligpris- eller diabetesdatasett fra scikit-learn). Lagre en fast kopi i repoet (eller i Blob med versjonsnummer) og en `DATA_VERSION`.
2. **`train.py`** – les data, del i trening/test med fast `random_state`, tren, beregn MAE, logg til MLflow, lagre `modell.joblib` og `metrics.json`.
3. **Kvalitetsport** `evaluate.py`:

```
        import json, sys
        ny = json.load(open("metrics.json"))["mae"]
        terskel = 55.0                                  # akseptabel feil (velg ut fra
datasettet)
        forrige = json.load(open("prod_metrics.json"))["mae"] if
__import__("os").path.exists("prod_metrics.json") else None
        print(f"Ny MAE: {ny:.2f}  Forrige: {forrige}")
        if ny > terskel or (forrige is not None and ny > forrige * 1.02):   # maks 2 %
dårligere
            print("Modellen besto IKKE kvalitetsporten"); sys.exit(1)
        print("Kvalitetsport OK")
```

4. **`serve.py`** – FastAPI fra kapittel 14 med `/predict`, `/health` (inkl. modellversjon) og Prometheus-metrikker (`prometheus-client`) for antall prediksjoner og latency. Logg et utvalg av input-verdier (ikke personopplysninger) for drift-analyse.
5. **Dockerfile** som kopierer inn `serve.py` og den **trente** modellen.
6. **Workflow:** test → train → evaluate → build → deploy (canary som i kapittel 12). Last opp `metrics.json` og MLflow-kjøringen som artefakter.
7. **Drift-sjekk:** et skript som sammenligner gjennomsnitt og standardavvik for hver input-variabel siste døgn mot treningsdataene, og feiler (→ varsel/issue) hvis avviket er stort. Kjør det med `schedule` i GitHub Actions.
8. **Simuler drift:** send forespørsler med systematisk forskjøvne verdier og se at drift-sjekken slår ut. Tren på nytt med «nye» data og se den nye modellen gå gjennom porten.

Forventet resultat: Hver endring i kode **eller data** gir en ny, evaluert, sporbar modellversjon; dårligere modeller stoppes automatisk; drift oppdages.

Vanlige feil: Ikke-reproduserbar trening (glemt `random_state`, ulike bibliotekversjoner – lås dem i `requirements.txt`); modellfil som er for stor for Git (bruk Git LFS, Blob eller MLflow-artefakter); trening og servering bruker forskjellig forbehandling (*training/serving skew*) – del koden for forbehandling mellom dem.

Opprydding: Slett ressursgruppen; `mlflow`-data lokalt kan slettes.

På CV: *«Bygget en MLOps-pipeline: reproduserbar trening sporet med MLflow, automatisk kvalitetsport som stopper dårligere modeller, modell servert som FastAPI-container med canary-utrulling via GitHub Actions, og overvåking av latency og data drift.»*

Prosjekt 12 – Observability og hendelsesøvelse (game day)

Læringsmål: Sette opp full observability for en mikrotjeneste-app lokalt (metrikker, logger, traces), definere SLO-er og varsler, og gjennomføre en kontrollert hendelsesøvelse med postmortem.

Forutsetninger: Kapittel 8, 9 og 13.

Arkitektur:

```
Docker Compose:
  frontend (Flask) ──HTTP──► api (FastAPI) ──► postgres
       │ OpenTelemetry SDK          │ OTel SDK
       └──────────────┬─────────────┘
                      ▼ OTLP
           OpenTelemetry Collector
           ├─► Prometheus (metrikker) ──► Alertmanager (varsler → webhook/e-post)
           ├─► Grafana Tempo eller Jaeger (traces)
           └─► Grafana Loki (logger)
                      ▼
                Grafana (dashboards: RED, SLO, traces ↔ logger)
```

Verktøy: Docker Compose, OpenTelemetry Collector, Prometheus, Grafana, Loki, Tempo/Jaeger.

Tid: 8–10 timer. **Kostnad:** Gratis (lokalt).

Steg for steg:

1. **To tjenester** som kaller hverandre (frontend → api → database). Instrumenter begge med OpenTelemetry (auto-instrumentering for Flask/FastAPI/requests/psycopg) og strukturerte JSON-logger med `trace_id`.
2. **Compose-oppsett** med Collector, Prometheus, Grafana, Loki og Tempo (eller Jaeger). Grafana har ferdige eksempeloppsett for denne stacken i dokumentasjonen sin – bruk dem som utgangspunkt og lås image-versjoner.
3. **Dashboards:** RED per tjeneste, p95/p99-latency, feilrate, databasekall per forespørsel.
4. **SLO og varsler:** SLO 99 % vellykkede forespørsler over 1 time (kort periode for øvelsen). Lag en Prometheus-regel for *burn rate* (hvor fort error budget brukes) og en for p95 > 1 s.
5. **Feilinjeksjon:** legg inn miljøvariabler i api-tjenesten: `FEILRATE` (andel forespørsler som gir 500) og `FORSINKELSE_MS`. Skriv en enkel lastgenerator (`while true; do curl ...; done` eller k6).
6. **Game day** – med en medstudent som «angriper»:
   - Medstudenten endrer i hemmelighet én ting: øker `FORSINKELSE_MS`, setter `FEILRATE=0.2`, stopper databasen, eller fyller disken i en container.
   - Du får bare varselet, og skal finne og løse problemet med dashboards, traces og logger. Noter tidspunkter: *oppdaget*, *diagnostisert*, *løst* (MTTD/MTTR).
   - Bytt roller.
7. **Postmortem:** skriv en blameless postmortem (mal i kapittel 8) for hvert scenario. Hva gjorde det lett/vanskelig å finne feilen? Hvilket dashboard/varsel manglet? Implementer minst én forbedring.

Forventet resultat: Du kan følge én forespørsel fra frontend til database i en trace, hoppe fra trace til tilhørende logger, se SLO-status, og du har praktisk erfaring med hendelseshåndtering.

Vanlige feil: Traces henger ikke sammen mellom tjenester (trace-kontekst propageres ikke – bruk OTel-instrumentering av HTTP-klienten); for mange varsler (juster terskler og `for:`); metrikker

med for mange unike labelverdier (*high cardinality*, f.eks. bruker-ID som label) som sprenger Prometheus.

Opprydding: `docker compose down -v`.

På CV: *«Implementerte observability for mikrotjenester med OpenTelemetry, Prometheus, Loki, Tempo og Grafana; definerte SLO-er med burn rate-varsler; gjennomførte hendelsesøvelser (game days) med blameless postmortems.»*

Slik presenterer du prosjektene

På GitHub:

- Fest (*pin*) de 4–6 beste repoene på profilen.
- Hver README: problem, arkitektur, teknologier, hvordan kjøre, hva du lærte, videre arbeid.
- Grønne CI-badges. Ryddig historikk med gode commit-meldinger.
- Ingen hemmeligheter, ingen `node_modules` / `.venv`, ingen state-filer.

På CV-en (prosjektseksjon):

```
Azure DevOps-plattform (sluttprosjekt)                          github.com/<deg>/plattform
Terraform · Azure Container Apps · PostgreSQL · Key Vault · GitHub Actions · OpenTelemetry
• Bygget hele infrastrukturen som kode med moduler og remote state for dev og prod
• CI/CD med OIDC (ingen lagrede hemmeligheter), sikkerhetsskanning, canary og automatisk
rollback
• SLO-basert overvåking og varsling; testet gjenoppretting (PITR) og full gjenoppbygging
på 18 min
```

Tallfest der du kan (byggetid, image-størrelse redusert fra X til Y MB, gjenoppbygging på Z minutter).

I intervju – STAR-metoden: *Situation* (hva var problemet), *Task* (hva skulle du oppnå), *Action* (hva gjorde **du**, konkret), *Result* (hva ble resultatet, og hva lærte du). Ha 2–3 historier klare fra prosjektene – særlig én om et vanskelig problem du feilsøkte.

Vedlegg A – Ordliste

Begrepene er sortert alfabetisk. Engelsk begrep står først når det er det som brukes i bransjen; tallet i parentes viser kapitlet der begrepet forklares nærmere.

Begrep	Forklaring
ACR (Azure Container Registry)	Privat register for container-images i Azure (7.2)
Agile (smidig)	Arbeidsmåte med korte iterasjoner og hyppig tilbakemelding (8)
AKS (Azure Kubernetes Service)	Azures administrerte Kubernetes (7.2, 10)
Alert / varsel	Automatisk melding når en betingelse i overvåkingen er oppfylt (7.3, 13)
Ansible	Agentløst verktøy for konfigurasjonsstyring over SSH (11)
API	Avtalt grensesnitt for at programmer kan snakke sammen, ofte over HTTP med JSON (1)
App Service	Azures PaaS for webapper og API-er (7.2)
Application Gateway	Azures lag 7-lastbalanserer med WAF (7.2)
Application Insights	Applikasjonsovervåking i Azure Monitor (7.3, 13)
ARM (Azure Resource Manager)	Kontrollplanet som tar imot alle forespørsler om Azure-ressurser (7.1)
Artefakt (*artifact*)	Ferdig byggeresultat: image, pakke, zip (12)
Autentisering (AuthN)	Å bekrefte hvem noen er (13)
Autorisasjon (AuthZ)	Å bestemme hva noen får gjøre (13)
Autoskalering	Automatisk justering av kapasitet etter last (6, 10)
Availability set	VM-er fordelt på feil- og oppdateringsdomener i ett datasenter (7.2)
Availability zone (tilgjengelighetssone)	Fysisk adskilt datasenter-lokasjon i en region (6)
Backend	Serverdelen av en applikasjon: logikk og data (1)
Bash	Vanlig Linux-shell og skriptspråk (3)
Bastion	Azure-tjeneste for SSH/RDP uten offentlig IP på VM-en (7.2)
Bicep	Azures deklarative IaC-språk (7.4, 11)
Blob Storage	Objektlagring i Azure (7.3)

Begrep	Forklaring
Blue-green deployment	To like miljøer; all trafikk byttes på én gang (8, 12)
Branch	Parallell utviklingslinje i Git (5)
Canary deployment	Ny versjon får gradvis økende andel trafikk (8, 12)
CapEx / OpEx	Investering på forhånd / løpende driftsutgift (6)
CI/CD	Continuous Integration / Delivery / Deployment (8, 12)
CIDR	Notasjon for IP-område, f.eks. `10.0.0.0/16` (2)
CLI	Kommandolinjegrensesnitt, f.eks. `az`, `kubectl` (3, 7)
Cloud-init	Konfigurasjon som kjøres ved første oppstart av en VM (7.2)
Cluster	Gruppe maskiner som samarbeider, f.eks. i Kubernetes (10)
CNI	Nettverksplugin for Kubernetes (10)
Commit	Lagret øyeblikksbilde i Git (5)
ConfigMap	Kubernetes-objekt for ikke-sensitiv konfigurasjon (10)
Container	Isolert, pakket applikasjon som deler vertens kjerne (9)
Control plane (kontrollplan)	Den styrende delen av et system (ARM i Azure, API-server i Kubernetes)
CPU	Prosessoren som utfører instruksjoner (1)
Cron	Planlegging av jobber på faste tidspunkter (3)
CVE	Offentlig registrert sårbarhet (13)
DaemonSet	Kubernetes-arbeidslast med én pod per node (10)
Data drift	At produksjonsdata endrer seg fra treningsdata (14)
Deallocate	Stoppe en Azure-VM slik at compute ikke faktureres (7.2)
Deklarativ	Beskriver ønsket sluttresultat, ikke stegene (10, 11)
Deployment	Utrulling av software; også Kubernetes-objekt for tilstandsløse apper (10)
DevSecOps	Sikkerhet integrert i hele DevOps-livssyklusen (8)
DHCP	Deler ut IP-adresser automatisk (2)

Begrep	Forklaring
DNS	Oversetter navn til IP-adresser (1, 2)
Docker / Dockerfile	Containerverktøy / oppskrift for å bygge et image (9)
DORA-metrikker	Fire mål for leveranseevne: frekvens, ledetid, feil-rate, gjenopprettingstid (8)
DR (*disaster recovery*)	Gjenoppretting etter større hendelser (6, 13)
Elastisitet	Automatisk skalering opp og ned etter behov (6)
Entra ID	Microsofts identitetstjeneste, tidligere Azure AD (7.1)
Environment	Deploymål med regler (GitHub/Azure DevOps) (12)
Error budget	Tillatt upålitelighet innenfor en SLO (8)
etcd	Databasen som lagrer Kubernetes-clusterets til-stand (10)
Feature flag	Slår funksjoner av/på uten ny deploy (8)
Firewall (brannmur)	Tillater eller blokkerer nettverkstrafikk (2)
Frontend	Delen av appen brukeren ser (1)
Functions	Azures serverless kodetjeneste (7.2)
Gateway API	Kubernetes' moderne etterfølger til Ingress (10)
Git / GitHub	Versjonskontrollverktøy / nettjeneste for Git-repoer (5)
GitOps	Git som sannhetskilde, agent synkroniserer miljøet (11)
GPU	Prosessor med tusenvis av kjerner for parallell be-regning (14)
Grafana	Visualisering og dashboards (13)
Helm / chart	Pakkehåndterer for Kubernetes / pakke med maler og verdier (10)
HPA	Horizontal Pod Autoscaler (10)
HTTP / HTTPS	Protokollen for web / kryptert med TLS (1, 2)
Hypervisor	Programvare som kjører virtuelle maskiner (1)
IaaS / PaaS / SaaS	Infrastruktur / plattform / programvare som tjeneste (6)
IaC	Infrastructure as Code (11)

Begrep	Forklaring
Idempotens	Samme operasjon flere ganger gir samme resultat (11)
Image	Skrivebeskyttet mal for containere (9)
Incident	Uplanlagt avbrudd eller forringelse av en tjeneste (8, 13)
Ingress	Kubernetes-objekt for HTTP-ruting inn i clusteret (10)
IP-adresse	Adresse for en enhet på et nettverk (2)
Key Vault	Azures hvelv for hemmeligheter, nøkler og sertifikater (7.3)
Kernel (kjerne)	Kjernen i operativsystemet (1, 3)
KQL	Spørrespråket i Log Analytics (7.3)
kubectl	Kommandolinjeverktøy for Kubernetes (10)
Latency	Forsinkelse / responstid (2, 13)
Least privilege	Minste nødvendige tilgang (13)
Load balancer (lastbalanserer)	Fordeler trafikk mellom servere (2, 7.2)
Log Analytics	Azures loggdatabase (7.3)
Managed identity	Azure-administrert identitet for ressurser – uten passord (7.1)
Management group	Gruppe av Azure-abonnementer (7.1)
Merge / merge conflict	Slå sammen brancher / når samme linjer er endret begge steder (5)
MFA	Multifaktorautentisering (7.1, 13)
Microservices	Applikasjon delt i mange små tjenester (1)
MLOps	DevOps for maskinlæring (14)
Namespace	Logisk område i Kubernetes; også Linux-isolasjonsmekanisme (9, 10)
NAT	Oversetting av IP-adresser, f.eks. privat → offentlig (2)
Node	En maskin i et cluster (10)
NSG	Network Security Group – Azures nettverksfilter (7.2)

Begrep	Forklaring
Observability	Evnen til å forstå et system ut fra metrikker, logger og traces (8, 13)
OIDC	Åpen standard for innlogging; brukes for hemmelighetsløs tilgang fra CI til Azure (12)
OOMKilled	Prosess drept fordi den gikk tom for minne (9, 10)
OpenTelemetry	Standard for innsamling av telemetri (13)
Peering	Kobling mellom VNets (7.2)
Pipeline	Automatisert kjede av steg fra kode til drift (12)
Pod	Minste enhet i Kubernetes (10)
Policy (Azure Policy)	Regler for hvilke ressurser og innstillinger som er tillatt (7.1)
Port	Nummer som identifiserer en tjeneste på en maskin (2)
Postmortem	Etteranalyse etter en hendelse, uten skyld (8)
Private endpoint	Privat IP i ditt VNet for en PaaS-tjeneste (7.2, 7.3)
Prometheus / PromQL	Metrikksystem / dets spørrespråk (13)
Probe	Helsesjekk (readiness, liveness) (10)
Pull request	Forespørsel om å merge en branch, med gjennomgang (5)
PVC / PV	Forespørsel om lagring / lagringsressurs i Kubernetes (10)
RAG	Retrieval-Augmented Generation (14)
RAM	Raskt, flyktig arbeidsminne (1)
RBAC	Rollebasert tilgangskontroll (7.1, 10)
Region	Geografisk område med datasentre (6)
Registry	Lager for container-images (9)
Remote state	Terraform-state lagret sentralt med låsing (11)
Repository (repo)	Prosjekt under versjonskontroll (5)
Resource group	Logisk mappe for Azure-ressurser (7.1)
Reverse proxy	Står foran servere og tar imot forespørsler (2)
Rollback	Gå tilbake til forrige versjon (12)
RPO / RTO	Maks datatap / maks gjenopprettingstid (6)

Begrep	Forklaring
SAS	Shared Access Signature – tidsbegrenset tilgang til Azure Storage (7.3)
Scrum	Rammeverk for smidig arbeid i sprinter (8)
SDLC	Livssyklusen for programvareutvikling (8)
Secret	Hemmelighet; også Kubernetes-objekt (10, 13)
Serverless	Kode som kjører ved hendelser uten serveradministrasjon (6, 7.2)
Service (Kubernetes)	Stabil adresse og lastbalansering for pods (10)
Service principal	Identitet for en applikasjon i Entra ID (7.1)
Shared responsibility	Delt sikkerhetsansvar mellom skyleverandør og kunde (6)
SLA / SLO / SLI	Avtale / mål / måling for tjenestekvalitet (8)
Smoke test	Rask test rett etter deploy (12)
SRE	Site Reliability Engineering (8)
SSH	Kryptert fjerninnlogging (2, 3)
Staging	Testmiljø som ligner produksjon (12)
State (Terraform)	Terraforms kobling mellom kode og ekte ressurser (11)
Subnet	Del av et nettverks adresseområde (2, 7.2)
Subscription	Azure-abonnement: fakturerings- og tilgangsgrense (7.1)
systemd / systemctl	Linux' tjenestesystem / kommandoen for det (3)
Tag	Nøkkel–verdi-merkelapp på ressurser; også versjonsmerke i Git/Docker
TCP / UDP	Pålitelig / rask transportprotokoll (2)
Terraform	Deklarativt IaC-verktøy fra HashiCorp (11)
TLS	Kryptering av nettverkstrafikk (2)
Toil	Manuelt, repeterende arbeid som bør automatiseres (8)
Trace / span	En forespørsels vei gjennom tjenester / ett ledd i den (13)
TTL	Hvor lenge et DNS-svar caches (2)

Begrep	Forklaring
UDR	User Defined Route – egen rute i Azure (7.2)
Virtualisering	Kjøre flere virtuelle maskiner på én fysisk (1)
VM	Virtuell maskin (1, 7.2)
VNet	Virtuelt nettverk i Azure (7.2)
Volume	Varig lagring for containere (9)
VPN	Kryptert tunnel mellom nettverk (2)
WAF	Web Application Firewall (2, 7.2)
Workload identity federation	Tillit mellom eksternt system og Entra ID uten hemmelighet (7.1, 12)
YAML	Menneskevennlig dataformat brukt i konfigurasjon (4)
Zero Trust	«Stol aldri, verifiser alltid» (13)

Vedlegg B – Hurtigreferanser

Linux

Oppgave	Kommando
Hvor er jeg / list filer	`pwd` · `ls -la`
Bytt mappe	`cd /sti` · `cd ..` · `cd ~`
Lag / slett mappe	`mkdir -p a/b` · `rm -r mappe`
Kopier / flytt	`cp -r kilde mål` · `mv gammel ny`
Les fil	`cat` · `less` · `head -n 20` · `tail -f`
Søk i tekst	`grep -rin "tekst" .`
Finn filer	`find / -name "*.log" -size +100M`
Rettigheter / eier	`chmod 640 fil` · `chown bruker:gruppe fil`
Brukere	`sudo useradd -m -s /bin/bash navn` · `sudo usermod -aG gruppe navn` · `id navn`
Prosesser	`ps aux` · `top` · `kill PID`
Tjenester	`sudo systemctl status/start/stop/restart/enable nginx`
Logger	`journalctl -u nginx -f` · `tail -f /var/log/syslog`
Pakker (Ubuntu)	`sudo apt update && sudo apt install -y pakke`
Disk / minne	`df -h` · `du -sh *` · `free -h`
Nettverk	`ip addr` · `ip route` · `ss -tlnp`
Fjerninnlogging / kopiering	`ssh bruker@vert` · `scp fil bruker@vert:/sti`
Cron	`crontab -e` · `crontab -l`
Brannmur (Ubuntu)	`sudo ufw allow 22/tcp` · `sudo ufw enable` · `sudo ufw status`

Nettverksfeilsøking

Oppgave	Kommando
DNS	`dig +short navn` · `nslookup navn`
Rekkevidde	`ping -c 4 vert` · `traceroute vert` · `mtr vert`
Port åpen?	`nc -zv vert 443`
HTTP	`curl -v https://vert` · `curl -I url`
TLS-sertifikat	`openssl s_client -connect vert:443 -servername vert`
Trafikk	`sudo tcpdump -i eth0 port 53`

Git

Oppgave	Kommando
Start / klon	`git init` · `git clone URL`
Status / forskjeller	`git status` · `git diff` · `git diff --staged`
Commit	`git add -p` · `git commit -m "melding"`
Historikk	`git log --oneline --graph --all`
Branch	`git switch -c ny` · `git switch main` · `git branch -d gammel`
Synkroniser	`git pull` · `git push -u origin branch` · `git fetch`
Merge / rebase	`git merge branch` · `git rebase main`
Angre	`git restore fil` · `git restore --staged fil` · `git revert HASH`
Midlertidig lagring	`git stash` · `git stash pop`
Tag	`git tag -a v1.0.0 -m "..."` · `git push origin v1.0.0`

Docker

Oppgave	Kommando
Bygg	`docker build -t navn:tag .`

Oppgave	Kommando
Kjør	`docker run -d -p 8080:80 --name web image:tag`
List	`docker ps -a` · `docker images`
Logger / shell	`docker logs -f web` · `docker exec -it web sh`
Stopp / slett	`docker stop web` · `docker rm web` · `docker rmi image`
Inspiser	`docker inspect web` · `docker stats`
Register	`docker login` · `docker tag` · `docker push`
Compose	`docker compose up -d --build` · `docker compose logs -f` · `docker compose down -v`
Rydd	`docker system df` · `docker system prune`

Kubernetes (kubectl)

Oppgave	Kommando
Kontekst og namespace	`kubectl config get-contexts` · `kubectl config set-context --current --namespace=ns`
List	`kubectl get pods,svc,deploy -n ns -o wide`
Detaljer og hendelser	`kubectl describe pod navn` · `kubectl get events --sort-by=.lastTimestamp`
Logger	`kubectl logs pod [-c container] [-f] [--previous]`
Shell	`kubectl exec -it pod -- sh`
Bruk YAML	`kubectl apply -f fil.yaml` · `kubectl delete -f fil.yaml`
Skaler / oppdater	`kubectl scale deploy/web --replicas=3` · `kubectl set image deploy/web web=img:2`
Utrulling	`kubectl rollout status/history/undo deploy/web`
Port-forward	`kubectl port-forward svc/web 8080:80`
Ressursbruk	`kubectl top nodes` · `kubectl top pods`
Tilgang	`kubectl auth can-i create pods --as=bruker`
Helm	`helm upgrade --install navn ./chart -f values.yaml` · `helm rollback navn 1`

Azure CLI

Oppgave	Kommando
Innlogging	`az login` · `az account set --subscription ID` · `az account show`
Ressursgruppe	`az group create -n rg -l norwayeast` · `az group delete -n rg --yes --no-wait`
VM	`az vm create ...` · `az vm deallocate -g rg -n vm` · `az vm list -d -o table`
Nettverk	`az network vnet create ...` · `az network nsg rule create ...`
Lagring	`az storage account create ...` · `az storage blob upload --auth-mode login ...`
Web-app	`az webapp up --runtime "PYTHON:3.12" --sku F1 -n app` · `az webapp log tail`
Key Vault	`az keyvault secret set/show --vault-name kv -n navn`
RBAC	`az role assignment create --assignee ID --role Reader --scope SCOPE`
Policy	`az policy assignment create --policy ID --scope SCOPE`
ACR	`az acr create ...` · `az acr login -n acr` · `az acr build -r acr -t img:tag .`
AKS	`az aks create ...` · `az aks get-credentials -g rg -n aks` · `az aks stop`
Deploy Bicep	`az deployment group what-if/create -g rg -f main.bicep`
Filtrere utdata	`--query "[].name" -o tsv` · `-o table`

Terraform

Oppgave	Kommando
Initialiser	`terraform init` (`-migrate-state` ved backend-bytte)
Format / valider	`terraform fmt -recursive` · `terraform validate`

Oppgave	Kommando
Planlegg / utfør	`terraform plan -out=tfplan` · `terraform apply tfplan`
Slett alt	`terraform destroy`
Outputs	`terraform output` · `terraform output -raw navn`
State	`terraform state list` · `terraform state show adresse`
Variabler	`-var-file=prod.tfvars` · `export TF_VAR_navn=verdi`

Python – mønstre du ofte trenger

```
import os, json, logging, argparse, subprocess, requests
verdi = os.environ.get("NAVN", "standard")                    # miljøvariabel
data = json.load(open("fil.json"))                            # les JSON
svar = requests.get(url, timeout=10); svar.raise_for_status() # HTTP-kall
logging.basicConfig(level=logging.INFO)                       # logging
subprocess.run(["ls", "-la"], check=True)                     # kjør kommando
```

KQL og PromQL

```
Tabell | where TimeGenerated > ago(1h) | summarize count() by bin(TimeGenerated, 5m) |
render timechart
```

```
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

Subnetting

CIDR	Adresser	Brukbare i Azure
/16	65 536	65 531
/24	256	251
/25	128	123
/26	64	59
/27	32	27

CIDR	Adresser	Brukbare i Azure
/28	16	11
/29	8	3

Formel: `2^(32 – prefiks)`. Azure reserverer 5 adresser per subnett.

Vedlegg C – Oversikt over sertifiseringer

> **Merk:** Sertifiseringer, eksamenskoder, innhold og priser endres jevnlig. Tabellen gjenspeiler situasjonen slik den var kjent per oktober 2026. Sjekk alltid den offisielle siden før du planlegger.

Sertifisering	Utsteder	Nivå	Relevans	Når i læringsplanen
AZ-900 Azure Fundamentals	Microsoft	Grunnleggende	Ordforråd og oversikt over Azure	Måned 4–5
AZ-104 Azure Administrator Associate	Microsoft	Associate	Praktisk Azure-drift – svært etterspurt	Måned 9–12
AZ-400 DevOps Engineer Expert	Microsoft	Expert	DevOps-prosesser og verktøy i Azure/GitHub (krever AZ-104 eller AZ-204)	Etter studiene / første jobb
AZ-305 Solutions Architect Expert	Microsoft	Expert	Arkitektur (krever AZ-104)	Etter noen års erfaring
AZ-500 Security Engineer	Microsoft	Associate	Sky-sikkerhet	Ved interesse for sikkerhet
AI-900 AI Fundamentals	Microsoft	Grunnleggende	AI-begreper i Azure	Ved interesse for AI/MLOps
DP-100 Azure Data Scientist	Microsoft	Associate	Azure Machine Learning	MLOps-retning
GitHub Foundations / GitHub Actions	GitHub	Grunnleggende/Associate	Git, GitHub og CI/CD	Måned 3 og 8
Terraform Associate	HashiCorp	Associate	IaC med Terraform	Måned 8–10
CKAD / CKA	CNCF / Linux Foundation	Praktisk (lab-eksamen)	Kubernetes for utviklere / administratorer	Etter AZ-104

Sertifisering	Utsteder	Nivå	Relevans	Når i læringsplanen
KCNA	CNCF	Grunnleggende	Kubernetes og cloud-native grunnlag	Måned 8
AWS Cloud Practitioner	AWS	Grunnleggende	Om du vil vise bredde	Valgfritt
Linux Foundation LFCS / Red Hat RHCSA	LF / Red Hat	Praktisk	Linux-administrasjon	Valgfritt

Råd om sertifiseringer:

- Sertifiseringer åpner dører, men **prosjekter og forståelse** får deg jobben. Kombiner dem.
- Ikke samle sertifikater uten praksis – intervjuere merker det raskt.
- Bruk **gratis ressurser**: Microsoft Learn (gratis læringsstier og *Practice Assessments*), studentrabatter, og gratis vouchere fra arrangementer.
- Microsoft Associate- og Expert-sertifiseringer må **fornyes årlig** med en gratis nettbasert test.

Vedlegg D – Kilder og videre læring

Informasjonen i boken bygger på offisiell dokumentasjon fra leverandørene. Lenkene under var gyldige per oktober 2026. Dokumentasjonen oppdateres jevnlig – når boken og den offisielle dokumentasjonen er uenige, er det dokumentasjonen som gjelder.

Offisiell dokumentasjon

Område	Kilde
Azure (alle tjenester)	Microsoft Learn: https://learn.microsoft.com/azure/
Azure CLI	https://learn.microsoft.com/cli/azure/
Azure-arkitektur og Well-Architected	https://learn.microsoft.com/azure/architecture/ og https://learn.microsoft.com/azure/well-architected/
Cloud Adoption Framework (navngivning, landingssoner)	https://learn.microsoft.com/azure/cloud-adoption-framework/
Bicep	https://learn.microsoft.com/azure/azure-resource-manager/bicep/
Azure Verified Modules	https://azure.github.io/Azure-Verified-Modules/
Azure-priser og kalkulator	https://azure.microsoft.com/pricing/ og https://azure.microsoft.com/pricing/calculator/
Azure SLA-er	https://www.microsoft.com/licensing/docs/view/Service-Level-Agreements-SLA-for-Online-Services
Microsoft-sertifiseringer	https://learn.microsoft.com/credentials/
Azure DevOps	https://learn.microsoft.com/azure/devops/
Microsoft Entra ID	https://learn.microsoft.com/entra/
Docker	https://docs.docker.com/
Kubernetes	https://kubernetes.io/docs/
Helm	https://helm.sh/docs/
Terraform og azurerm-provider	https://developer.hashicorp.com/terraform/docs og https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs
OpenTofu	https://opentofu.org/docs/
Ansible	https://docs.ansible.com/
Git	https://git-scm.com/doc (inkl. gratis boken *Pro Git*)
GitHub og GitHub Actions	https://docs.github.com/

Område	Kilde
Python	https://docs.python.org/3/
Prometheus	https://prometheus.io/docs/
Grafana	https://grafana.com/docs/
OpenTelemetry	https://opentelemetry.io/docs/
Argo CD / Flux	https://argo-cd.readthedocs.io/ og https://fluxcd.io/docs/
OWASP Top 10 og LLM Top 10	https://owasp.org/www-project-top-ten/ og https://genai.owasp.org/
DORA-forskningen	https://dora.dev/
Google SRE-bøkene (gratis på nett)	https://sre.google/books/
The Twelve-Factor App	https://12factor.net/
NIST-definisjonen av cloud (SP 800-145)	https://csrc.nist.gov/pubs/sp/800/145/final
Ubuntu Server	https://ubuntu.com/server/docs

Bøker

- *The Phoenix Project* – Gene Kim, Kevin Behr, George Spafford (DevOps som roman – start her).
- *The DevOps Handbook* – Gene Kim, Jez Humble, Patrick Debois, John Willis, Nicole Forsgren.
- *Accelerate* – Nicole Forsgren, Jez Humble, Gene Kim (forskningen bak DORA).
- *Site Reliability Engineering* – Google (gratis på sre.google).
- *Kubernetes Up & Running* – Brendan Burns, Joe Beda, Kelsey Hightower m.fl.
- *Terraform: Up & Running* – Yevgeniy Brikman.
- *Designing Machine Learning Systems* – Chip Huyen (for MLOps).
- *Team Topologies* – Matthew Skelton, Manuel Pais (for Platform Engineering).

Gratis øvingsressurser

- **Microsoft Learn** – læringsstier, sandkasser og *Practice Assessments* for sertifiseringer.
- **Offisielle AZ-104-labber** – søk etter «MicrosoftLearning AZ-104» på GitHub.
- **Kubernetes:** kind, minikube, Killercoda (interaktive scenarioer i nettleseren).
- **Linux:** OverTheWire Bandit (terminalspill), Linux Journey.
- **Git:** Learn Git Branching (visuell øving).
- **Generelt:** roadmap.sh (veikart for DevOps og andre roller).

Norske fagmiljøer

- Meetups og fagnettverk innen DevOps, Kubernetes, Azure og sky i Oslo, Bergen, Trondheim og Stavanger (søk på meetup.com).
- Konferanser som NDC Oslo, JavaZone og Booster (Bergen) har mye sky- og DevOps-innhold; mange har studentrabatter eller frivillige-ordninger.
- Studentforeninger og Noroffs eget fagmiljø – still spørsmål og del prosjekter.

Vedlegg E – Personlig 12-måneders læringsplan

Planen er laget for en Noroff-student i DevOps Engineering som bruker **ca. 8–12 timer i uken** på selvstudier og prosjekter **i tillegg til** studiets pensum. Juster tempoet etter emnene dine – når studiet dekker et tema, bruk boken som støtte og prosjektene som påbygg. Det viktigste er **jevn innsats**: litt hver uke slår mye av og til.

```
 Mnd:  1    2    3    4    5    6    7    8    9    10   11   12
       ├────Grunnmur─────┤├─Plattform──┤├────Verktøy──────┤├Spesialisering┤
       IT/Nett  Linux Py/Git Cloud Azure DevOps Docker K8s IaC CI/CD Sikk/Mon
AI/Sluttprosj.
 Sert:                (GH Found.) AZ-900                     (Terraform)  AZ-104
 Jobb:                                    LinkedIn/CV    Søk sommerjobb/internship ──►
```

Ukentlig rytme (forslag):

Dag	Aktivitet	Tid
Man	Les kapittel/del av kapittel	1,5 t
Ons	Gjør øvingsoppgaver i terminal/Azure	2 t
Lør	Prosjektarbeid	3–4 t
Søn	Repetisjon: quiz, notater, oppdater GitHub	1 t

Måned 1 – IT-grunnlag og nettverk

Hva: Datamaskiner, OS, servere, internett, DNS, HTTP, IP, subnetting, TCP/UDP, porter, brannmur, TLS. **Hvorfor:** Alt annet i DevOps bygger på dette; nettverk er den vanligste feilkilden i skyen. **Kapitler:** 1 og 2.

Uke	Fokus	Øving
1	Kapittel 1: datamaskin, OS, server, app, database	Installer VirtualBox/WSL og Ubuntu
2	Kapittel 1: internett, DNS, HTTP, API, virtualisering	`curl`, `nslookup`, forklar «hva skjer når jeg åpner en nettside» høyt
3	Kapittel 2: IP, CIDR, subnetting, DNS, DHCP	20 subnetting-oppgaver til du klarer dem uten hjelp
4	Kapittel 2: TCP/UDP, porter, NAT, LB, proxy, TLS, SSH, feilsøking	Lag SSH-nøkkel; feilsøk med `dig`, `nc`, `curl -v`

Test deg selv: Quizene i kapittel 1 og 2 med minst 80 % riktig. Tegn nettverksdiagrammet for et hjemmenett. **GitHub:** Opprett kontoen og et repo `devops-notater` med notatene dine i Markdown.

Måned 2 – Linux

Hva: Terminal, filsystem, rettigheter, brukere, prosesser, systemd, pakker, logger, Bash, cron, sikkerhet. **Hvorfor:** Nesten alle servere, containere og CI-runnere kjører Linux. **Kapitler:** 3.

Uke	Fokus	Øving
1	Filsystem, filer, grep/find, pipes	Bruk bare terminalen en hel uke
2	Brukere, rettigheter, prosesser, tjenester, pakker	Øvingsoppgavene i kapittel 3
3	Bash-skript, cron, miljøvariabler, logger	Skriv `backup.sh` og `diskvarsel.sh`
4	Sikkerhet og feilsøking	**Prosjekt 1**

Test deg selv: Kan du sette opp en webserver med herdet SSH og brannmur uten å se i boken? **GitHub:** Repo for prosjekt 1 med README.

Måned 3 – Python, Bash og Git

Hva: Python-grunnlag, JSON/YAML, API-kall, feilhåndtering, logging; Git og GitHub-arbeidsflyt. **Hvorfor:** Automatisering er kjernen i DevOps; Git er hverdagsverktøyet i alle team. **Kapitler:** 4 og 5.

Uke	Fokus	Øving
1	Python: variabler, typer, betingelser, løkker, funksjoner	Små oppgaver hver dag
2	Lister, ordbøker, filer, JSON/YAML, API-kall, feilhåndtering	`uptime_check.py` fra kapittel 4
3	Git: commit, branch, merge, konflikter, PR	Prosjektet i kapittel 5
4	GitHub Actions-grunnlag, tester med pytest	**Prosjekt 4** (GitHub-profil og portefølje)

Test deg selv: Skriv et skript som henter data fra et API, filtrerer det og lagrer resultatet – med tester og CI. **GitHub:** Profil-README, porteføljerepo, Python-skriptene med CI-badge. **Sertifisering (valgfritt):** GitHub Foundations.

Måned 4 – Cloud Computing og Azure-grunnmur

Hva: Sky-konsepter, IaaS/PaaS/SaaS, regioner, HA/DR, kostnader; Azure-hierarki, Entra ID, RBAC, Policy, kostnadsstyring. **Hvorfor:** Dette er fundamentet for alt skyarbeid og pensum for AZ-900. **Kapitler:** 6 og 7.1.

Uke	Fokus	Øving
1	Kapittel 6	Pricing Calculator-øvelsene
2	Azure-konto (Azure for Students), portal, CLI, **budsjettvarsel**	Ressursgrupper, tags, låser
3	Entra ID, RBAC, managed identities	Testbruker + gruppe + Reader-tildeling
4	Azure Policy, Cost Management	**Prosjekt 2** (kostnadsjeger)

Test deg selv: Microsoft Learn sin AZ-900 *Practice Assessment*. **GitHub:** Prosjekt 2.

Måned 5 – Azure-tjenester og AZ-900

Hva: VNet, NSG, VM, Load Balancer, App Service, Functions, Storage, SQL, Key Vault, Monitor, Backup. **Hvorfor:** De mest brukte Azure-tjenestene; gir deg praktisk erfaring før DevOps-verktøyene. **Kapitler:** 7.2, 7.3, 7.5.

Uke	Fokus	Øving
1	Nettverk og VM-er	**Prosjekt 3**
2	App Service, Functions, ACR	Deploy Flask-appen med `az webapp up`
3	Storage, SQL, Key Vault, Monitor, Log Analytics	Øvelsene i 7.3
4	AZ-900-repetisjon og prøveeksamen	**Ta AZ-900**

Test deg selv: Prøveeksamenen i 7.5 med 80 %+. **GitHub:** Prosjekt 3. **Sertifisering:** AZ-900.

Måned 6 – DevOps-kultur, Azure DevOps og Bicep

Hva: DevOps-prinsipper, Agile/Scrum, CI/CD-begreper, deploy-strategier, SRE; Azure DevOps, Pipelines og Bicep. **Hvorfor:** Du skal kunne forklare *hvorfor* DevOps – det spørres om i alle intervjuer. **Kapitler:** 8 og 7.4.

Uke	Fokus	Øving
1	Kapittel 8: kultur, DORA, Agile/Scrum	Kanban-tavle for dine prosjekter
2	Kapittel 8: CI/CD, deploy-strategier, SRE, DevSecOps	Skriv en postmortem (øvingsoppgave)
3	Azure DevOps og Pipelines	Pipeline i Azure DevOps for Flask-appen
4	Bicep	`main.bicep`-øvelsen + **Prosjekt 6** (nettside på Azure)

Test deg selv: Forklar CI vs. Continuous Delivery vs. Continuous Deployment, SLO vs. SLA, og blue-green vs. canary – muntlig, uten notater. **GitHub:** Prosjekt 6 (din CV-nettside!). **Jobb:** Oppdater CV og LinkedIn med prosjektene så langt.

Måned 7 – Docker

Hva: Images, containere, Dockerfile, multi-stage, volumes, nettverk, Compose, registre, sikkerhet. **Hvorfor:** Containere er standard leveringsformat for moderne applikasjoner. **Kapitler:** 9.

Uke	Fokus	Øving
1	Images, containere, grunnkommandoer	Øvingsoppgave 1–3 i kapittel 9
2	Dockerfile, cache, multi-stage	Optimaliser Flask-imaget
3	Volumes, nettverk, Compose	Flask + PostgreSQL + Redis i Compose
4	Sikkerhet, skanning, publisering	**Prosjekt 5**

Test deg selv: Quiz kapittel 9; feilsøk en container som krasjer uten å se i boken. **GitHub:** Prosjekt 5. **Jobb:** Begynn å se etter **sommerjobber og internships** – mange norske selskaper har søknadsfrist på høsten og tidlig vinter for neste sommer. Følg med på karrieresidene til konsulentselskaper, banker, telekom og offentlige virksomheter.

Måned 8 – Kubernetes

Hva: Arkitektur, pods, deployments, services, ingress/Gateway API, config, secrets, lagring, HPA, Helm, sikkerhet, AKS. **Hvorfor:** Kubernetes er standard for drift av containere i stor skala, og etterspurt i jobbannonser. **Kapitler:** 10.

Uke	Fokus	Øving
1	Arkitektur, pods, deployments	kind-cluster, nginx-deployment
2	Services, ingress, ConfigMaps, Secrets, namespaces	Flask-appen i Kubernetes
3	Lagring, HPA, probes, Helm	Lag din egen Helm chart
4	Sikkerhet, AKS	**Prosjekt 9**

Test deg selv: Quiz kapittel 10; fiks `CrashLoopBackOff` og `ImagePullBackOff` på egen hånd. **GitHub:** Prosjekt 9. **Sertifisering (valgfritt):** KCNA.

Måned 9 – Infrastructure as Code

Hva: Terraform (providers, resources, variables, state, modules, remote state), Ansible, GitOps. **Hvorfor:** All profesjonell skyinfrastruktur bygges som kode. **Kapitler:** 11.

Uke	Fokus	Øving
1	Terraform-grunnlag	Ressursgruppe + VNet + subnett med `for_each`
2	State, remote state, moduler	Flytt state til Azure Blob; lag en modul
3	Det komplette eksemplet i kapittel 11; Ansible	Webapp + Key Vault + overvåking; Ansible-playbook
4	Miljøer og pipeline	**Prosjekt 8** (påbegynn)

Test deg selv: Quiz kapittel 11; forklar state-filen og hvorfor den må beskyttes. **GitHub:** Terraform-repo. **Sertifisering (valgfritt):** Terraform Associate.

Måned 10 – CI/CD og AZ-104-forberedelse

Hva: GitHub Actions i dybden, OIDC, artefakter, rollback, blue-green, canary; AZ-104-pensum. **Hvorfor:** Pipelines binder alt sammen – det er her DevOps blir synlig. **Kapitler:** 12 og 7.6.

Uke	Fokus	Øving
1	GitHub Actions i dybden, testing	Workflow med matrix, services og cache
2	OIDC til Azure, environments	**Prosjekt 7**

Uke	Fokus	Øving
3	Den komplette pipelinen i kapittel 12 (canary)	Fullfør **prosjekt 8**
4	AZ-104-studieplan uke 1–2 (identitet, styring)	Lab 1 og 2 i 7.6

Test deg selv: Quiz kapittel 12; tegn hele pipelinen din fra hukommelsen. **GitHub:** Prosjekt 7 og 8. **Jobb:** Søk aktivt på sommerjobber/internships og junior-/trainee-programmer. Bruk prosjektene som samtaletema.

Måned 11 – Sikkerhet, monitoring og AZ-104

Hva: IAM, Zero Trust, hemmeligheter, kryptering, skanning, backup/DR, logger, metrikker, traces, Prometheus, Grafana, hendelseshåndtering; resten av AZ-104. **Hvorfor:** Sikkerhet og driftbarhet skiller en hobbyløsning fra en produksjonsløsning. **Kapitler:** 13 og 7.6.

Uke	Fokus	Øving
1	Kapittel 13 del A (sikkerhet)	Gå gjennom Defender for Cloud-anbefalinger; Trivy/pip-audit
2	Kapittel 13 del B og C (monitoring, hendelser)	**Prosjekt 12**
3	AZ-104 lab 3–6 (lagring, compute, nettverk)	Labbene i 7.6
4	AZ-104 lab 7–8, prøveeksamen	Microsoft *Practice Assessment* til 80 %+

Test deg selv: Prøveeksamenen i 7.6. **Sertifisering:** Ta **AZ-104** når du jevnt over får 80 %+ (i måned 11 eller 12). **GitHub:** Prosjekt 12.

Måned 12 – AI-infrastruktur, sluttprosjekt og jobbsøking

Hva: MLOps, AI-infrastruktur, LLM-drift; sluttprosjektet som binder alt sammen. **Hvorfor:** Sluttprosjektet er porteføljens hovedverk; AI-kunnskap gir deg et fortrinn. **Kapitler:** 14 og 15 (prosjekt 10 og 11).

Uke	Fokus	Øving
1	Kapittel 14	Ollama lokalt; modell-API i Docker
2	**Prosjekt 10** fase 1–3	Design, IaC, app

Uke	Fokus	Øving
3	**Prosjekt 10** fase 4–6	CI/CD, overvåking, gjenoppretting
4	**Prosjekt 10** fase 7 + jobbsøking	README, demovideo, oppryd-ding; søknader

Test deg selv: Kan du forklare hele sluttprosjektet på 5 minutter – og svare på «hvorfor valgte du X i stedet for Y?» **GitHub:** Prosjekt 10 (festet øverst), eventuelt prosjekt 11. **Videre:** Velg retning – dy-pere i Azure/plattform (AZ-400, CKA), sikkerhet (AZ-500) eller AI/MLOps (DP-100, prosjekt 11 og læringsveien i kapittel 14).

Når kan jeg begynne å søke?

Type	Når	Hva du bør ha
Sommerjobb / internship	Søk fra måned 7–10 (mange fris-ter er på høsten/vinteren for neste sommer)	3–5 prosjekter på GitHub, AZ-900, god forståelse av Linux, Git, Docker og Azure-grunnlag
Deltidsjobb / studentstilling (drift, support, IT-avdeling)	Når som helst fra måned 3	Grunnleggende Linux/nettverk og vilje til å lære – praktisk erfaring er gull verdt
Trainee / junior DevOps / Cloud	Siste studieår	Sluttprosjekt, AZ-104, 6–10 pro-sjekter, gjerne praksisplass eller sommerjobb

Søknadstips:

- Skreddersy CV-en til annonsen; fremhev prosjekter som matcher teknologiene de nevner.
- Lenk til GitHub og CV-nettsiden din (prosjekt 6) øverst.
- Forbered 2–3 STAR-historier (kapittel 15).
- Vær ærlig om hva du kan – og vis at du lærer raskt og systematisk.
- Bygg nettverk: meetups, LinkedIn, studentarrangementer, bedriftspresentasjoner.

Månedlig sjekkliste

- ☐ Har jeg lest kapitlene for måneden og tatt quizene?
- ☐ Har jeg gjort øvingsoppgavene i terminalen/Azure – ikke bare lest?
- ☐ Er månedens prosjekt på GitHub med README og «Hva jeg lærte»?
- ☐ Har jeg ryddet opp alle sky-ressurser og sjekket Cost Management?
- ☐ Har jeg oppdatert CV/LinkedIn med nye ferdigheter?

- [ ] Hva var vanskeligst denne måneden – og hva gjør jeg for å styrke det?

Avslutning

Du har nå et kart over hele landskapet – fra hvordan en CPU fungerer til hvordan en språkmodell driftes i Kubernetes. Kartet er ikke terrenget. Ferdighetene kommer når du går terrenget selv: skriver kommandoene, ser dem feile, leser feilmeldingene, og prøver igjen.

Husk tre ting:

1. **Grunnmuren betyr mest.** Linux, nettverk og forståelse av *hvorfor* gjør alt det andre lettere.
2. **Bygg, dokumenter, del.** Et prosjekt på GitHub med en ærlig README er verdt mer enn ti kurs du bare har sett.
3. **Rydd opp i skyen.** Hver gang.

Lykke til med studiene – og velkommen til DevOps.